Site Automation

Operations runbooks: health checks, troubleshooting, ServiceNow, POA&M and patching - with AAP and, later, ACT

Setup guide for new AAP admins | site-automation 0.15.5 | October 2026

What this is. A step-by-step guide to setting up the site's operations runbooks in Ansible Automation Platform: health checks (system, compliance, MariaDB / MySQL, certificates, patching), troubleshooting, ServiceNow tickets, POA&M status and patching, and later ACT (GenAI). It starts with what each AAP object *is* and walks through every click, saying what each step does, why, and what you should see.

How to read it

New to Ansible or AAP: read chapter 1 first, then do chapter 2 in order. Inventory already in AAP? Chapter 3 replaces the inventory parts of chapter 2. The later chapters are reference: open them when you build a workflow, read a finding, add ACT, change a setting, approve commands or set up the MariaDB / MySQL check.

Every setting, with its default and what it does - and where to put it - is in its own PDF, the *Settings Reference*: keep it open next to this one.

Rendered from `docs/` in the site-automation repository (version 0.15.5). The repository is the source of truth.

Host names and addresses in this guide are placeholders. Copy YAML from the repository files, not from the PDF.

Contents

1. Start here: the pieces and how they fit 8

What the runbooks do . . . 9
The ideas behind Ansible, in plain words . . . 9
The AAP objects, and what each one is for . . . 10
How a health check works, start to finish . . . 11
The findings: the one idea that makes everything modular . . . 11
Where things are . . . 12
The order to do things in . . . 12

2. Setting up AAP, step by step 13

Before you start . . . 13
Step 1. Put the repository in your work Git, with your inventory . . . 13
Step 2. Credential types . . . 14
Step 3. Credentials . . . 14
Step 4. Project . . . 15
Step 5. Inventory (filled from the repository) . . . 15
Step 6. The first job template: Health check . . . 16
Step 7. Run it on one host and read the output . . . 17
Step 8. Tune the thresholds . . . 17
Step 9. The other job templates . . . 18
Step 10. ServiceNow . . . 22
Step 11. Workflows and schedules . . . 23
Step 12. Who can do what . . . 23
If something goes wrong . . . 24

3. Using the inventory you already have in AAP 25

The easy way: your settings files, already named after your groups . . . 25
How variables work in AAP (two minutes) . . . 27
Step 1. Look at what you have . . . 27
Step 2. Create the groups the runbooks use . . . 28
Step 3. Settings for every host (inventory variables) . . . 29
Step 4. Settings for particular groups . . . 29
Step 5. Settings for one host (only when one host is different) . . . 30
Step 6. Use your inventory in the job templates . . . 30

Step 7. Check that it worked . . . 31
Which YAML files in the repository do I change? . . . 31
4. ServiceNow: set up, test and verify 32
What you need . . . 32
Step 1. Ask the ServiceNow admins for an API account . . . 32
Step 2. The addresses . . . 33
Step 3. The network path from AAP . . . 34
Step 4. Check from a shell first (optional, fastest) . . . 34
Step 5. The credential in AAP . . . 35
Step 6. Your settings . . . 35
Step 7. The ServiceNow - test ticket template . . . 36
Step 8. Run it: first a dry run, then for real . . . 36
Step 9. Check the ticket in ServiceNow . . . 37
When a step fails . . . 38
Next . . . 38
5. Workflows and schedules 40
How to build any workflow, click by click . . . 40
Adding ACT to a workflow: what "on fail" means, and where site_act goes . . . 41
Linux workflows . . . 42
Windows workflows . . . 48
Linux and Windows together . . . 49
VMware workflows . . . 49
Schedules . . . 50
Things that trip people up . . . 51
6. Dry runs: try anything, change nothing 53
Where to set it . . . 53
What each template does in a dry run . . . 54
Examples . . . 55
What a dry run cannot tell you . . . 56
7. The runbooks: every check and what to do 57
Health check (playbooks/health_check.yml) . . . 57
Troubleshoot (playbooks/troubleshoot.yml) . . . 70
Certificate report (playbooks/cert_report.yml) . . . 71

Certificate renewal: the CSRs (playbooks/cert_renew_csr.yml) . . . 72
POA&M status (playbooks/poam_status.yml) . . . 75
ServiceNow tickets (playbooks/servicenow_tickets.yml) . . . 75
ServiceNow - test ticket (playbooks/servicenow_test_ticket.yml) . . . 76
ServiceNow health (playbooks/servicenow_health.yml) . . . 76
Patch hosts (playbooks/patch_hosts.yml) . . . 76
Apply approved ACT fix (playbooks/act_fix_approved.yml) . . . 77
VMware jobs and reports (playbooks/vm_*.yml) . . . 77
Windows hosts . . . 77
Not built yet (and why) . . . 77

8. Adding ACT (GenAI) later 79

How they fit: checks find WHAT, ACT finds WHY . . . 79
The three levels . . . 79
Turn it on . . . 80
Both endpoint formats and :probe (ACT 0.6.19 and newer) . . . 83
ACT for vCenter alarms (it runs on the AAP side) . . . 86
Write your own check (it works with the report, tickets and ACT automatically) . . . 86
Use ACT in a playbook of your own . . . 87
What ACT is told, and what it cannot do . . . 88

9. Windows servers 89

Step 1. WinRM on the servers . . . 90
Step 2. The execution environment . . . 90
Step 3. The credential . . . 91
Step 4. The Windows inventory . . . 91
Step 5. Test the connection . . . 92
Step 6. The job templates . . . 92
Step 7. Workflows and schedules . . . 93
Start small: the pilot order . . . 94
Patching through Software Center . . . 94
ACT on the servers (C:\ProgramData\act) . . . 95
Settings: what each check looks at, and how to change it . . . 96
If something goes wrong . . . 98
Not in this release (next) . . . 98

10. VMware jobs (vCenter): restart, snapshots, VLAN, reports, alarms and ACT 99

Before you start . . . 101
Which VMs a job acts on . . . 103
The templates . . . 104
Snapshots: the report, the cleanup, and the approval workflow . . . 105
Health: every datacenter, cluster and host . . . 108
VMs created: the weekly report . . . 110
Alarms: the report, and ACT's analysis . . . 111
Capacity planning: overall and per cluster . . . 114
Deploy a VM from a ServiceNow ticket . . . 116
ESXi security settings (every night): SSH off, timeouts, lockdown . . . 141
Email the report . . . 143
What you see . . . 145
Troubleshooting . . . 146
Limits . . . 159

11. NetApp ONTAP health report 160

What the report shows . . . 160
Before you start . . . 161
Set it up . . . 162
Settings . . . 163
Troubleshooting . . . 164
Limits . . . 165

12. Emailed reports: the formatted email and its look 166

Which jobs email . . . 167
Turning it on . . . 168
Green or red: when a report finds problems . . . 169
What a report looks like . . . 170
Changing the look . . . 170
Emailing a report from another playbook . . . 170
Troubleshooting . . . 171

13. Secrets: where they live 173

Recommendation . . . 173
Where each secret lives . . . 174

Rules for every playbook in this repository . . . 175
The real trust boundary . . . 176
Keeping secrets out of Git . . . 176
Rotation checklist . . . 177
14. Approved commands: what ACT may run by itself 178
Read this first: two safety rules that you cannot switch off . . . 178
How a command becomes "approved": the three ways . . . 178
Add an approved command, step by step . . . 179
How to verify it worked . . . 181
Reference: what the guard rails are . . . 183
15. The MariaDB and MySQL check, step by step 184
The short version . . . 184
1. Which hosts the check runs on . . . 184
2. Tell it where MariaDB is . . . 185
3. Where to put the settings . . . 186
4. The login: the "MariaDB monitor" credential . . . 186
5. What it checks, and the limits you can change . . . 187
How to verify . . . 189
What was tested, and what was not . . . 189
If it does not work . . . 190
16. Active Directory health report 191
What the report shows . . . 191
Before you start . . . 193
Set it up . . . 194
Settings . . . 195
Troubleshooting . . . 196
Limits . . . 197
Appendix: credential types to paste into AAP 198
ServiceNow API (servicenow_api.yml) . . . 198
MariaDB monitor (mariadb_monitor.yml) . . . 198
Keystore password (keystore_password.yml) . . . 199
STIG Manager API (stigman_api.yml) . . . 199
ACT model key (act_model_key.yml) . . . 199

ACT GenAI beta key (act_genai_beta_key.yml) 200
STIG Manager database (stigman_database.yml) 200
TLS certificate (tls_certificate.yml) 201
Container registry login (hosts) (registry_login.yml) 201
Teams webhook (teams_webhook.yml) 201
SMTP relay (smtp_relay.yml) 202

1. Start here: the pieces and how they fit

> **Confused by all the moving pieces?**
> Read HOW_IT_FITS_TOGETHER.md (or its PDF in `docs/pdf/`). It shows what the code, the inventory, the credentials and the survey each do, how a setting travels from a YAML file to a host, and how to update the repository safely.

This repository holds **runbooks**: jobs that check Linux servers and help troubleshoot them. You run them from **Ansible Automation Platform (AAP)**, by hand or on a schedule. This page explains the few ideas you need. SETUP_AAP.md then walks you through the setup, one click at a time.

What the runbooks do

Runbook	What it answers	When it runs
Health check	Is anything wrong on these servers right now? Disk, mounts, services, load/memory, time, network, logging, SELinux, fapolicyd, auditd, accounts, certificates, patching, MariaDB / MySQL	Daily (the "daily" set) and weekly (the "weekly" set)
Database health	Are the MariaDB / MySQL databases (on the host or in podman containers) running, with connection room, no stuck replication, and disk space?	Hourly or daily, on its own
Troubleshoot	Someone reports a problem on a host. What does it look like?	On demand
Certificate report	Which certificates expire soon, on every server, in one list?	Weekly
POA&M status	Which POA&M items are overdue or due soon? Which open CAT I/II findings have no POA&M item?	Weekly
ServiceNow tickets	Opens or updates a ticket for each problem a check found. Notes the ones that cleared	After every check (as a workflow step)
ServiceNow health	Is ServiceNow answering? Are the MID Servers up?	Hourly
Patch hosts	Installs updates, and proves every service came back. Reboots only with `automatic_restarts: yes`	Monthly, after an approval
Apply approved ACT fix	Applies exactly the fix a person approved	Only inside a workflow, after an approval
NetApp ONTAP health report	Is any node, disk, port, volume, aggregate, SnapMirror or certificate in trouble on the storage clusters? (NETAPP.md)	Daily
VMware jobs	Restart, shut down, snapshot, notes or change the VLAN of VMs through vCenter. Reports: secure boot, datastores, snapshots (with an approved cleanup), alarms and events, and ACT's analysis of the alarms: likely cause, fix and confidence (VMWARE.md)	On demand; the reports daily or weekly

Every check is **read-only**. It looks and reports, and changes nothing. Patching changes servers, and waits for an approval first; the VMware jobs change only what an operator asked for, and never the AAP server's VM.

The ideas behind Ansible, in plain words

- **Host**: a server Ansible connects to, over SSH, the way you would.

- **Inventory**: the list of hosts, sorted into **groups** such as `stigman` (the STIG Manager servers) or `mariadb`. The built-in group `all` holds every host. A group can also carry **settings (variables)**, for example "on the database hosts, the MariaDB container is called servicenow-mariadb".
- **Task**: one step, such as "run `df` and read the result". A **module** is the tool a task uses (`command`, `shell`, `uri`, `set_fact`...).
- **Role**: a folder of tasks that does one job, plus its settings. `roles/check_disk/` is "check the disks". Its settings, with their defaults, are in `roles/check_disk/defaults/main.yml`.
- **Playbook**: the file you run. It says which hosts, and which roles or tasks to run on them. `playbooks/health_check.yml` says "on every host, run the checks I was given".
- **become**: run as root through sudo. The checks need it to read root-only information (the audit log, /etc/shadow).
- **Check mode**: a dry run. Ansible shows what it would change and changes nothing. Every runbook here works in check mode, and the checks give the same answers in both modes.
- **Idempotent**: running it twice does no harm. The second run changes nothing.

The AAP objects, and what each one is for

AAP is a web front end that runs playbooks for you. It keeps the passwords, records every run, and runs things on a schedule. You set up a handful of objects once. Each one points at the next:

AAP object	What it is	Here
Project	A copy of a Git repository that AAP keeps in sync	`site-automation` (this repository)
Inventory	The hosts and groups, and their settings	`Linux servers`, filled from `inventories/site/` in this repository, or the inventory you already have in AAP (USING_YOUR_AAP_INVENTORY.md)
Credential type	A form for a kind of secret, and how to hand it to the job	`ServiceNow API`, `MariaDB monitor`, `ACT model key`... (`aap/credential_types/`)
Credential	One filled-in form: the actual secret. Nobody can read it back	`Linux ssh (sudo)`, `ServiceNow API`...
Execution environment	The container a job runs in (Ansible, Python, tools)	The **Minimal** execution environment is enough. This repository needs no extra collections
Job template	"Run this playbook, on this inventory, with these credentials" plus the questions to ask (**survey**). This is what you launch	`Health check`, `Troubleshoot`, `Patch hosts`...
Workflow	Job templates chained together. The next step can depend on the last one's result (success, failure, always). It can stop for an **approval**	`Daily health` = Health check, then ServiceNow tickets
Schedule	"Launch this template or workflow at these times"	Daily health every day at 06:00

Three more words you will see:

- **Survey**: the questions a template asks when you launch it, such as "which checks?". Each answer becomes a variable for the playbook.
- **Limit**: run on only some hosts or groups, for example `snowdb01.example.mil` or `mariadb_hosts`.
- **Artifacts**: what a job hands to the next step of a workflow. The health check hands its findings to the ServiceNow step this way (`set_stats` in the playbook).

How a health check works, start to finish

1. You click **Launch** on the *Health check* template (or its schedule fires).
2. AAP syncs the project, starts the execution environment, and connects to each host over SSH with the Machine credential.
3. On each host, each chosen check runs read-only commands and turns what it sees into **findings**. One finding is one thing that is wrong, in plain words. It includes a command you can paste to look further:

```
[CRITICAL] disk: /var is 96% full (critical at 95%), 1.2 GiB free of 40.0 GiB
      look: timeout 60 du -xh --max-depth=2 /var 2>/dev/null | sort -h | tail -15
```

4. The job prints a summary per host and for the whole fleet. It then **fails the hosts that have findings**. The red status is on purpose: it is how a workflow knows it should open tickets (or ask ACT, or ask for approval).
5. In the *Daily health* workflow, the *ServiceNow tickets* step runs next. It opens a ticket for each finding, or updates the ticket that is already open.

The findings: the one idea that makes everything modular

Every check produces the same kind of finding:

Field	Example	Used by
`check`	`disk`	reports, tickets
`id`	`disk:/var`	ServiceNow updates the same ticket every run, instead of opening a new one
`severity`	`critical` or `warning`	pass/fail, ticket urgency
`summary`	`/var is 96% full (critical at 95%)`	people
`hint`	`du -xh --max-depth=2 /var ...`	people, and ACT, which runs it to collect evidence

The report, the ServiceNow tickets and ACT only ever read findings. That is why:

- a new check you write shows up in reports and tickets with no other change;
- ACT works on top of **any** check. Add `use_act=true` and ACT reads the findings and the hint commands' output, then explains the root cause (ADDING_ACT.md).

Where things are

Path	What
`playbooks/`	the files AAP runs (one per job template)
`playbooks/group_vars/`	**your settings**, one file per AAP group: edit these (VARIABLES.md, USING_YOUR_AAP_INVENTORY.md)
`roles/check_*/`	one folder per check. Its settings are in `defaults/main.yml`
`roles/site_findings/`	starts, reports and publishes findings, and decides pass/fail
`roles/servicenow/`, `roles/troubleshoot/`, `roles/patch/`, `roles/poam/`	the other runbooks
`roles/site_act/`	the bridge to ACT (GenAI)
`inventories/example/`	a sample inventory and settings. Copy it to `inventories/site/`
`aap/credential_types/`	the credential types to create in AAP
`poam/poam.csv`	your POA&M list (for the POA&M runbook)
`docs/`	this guide, SETUP_AAP.md, USING_YOUR_AAP_INVENTORY.md, VARIABLES.md, RUNBOOKS.md, MARIADB.md, WORKFLOWS_AND_SCHEDULES.md, ADDING_ACT.md, APPROVED_COMMANDS.md; PDFs in `docs/pdf/`

The order to do things in

1. **Set up and run one health check by hand** (SETUP_AAP.md, steps 1 to 8). Your inventory is already in AAP? Use USING_YOUR_AAP_INVENTORY.md for the inventory part. Read its output. Adjust thresholds in your settings files (VARIABLES.md) until the findings are the ones you care about.
2. **Add the other job templates** (step 9) and try each once.
3. **Add ServiceNow** (step 10), and build the *Daily health* workflow (WORKFLOWS_AND_SCHEDULES.md).
4. **Schedule** the workflows.
5. **Later: add ACT** to the runbooks you trust (ADDING_ACT.md).

2. Setting up AAP, step by step

Every step says **what** you are doing, **why**, the **clicks**, and what you should **see** when it worked. Menu names are from AAP 2.7. Older versions (2.5, 2.6) may word a label or a menu differently: follow the meaning, not the exact word. If you have not yet, read START_HERE.md first. It explains the words used here.

Before you start

You need:

- An AAP account that can create projects, inventories, credentials and templates in your organization (an **Organization admin** does it all).
- A **Linux account on the servers** that AAP logs in as, for example `svc_aap`. It needs an SSH key and sudo. With `NOPASSWD` sudo, add it to `check_accounts_nopasswd_allowed`, or the accounts check will report it.
- A **Git repository at work** that AAP can reach (GitLab, Bitbucket, GitHub Enterprise).
- For ServiceNow: an **API account** (step 10).

Step 1. Put the repository in your work Git, with your inventory

> **Already have your inventory in AAP?**
> Then you do not need `inventories/site/`. Push the repository without it (skip items 1-3 below), skip step 5, and follow USING_YOUR_AAP_INVENTORY.md for the groups and settings to add.

What. Copy this repository into your work Git, and create your real inventory in it. (Already pushed an earlier release at work? Update it with `scripts/update-from-release.ps1` in VS Code, as in HOW_IT_FITS_TOGETHER.md, "Updating the repository at work". Then skip to step 2.)

Why. AAP reads playbooks from Git, never from your laptop. The inventory lives in the same repository, so host lists and thresholds are versioned and reviewed like code. Every change has a who, a when and a why.

1. Extract the release zip (right-click › **Extract All**). In the extracted folder, copy the folder `inventories\example` and name the copy `inventories\site`.
2. Edit `inventories/site/hosts.yml`. Replace the example host names with yours and keep the group names: the playbooks use them. A host can be in several groups.
3. Edit `inventories/site/group_vars/all.yml`. These settings apply to every host: the LDAP and SIEM servers each host must reach, the account AAP logs in as (`svc_aap`), and the thresholds. Every setting and its default is listed in VARIABLES_REFERENCE.md. **Two places can hold a setting:** this inventory file (it becomes AAP's *inventory* Variables box) and the project's own settings files `playbooks/group_vars/*.yml` (`all.yml` is only switched-off placeholders; a few group files, such as `stigman.yml` and `aap.yml`, ship real values). If a setting is in both, the project file wins. Use one place per setting: VARIABLES.md explains the order.
4. Put it in your work Git, with VS Code:
   - Create an empty repository on your Git server (for example `site-automation`) and copy its clone URL.
   - In VS Code: **Source Control › Clone Repository** › paste the URL › pick a folder, such as `C:\git\site-automation`.
   - Copy everything from the extracted release folder into that folder.

- In **Source Control**: type a message, click **Commit** (answer *Yes* to "stage all changes"), then **Sync Changes** / **Publish Branch**.

The same from a command line: `git add -A`, `git commit -m "site-automation <version>"`, then `git push`.

5. Note the two values AAP needs in step 4: the **clone URL** (the one you pasted above) and the **branch** (shown at the bottom left of VS Code, usually `main`). AAP does not read them from your copy; you type them in once.

You should see the files in your Git server's web page, including `inventories/site/`.

Step 2. Credential types

What. Teach AAP the kinds of secrets these runbooks use, one form per kind.

Why. AAP ships with types for SSH (Machine) and Git (Source Control). The ServiceNow login and the others are custom. A credential type says which fields the form has and how the secret reaches the job. Every type here passes it as an **environment variable** in the execution environment. The playbook must ask for it on purpose, and it never shows in job output or Git.

For each file in `aap/credential_types/` that you need now:

1. **Automation Execution › Infrastructure › Credential Types › Create credential type**.
2. **Name**: the name at the top of the file (for example `ServiceNow API`).
3. **Input configuration**: paste the `inputs:` part of the file, *without* the `inputs:` line.
4. **Injector configuration**: paste the `injectors:` part, *without* the `injectors:` line.
5. Click **Create credential type**.

File	Name	Needed for
`servicenow_api.yml`	ServiceNow API	tickets, ServiceNow health (step 10)
`mariadb_monitor.yml`	MariaDB monitor	the MariaDB check on a containerized database (root login refused there); optional on host installs
`keystore_password.yml`	Keystore password	optional: certs check on a PKCS12 keystore
`stigman_api.yml`	STIG Manager API	optional: POA&M cross-check with STIG Manager
`act_model_key.yml`	ACT model key	later, for ACT (ADDING_ACT.md): GenAI.mil and Ask Sage keys
`act_genai_beta_key.yml`	ACT GenAI beta key	only for ACT with the GenAI.mil beta provider

You should see each new type in the Credential Types list.

Step 3. Credentials

What. Store the actual secrets, once.

Why. This is the only place a password or key is kept. Templates point at the credential by name. Nobody, including you, can read the secret back, and it is never in Git or job output.

Automation Execution › Infrastructure › Credentials › Create credential:

Name	Credential type	Fill in
Linux ssh (sudo)	Machine	**Username** `svc_aap`, **SSH Private Key**, **Privilege Escalation Method** `sudo` (plus the sudo password if sudo asks for one)
site-automation git	Source Control	only if the repository is private: user name and token, or an SSH key
ServiceNow API	ServiceNow API	instance URL, API user, password (step 10)
MariaDB monitor	MariaDB monitor	the read-only database account, for MariaDB **and MySQL**; needed when the database runs in a container (the ServiceNow database), usually needed for MySQL on a host, optional for MariaDB on a host. See MARIADB.md for the setup and the account to ask the DBA for

You should see the credentials listed, with secret fields shown as `ENCRYPTED`.

Step 4. Project

What. Tell AAP where the repository is.

Why. A project is AAP's copy of the repository. Templates pick their playbook from it.

1. **Automation Execution › Projects › Create project**.
2. **Name** `site-automation`. **Organization**: yours.
3. **Execution environment**: **Minimal execution environment** (`ee-minimal`). These runbooks use only modules that come with Ansible itself, so no extra collections or custom execution environment are needed (useful on a disconnected network).
4. **Source control type** `Git`. **Source control URL**: from step 1. **Source control branch/tag/commit**: `main`. Later, pin a release tag such as `v<version>` (the tags are listed on the repository's Releases page). **Source control credential**: `site-automation git` if the repository is private.
5. Tick **Update revision on launch**. Every job then syncs first, so a change you push is used right away.
6. Click **Create project**.

You should see the project's **Last job status** turn **Successful** after a few seconds. If it fails, open the sync job: it is almost always the URL, the branch name, or the credential.

Step 5. Inventory (filled from the repository)

(Skip this step if your inventory is already in AAP: see USING_YOUR_AAP_INVENTORY.md.)

What. Create the `Linux servers` inventory and fill it from `inventories/site/hosts.yml`.

Why. Sourcing the inventory from the project brings in hosts, groups and the settings in `group_vars/`, all at once, and keeps them in step with Git. You do not type hosts into AAP by hand.

1. **Automation Execution › Infrastructure › Inventories › Create inventory › Create inventory**. **Name** `Linux servers`. Click **Create inventory**.
2. Open it › **Sources** tab › **Create source**:
   - **Name** `site inventory from Git`
   - **Source** `Sourced from a Project`

- **Project** `site-automation`
- **Inventory file** `inventories/site/hosts.yml`
- Tick **Overwrite**, **Overwrite variables** and **Update on launch**.

3. Click **Create source**, then **Launch inventory update**. The rocket icon also starts it.

You should see, under **Groups**, `stigman`, `mariadb_hosts`, `patch_hosts`... Under **Hosts** you should see your servers. Open a group: its **Variables** are the ones from `group_vars/<group>.yml`.

Step 6. The first job template: Health check

What. A template that runs `playbooks/health_check.yml`.

Why. A job template is the thing you launch. It fixes which playbook, which inventory and which credentials. Its survey asks which checks to run.

1. **Automation Execution › Templates › Create template › Create job template**.
2. Fill in:

Field	Value	Why
Name	`Health check`	
Job type	Run	(Check also works: the checks read the same way)
Inventory	`Linux servers`	
Project	`site-automation`	
Playbook	`playbooks/health_check.yml`	
Execution environment	Minimal execution environment	
Credentials	`Linux ssh (sudo)` (+ `MariaDB monitor` if you made it)	how AAP logs in
Limit	leave empty, tick **Prompt on launch**	so you can run it on one host
Privilege escalation	ticked	the checks read root-only files

3. Click **Create job template**.
4. **Survey** tab › **Create survey question**:
   - **Question** `Which checks?`
   - **Answer variable name** `health_checks`
   - **Answer type** *Multiple Choice (multiple select)*
   - **Choices**, one per line: `daily`, `weekly`, `all`, `disk`, `mounts`, `services`, `performance`, `time`, `network`, `logging`, `selinux`, `fapolicyd`, `auditd`, `accounts`, `certs`, `patching`, `mariadb`, `mysql`, `database`, `containers`
   - **Default answer** `daily`. Tick **Required**. Click **Create question**.

Made this survey with an earlier release? AAP never updates a survey for you: open the **Health check** template › **Survey** tab › click the `health_checks` question › **Edit** › add `containers` on a new line under **Choices** › **Save**. (`daily` and `all` already include the containers check without that; the choice is for running

it on its own.)

5. Turn the survey **on** (the switch at the top of the Survey tab).

You should see the template in the Templates list with a rocket icon.

Step 7. Run it on one host and read the output

What. Launch the health check on a single server.

Why. Start small. You will see what the checks find on a real host, and whether the thresholds suit you.

1. Click the rocket on **Health check**. **Limit**: one host name. Survey: `daily`. Click **Next**, then **Launch**.
2. The job output scrolls by. The parts to read:
   - `Check disk | run`, `Check services | run`...: each check running (green `ok` lines are normal).
   - `Report | result for this host`: the verdict in plain words. Either `HEALTHY. Checks run: ...` or a list of `[CRITICAL]` / `[WARNING]` findings. Each finding has a `look:` line: a command you can paste on the host to see more.
   - `Report | fleet summary`: one line for all hosts, for example `3 healthy, 1 with findings`.
   - A red `FAIL this host because it has findings` at the end. This is **on purpose**: a job with findings fails, so a workflow can react (open tickets). It does not mean the playbook broke.
3. The **Details** tab shows who launched it, when, and the exact revision of the repository.

You should see a finished job, green if the host is healthy, red with findings otherwise.

Step 8. Tune the thresholds

What. Make the findings match what you care about.

Why. Defaults are a starting point. If a warning is expected on some host, change the setting for that host or group, not the code.

1. Change the setting in the project's settings file `playbooks/group_vars/<group>.yml` (every host: `all.yml`; one host: `playbooks/host_vars/<host>.yml`, create it). This is the place VARIABLES.md recommends, and it wins over the same setting in `inventories/site/group_vars/` or in an AAP group or inventory Variables box. The file name must be the exact group or host name. Examples:

```yaml
check_disk_overrides:
  /var/log/audit: {warn: 70, crit: 85}      # this filesystem: warn earlier
check_services_ignore_failed: ['^dnf-makecache']
site_fail_on: [critical]                    # warnings are reported but do not fail the job
```

(With `[critical]`, a check that could not run, `<check>:check-error`, still fails the host: a check that did not run is not a healthy result.)

2. Commit and push. `playbooks/group_vars/` is read straight from the project, which syncs on launch: nothing else to do. Only if you changed `inventories/site/group_vars/` instead, run the inventory source again in AAP (**Inventories › Linux servers › Sources › Launch inventory update**): those files reach AAP only through the inventory source.
3. Launch the health check again.

You should see the finding gone, or at its new threshold.

Step 9. The other job templates

Create each the same way as step 6: **Inventory** `Linux servers`, **Project** `site-automation`, **Execution environment** Minimal. For the ones that log in to hosts, tick **Privilege escalation** and leave **Limit** empty with **Prompt on launch** ticked. Only the differences are listed. A survey question is *Multiple Choice (single select)* unless it says otherwise.

> **Three templates run on the controller, not on hosts:**
> POA&M status, ServiceNow tickets and ServiceNow health. For these, do **not** tick Prompt on launch for Limit. A workflow's Limit (for example `patch_hosts`) is passed to every step that prompts for one. These playbooks run on `localhost`, so the Limit would match nothing and the step would silently do nothing, and still show green.

Troubleshoot

On demand, when someone reports a problem. Always launch it with a **Limit**: the host with the problem. Findings do not fail this job.

Field	Value
Playbook	`playbooks/troubleshoot.yml`
Credentials	`Linux ssh (sudo)`
Survey: `ts_area`	"What kind of problem?" Choices: `overview`, `disk`, `performance`, `service`, `network`, `selinux`, `fapolicyd`, `login`, `time`, `logs`, `mariadb`, `mysql`. Default `overview`
Survey: `ts_service`	*Text*, not required. "Service name (for area = service), e.g. nginx"
Survey: `ts_target`	*Text*, not required. "host:port it cannot reach (for area = network)"
Survey: `ts_since`	*Text*. "How far back to read logs". Default `-2h`

Database health - MariaDB / MySQL

The database check on its own, for **MariaDB and MySQL**, installed on the host or in **podman containers** (one, several, found automatically, rootless). Give it to the database team. It only reads: no data is read, nothing is changed. Set the databases up first (MARIADB.md): the container name in a settings file, and the account for the credential.

Field	Value
Name	`Database health - MariaDB / MySQL`
Playbook	`playbooks/database_health.yml`
Credentials	`Linux ssh (sudo)` and `MariaDB monitor` (the read-only database account)
Privilege escalation	ticked
Job type	Run, with **Prompt on launch** ticked, so you can pick *Check* first

Field	Value
Limit	empty, **Prompt on launch** ticked. The job runs on the groups `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb` and `mysql`; a Limit narrows that to one host or group. A Limit cannot add a server outside those groups: for that, put `target: <group or host>` in the template's **Variables**
Survey	none. (Optional: a *Multiple Choice* `use_act`, choices `no` and `yes`, to add ACT's explanation; then also attach the *ACT model key* credential, ADDING_ACT.md)

Steps: create the template as in step 6, then launch it twice.

1. Launch with **Job type** `Check`. You should see, for every database host, a `MariaDB | status` line such as `MySQL 8.4.11 - container snow-mysql (podman, image ...): up 312.4 h, 23/151 connections ...`, then `HEALTHY` (or a list of findings, each with a command to look further), and at the end `N healthy, M with findings`. *Check* is safe here: the job only reads either way.
2. Launch with **Job type** `Run`. The result is the same. A host with findings shows as failed, so a workflow can open a ticket (workflow 8).

If a host prints `mariadb:connect` with `Access denied`, the credential is missing or the account lacks a privilege: MARIADB.md.

Service watch

Two templates and a workflow: the steps are in SERVICE_WATCH_DEMO.md (Parts 5 and 6). The one field to decide here is which hosts it watches.

Field	Value
Playbooks	`playbooks/service_watch.yml` (*Service watch - check*) and `playbooks/service_fix_approved.yml` (*Service watch - apply approved fix*)
Credentials	`Linux ssh (sudo)`, plus `ACT model key` on the check template
Variables	optional: `target: <group or host>`. Without it the jobs run on the group `stigman` - a **group inside the template's inventory**, not an inventory called stigman (if there is no such group, the job stops and lists the groups the inventory has). To watch the containers of other servers, put that group (or `all`) here, on **both** templates. Which containers are watched on each host is found by itself: every container that should be running, root's and each user's (SERVICE_WATCH_DEMO.md, "Which containers are watched")
Survey	on the workflow: `watch_mode` (`approval` / `self-heal`)

Certificate report

Weekly: one table of every certificate on every host, soonest expiry first.

Field	Value
Playbook	`playbooks/cert_report.yml`
Credentials	`Linux ssh (sudo)` (+ `Keystore password` if you use it)
Survey: `check_certs_warn_days`	*Integer*. "Warn this many days before expiry". Default `30`

Certificate renewal - CSRs

A new key and a CSR on the host for each certificate file that expires, with the CSR attached to the certificate's ServiceNow incident (step 3 of workflow 7; RUNBOOKS.md, "Certificate renewal").

Field	Value
Playbook	`playbooks/cert_renew_csr.yml`
Credentials	`Linux ssh (sudo)`, `ServiceNow API` (+ `Keystore password` if the certificate report uses it)
Variables	`servicenow_assignment_group: <your PKI or Linux team>`: who gets a ticket this job opens (a certificate with no ticket yet). `cert_skip_hosts: [<your Satellite and Capsules>]` if they are not in a group `sat`. `cert_renew_self_signed: true` for CSRs for self-signed certificates too (never postfix's placeholder). Better in `all.yml`, so the certificate report gets the same
Job type	try it as **Check** first: it makes and sends nothing

POA&M status

Weekly. Runs on the controller and reads `poam/poam.csv` from the project.

Field	Value
Playbook	`playbooks/poam_status.yml`
Credentials	none (+ `STIG Manager API` for the cross-check)
Privilege escalation	not needed
Limit	empty, and **no** Prompt on launch (it runs on the controller)
Survey: `poam_stigman_api`	*Text*, not required. "STIG Manager API URL (blank = skip the cross-check)"

ServiceNow - test ticket

Run once before the tickets step, and again after a password or certificate change. It opens a test incident, reads it back, adds a work note and resolves it, with PASS / WARN / FAIL for each step (SERVICENOW_SETUP.md). Runs on the controller.

Field	Value
Playbook	`playbooks/servicenow_test_ticket.yml`
Credentials	`ServiceNow API`
Job type	Run, with **Prompt on launch** ticked, so you can pick *Check* (a dry run: it only logs in and reads)
Privilege escalation	not needed
Limit	empty, and **no** Prompt on launch (it runs on the controller)
Survey	none

ServiceNow tickets

Only useful as a workflow step after a check. Runs on the controller.

Field	Value
Playbook	`playbooks/servicenow_tickets.yml`
Credentials	`ServiceNow API`
Privilege escalation	not needed
Limit	empty, and **no** Prompt on launch (it runs on the controller)
Survey	none

ServiceNow health

Hourly. Runs on the controller.

Field	Value
Playbook	`playbooks/servicenow_health.yml`
Credentials	`ServiceNow API`
Privilege escalation	not needed
Limit	empty, and **no** Prompt on launch (it runs on the controller)
Survey	none

Patch hosts

Monthly, inside the *Patch with checks* workflow. Run it as *Check* first: it lists what would be updated and changes nothing.

Field	Value
Playbook	`playbooks/patch_hosts.yml`
Credentials	`Linux ssh (sudo)`
Job type	Run, with **Prompt on launch** ticked, so you can pick *Check* (a dry run)
Variables	**required**: `target: <group>`, the name of a **group (or host) inside the inventory** - not the inventory's own name. For example `target: stigman`, or `target: all` when the template's inventory holds only the servers to patch. Without it the job looks for a group called `patch_hosts`, and stops with a message that lists the groups your inventory does have
Survey: `patch_security_only`	"Security updates only?" Choices `false`, `true`. Default `false`
Survey: `automatic_restarts`	"Reboot hosts automatically when the updates need it?" Choices `no`, `yes`. Default `no` (no automatic restarts: the job lists the hosts that need a reboot)

Apply approved ACT fix

Only inside the approval workflow (ADDING_ACT.md). Create it when you add ACT.

Field	Value
Playbook	`playbooks/act_fix_approved.yml`
Credentials	`Linux ssh (sudo)` (no ACT key: it runs the approved commands itself)
Job type	Run, with **Prompt on launch** ticked, so a workflow can run it as *Check* (a dry run that only shows what it would run)
Variables	leave **Prompt on launch** unticked
Survey	none

VMware jobs and reports

They run against vCenter, not your hosts, and have their own setup: the VMware execution environment, a vCenter credential, one template per job. Follow VMWARE.md; the emails are in EMAIL_REPORTS.md.

NetApp ONTAP health report

It reads the storage clusters' REST API, not your hosts: a read-only ONTAP account, a NetApp ONTAP credential, one template. Follow NETAPP.md.

Try each one

With a Limit of one host:

- **Troubleshoot**: `ts_area` = `disk`. Each command's output appears under a `### what it shows` line.
- **Patch hosts**: Job type **Check**. It lists what would be updated and changes nothing.
- **Certificate report**: read the table under `All certificates on all hosts, soonest expiry first`.

Step 10. ServiceNow

What. An API account, its credential, and a test ticket that proves it works.

Why. The tickets step and the ServiceNow health check use ServiceNow's REST Table API with a user name and password. No plugin or collection is installed.

The whole setup, step by step, is in SERVICENOW_SETUP.md: the account to ask for (with a request you can copy), the instance URL and every API call, the network path (firewall, proxy, the DoD CA), the credential, the settings, the test ticket, and how to check the ticket in ServiceNow. In short:

1. Ask the ServiceNow admins for an **integration account** (web service access only) with:
   - `itil` (or a role that can create and update incidents);
   - `mid_server` (read the MID Server list), if you want the MID Server check.
2. Create the **ServiceNow API** credential (step 3) with the instance URL, for example `https://servicenow.example.mil`, and the account.
3. In your settings file (`playbooks/group_vars/all.yml`; the example inventory's `inventories/site/group_vars/all.yml` already has these two lines, so change the value in one place only), set who gets the tickets:

```yaml
servicenow_assignment_group: Linux Operations   # a group that exists in ServiceNow
servicenow_min_severity: warning                # or critical: tickets for critical findings only
```

4. Create **ServiceNow - test ticket** (step 9) and launch it, first as *Check*, then as *Run*. **You should see** six steps that say PASS, and a link to the test incident. It is resolved by the same job.
5. Launch **ServiceNow health** once: `API answers in N s`, then the MID Servers and their status.
6. Build the *Daily health* workflow (WORKFLOWS_AND_SCHEDULES.md). Its tickets step then runs after every check.

Try it safely first: launch the workflow **without** the ServiceNow API credential on the tickets template. The step then only prints `NOT CONNECTED - what would be opened` and sends nothing.

Step 11. Workflows and schedules

See WORKFLOWS_AND_SCHEDULES.md: *Daily health*, *Weekly compliance*, *Patch with checks*, *Fix with approval (ACT)*, and when to schedule each.

Going further, once the basics work:

- ACT (the model): ADDING_ACT.md. If a model answers `HTTP 400`, see its section Both endpoint formats and `:probe`.
- Letting ACT run fixes you trust without asking: APPROVED_COMMANDS.md.
- Every setting you can change, and where to put it: VARIABLES.md and VARIABLES_REFERENCE.md (in print: the *Settings Reference* PDF).
- MariaDB and MySQL checks (also in podman containers): MARIADB.md.

Step 12. Who can do what

Open a template or workflow › **User Access** (or **Team Access**) › **Add roles**:

Role	Can
Execute	launch it (and answer its survey)
Approve	approve or deny its approval steps (workflows)
Read	see it and its job output
Admin	change it

For example, give the help desk **Execute** on *Troubleshoot*, and only the server team **Execute** on *Patch hosts* and **Approve** on the patch workflow.

If something goes wrong

You see	It means	Do
Project sync fails	wrong URL, branch or Git credential	open the sync job and read the last lines
`UNREACHABLE! ... Permission denied (publickey)`	SSH key or user wrong for that host	check the Machine credential; test `ssh svc_aap@host` from a workstation
`Missing sudo password` / `a password is required`	sudo wants a password	put it in the Machine credential (Privilege Escalation Password)
`Unknown check(s): ...`	a survey choice that is not a check name	fix the survey choices (step 6)
`the <name> check could not run: ...`	that check hit an error. The others still ran	the message says why, often a missing command such as `semanage`
`skipped: the MariaDB/MySQL check is off for <host>`	the database check only runs on database hosts	add the host to `mariadb_hosts`, `mysql_hosts` or `database_hosts` in the inventory, or name its container (MARIADB.md)
Findings you do not care about	defaults do not fit that host	step 8: change the setting for its group or host
`NOT CONNECTED - what would be opened`	the tickets template has no ServiceNow API credential	attach it (step 10)
Tickets step fails: `No check results reached this job`	the check step before it failed before checking any host, or it was launched on its own	read the first red job in the workflow; run the tickets step only inside a workflow
A controller step (tickets, POA&M, ServiceNow health) shows `skipping: no hosts matched`	its template prompts for a Limit and the workflow passed one	untick Prompt on launch for Limit on that template (step 9)
A VMware job or report fails	see the troubleshooting tables	VMWARE.md, "Troubleshooting"
No email, or an email error	see the troubleshooting table	EMAIL_REPORTS.md, "Troubleshooting"

3. Using the inventory you already have in AAP

You imported your inventory into AAP. You do not need `inventories/site/` in Git, and you can skip the Git inventory parts of SETUP_AAP.md (the `cp inventories/example ...` in step 1, and all of step 5). Your hosts and groups stay in AAP. Your **settings** can go in either of two places, and you can use both as long as **one setting lives in one place**:

- **this project's settings files** in `playbooks/group_vars/` (edited in VS Code, reviewed in Git, never overwritten by an update): the next section; or
- **AAP's Variables boxes** (inventory, group, host): the steps after it. What you paste there is YAML, and the blocks to paste are below.

Which place wins when a setting is in both, and the full list of places, is in VARIABLES.md. Every setting and its default: VARIABLES_REFERENCE.md.

The runbooks need two things from your inventory:

1. **Groups**: which hosts are which (steps 1-2). The safety lists are built in and already cover the usual group names (`aap`, `aap_hosts`, `sat`, `idm`, `netapp`, `netapp_console_hosts`...).
2. **Settings (variables)**: your LDAP server, your SIEM, the account AAP logs in as, the ServiceNow assignment group... (steps 3-5).

The easy way: your settings files, already named after your groups

The repository has one settings file per AAP group, in `playbooks/group_vars/`. **The file name is the group name**: `mariadb.yml` applies to the hosts in AAP group `mariadb`, and `all.yml` to every host. Ansible reads these files by itself on every run, and the hosts and groups stay in AAP. **If your group has another name, copy the file to your group's name** (for example `mariadb.yml` to `mariadb_hosts.yml`, `aap.yml` to `aap_hosts.yml`, `netapp.yml` to `netapp_console_hosts.yml`, which are the names the example inventory uses). A file for a group your inventory does not have is never used, without any error. The files are:

File	For AAP group	What is in it
`all.yml`	every host (and the ServiceNow / POA&M jobs)	placeholders, all switched off: the safety lists (already built in), ACT provider and models, LDAP/log server, service account, ServiceNow assignment group
`stigman.yml`	`stigman`	the containers service watch watches (`stigman`, `nginx`); the web certificate
`mariadb.yml`, `mysql.yml`	`mariadb`, `mysql`	placeholders: turn the database check on, and the container's name (MariaDB or MySQL; MARIADB.md)
`aap.yml`, `sat.yml`, `idm.yml`, `logstash.yml`	the same names	disk limits, certificates, services that must run
`netapp.yml`	`netapp`	nothing yet (the group is protected by the built-in lists)
`rhel8_all.yml`	`rhel8_all`	a placeholder for Python 3.9 (only with a newer AAP)

To change a setting:

1. Open the file in VS Code (for example `playbooks\group_vars\mariadb.yml`).
2. A line that starts with `#` is **off**. To turn a placeholder on, delete the `#` in front of it (and in front of the lines that belong to it), and replace `CHANGE-ME` with your value. Keep the indentation exactly as shown: spaces, never tabs.
3. **Commit** and **Sync Changes**. AAP uses it on the next run: the project syncs itself when **Update revision on launch** is ticked.

Example (the full guide, with several containers and rootless ones, is MARIADB.md): MariaDB runs in a container called `snow-mariadb`. In `mariadb.yml`, change

```
check_mariadb_enabled: true
# check_mariadb_container: CHANGE-ME
```

into

```yaml
check_mariadb_enabled: true
check_mariadb_container: snow-mariadb
```

A group that has no file yet (`sn_prod` here is only an example name; use one of yours): create `playbooks/group_vars/sn_prod.yml` with the settings for it. The same name as the AAP group, `.yml` at the end. For one host only: `playbooks/host_vars/<host name>.yml` (create the folder).

One thing the files cannot do: choose which hosts a template runs on. That is decided before any setting is read. Each playbook therefore has a built-in choice, and the job template's own **Variables** box (the *Extra variables* on its Details page, not the inventory's) can change it with one `target:` line:

Job templates	Variables box empty: runs on	To run on something else
Health check, Troubleshoot, Certificate report, Apply approved ACT fix	**every host** in the template's inventory (the built-in group `all`)	one run: the **Limit** at launch (tick **Prompt on launch** next to Limit). Always: `target: "rhel8_all:rhel9_all"`
Database health - MariaDB / MySQL	the groups `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb`, `mysql`	`target: <group or host>` (a Limit only narrows those groups; it cannot add a server outside them)
Service watch, Service watch - apply approved fix, STIG Manager - deploy	the group `stigman`	`target: <group>`
Patch hosts	the group `patch_hosts` only (no such group: the job stops and lists your groups)	**always set it**, for example `target: "rhel8_all:rhel9_all"`, and pick the hosts with the Limit at launch. `target` is a group or host **inside** the inventory, never the inventory's own name

`target` and the Limit take group names, host names, or several joined with `:`. `stigman:mariadb` means either group, `rhel9_all:&sn_prod` only hosts in both, and `rhel9_all:!sn_dev` all of `rhel9_all` except `sn_dev`. Put a space after `target:`, quote a value that has a `:` in it, and use the group name exactly as the inventory's **Groups** tab shows it (`rhel9_all`, not `rhel9`).

Is every host in this inventory a Linux server? If some are not (firewalls, switches, appliances), an empty Variables box tries them too: they show as failed, and in a workflow that opens ServiceNow tickets each of them gets a ticket (a host that never reports is ticketed), on every run. Then put `target: "rhel8_all:rhel9_all"` on those templates **before** you schedule anything.

These files are yours. The update script adds a settings file you do not have yet, and never changes one you already have. A new release can therefore never overwrite your settings.

Or put the same lines in AAP instead. Everything below explains the AAP Variables boxes, which take exactly the same lines. **Keep each setting in one place.** If the same setting is in both a file here and an AAP *group* or *inventory* box, **the file here wins** (every job prints a note when that happens). The exception is one host: a *host's* Variables box in AAP beats a group file, and only a `playbooks/host_vars/` file beats the host box. Full order: VARIABLES.md.

How variables work in AAP (two minutes)

A variable is a setting with a name, such as `check_disk_warn_pct: 85`. Where you type it decides which hosts get it:

Where in AAP	Applies to	The Git-file equivalent
Inventories › your inventory › Edit inventory › Variables	every host in the inventory	`group_vars/all.yml`
… › Groups › a group › Edit group › Variables	the hosts in that group (and in its child groups)	`group_vars/<group>.yml`
… › Hosts › a host › Edit host › Variables	that one host	`host_vars/<host>.yml`

- **Inside AAP the more specific one wins**: host beats group, a child group beats its parent group, and a group beats the inventory. A survey answer, or the template's own **Variables**, beats all of them. **Files in this project's `playbooks/group_vars/` beat the AAP group and inventory boxes** (see the previous section).
- **Anything you do not set uses the default** in `roles/<role>/defaults/main.yml`. Every setting is explained in VARIABLES_REFERENCE.md. Do not edit the role files: set the value in a settings file or in AAP instead. A new release would overwrite edits in the roles, but never your settings files or AAP variables.
- The boxes take YAML. Keep the indentation exactly as shown (spaces, not tabs). AAP refuses to save YAML it cannot read.

Step 1. Look at what you have

Inventories › your inventory: open the **Groups** tab, then the **Hosts** tab. Write down which of your groups (or hosts) are:

- all your Linux servers;
- the AAP server(s);
- the NetApp Console host;
- the STIG Manager host(s);
- the ServiceNow database host(s);

- the ServiceNow MID Server host(s);
- the servers monthly patching may touch.

On the old ansible-core machine, `ansible-inventory -i <your inventory> --graph` prints the same tree.

Step 2. Create the groups the runbooks use

A host can be in many groups. Creating these groups leaves your existing groups as they are.

Create group	Put in it	Why	Needed?
`aap_hosts`	the AAP server(s)	**safety**: ACT never fixes anything here by itself, and patching never reboots it	**yes**
`netapp_console_hosts`	the NetApp Console host	**safety**: ACT only diagnoses here (vendor-managed)	**yes**, if you have one
`no_patch`	the groups `aap_hosts` and `netapp_console_hosts` (as children)	**safety**: *Patch hosts* skips these even when a job targets them	**yes**
`patch_hosts`	the servers monthly patching may touch	*Patch hosts* only patches these	only for patching
`mariadb_hosts`, `mysql_hosts`, `database_hosts`	the database host(s), MariaDB or MySQL (any of the three, or the groups `mariadb` / `mysql`; the check treats them alike)	the database check runs here, and the **Database health** template targets these by default	only for the database check
`stigman`	the STIG Manager host(s)	service watch and STIG Manager deploy run here	only for those

The clicks:

1. **Inventories › your inventory › Groups › Create group**. Type the **Name** (exactly as in the table), then **Create group**.
2. Add hosts: open the new group › **Hosts** tab › **Add existing host** › tick the hosts › **Add**.
3. Or add one of your groups inside it: open the new group › **Related Groups** tab › **Add existing group** › tick your group › **Add**. Its hosts are now in the new group too.

Keep your own names instead? If you already have, say, an `aap` group and do not want a second group, tell the runbooks your names. Put these in the inventory **Variables** (step 3). A list you write **replaces** the shipped one, so repeat every name you want kept; `aap` and `aap_hosts` are always protected, whatever you write:

YAML

```
site_act_diagnose_only_groups: [aap, netapp]   # ACT never fixes these
patch_never_reboot_groups: [aap]               # patching never reboots these
patch_never_patch_groups: [aap, netapp]        # patching skips these
```

Updated from 0.4.1 or older? Your `playbooks/group_vars/all.yml` then still sets these three lists (and `site_act_provider`, `site_act_models`) as real lines, and that file silently wins over the inventory Variables box. Open it and put `#` in front of those lines (or delete them), or make your change there instead. The job's `NOTE - settings in this project's files win ...` line lists what the file still sets.

The checks run on every host of the inventory. To keep them to your Linux servers, put `target: linux_servers` (your all-Linux group) in the **Variables** of the Health check, Troubleshoot, Certificate report and Apply approved ACT fix templates.

Step 3. Settings for every host (inventory variables)

Inventories › your inventory › Edit inventory › Variables. If something is already there, add below it; do not delete what is there. Paste, then change every line marked `CHANGE`:

```
---- what every host must be able to reach, from itself ----
check_network_dns_names: [ldap.yoursite.mil]              # CHANGE: names that must resolve
check_network_tcp:                                        # CHANGE: services every host needs
  - {name: LDAP, host: ldap.yoursite.mil, port: 636}
check_logging_remote:                                     # CHANGE: your SIEM / log collector
  - {name: SIEM, host: siem.yoursite.mil, port: 6514}

# ---- the account AAP logs in as ----
check_accounts_nopasswd_allowed: [svc_aap]                # CHANGE: your AAP service account
check_accounts_expiry_watch: [svc_aap]                    # CHANGE: the same account

# ---- STIG ----
check_fapolicyd_required: true                            # false if some hosts do not run
  fapolicyd

# ---- where tickets go ----
servicenow_assignment_group: Linux Operations             # CHANGE: a group that exists in
  ServiceNow
servicenow_min_severity: warning                          # or critical: tickets for critical
  only
```

The ACT provider and models are **not** set here: they are chosen in `playbooks/group_vars/all.yml` (lines ready as comments) or per template (ADDING_ACT.md, step 2). Do not also set them here: if the file ever sets them, the file wins and the value here is ignored.

Not sure about a value yet? Leave the line out. The check then uses its default, or checks nothing for it (no `check_network_tcp` means no TCP checks).

Step 4. Settings for particular groups

Open the group › **Edit group** › **Variables**, paste, change the `CHANGE` lines, **Save**. Put these on your own groups or on the groups from step 2. Both work.

The AAP server(s), for example on `aap_hosts`:

```yaml
check_disk_overrides:
  /var: {warn: 80, crit: 90}          # AAP keeps job output and container images under /var
```

The STIG Manager host(s), on `stigman`:

```yaml
check_certs_endpoints:                  # the certificate browsers actually see
  - {name: STIG Manager, host: 127.0.0.1, port: 443, servername: stigman.yoursite.mil}   # CHANGE
check_certs_files: [/etc/stigman/*.crt, /etc/stigman/*.pem]                          # CHANGE:
  where your cert files are
watch_containers:                       # optional since 0.6.0: service watch finds the containers
  - name: stigman                       # that should run by itself; this list is watched first,
  - name: nginx                         # in this order (dependencies first), and a missing one is
    reported
```

If STIG Manager runs as systemd units (`systemctl list-units | grep -i stig` shows them), also add them, so the services check watches them: `check_services_required_extra: [<unit>, <unit>]`.

The ServiceNow database host(s) (MariaDB or MySQL): also add this group to `mariadb_hosts` or `mysql_hosts` (step 2).

```yaml
check_mariadb_container: servicenow-mariadb     # CHANGE: the container name from `sudo podman ps
  -a`
```

Then attach the **MariaDB monitor** credential to the **Database health**, Health check and Troubleshoot templates (it serves MySQL too). Container images give root a password, so the check logs in with a monitoring account instead. The whole setup (several containers, discovery, rootless containers, the account to ask the DBA for, and how to verify) is in MARIADB.md.

The ServiceNow MID Server host(s):

```yaml
check_services_required_extra: [mid]            # CHANGE: its unit name (systemctl list-units |
  grep -i mid)
check_certs_keystores:
  - path: /opt/servicenow/mid/agent/security/agent_keystore     # CHANGE: your MID Server's path
```

The NetApp Console host: nothing is needed. It is checked like every host; being in `netapp_console_hosts` and `no_patch` keeps automation from changing it.

Step 5. Settings for one host (only when one host is different)

Hosts › the host › Edit host › Variables. For example, a host whose `/data` is always nearly full on purpose:

```yaml
check_disk_overrides:
  /data: {warn: 95, crit: 98}
```

Or a host with a known-broken vendor unit you want ignored:

```yaml
check_services_ignore_failed: ['^vendor-agent']
```

Step 6. Use your inventory in the job templates

On every job template (and workflow), set **Inventory** to your inventory. That is the only link between them. Everything else (project, playbook, credentials, survey) is as in SETUP_AAP.md, steps 6 and 9.

Step 7. Check that it worked

1. Launch **Health check** with **Limit** = one server and the survey set to `daily`.
2. In `Report | result for this host`, look for your own values. A network finding names *your* LDAP server; an accounts finding names *your* service account. If a value is not there, the variable did not reach the host: check its spelling, and which group or host it is on.
3. `skipping: no hosts matched` (a green job that did nothing) means the Limit or `target` names a group or host this inventory does not have. Check the spelling against the inventory's **Groups** tab, and that the template uses the right **Inventory**.
4. `skipped: the MariaDB/MySQL check is off for <host>` means the host is in none of `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb` or `mysql` and has no container name set (MARIADB.md).
5. Findings you do not care about: change the setting for that group or host (steps 4-5), not the code.

Which YAML files in the repository do I change?

File	Change it?
`playbooks/group_vars/*.yml`, `playbooks/host_vars/*.yml`	**Yes, these are your settings** (the update script never overwrites them)
`inventories/example/...`	**No.** Only a reference to copy blocks from; AAP does not read it
`roles/*/defaults/main.yml`	**No.** Read VARIABLES_REFERENCE.md to see every setting; set values in a settings file or in AAP instead
`playbooks/*.yml`	**No**
`aap/credential_types/*.yml`	**No.** Paste their contents into AAP (credential types)
`ansible.cfg`	**No.** AAP ignores its `inventory =` line and uses the template's inventory
`poam/poam.csv`	**Yes.** Your POA&M export (see `poam/README.md`)

4. ServiceNow: set up, test and verify

The runbooks open ServiceNow incidents for the problems the checks find (the **ServiceNow tickets** step), and check that ServiceNow itself is healthy (**ServiceNow health**). This page takes you from nothing to a **verified test ticket**, one step at a time:

1. an API account from the ServiceNow admins;
2. the addresses: the instance URL and the API calls;
3. the network path from AAP to ServiceNow;
4. a quick check from a shell (optional);
5. the credential in AAP;
6. your settings: which assignment group gets the tickets;
7. the **ServiceNow - test ticket** template;
8. a dry run, then a real run;
9. checking the ticket in ServiceNow.

Sign-in method. This integration uses **basic authentication only** (an account name and password over HTTPS). OAuth, CAC/PKI login and SSO are **not supported**. If your instance requires one of them for the API, ask the ServiceNow admins for an integration account that is allowed to use basic authentication for the Table API, and exempt from SSO.

Plan about 30 minutes, plus the time the ServiceNow admins need for step 1.

What you need

Item	Example	Who gives it to you
The instance URL	`https://yourinstance.servicenowservices.com` or `https://servicenow.example.mil`	the ServiceNow admins (or your browser's address bar)
An API account and its password	`svc_aap_sn`	the ServiceNow admins
Roles on that account	`itil` (and `mid_server` for the MID Server check)	the ServiceNow admins
The assignment group for the tickets	`Linux Operations`	the ServiceNow admins or your team lead
A network path from AAP to the instance	TCP 443 from the AAP execution nodes	the network or firewall team
Trust for the instance's certificate	the DoD root CA, if the instance uses a DoD certificate	you (step 3)

Step 1. Ask the ServiceNow admins for an API account

What. An account that only the automation uses, allowed to read, open and update incidents.

Why. The playbooks log in to ServiceNow's REST API with a user name and password (basic authentication). A dedicated account keeps the tickets traceable ("opened by svc_aap_sn"), and its access can be limited to what the automation needs.

Send them this (change the names in brackets):

```
Request: a ServiceNow integration account for Ansible Automation Platform (AAP)

- User ID: svc_aap_sn (or your naming standard)
- "Web service access only": ticked (API use only, no interactive login)
- "Internal Integration User": ticked, if our release has it
- Roles: itil (read, create and update incidents; read groups and users)
         mid_server (read the MID Server list; only if we want the MID Server check)
  No admin role.
- Authentication: basic authentication over HTTPS for the Table API. If MFA or single
  sign-on is enforced, this account must be exempt, as web-service accounts usually are.
- If IP address access control is on: allow the AAP execution nodes' addresses: [addresses]
- Password: set, with "Password needs reset" unticked. Tell me the expiry or rotation
  rule, so the AAP credential can be updated in time.
- Tickets go to the assignment group: [Linux Operations]. Please confirm its exact name.
- If our instance has custom incident states or close codes: the value for "Resolved"
  and a valid close code.
```

You should get back the user ID, the password (through your normal secure channel), the exact assignment group name, and, if they are custom, the resolved state and close code.

Step 2. The addresses

The instance URL is the start of the address you see in your browser on ServiceNow: the `https://` and the host name, **nothing after it**.

In the browser	Instance URL for AAP
`https://yourinstance.servicenowservices.com/now/nav/ui/home`	`https://yourinstance.servicenowservices.com`
`https://servicenow.example.mil/nav_to.do?uri=incident_list.do`	`https://servicenow.example.mil`

The API calls. Everything goes to that host over HTTPS (TCP 443), with basic authentication and JSON. These are the only calls the playbooks make:

Method and path	What for	Used by
`GET /api/now/table/incident?sysparm_limit=1`	log in and read (the connection test)	test ticket, health
`GET /api/now/table/sys_user_group?sysparm_query=name=<group>`	does the assignment group exist?	test ticket
`GET /api/now/table/sys_user?sysparm_query=user_name=<caller>`	does the caller exist?	test ticket
`GET /api/now/table/incident?sysparm_query=correlation_id=...`	is a ticket for this problem already open?	tickets
`POST /api/now/table/incident`	open an incident	test ticket, tickets
`GET /api/now/table/incident/<sys_id>`	read an incident back	test ticket
`PATCH /api/now/table/incident/<sys_id>`	add a work note; resolve	test ticket, tickets

Method and path	What for	Used by
`GET /api/now/table/ecc_agent`	MID Servers: up and validated?	health

Step 3. The network path from AAP

Which machine talks to ServiceNow. The ServiceNow jobs run on the **AAP execution node**, inside the execution environment container, not on your managed hosts. To see the nodes, open **Automation Execution › Infrastructure › Instances** (or **Instance Groups**). Those are the addresses the firewall (and ServiceNow's IP access control, if any) must allow.

Firewall. Allow the execution nodes › the instance, TCP 443.

A proxy. If the execution nodes reach the outside only through a proxy, give the jobs the proxy settings. Go to **Settings › Automation Execution › Job › Edit › Extra Environment Variables**, and set (add to what is already there):

```
{"https_proxy": "http://proxy.example.mil:8080", "no_proxy": "localhost,127.0.0.1,.example.mil"}
```

This applies to every job. Leave hosts that must not go through the proxy in `no_proxy`.

The certificate. A ServiceNow instance with a DoD certificate is only trusted if the execution environment trusts the DoD root CA. Choose one:

Option	How	When
A. The CA in the execution environment	have the AAP admins build the execution environment with the DoD root CAs in its trust store	best: every job trusts it
B. A CA file in the repository	put the CA certificate (PEM) in the project, for example (an example path: the folder does not exist until you create it) `playbooks/files/ca/servicenow-ca.pem`, and set `servicenow_ca_path: "{{ playbook_dir }}/files/ca/servicenow-ca.pem"` (step 6)	when you cannot change the execution environment. CA certificates are public, so this is fine
C. Switch the check off	`servicenow_validate_certs: false`	only for one test, never for real use

Step 4. Check from a shell first (optional, fastest)

On the AAP execution node, or any Linux machine on the same network:

shell

```
curl -sS -u svc_aap_sn -H 'Accept: application/json' \
  'https://yourinstance.servicenowservices.com/api/now/table/incident?sysparm_limit=1&sysparm_fiel
    ds=number'
```

`curl` asks for the password, so it is not left in your shell history.

You should see `{"result":[{"number":"INC0012345"}]}` (any number). Anything else:

What you get	What it means
`SSL certificate problem: unable to get local issuer certificate`	the CA is not trusted (step 3, the certificate)
`Could not resolve host`	DNS: check the host name
nothing, then `Connection timed out`, or `Connection refused`	firewall or proxy (step 3)
`{"error":{"message":"User Not Authenticated"...}}` (HTTP 401)	wrong user or password, the account is locked, or it is not allowed to use basic authentication
`{"error":{"message":"Operation Failed","detail":"ACL Exception..."}}` (HTTP 403)	the account lacks the `itil` role
an HTML page	the URL has something after the host name, or a login page (SSO) answered: the account must be exempt from SSO

Step 5. The credential in AAP

1. If you have not yet, create the **ServiceNow API** credential type (SETUP_AAP.md, step 2). It uses the file `aap/credential_types/servicenow_api.yml`.
2. **Automation Execution › Infrastructure › Credentials › Create credential**:
   - **Name** `ServiceNow API`;
   - **Credential type** `ServiceNow API`;
   - **Instance URL**: from step 2 (only `https://` and the host name);
   - **API user name** and **API password**: from step 1.
3. **Create credential**.

You should see the credential listed, with the password shown as `ENCRYPTED`. The job gets the three values as `SN_HOST`, `SN_USERNAME` and `SN_PASSWORD`. They never appear in job output, and never reach your managed hosts.

Step 6. Your settings

In VS Code, open `playbooks/group_vars/all.yml`. (If your inventory comes from Git, the same lines in `inventories/site/group_vars/all.yml` end up in AAP's inventory Variables box, and the project's file wins over them: keep each setting in **one** place. See VARIABLES.md.) In the ServiceNow block, delete the `#` in front of the lines and fill in your values:

```
---- ServiceNow tickets ------------------------------------------------------------
servicenow_assignment_group: Linux Operations   # the exact name from step 1
servicenow_min_severity: warning                # or critical: tickets for critical findings only
# servicenow_caller: svc_aap_sn                  # optional; empty = the API account is the caller
# servicenow_ca_path: "{{ playbook_dir }}/files/ca/servicenow-ca.pem"   # step 3, option B
```

Fields your instance makes mandatory. If your incident form requires more fields (a red asterisk: Location, Subcategory, Channel ...), set them for every new ticket, and the fields a resolved ticket needs for the resolve step. Use each field's **column name**: right-click its label on the form and choose **Show - '<column name>'** (Channel is `contact_type`). Names and choice labels work as values, as they appear on

the form:

```yaml
servicenow_category: Software
servicenow_fields:
  location: "Building 100"
  subcategory: "Operating System"
  contact_type: "Self-service"         # Channel
servicenow_close_code: "Solution provided"     # Resolution code, as your instance has it
servicenow_resolve_fields:
  resolved_by: svc_aap_sn
```

Mandatory fields enforced only by the form (UI policies) do not stop the API, but the ticket then cannot be closed by hand until someone fills them in, so set them here anyway. A data policy does stop the API: step 3 or 6 of the test then fails with `Data Policy Exception` and names the field.

Commit and **Sync Changes**. Every other ServiceNow setting (urgency, impact, resolve, close code) is described in `roles/servicenow/defaults/main.yml`.

All settings, with their defaults: servicenow.

Step 7. The ServiceNow - test ticket template

Automation Execution › Templates › Create template › Create job template:

Field	Value
Name	`ServiceNow - test ticket`
Inventory	any (for example `rhel-inventory`); it runs on AAP itself, not on these hosts
Project	`site-automation`
Playbook	`playbooks/servicenow_test_ticket.yml`
Credentials	`ServiceNow API`
Job type	Run, with **Prompt on launch** ticked (so you can pick *Check*, a dry run)
Limit	empty, and **no** Prompt on launch
Privilege escalation	not needed

Create job template.

Step 8. Run it: first a dry run, then for real

The dry run. Launch › **Job type** `Check` › **Launch**. Steps 1 and 2 run for real, because they only read. Steps 3 to 6 are only printed:

```
PASS 1 connection: the instance answers and svc_aap_sn logged in and read the incident table (0 s)
PASS 2 settings: assignment group "Linux Operations" exists; caller not set: the tickets show the
  API account as the caller
DRY RUN (Check mode): steps 1-2 ran (they only read). Steps 3-6 would:
3 open: POST https://yourinstance.servicenowservices.com/api/now/table/incident {...}
```

Nothing was created in ServiceNow.

The real run. Low priority does not mean silent: ServiceNow may still send its usual emails, to the assignment group for the new incident and to the caller when it is resolved. Warn the group first, or point the test at a test group (`servicenow_assignment_group: <test group>` in the template's **Variables**), or at a test instance if you have one.

Launch › **Job type** `Run`. At the end it prints a summary like this:

```
ServiceNow test at https://yourinstance.servicenowservices.com as svc_aap_sn:
PASS | 1 connection | the instance answers and svc_aap_sn logged in and read the incident table (0
  s)
PASS | 2 settings | assignment group "Linux Operations" exists; caller not set: ...
PASS | 3 open | opened INC0012346 (sys_id 5f1c...)
PASS | 4 read back | INC0012346: state New, priority 5 - Planning, assignment group Linux
  Operations, ...
PASS | 5 work note | added a work note to INC0012346 (see its Activity stream)
PASS | 6 resolve | resolved INC0012346 (state 6)
```

Step 4 also prints a link: `Open it in ServiceNow: https://.../nav_to.do?uri=incident.do?sys_id=...`.

Result	Meaning
PASS	that step works
WARN	it works, but look at it (for example, no assignment group is set, or the instance would not resolve the ticket). The job stays green
FAIL	that step does not work. The job is red, and the line says why and what usually causes it

To leave the test ticket open and look at it first, add `servicenow_test_resolve: false` to the template's **Variables**.

Step 9. Check the ticket in ServiceNow

1. Click the link from step 4 of the job output, or go to **Incident › All** and search for the number (`INC0012346`).
2. Check:
   - **Short description**: `[site-automation test] test ticket from AAP job <number> - safe to close`;
   - **Assignment group**: yours (step 6);
   - **Caller**: your `servicenow_caller`, or the API account;
   - **Urgency** 3 and **Impact** 3, so normally **Priority** 5 - Planning, the lowest (your instance's priority table decides). The real tickets use `servicenow_urgency` and `servicenow_impact`;
   - **Activity**: the work note `Test work note from AAP job ...`;
   - **State**: Resolved, with the close notes `Test ticket from AAP job ...: resolved automatically by the same job` (unless you set `servicenow_test_resolve: false`).
3. To find every test ticket: filter **Incident › All** on **Correlation display** = `site-health-test`.

The test passes when all six steps say PASS (or WARN for step 6, if you do not resolve tickets automatically), and the ticket in ServiceNow looks like the list above.

When a step fails

Step and message	Cause	Fix
1: `Attach the "ServiceNow API" credential`	the template has no ServiceNow credential	step 7: add it under Credentials
1: `HTTP -1 ... certificate verify failed`	the execution environment does not trust the instance's CA	step 3, the certificate
1: `HTTP -1 ... Name or service not known`	DNS	the Instance URL's host name; DNS from the execution node
1: `HTTP -1 ... timed out` or `Connection refused`	firewall or proxy	step 3
1: `HTTP 401`	wrong user or password; account locked or inactive; "Password needs reset"; basic authentication not allowed	step 1 with the admins, then update the credential
1: `HTTP 403`	the account may not read incidents	the `itil` role
1: `HTTP 404`	the Instance URL has a path after the host name	only `https://` and the host name
2: `assignment group "..." does not exist`	the name differs from ServiceNow's, even in capitals or spaces	copy the exact name from ServiceNow into `servicenow_assignment_group`
2: `could not look up ...` (WARN)	the account may not read groups or users	fine if step 4 shows the group; otherwise ask for read access to `sys_user_group`
3: `HTTP 403 ... ACL Exception Insert Failed`	the account may read but not create incidents	the `itil` role
3: `HTTP 400`	ServiceNow refused a value, and says which	usually `servicenow_category` or a custom field
4: `did not stick` (WARN)	the ticket was opened without the group	the group name (step 6)
5: `HTTP 403`	the account may create but not update	the `itil` role, or an ACL on work notes
6: `could not resolve ... Data Policy Exception` (WARN)	the instance needs other values to resolve	ask the admins for the resolved state and close code; set `servicenow_resolved_state` and `servicenow_close_code`. Only needed if you use `servicenow_resolve_cleared: true`

Next

- **ServiceNow health** (hourly): the instance's API and the MID Servers (RUNBOOKS.md).
- **The tickets step** in the *Daily health* workflow (WORKFLOWS_AND_SCHEDULES.md): one incident per problem, updated on later runs, noted or resolved when it clears.

- Keep **ServiceNow - test ticket**. Run it again after a password change, a new certificate, or a ServiceNow upgrade.

5. Workflows and schedules

A **workflow** chains job templates. Each arrow says when the next step runs:

- **Run on success**: only if the step before it succeeded.
- **Run on fail**: only if it failed.
- **Run always**: either way.

The checks *fail on purpose* when they find something. So "Run on fail" means "a problem was found", and "Run always" means "whatever happened". A step also receives what the steps before it published (**artifacts**). That is how the tickets step learns what the health check found.

This chapter first shows how to build any workflow, then a catalog of ready-made workflows for Linux, for Windows, and for both, and how to schedule them.

How to build any workflow, click by click

1. **Automation Execution › Templates › Create template › Create workflow job template**.
2. **Name** (e.g. `Daily health`), **Organization**.
   - **Inventory: leave it empty.** Each step then uses its own template's inventory, so one workflow can hold Linux and Windows steps. A workflow inventory only reaches steps whose template prompts for the inventory.
   - Tick **Prompt on launch** next to **Limit** if you want to pick hosts at launch.
   - Leave **Variables** empty: workflow variables override every step's own answers.
3. **Create workflow job template**. The **workflow visualizer** opens.
4. **The first step: Add step** › **Node type**:
   - *Job Template*: one of the templates (the usual case);
   - *Approval*: a person must click Approve (give it a **Timeout**);
   - *Workflow Job Template*: another workflow, as one step;
   - *Project Sync* / *Inventory Source Sync*: refresh the code or the inventory first.

Pick the template, then fill in **this step's** answers:

- **Survey answers**, e.g. `health_checks` = `daily`;
- **Job type** `Check` for a dry run (DRY_RUNS.md);
- a **Limit**.

The wizard only offers what the template asks in its survey or has **Prompt on launch** ticked for. › **Finish**.

5. **Next steps:** hover over a box › **Add step and link** (the plus sign) › the node, and the **link type** (success / fail / always).
   - **Two things at once:** add a second first step with **Add step**. Both start together.
   - **Two arrows into one step:** open the step › **Convergence**: *Any* runs it when one parent finishes as linked, *All* waits for all of them.
6. **Save** (top right). Closing without saving throws the graph away.
7. **Try it:** launch it once with the risky steps set to `Check` (step 4), then set them back.

Who may approve: open the workflow › **User Access** (or **Team Access**) › **Add roles** › **Approve**. Pending approvals are in **Automation Execution › Administration › Workflow Approvals**, and in the workflow

job's view.

Which templates fit where:

Template	Fails when	Publishes (for the next step)	Typical next step
Health check / Windows health check	a finding (`site_fail_on`), or ACT proposes a fix	the findings (`site_health`)	ServiceNow tickets (**always**); an approval (**on fail**)
Certificate report / Windows certificate report	a certificate expires soon	the findings	ServiceNow tickets (**always**)
POA&M status	an item is overdue / due soon	the findings	ServiceNow tickets (**always**)
Troubleshoot / Windows troubleshoot	never for findings (a look, not a verdict)	the findings	a health check, tickets if you want them
Patch hosts / Windows patch	a host fails, or its services do not come back	-	a health check (**always**)
Apply approved ACT fix	the problem is still there	the findings of the re-check	ServiceNow tickets (**always**)
Service watch - check	a container is down (NEEDS APPROVAL)	the fix to approve	an approval (**on fail**)
ServiceNow tickets	problems exist (it opened or updated tickets)	-	- (the end)

Adding ACT to a workflow: what "on fail" means, and where site_act goes

You do not add `site_act` to the workflow. There is no ACT template and no ACT box. ACT runs **inside the check job**. The check playbooks (Health check, Troubleshoot, Service watch, the Windows ones) already include `roles/site_act`, and it runs when that job's answers say `use_act` = `yes`, at the level in `site_act_level`. In the workflow you only give the check box those two answers.

"On fail" is the check job's own result. Nothing extra is checked. The check playbook decides by itself, against its thresholds (the role defaults and your `group_vars`: disk full %, certificate days, ...). The check box ends **red** when:

- it found a problem at or above `site_fail_on` (default: `critical` and `warning`), or
- ACT (`diagnose`) proposed a fix that waits for a person: `NEEDS APPROVAL ... ACT proposes: <commands>`, or
- a host could not be checked (sudo failed, the job stopped on it), or no host at all could be reached. One host that cannot be reached does not make it red: it is listed as NOT REACHED and gets its "no result" ticket (`site_unreachable_fails: true` = red, as before).

A **Run on fail** link from the check box therefore leads to the approval. **Run on success** means the check box ended green; **Run always**, either way.

Box	Link into it	Runs when
1. Health check (`use_act` = `yes`, `site_act_level` = `diagnose`)	(start)	always

Box	Link into it	Runs when
2. Approval	**Run on fail**, from 1	the check job ended red
3. Apply approved ACT fix	**Run on success**, from 2	a person clicked Approve
4. ServiceNow tickets	**Run always**, from 1 and from 3	after the check, and after the fix

The steps in words:

1. **Health check** box: answers `use_act` = `yes`, `site_act_level` = `diagnose`. This needs those two survey questions on the template (ADDING_ACT.md, step 3).
2. **Approval** box, **Run on fail** from 1.
3. **Apply approved ACT fix** box (`playbooks/act_fix_approved.yml`), **Run on success** from 2. It needs no answers: AAP hands it ACT's proposed commands from the health check job, and it runs exactly those.
4. **ServiceNow tickets** boxes, **Run always**, from 1 and from 3.

The approver opens the health check job and reads `ACT | what ACT says` and the red `NEEDS APPROVAL` line. If there is no such line (the job is red because of findings ACT had no fix for), there is nothing to approve: **Deny**. Approving does no harm either: the apply step skips every host with nothing approved.

Without approvals: `site_act_level` = `self-heal` with the fixes you allow in `site_act_allow`. ACT applies those itself, inside the check job, and the checks run again. Then the workflow is simply `[Health check] --Run always--> [ServiceNow tickets]`.

A playbook of your own gets ACT in the **playbook**, not in the workflow: it includes `site_findings` (start, your check), then `site_act`, then `site_findings` report. See ADDING_ACT.md, "Use ACT in a playbook of your own". Its job template then gets the same two survey questions, and it fits the same workflow.

Linux workflows

1. Daily health

```
[Health check]  --Run always-->  [ServiceNow tickets]
 health_checks = daily
```

Step	Template	Answers	Link
1	Health check	`health_checks` = `daily`	(start)
2	ServiceNow tickets	none	**Run always**, from step 1

Why "always". The tickets step opens tickets for what failed, and also notes the tickets of problems that cleared. That needs the healthy runs too. It fails when problems exist, so the workflow shows red until they are fixed.

Schedule: every day at 06:00.

2. Weekly compliance

Three independent chains in one workflow. Add each first box with **Add step** (not "Add step and link"), so they all start together:

```
[Health check]        --Run always-->  [ServiceNow tickets]
 health_checks = weekly
[Certificate report]  --Run always-->  [ServiceNow tickets]
[POA&M status]        --Run always-->  [ServiceNow tickets]
```

Give each chain **its own** tickets box. A box with two parents gets only one parent's findings: the artifacts overwrite each other.

Schedule: Mondays at 07:00.

3. Patch with checks

```
[Health check: pre]  --success-->  [Approval: patch?]  --success-->  [Patch hosts]
 disk, mounts, services                                                    |
        |                                                             Run always
     Run on fail                                                          v
        v                                                       [Health check: post] --always-->
          [ServiceNow tickets]
 [ServiceNow tickets]                                             health_checks = daily
```

Step	Template	Answers	Link
1	Health check	`health_checks` = `disk`, `mounts`, `services`	(start)
2	*Approval*	Name `Patch these hosts now?`, **Timeout** 8 hours	**Run on success**, from 1
3	Patch hosts	`patch_security_only` as you like; `automatic_restarts` (see below)	**Run on success**, from 2
4	Health check	`health_checks` = `daily`	**Run always**, from 3
5	ServiceNow tickets		**Run always**, from 4
6	ServiceNow tickets		**Run on fail**, from 1 (tickets for what the pre-check found)

Why these links:

- **Healthy first.** Patching a host that is already failing makes the next problem impossible to diagnose. The pre-check must pass before anyone is asked. **If any one host fails it, the whole run stops before the approval and no host is patched.** Fix that host (the ticket says what is wrong), or launch again with a Limit that leaves it out. Warnings count too while `site_fail_on` includes `warning`. A disk at 86% blocks patching, so clear it before the maintenance window.
- **A person approves.** The approval step waits (up to its **Timeout**). Deny, or no answer, stops the workflow and nothing is changed.
- **Always check after.** The post-check runs whatever happened in patching, so a half-patched host is still reported.

No automatic restarts unless you ask for them. Patching installs the updates; a host that then needs a reboot is **not** rebooted, and the job says so. The post-check's `patching` finding ("reboot needed") keeps it visible until someone reboots it. To let the job reboot hosts that need it, give step 3 `automatic_restarts` = `yes`. Add a survey question for it on the Patch hosts template (*Multiple Choice*, `no` / `yes`, default `no`), or put `automatic_restarts: true` in the step's variables. Hosts in `patch_never_reboot_groups` are never rebooted

either way.

Launch with **Limit** `patch_hosts`, or one group at a time (`stigman` this week, `servicenow_mid_hosts` next week). **Schedule:** monthly, in your maintenance window, for example the second Wednesday at 20:00. The approval then waits for the on-call person.

4. Patch with a dry run first

The same as 3, but the approver sees exactly what will be installed before approving:

```
[Health check: pre]              health_checks = disk, mounts, services
    |  Run on success
[Patch hosts]                    Job type = Check: lists what it would install
    |  Run on success
[Approval]                       "Install what the dry run listed?"
    |  Run on success
[Patch hosts]                    Job type = Run
    |  Run always
[Health check: post]             health_checks = daily
    |  Run always
[ServiceNow tickets]
```

Step	Template	Answers	Link
1	Health check	`health_checks` = `disk`, `mounts`, `services`	(start)
2	Patch hosts	**Job type** `Check`	**Run on success**, from 1
3	*Approval*	Name `Install what the dry run listed?`. Description: `Open the "Patch hosts" dry-run job: it lists every package per host.`	**Run on success**, from 2
4	Patch hosts	**Job type** `Run`; `automatic_restarts` as you decide	**Run on success**, from 3
5	Health check	`health_checks` = `daily`	**Run always**, from 4
6	ServiceNow tickets		**Run always**, from 5

Needs **Prompt on launch** for **Job type** on the Patch hosts template (steps 2 and 4 set it).

5. Fix with approval (ACT)

For when you use ACT. See ADDING_ACT.md for the three levels.

```
[Health check]  --Run on fail-->  [Approval: apply ACT's fix?]  --success-->  [Apply approved ACT
  fix]
 use_act = yes                                                                          |
 site_act_level = diagnose                                                       Run always
        |                                                                              v
   Run always                                                                [ServiceNow tickets]
        v
 [ServiceNow tickets]
```

Step	Template	Answers	Link
1	Health check (with the ACT survey questions)	`use_act` = `yes`, `site_act_level` = `diagnose`	(start)
2	ServiceNow tickets		**Run always**, from 1
3	*Approval*	Name `Apply the fix ACT proposed?`, **Timeout** 30 minutes. Description: `Read ACT's report and the NEEDS APPROVAL line in the health check job first.`	**Run on fail**, from 1
4	Apply approved ACT fix		**Run on success**, from 3
5	ServiceNow tickets		**Run always**, from 4 (notes the tickets whose problem the fix cleared)

Caveat: not yet proven in a live AAP. The whole approve-and-fix path depends on the check job's result (the proposed commands) reaching the apply job through the Approval node. The playbooks were tested on their own, but that hand-over inside AAP itself has not been run end to end. Rehearse it first with the dry run below, and read SERVICE_WATCH_DEMO.md (Part 9) if the apply job says `Nothing to apply`.

The approver reads, in the health check job, `ACT | what ACT says` (root cause, evidence, the fix) and the red `NEEDS APPROVAL ... ACT proposes: <command>` line. **That exact command** is all the apply step can run: it runs it itself (no model, no key), stops at the first command that fails, and runs the same checks again. The tickets carry ACT's analysis too.

Rehearse it: see ACT's fixes, change nothing.

- **One job, no workflow.** Launch **Health check** (or **Troubleshoot**) with `use_act` = `yes` and `site_act_level` = `diagnose`. ACT investigates with read-only commands and proposes fixes; every change is refused. The job ends red with `NEEDS APPROVAL ... ACT proposes: <commands>`: that list is what would run. Nothing on the host changed.
- **The whole workflow, as a dry run.** On the **Apply approved ACT fix** template (and *Service watch - apply approved fix*) tick **Prompt on launch** next to **Job type**. In the workflow visualizer click the apply box › **Edit** (pencil) › **Job type** `Check` › **Save**. Now run the workflow and approve: the apply job prints `DRY RUN on <host>: approved, and would run: <commands>. Nothing was changed (Check mode).` and ends green. When you trust it, set that node's **Job type** back to `Run`.

6. Security posture (weekly)

The STIG-related checks, and the POA&M list, with tickets:

```
[Health check]  --Run always-->  [ServiceNow tickets]
 selinux, fapolicyd, auditd, accounts
[POA&M status]  --Run always-->  [ServiceNow tickets]
```

Step	Template	Answers	Link
1	Health check	`health_checks` = `selinux`, `fapolicyd`, `auditd`, `accounts`	(start)

Step	Template	Answers	Link
2	ServiceNow tickets		**Run always**, from 1
3	POA&M status		(start, with **Add step**)
4	ServiceNow tickets		**Run always**, from 3

Schedule: Wednesdays 07:00. Use it instead of the `weekly` set in *Weekly compliance* if your security team wants these on their own day.

7. Certificate watch (daily)

Certificates expire on a day, not in a week:

```
[Certificate report]  --Run always-->  [ServiceNow tickets]  --Run always-->  [Certificate renewal
  - CSRs]
```

Step	Template	Answers	Link
1	Certificate report		(start)
2	ServiceNow tickets		**Run always**, from 1
3	Certificate renewal - CSRs (optional)		**Run always**, from 2

Schedule: every day 06:30. Put `check_certs_warn_days` (survey, 30) on the **workflow**, so every step uses the same number. Tickets then start a month ahead. The de-duplication keeps it to one ticket per certificate.

Step 3 makes a new key and a CSR on the host for each certificate file that expires, and attaches the CSR to that certificate's ticket from step 2 (RUNBOOKS.md, "Certificate renewal"). It must be **Run always**: steps 1 and 2 are red on purpose while certificates expire. Hosts in `cert_skip_hosts` or `cert_skip_groups` (Satellite) are left out of every step: set them in `all.yml`, or on the workflow, so every step gets them.

8. Database watch: MariaDB and MySQL (hourly)

```
[Database health - MariaDB / MySQL]  --Run always-->  [ServiceNow tickets]
```

Step	Template	Answers	Link
1	Database health - MariaDB / MySQL	none; it targets `mariadb_hosts`, `mysql_hosts` and `database_hosts` by default (a **Limit** narrows it)	(start)
2	ServiceNow tickets		**Run always**, from 1

The database template runs only the MariaDB / MySQL check (host installs and podman containers), so it is quick and the database team can own it and its tickets (MARIADB.md). Prefer to keep it in the general Health check? Use the *Health check* as step 1 with `health_checks` = `disk`, `services`, `mysql` (`mariadb` and `database` are the same check) and a **Limit** of your database group. The template *Database health* also supports `use_act=true`.

Schedule: hourly. It is quick, and a ticket comes within the hour of a problem.

9. Investigate a host (on demand)

One launch gives the troubleshooting commands, ACT's explanation and every check:

```
[Troubleshoot]  --Run always-->  [Health check]
 ts_area, use_act = yes,          health_checks = all
 site_act_level = explain
```

Step	Template	Answers	Link
1	Troubleshoot	`ts_area` = `overview` (or asked at launch), `use_act` = `yes`, `site_act_level` = `explain`	(start)
2	Health check	`health_checks` = `all`	**Run always**, from 1

Give the workflow **Prompt on launch** for **Limit** and launch it with one host. No tickets: you are already looking. Add `[ServiceNow tickets]` after step 2 (**Run always**) if you want them.

10. STIG Manager deploy with checks

```
[STIG Manager - deploy]         Job type = Check (+ Show changes): what would change
    |  Run on success
[Approval]                      "Apply the changes the dry run listed?"
    |  Run on success
[STIG Manager - deploy]         Job type = Run
    |  Run always
[Service watch - check]
    |  Run always
[Health check]                  health_checks = certs, services
```

Step	Template	Answers	Link
1	STIG Manager - deploy	**Job type** `Check` (and **Show changes**)	(start)
2	*Approval*	`Apply the changes the dry run listed?`	**Run on success**, from 1
3	STIG Manager - deploy	**Job type** `Run`	**Run on success**, from 2
4	Service watch - check		**Run always**, from 3
5	Health check	`health_checks` = `certs`, `services`	**Run always**, from 4

The dry run shows which files and units would change, and how. The approver approves only changes they expected. (This dry run has not been tried on a real server yet: run step 1 on its own once, on the test server, before you build the workflow.)

11. Service watch

The approval or self-heal workflow for containers: SERVICE_WATCH_DEMO.md, Part 6.

If you schedule it every 15 minutes, turn **off** "Enable concurrent jobs" on the workflow (edit the workflow), because its approval waits up to 30 minutes: without that, a new run starts every 15 minutes while the last one still waits for a person.

Windows workflows

Build them the same way, with the Windows templates (WINDOWS.md). The ServiceNow tickets step is the same template as for Linux: it runs on AAP itself.

12. Windows daily health

```
[Windows health check]  --Run always-->  [ServiceNow tickets]
 health_checks = daily
```

Schedule: every day 06:15.

13. Windows weekly compliance

```
[Windows health check]        --Run always-->  [ServiceNow tickets]
 health_checks = weekly
[Windows certificate report]  --Run always-->  [ServiceNow tickets]
```

Schedule: Mondays 07:15.

14. Windows patch with a dry run first

```
[Windows health check: pre]     health_checks = disk, services
    |  Run on success
[Windows patch]                 Job type = Check: lists the updates Software Center has
    |  Run on success
[Approval]                      "Install what the dry run listed?"
    |  Run on success
[Windows patch]                 Job type = Run (automatic_restarts?)
    |  Run always
[Windows health check: post]    health_checks = daily
    |  Run always
[ServiceNow tickets]
```

Step	Template	Answers	Link
1	Windows health check	`health_checks` = `disk`, `services`	(start)
2	Windows patch	**Job type** `Check`	**Run on success**, from 1
3	*Approval*	`Install what Software Center offers (listed in the dry run)?`, **Timeout** 8 hours	**Run on success**, from 2
4	Windows patch	**Job type** `Run`; `automatic_restarts` as you decide	**Run on success**, from 3
5	Windows health check	`health_checks` = `daily`	**Run always**, from 4
6	ServiceNow tickets		**Run always**, from 5

No automatic restarts unless step 4 gets `automatic_restarts` = `yes`: the servers that need a restart are listed, and the post-check's `patching` finding ("a restart is pending") keeps them visible. **Schedule:** inside the servers' ConfigMgr maintenance window. Outside it, ConfigMgr holds the updates and the patch step stops and says so.

15. ACT for Windows rollout

```
[ACT for Windows - install]     Job type = Check: would install / update / already current
    |  Run on success
[Approval]
    |  Run on success
[ACT for Windows - install]     Job type = Run
```

The dry run lists, per server, whether ACT would be installed, updated, or is already current. Launch it with a Limit (a group) to roll out group by group. Run it again after every update of this repository.

16. Windows: investigate a server (on demand)

```
[Windows troubleshoot]  --Run always-->  [Windows health check]
 ts_area                                  health_checks = all
```

Launch with one server in the Limit.

Linux and Windows together

17. Everything, every morning

One workflow, two independent chains (each first box added with **Add step**). Each step uses its own template's inventory, so leave the workflow's Inventory empty:

```
[Health check]           --Run always-->  [ServiceNow tickets]
 health_checks = daily
[Windows health check]   --Run always-->  [ServiceNow tickets]
 health_checks = daily
[Certificate report]     --Run always-->  [ServiceNow tickets]
```

Schedule: every day 06:00. It replaces workflows 1, 7 and 12.

VMware workflows

The VMware templates run on the AAP side against vCenter: no Limit, and each uses its own template's credentials (VMWARE.md).

18. VMware daily checks: health, alarms, then ACT's analysis (daily)

```
[VM - health report]  --Run always-->  [VM - alarms report]  --Run always-->  [VM - alarms ACT
  analysis]
```

Always, not On success: a report is red when `vm_health_fail` / `vm_alarm_fail: true` is set, and the next step should run anyway. The health report covers every datacenter, cluster and host (HA, DRS, vCLS, hardware); the alarms report the alarms and the day's events; the analysis ACT's likely cause and fix for each. Each step sends its own email (`report_email_to` on each template, or a workflow survey question `report_email_to` for all). **Schedule:** every day 06:00. Only the analysis is wanted? Schedule `VM - alarms ACT analysis` on its own: its email lists every problem.

19. Snapshot cleanup with approval

```
[VM - snapshot report]  --On success-->  [Approval]  --On success-->  [VM - snapshot cleanup]
```

Click by click, with the survey and the dry run: VMWARE.md, "Snapshots".

20. ESXi security settings (nightly)

No workflow needed: a schedule on the `ESXi - security settings` template, every day at 00:00 in **your** time zone (VMWARE.md, "ESXi security settings").

22. vSphere capacity planning (weekly)

No workflow needed: a schedule on `VM - capacity planning`, e.g. every Monday at 07:00. One template per cluster (`vm_capacity_clusters`) gives each cluster's owners their own report (VMWARE.md, "Capacity planning").

23. VMs created this week (weekly)

No workflow needed: a schedule on `VM - VMs created report`, every Friday at 12:00 in your time zone (Repeat frequency *Week*, on *Friday*). It reports the VMs created in the 7 days up to the run (VMWARE.md, "VMs created").

21. NetApp ONTAP health (daily)

No workflow needed: a schedule on the `NetApp - ONTAP health report` template, e.g. every day at 06:00 in your time zone (NETAPP.md). In a workflow with `ontap_report_fail: true`, its "On failure" path can open a ticket.

24. Active Directory health (daily)

No workflow needed: a schedule on the `Active Directory health report` template, every day at 06:30 in your time zone, with `ad_report_hours: 24` (the default) so each change is in exactly one report (ACTIVE_DIRECTORY.md). In a workflow with `ad_report_fail: true`, its "On failure" path can open a ticket.

Schedules

What. A schedule launches a template or workflow at set times.

How. Open the workflow (or template) › **Schedules** tab › **Create schedule**:

1. **Name**, **Start date/time**, **Time zone** (use your site's, not UTC, so "06:00" means 06:00).
2. **Repeat frequency** and its details: *Day* (every 1 day), *Week* (on Monday), *Month* (on the second Wednesday)...
3. **Prompts**: the survey answers, the Limit and the Job type **for this schedule**. Without them, the schedule uses the template's **defaults**. A schedule can only change what the template **prompts on launch** or asks in its survey.
4. **Finish**. The schedule's page shows the next run times. Check them once.

What	When	Launches
Daily health (or *Everything, every morning*)	every day 06:00	workflow 1 (or 17)
Certificate watch	every day 06:30	workflow 7
Database watch	hourly	workflow 8
ServiceNow health	hourly	template *ServiceNow health*, with an email **notification on failure**
Weekly compliance	Monday 07:00	workflow 2

What	When	Launches
Security posture	Wednesday 07:00	workflow 6
Patch preview	the Monday before patch night	template *Patch hosts*, **Prompts › Job type** `Check`
Patch with checks	monthly, in your window	workflow 3 or 4
Service watch	every 15 minutes (with **Enable concurrent jobs** switched off on the workflow)	workflow *Service watch - approve or self-heal*
Windows daily health / weekly compliance	06:15 daily / Monday 07:15	workflows 12 and 13
Windows patch	monthly, in the ConfigMgr maintenance window	workflow 14
VMware daily checks (health, alarms, ACT analysis)	every day 06:00	workflow 18
Active Directory health	every day 06:30	template *Active Directory health report* (24)

Notifications (optional). **Automation Execution › Administration › Notifiers › Create notifier** (type *Email*, your mail relay). Then on a template or workflow › **Notifications** tab › switch on **Failure**. Use it where a ticket cannot help. ServiceNow health is the main case: if ServiceNow is down, it cannot take a ticket about itself.

Things that trip people up

- **Red is not broken.** A check job with findings is red by design. Read `Report | result for this host`: findings mean the playbook worked.
- **Save the visualizer.** Closing without **Save** loses the graph.
- **One tickets box per check box.** Artifacts from two parents overwrite each other.
- **A step can only be set to Check** (or given a Limit, or survey answers) if its template has **Prompt on launch** for it, or a survey.
- **Schedules use survey defaults** unless the schedule sets its own answers (Prompts step).
- **Approvals need a timeout.** Otherwise a forgotten approval keeps the workflow open forever. After the timeout it counts as denied.
- **Workflow variables override node answers.** Leave the workflow's **Variables** empty, and set answers on each step.
- **Leave the workflow's Inventory empty** unless every step should use it: it only reaches steps whose template prompts for the inventory, and a Linux inventory on a Windows step matches no host.
- **Limit on the workflow** applies to every step whose template prompts for a Limit. The controller-side templates (ServiceNow tickets, ServiceNow health, POA&M status) must **not** prompt for one, or a workflow Limit makes them match no host and silently do nothing.
- **A tickets step fails when nothing reached it.** If the check step died before checking any host (credential, project sync, execution environment), the tickets step fails too, so a broken schedule shows red instead of green. Read the first red job.

- **No automatic restarts** unless a patch step gets `automatic_restarts` = `yes`.

6. Dry runs: try anything, change nothing

A **dry run** runs a job without changing anything. Ansible calls it *check mode*; AAP calls it **Job type: Check**. Every step that would change something only says what it *would* change. The steps that only read (the health checks, looking at logs, asking ServiceNow what is open) still run, so the answers are real.

Use a dry run the first time you use any template, on one host, and before every change you are not sure about: patching, applying an approved fix, installing ACT.

Where to set it

There are five places. Pick the one that fits:

Where	How	Good for
1. The template	Template › **Edit** › **Job type** `Check`	a template that is *always* a dry run, e.g. a copy named `Patch hosts (dry run)` that more people may launch
2. At launch	Template › **Edit** › tick **Prompt on launch** next to **Job type**. Then at launch, pick `Check` or `Run`	the usual way: the same template, you choose each time
3. A workflow step	(needs 2 on the template) Workflow › **Visualizer** › click the step › **Edit** (pencil) › **Job type** `Check` › **Save**	rehearsing a whole workflow, or a "dry run first, then real" workflow
4. A schedule	(needs 2) Template or workflow › **Schedules** › the schedule › **Prompts** › **Job type** `Check`	a regular preview, e.g. "what would patch night install" a week before
5. Command line	`ansible-playbook ... --check` (add `--diff` to see file changes)	testing from a shell

Show changes. Tick **Show changes** on the template (or `--diff`) and a dry run also shows the line-by-line changes it would make to files, for example in STIG Manager's configuration.

How to tell. The job's **Details** tab says **Job type: Check**. Our playbooks also print `DRY RUN (Check mode): would ...` where it matters.

What each template does in a dry run

Template	In a dry run it...
Health check, Database health, Troubleshoot, Certificate report	runs **exactly as usual**: they only read, so you get the same findings. Nothing is written: no syslog lines, no disk-forecast sample, and on Windows no event log entries
POA&M status	as usual: reads the CSV and asks STIG Manager
ServiceNow tickets	asks ServiceNow what is already open, then prints `would open: ...`, `would add a note to: ...`, `would resolve: ...`. **Nothing is sent**
ServiceNow health	as usual: only reads
ServiceNow - test ticket	logs in and checks the assignment group and caller (it only reads), then prints the incident it would open, note and resolve. **Nothing is created**
Patch hosts (Linux)	lists the packages dnf would update. Installs nothing, restarts nothing
Apply approved ACT fix	`DRY RUN on <host>: approved, and would run: <commands>`. Runs nothing. A command the apply-time guard would refuse (reboot, `mkfs`, ...) shows `Refused on <host>: ...` and that host goes red, as in the real run
Service watch - check	checks the containers and shows what it would propose. Writes no incident, does not call ACT
Service watch - apply approved fix	`DRY RUN on <host>: approved, and would run: <commands>` (or `Refused on <host>: ...`, as above)
STIG Manager - deploy	says which files and units would change (with Show changes: how), and verifies nothing (nothing was started). Changes nothing. *Not yet tried on a real server: try it once on the test server*
Windows health check, troubleshoot, certificate report, connection test	as usual: they only read
Windows patch	asks Software Center to rescan (without starting anything) and prints `DRY RUN (Check mode): would install <updates>`
ACT for Windows - install	`DRY RUN: would install ACT in C:\ProgramData\act because ...`, or that it is already current
VMware jobs (restart, shut down, snapshot, delete snapshot, notes, change VLAN)	finds the VMs and runs every check (the AAP VM is refused as in a real run), then prints `Would restart ...`, `Would take snapshot ...`, `would delete ...`, the notes `AFTER`, `would move ...`. **vCenter is not changed**, and no report email is sent (`DRY RUN: would email ...`)

Template	In a dry run it...
VM - secure boot report, VM datastore report, VM snapshot report, VM alarms report	as usual: only read. No report email is sent
VM - alarms ACT analysis	reads vCenter and lists the problems it would give ACT (`A dry run (Check): ACT was not called`). ACT is not called, no email is sent
NetApp - ONTAP health report	as usual: it only reads (GET requests), and prints the same report. No email is sent
VM - capacity planning	as usual: it only reads. With `vm_capacity_act`, ACT is not called (`ACT did not run: A dry run`). No email
ESXi - security settings	reads every host's SSH service, timeouts and lockdown mode and lists what it would change (`ESXi security settings (dry run): 3 of 40 host(s) would change`). **No host is changed**, no email is sent
VM - snapshot cleanup	lists what it would delete (`WOULD DELETE ...`), deletes nothing, sends no email. In the approval workflow: set the cleanup step's Job type to Check (docs/VMWARE.md, "Dry run")

ACT does not run in a dry run. It would call the model, and ACT could change things. A dry run with `use_act` = `yes` reports the checks and says ACT gave no result. To see ACT's ideas without changes, do a *real* run with `site_act_level` = `diagnose`: ACT then only reads and proposes (ADDING_ACT.md).

Examples

Before patching one server. Launch **Patch hosts** › **Job type** `Check`, **Limit** `web01`. The output lists every package that would be updated. Nothing is installed.

```
TASK [patch : Patch | what was updated (dry run - what WOULD be updated)]
ok: [web01] => {"msg": "DRY RUN (Check mode), nothing was installed - would make 4 change(s):
  Installed: expat-2.5.0-6.el9_8.5.x86_64; Installed: libxml2-2.9.13-14.el9_8.5.x86_64;
  Removed: libxml2-2.9.13-14.el9_8.4.x86_64; Removed: expat-2.5.0-6.el9_8.3.x86_64"}
```

(`Removed` is the old version that the new one replaces.)

Before patch night, every month. On **Patch hosts** add a schedule for the Monday before patch night, with **Prompts** › **Job type** `Check`. The job output shows what patch night will install on every host, in time to exclude something.

A dry-run copy for the team. Template **Patch hosts** › **…** › **Copy**. Rename the copy `Patch hosts (dry run)`, set **Job type** `Check`, untick **Prompt on launch** for it, and give your team **Execute** on the copy only. They can preview; only you can patch.

Which tickets would a run open? In the *Daily health* workflow click the **ServiceNow tickets** step › **Edit** › **Job type** `Check`. Run the workflow: the tickets job prints what it would open, note and resolve. Set it back to `Run` when the tickets look right.

A Windows server, before its first real patch. Launch **Windows patch** › **Job type** `Check`, **Limit** one server:

```
"DRY RUN (Check mode): would install 2026-09 Cumulative Update (KB5040001); Malicious Software
  Removal Tool (KB890830)"
```

The approve-and-fix path, end to end. See WORKFLOWS_AND_SCHEDULES.md, "Fix with approval (ACT)": set the apply step to `Check`, run the workflow, approve. The apply job shows what it would run.

From a shell (on a machine with Ansible and this repository):

```shell
ansible-playbook -i <inventory> playbooks/patch_hosts.yml --check --diff -l web01
ansible-playbook -i <windows inventory> playbooks/win_patch.yml --check -l web01
```

What a dry run cannot tell you

- Whether the updates will **install cleanly**, or how long they take: it only lists them.
- Whether a **restart** would succeed: nothing restarts in a dry run.
- Anything that **depends on an earlier change**. In a dry-run workflow the post-check after "patching" still sees the unpatched host, because nothing was patched.
- What **ACT** would say (see above).

7. The runbooks: every check and what to do

For each runbook: what it answers, its settings (defaults in `roles/<role>/defaults/main.yml`; change them in your settings files or AAP's Variables boxes, as VARIABLES.md shows), and each finding with what usually fixes it. The `look:` command printed with a finding is always the first thing to run.

Health check (playbooks/health_check.yml)

Runs the checks you pick on each host: a list of names, or the shortcuts:

- `daily` = disk, mounts, services, performance, time, network, logging, mariadb (the database check: MariaDB or MySQL; `mysql` and `database` also pick it), containers (podman containers, rootful and rootless)
- `weekly` = selinux, fapolicyd, auditd, accounts, certs, patching
- `all` = both

A host with a finding at a severity in `site_fail_on` (default: critical and warning) fails. That is what a workflow reacts to. A template that only emails its report to people: `site_fail_on: []` keeps the job green (EMAIL_REPORTS.md). A check that itself breaks becomes a warning finding (`the X check could not run: ...`, id `<check>:check-error`) and the other checks still run. That finding fails the host even when `site_fail_on` is `[critical]` (a check that did not run is not a healthy result); only `site_fail_on: []` never fails. Its ServiceNow incidents from earlier runs stay open: a check that did not run to its end clears nothing it found before (no "passes again" note, nothing resolved).

A host that cannot be reached (switched off, no network, ssh or WinRM refused, its login refused) does not fail the job. It is listed as `NOT REACHED - <why>` and left out; the other hosts are checked as usual. It still counts as checked-for, so the ServiceNow step opens its "no result" ticket. The same goes for the certificate report and renewal, database health, the service watch and troubleshoot, Linux and Windows. Two cases still fail the job:

- **no host at all is reached**: that is what a wrong or expired machine credential looks like;
- `site_unreachable_fails: true`: a host not reached fails the job, as before.

Patching, the ACT and service fixes, the STIG Manager deploy and the Windows connection test always fail for a host they cannot reach: a workflow must not go on as if it had been patched or fixed. A host lost later in a job (it went down halfway) still fails it.

All settings of this job, with their defaults: site_findings.

disk

Finding	Means	Usually
`/var is 96% full`	local filesystem space (`check_disk_warn_pct` 85, `check_disk_crit_pct` 95)	find what grew with the `look:` command: old logs, core dumps, a runaway log file. `journalctl --vacuum-size=500M` for the journal
`... has used 97% of its inodes`	too many small files (space may still be free)	usually a directory with millions of small files (session/cache/spool); `look:` finds it
`/var will be full in about 3.2 days`	growth-rate forecast from earlier runs (needs a day of history)	same as above, before it becomes an outage
`could not read filesystem usage`	`df` failed or timed out	a hung mount; see the mounts check

Per-mount limits: `check_disk_overrides: {/var/log/audit: {warn: 70, crit: 85}}`. The forecast keeps a small history in `/var/lib/site-health/disk-history.json` (the only file the checks write; a Check-mode run does not write it).

All settings of this check, with their defaults: check_disk.

mounts

Finding	Means	Usually
`/data (xfs) is in /etc/fstab but is not mounted`	it did not mount at boot, or was unmounted	`journalctl -b` for the mount error; a missing disk or LUN; a typo in fstab
`/var is mounted READ-ONLY`	the kernel remounted it read-only after disk/filesystem errors	**urgent**: `journalctl -k` for I/O errors; storage team; plan an fsck
`network mount /mnt/share does not answer within 5s`	NFS/CIFS server down, network, or a stale handle	check the server and the network; diagnose only (NetApp is vendor-managed)
`network mount ... is 96% full`	the share is full	the storage owner

All settings of this check, with their defaults: check_mounts.

services

Finding	Means	Usually
`systemd unit X has FAILED`	a unit crashed or exited with an error	`systemctl status X`, `journalctl -u X`; fix the cause, then `systemctl restart X`
`required service X is inactive`	a service that must run (`check_services_required` + `_extra`) is stopped	same; find out why it stopped before starting it
`required service X is not installed`	the unit does not exist here	remove it from the list for this group, or install it
`required service X is disabled`	it will not start after a reboot	`systemctl enable X`
`X has restarted 5 times`	a restart loop: it keeps crashing	its log shows the crash

Per group: `check_services_required_extra: [mariadb]`. To ignore a failed unit: `check_services_ignore_failed: ['^dnf-makecache']`.

All settings of this check, with their defaults: check_services.

performance

Finding	Means	Usually
`15-minute load is 14.2 on 4 CPUs (3.55 per CPU)`	more work than CPUs, for 15 minutes	the top CPU processes are in the finding; a runaway job, or undersized
`only 4% of memory is available`	memory is nearly exhausted	the top memory processes are in the finding; a leak, or undersized
`63% of swap is in use`	the host is short of memory	as above
`CPUs spend 35% of their time waiting for disk`	slow or overloaded storage	`iostat -x` shows which device; storage latency
`the hypervisor takes 15% of CPU time (steal)`	the VM host is overcommitted	the virtualization team
`the kernel killed processes for lack of memory`	OOM kills in the last 24 h (with the victims)	the victim's memory limit, or the host's memory
`27 zombie processes`	a parent process is not reaping its children	restart the parent (`look:` shows it)

All settings of this check, with their defaults: check_performance.

time

Finding	Means	Usually
`the clock is NOT synchronized`	chrony uses no time source	`chronyc sources -v`: unreachable servers, firewall (UDP 123)
`chrony is installed but not answering`	chronyd is not running	`systemctl start chronyd`
`the clock is 2.4s ahead of NTP time`	offset above `check_time_offset_warn`	chrony corrects it slowly; very large offsets break Kerberos/AD logins at 5 minutes

All settings of this check, with their defaults: check_time.

network

Finding	Means	Usually
`there is no default route`	the host cannot reach other networks	NetworkManager connection, gateway
`this host's own name ... does not resolve`	its FQDN is not in DNS or /etc/hosts	DNS record
`ldap.example.mil does not resolve (DNS)`	a name in `check_network_dns_names` fails	`/etc/resolv.conf`, DNS server
`LDAP (ldap.example.mil:636) does not answer within 5s`	a TCP port in `check_network_tcp` is unreachable **from this host**	firewall (local or network), the service down, routing

All settings of this check, with their defaults: check_network.

logging

Finding	Means	Usually
`the journal is kept in memory only`	`/var/log/journal` missing: logs vanish at reboot	`mkdir /var/log/journal && systemctl restart systemd-journald` (or `Storage=persistent`)
`log server SIEM (...) does not answer`	forwarding target unreachable	network/firewall; the SIEM collector
`rsyslog has 850 MB of undelivered messages queued`	the log server has been unreachable for a while	as above; the queue drains once it answers
`1203 error messages in the journal`	something complains a lot (top senders listed)	look at the top sender first

All settings of this check, with their defaults: check_logging.

selinux

Finding	Means	Usually
`SELinux is Permissive, it must be enforcing`	not enforcing (STIG)	`setenforce 1` after checking denials; `SELINUX=enforcing` in `/etc/selinux/config`
`/etc/selinux/config says SELINUX=permissive`	a reboot would change the mode	fix the config file
`locally added permissive domain(s)`	SELinux only logs for those domains	`semanage permissive -d <domain>` once the policy is right
`12 SELinux denial(s) since yesterday - top: nginx (httpd_t) denied read on a file labeled user_home_t`	SELinux blocked something	a wrong label (fix: `restorecon -Rv <path>`), a boolean (`setsebool -P ...`), or a real policy need. `audit2why` says which
`SELinux boolean X is off, expected on`	a boolean in `check_selinux_booleans` differs	`setsebool -P X on`
`N file(s) under /etc/nginx have the wrong SELinux label`	a dry-run `restorecon` would relabel them (`check_selinux_relabel_paths`)	`restorecon -Rv <path>`

All settings of this check, with their defaults: check_selinux.

fapolicyd

Finding	Means	Usually
`fapolicyd is inactive`	application allow-listing is off	`systemctl start fapolicyd`; its log says why it stopped
`fapolicyd is in permissive mode`	it logs but blocks nothing	`permissive = 0` in `/etc/fapolicyd/fapolicyd.conf`
`N file(s) no longer match the fapolicyd trust database`	files changed outside RPM	`fapolicyd-cli --update`, or re-add the file's trust entry
`N fapolicyd denial(s) since yesterday - top: /usr/bin/bash -> /opt/app/tool`	it blocked programs	a legitimate program needs a trust entry (`fapolicyd-cli --file add <path>`), after review

`check_fapolicyd_required: true` (the example inventory sets it) makes a missing fapolicyd a finding.

All settings of this check, with their defaults: check_fapolicyd.

auditd

Finding	Means	Usually
`auditd is failed`	nothing is being audited	`systemctl status auditd`; often a full audit log partition
`auditing is DISABLED in the kernel`	`auditctl -s` says enabled 0	`audit=1` on the kernel command line
`only 1 audit rule(s) are loaded`	the STIG rules are missing or failed to load	`augenrules --check`, `augenrules --load`; a syntax error in `/etc/audit/rules.d/`
`N audit event(s) were LOST`	the kernel backlog overflowed	raise `-b` (backlog) in the rules
`the audit log filesystem is 92% full`	auditd may stop, or halt the host, when it fills	rotate/offload; check `space_left_action`
`audit records are not forwarded`	au-remote is not active (`check_auditd_remote_required`)	configure audisp-remote
`audit volume is 850 MB a day (... events in 24 h, at most N in one hour) - mostly account svc_x (60%), rule key delete (55%), program /usr/bin/python3 (40%)`	more audit records than `check_auditd_volume_warn_mb_day` (200), excluded accounts not counted	what the named account/program does that the named STIG rule records: a job deleting temp files in a loop, an app in a rootless container (its processes keep the user's login ID)
`the audit logs on disk reach back only 9.5 h (5 files of 8 MB, then the oldest is deleted)`	with `max_log_file_action = ROTATE` older records are gone (`check_auditd_retention_warn_hours` 24)	bigger `max_log_file` / more `num_logs` in `/etc/audit/auditd.conf`, or fewer records (the finding names who makes them)
`auditd dropped records N time(s) ... on the way to its plugins`	the queue to au-remote / the SIEM forwarder was full: the SIEM copy is missing records	fewer records; a larger `q_depth` in `auditd.conf`
`audit rule(s) stop recording auid=1234 (someone) altogether`	a `-a never` (or `-a always,exclude`) rule hides an account that is not in `check_auditd_exclude_accounts`	remove the rule, or approve the account (below)
`too much to break down by account within 120 s`	very large logs	`aureport` with the `look:` command, or a larger `check_auditd_volume_timeout`

How much is audited, and the vulnerability scanner. The volume, retention and rule findings come from the audit logs themselves. The breakdown (which account, which STIG rule key, which program) is read only when the volume or the retention is over its limit, at low priority, and is always printed in the job output (`Auditd | how much is audited`), with excluded accounts shown separately.

A credentialed vulnerability scan logs in to every host and runs thousands of commands with sudo: during a scan its account usually makes most of the audit records. List it in `check_auditd_exclude_accounts` (in `playbooks/group_vars/all.yml`, so it applies to every host) and it no longer counts toward the volume warning. It still counts for retention and disk space, which are physical: if the scan rotates a week of records

away in an hour, that is still reported, with the scanner named as the cause. The job output shows how much of the scanner's volume comes from syscall rules and how much from sudo/PAM records, which tells you what an audit rule exclusion would remove.

To stop auditd recording that account at all (a host change, not part of the health check):

- It is a deliberate exception to the STIG audit requirements: get it approved and recorded first.
- The rules `-a never,exit -F arch=b64 -S all -F auid=<UID>` and the same with `arch=b32` (the form of audit's own sample rules), placed before the STIG rules, remove the syscall records: a file like `/etc/audit/rules.d/05-exclude-scanner.rules` (`augenrules` merges the files in name order). The sudo/PAM records (USER_CMD, USER_START, CRED_*) come from user space and need `-a never,user -F auid=<UID>`. Leaving that one out keeps the sudo records, which still show what the account ran.
- Use the numeric UID. A name is looked up when the rules load at boot, before IdM/AD (sssd) can answer, and a rule that fails to load can stop the rest of the rules from loading.
- STIG hosts make the rules immutable (`-e 2`): a new rule takes effect only at the next reboot.
- The auditd check then reports rules like this only for accounts that are NOT on `check_auditd_exclude_accounts`.

All settings of this check, with their defaults: check_auditd.

accounts

Finding	Means	Usually
`account(s) other than root have UID 0`	full root power under another name	**investigate first**; remove or renumber the account
`account(s) with an EMPTY password`	log in without a password	`passwd -l <user>`
`3 account(s) not used for 35+ days`	inactive and not locked (STIG)	lock them (`usermod -L`), or list break-glass accounts in `check_accounts_ignore_inactive`
`sudo without a password (NOPASSWD) for: alice`	a sudoers entry skips the password	remove it, or allow it on purpose in `check_accounts_nopasswd_allowed` (e.g. `svc_aap`)
`the password of svc_aap expires in 6 days`	an account in `check_accounts_expiry_watch` expires soon	**change it before it breaks automation** (and update the Machine credential)

All settings of this check, with their defaults: check_accounts.

certs

Finding	Means	Usually
`certificate CN=... in /etc/stigman/stigman.crt expires in 9 days`	a certificate file expires within `check_certs_crit_days` (critical) / `warn_days` (warning)	renew it. For STIG Manager: update the *TLS certificate* credential and run *STIG Manager - deploy*; it restarts only nginx
`... in STIG Manager (https://127.0.0.1:443) expires in ...`	the certificate a TLS endpoint serves	the service's certificate, even if the file on disk was already replaced (restart the service)
`... in /opt/.../agent_keystore (alias mid) expires in ...`	a Java keystore entry	renew in the keystore (MID Server: ServiceNow's procedure)
`could not check the certificate in X`	unreadable file, endpoint not answering, or a PKCS12 keystore without its password	fix the path, or add the Keystore password credential

What it looks at: `check_certs_files` (globs), the usual directories (`check_certs_discover_dirs`, skipping CA bundles), `check_certs_endpoints`, `check_certs_keystores`. For a PKCS12 keystore, name the variable that holds the password (`password_env: KEYSTORE_PASSWORD`), never the password.

All settings of this check, with their defaults: check_certs.

patching

Finding	Means	Usually
`12 security advisories pending, 2 of them rated Critical`	updates that fix security issues are waiting	patch (the *Patch with checks* workflow)
`a reboot is needed to finish earlier updates`	new kernel or core libraries are not in use yet	reboot in a window
`no package has been updated for 60 days`	older than `check_patching_max_age_days`	patch
`cannot check for updates`	dnf cannot reach its repositories (Satellite, mirror)	repository configuration, network, subscription

All settings of this check, with their defaults: check_patching.

mariadb and mysql (health only): the database check

Checks **MariaDB and MySQL** (5.7, 8.0, 8.4; MariaDB 10.x and 11.x; Percona). Pick it in the Health check as `mariadb`, `mysql` or `database` (the same check), or run it alone with the **Database health - MariaDB / MySQL** template (`playbooks/database_health.yml`: only this check, on every database server it is pointed at; it fails a host that has findings, so a workflow can open a ticket). In the Health check it runs on hosts in `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb` or `mysql`, and on any host that has a container name set. It runs status queries only: no data is read and nothing is changed. It handles a database installed on the host and one in **podman containers** (one, several, found automatically, or rootless): the client inside each container is used. **How to set it up, step by step, and how to verify it: MARIADB.md.**

Finding	Means	Usually
`MariaDB is not running`	the service or container is down	its log; disk full; a crash
`MariaDB is running but the status query failed`	cannot log in, or it hangs	the login (see below); `max_connections` reached
`MariaDB restarted 12 minutes ago`	a restart nobody may have planned	the error log: crash, OOM kill
`143 of 151 connections in use (95%)`	clients are about to be refused	the `look:` query shows which user and client hold them; a connection leak in an app
`N client connection(s) were REFUSED`	`max_connections` was reached since the last start	as above; raise `max_connections` only if the load is real
`40 queries are executing at this moment`	a pile-up: locking or one slow query	`look:` lists the oldest
`2 query/queries running longer than 300s`	long-running queries (user / db / seconds / state; never the query text)	the owner decides; `KILL <id>` only after that
`replication (default) is broken: IO thread No`	a replica stopped copying	`Last_IO_Error` / `Last_SQL_Error` in the finding
`replica ... is 1900 seconds behind`	replication lag	load on the replica; long transactions on the primary
`Galera node is not healthy` / `cluster has 2 node(s), expected 3`	Galera cluster trouble	the node's error log; network between nodes
`MySQL Group Replication is not healthy` / `has 2 ONLINE member(s), expected 3`	a Group Replication member is not ONLINE, or the group is short (`check_mariadb_group_size`)	`SELECT ... FROM performance_schema.replication_group_members` (in the finding); the member's error log
`could not read the replication status` / `... Group Replication members`	the monitoring account lacks a privilege	the grant named in the finding (MARIADB.md)
`only 91% of reads come from memory`	`innodb_buffer_pool_size` may be too small	the DBA
`3 [ERROR] line(s) in the MariaDB log`	errors in its log in the last day (latest shown)	read them
`the MariaDB data directory is on a filesystem 93% full`	MariaDB stops writing when it fills	disk space

Performance (all warnings; `check_mariadb_performance: false` turns them off). Counters that count since the server started are judged as an average since then, and only after it has run

`check_mariadb_perf_min_uptime_hours` (2): cold caches right after a start are normal. Old transactions, lock waits, purge lag and open files are judged as they are now. The job's status line prints the numbers either way: `performance since start, 6 days: 100 queries/s, 3 slow/h, 4% temporary tables on disk, ...`.

Finding	Means	Usually
`mariadb:slow-queries: 504 slow queries per hour ...`	queries slower than `long_query_time`, on average since the start	the slow query log (the finding says whether it is on) or `performance_schema` (the `look:` query lists the costliest statements) shows which; an index or a rewrite
`mariadb:tmp-disk: 60% of the temporary tables ... went to disk`	sorts and GROUP BYs spill to disk	`tmp_table_size` and `max_heap_table_size` (the smaller one counts); TEXT/BLOB columns always spill
`mariadb:full-joins: 1,440 joins per hour ... read a whole table`	joins with no usable index	the DBA adds the index; they get slower as the tables grow
`mariadb:table-locks`	MyISAM / Aria tables locked as a whole are busy	convert them to InnoDB (the `look:` query lists them)
`mariadb:row-locks: queries waited on average 2,000 ms for a row ...`	transactions wait for each other's row locks	long transactions, or many writers on the same rows
`mariadb:deadlocks`	InnoDB rolled back one transaction of each deadlock	the application's order of updates; `SHOW ENGINE INNODB STATUS` shows the latest
`mariadb:lock-waits: waiting for a lock for 30 s or longer right now ...`	sessions blocked NOW (metadata, table or row locks; who / db / seconds / state, never the query)	the session holding the lock: usually an open transaction or an `ALTER TABLE`. The owner decides before any `KILL`
`mariadb:long-transactions: 2 transaction(s) open longer than 600 s ...`	transactions left open hold their locks and stop purge; `idle (Sleep)` = the application never committed	the application owner; a connection pool that does not commit
`mariadb:purge-lag: 2,500,000 old row versions wait to be purged`	InnoDB history list length: reads slow down, undo grows	the long transaction above
`mariadb:open-files`	close to `open_files_limit`	raise `open_files_limit` (and the unit's `LimitNOFILE`)
`mariadb:table-cache`	the table cache is full and tables are reopened all the time	`table_open_cache`
`mariadb:bufferpool-wait`, `mariadb:log-waits`	InnoDB waited for a free buffer pool page / a full log buffer	`innodb_buffer_pool_size` / `innodb_log_buffer_size`
`mariadb:aborted-connects`	many failed connection attempts per hour	wrong passwords, a health probe that does not log in, or a scan

The performance query reads `information_schema.INNODB_TRX` (needs `PROCESS`, which the monitoring account already has) and `INNODB_METRICS`; without the privilege those findings are simply not made, the

rest still are.

Logging in (the full steps are in MARIADB.md). Without a credential, the check logs in as root through the local socket. That is MariaDB's default for a host install on RHEL (no password needed with sudo); MySQL's root usually has a password. **A database in a container usually gives root a password** (the official images do), and the check then reports `Access denied ... attach the "MariaDB monitor" credential`. For containers, or whenever you prefer not to use root, ask the DBA to create a monitoring account with **no data access**, inside that database with host `localhost`, and attach the *MariaDB monitor* credential to the Health check template:

```
SQL
CREATE USER 'aap_monitor'@'localhost' IDENTIFIED BY '<password>';
GRANT PROCESS, REPLICATION CLIENT ON *.* TO 'aap_monitor'@'localhost';   -- MySQL; MariaDB before
  10.5.9
GRANT PROCESS, REPLICA MONITOR ON *.* TO 'aap_monitor'@'localhost';      -- MariaDB 10.5.9 and
  newer instead
GRANT SELECT ON performance_schema.* TO 'aap_monitor'@'localhost';       -- MySQL Group
  Replication only
```

`PROCESS` shows other sessions (for long queries). Replication status needs `REPLICATION CLIENT` (MySQL) or, on MariaDB 10.5.9 and newer, `REPLICA MONITOR`: `REPLICATION CLIENT` **alone is not enough there.** The password reaches the client only through its environment, never its command line.

All settings of this check, with their defaults: check_mariadb.

containers: podman containers, root's and every user's

Finds the podman containers on the host by itself - you list nothing - and checks each one. Hosts without podman are skipped (`skipped: podman is not installed`, no finding). It is read-only: it runs `podman ps`, one `podman container inspect` and `systemctl show`, and nothing that starts, stops or creates anything.

Which containers. Root's (rootful podman) and each user's (rootless podman: only that user can see them). A user is looked at when a podman process runs as them, when they have *linger* on (a file in `/var/lib/systemd/linger/`), or when they are in `/etc/subuid` and have container storage in their home (`~/.local/share/containers/storage`). A user's containers are read **as that user** (`runuser -u USER -- env XDG_RUNTIME_DIR=/run/user/UID podman ...`), and only when `/run/user/UID` exists: otherwise podman would create it, and a check must not change anything. Such a user is listed as `not checked` instead (and reported, below).

Which should be running. A container should be running when a systemd unit runs it and starts it at boot (a Quadlet `.container` file with an `[Install]` section, or an enabled unit from `podman generate systemd` or written by hand), or when its restart policy is `always` or `unless-stopped`. A container that has neither and stopped with exit code 0 was stopped on purpose: no finding. A Quadlet or `--new` unit removes its container while the unit is stopped, so the check also reads the Quadlet files and unit files: a container that should run but does not exist at all is found too (state `missing`).

Pods and compose projects too. Some units start several containers whose names the unit does not say: a Quadlet `.kube` file or a `podman-kube@` unit (`podman kube play` of Kubernetes YAML), a unit that runs `podman pod start`, `podman-compose` / `podman compose` / `docker-compose`, or `podman run` without `--name`. Each such unit is a row of its own (the name of the unit, kind `pod`, `compose project` ...), and it is down when its unit is not active. A `Type=oneshot` unit that finished successfully since boot (`podman-compose up -d`) counts as up: its containers run on without it. Root's units are read from `podman_discover_unit_dirs` (`/etc/systemd/system`, `/usr/local/lib/systemd/system`), a user's from `~/.config/systemd/user`.

Podman's own tools (`podman-auto-update`, `podman-restart`, the API service) are left out. The containers of a running pod are listed as containers when they carry the unit's label (Quadlet sets it).

The output has a table of what was found:

```
OWNER        NAME                      STATE             UNIT                          SHOULD
  RUN
root         systemd-db                running           db.service (active)           yes
root         web                       running           -                             yes
alice        web                       missing           web.service (failed)          yes
bob          web                       exited (3)        -                             no
not checked: carol - no /run/user/1003 (the user is not logged in and linger is off): ...
```

Every finding names the owner, and its `look:` command runs as that owner (paste it as root):

Finding	Means	Usually
`container X (owner O) should be running (...) but it does not exist / it exited with code N` (critical)	a container that should run is down, or its systemd unit failed	read the `look:` output (the unit's status, the container's log, the journal). Start it the right way: `systemctl start UNIT` for a unit (as a user: `runuser -u O -- env XDG_RUNTIME_DIR=/run/user/UID systemctl --user start UNIT`), `podman start X` for a plain container. Service watch does this for you, with approval
`required container X ... does not exist` / `is not running` (critical)	a container in `check_containers_required` is missing or stopped	create or start it; fix its name in the setting
`container X (owner O) is running but its healthcheck reports unhealthy` (critical)	its own healthcheck fails	`podman inspect X` shows the last healthcheck output (`State.Health.Log`); the container's log
`container X (owner O) stopped with an error: it exited with code N, 2.5 hours ago` (warning)	a container that is not meant to run all the time stopped with an error in the last `check_containers_recent_hours` (24) hours	its log. Older crashes are not reported
`container X (owner O) has restarted N times - a restart loop` (warning)	more restarts than `check_containers_restart_warn` (3), counted by podman or by its systemd unit	its log shows why it keeps stopping
`container X (owner O) was killed by the out-of-memory killer` (warning)	the kernel killed it for memory	more memory on the host, or a higher memory limit on the container
`user U (uid N) has containers that should run (...) but linger is off` (warning)	the user's containers stop when they log out and do not start at boot	`loginctl enable-linger U` (as root)

Finding	Means	Usually
`user U (uid N) has podman containers ... and N unit(s) that should start at boot, but they could not be checked: no /run/user/N` (warning)	the user has units that should start at boot (a Quadlet with `[Install]`, an enabled unit) but is not logged in and has no linger: none of their containers run now. A user with only container storage (an admin who once ran podman) is listed under `not checked:` and gets no finding	if they should run: `loginctl enable-linger U`. If not, leave them out: `podman_discover_ignore: ['U/.*']`
`the containers of O could not be read: podman ps failed ...` (warning)	podman (or systemctl) failed for that owner: a blind spot	run the `look:` command to see the error (storage problems, `podman system migrate` needed after an upgrade ...)

Settings (roles `podman_discover` and `check_containers`; VARIABLES_REFERENCE.md):

Setting	Default	What it does
`podman_discover_enabled`	`true`	`false` = do not look for containers (the check says `skipped`)
`podman_discover_rootless`	`true`	also look at users' containers
`podman_discover_users`	`[]`	more users to look at, e.g. `[appsvc]` when the rules above miss one
`podman_discover_unit_dirs`	`[/etc/systemd/system, /usr/local/lib/systemd/system]`	where root's own units that run podman are looked for (a user's: `~/.config/systemd/user`)
`podman_discover_ignore`	`[]`	containers to leave out: regular expressions that must match the **whole** name or `OWNER/NAME`, e.g. `['test-.*', 'alice/.*', 'root/scratch']`
`check_containers_required`	`[]`	containers that must exist and run: a name (any owner) or `OWNER/NAME`, e.g. `[stigman-api, alice/web]`
`check_containers_recent_hours`	`24`	how recent a crash must be to be reported
`check_containers_restart_warn`	`3`	restarts above this = a restart loop

To verify it on one host: run the Health check with `health_checks: containers` and a Limit of that host; compare the table with `sudo podman ps -a` and, for a user, `sudo runuser -u alice -- env XDG_RUNTIME_DIR=/run/user/$(id -u alice) podman ps -a`. Service watch uses the same way of finding containers (SERVICE_WATCH_DEMO.md).

All settings of this check, with their defaults: check_containers, podman_discover.

containers, database, service watch: which one does what

Three jobs look at podman containers. They find the containers the same way (`roles/podman_discover`), but each answers a different question:

	containers (Health check)	mariadb (Health check, or *Database health* alone)	Service watch
Asks	is every container alive and stable?	is the database *inside* working, and fast?	something is down: why, and what fixes it
Looks at	every container, pod and compose project (`podman ps -a`, Quadlet files, systemd units): down, unhealthy, crashed, restart loops, OOM kills, linger	connections, long queries, replication, Galera / Group Replication, its log, its disk, and performance (slow queries, lock waits, open transactions ...)	the containers that should run: down or unhealthy
Then	a finding (a ticket)	a finding (a ticket)	ACT's root cause and a recommended fix, which waits for approval (`watch_mode: approval`, the default)
Run it	daily, every host	daily with the Health check; hourly alone for the database team	from its AAP workflow, when you want a diagnosis and a fix to approve (on demand, or on a schedule for the hosts that matter)

One outage, one finding. When a database container is down and both checks run in the same Health check, the outage is reported once: as `containers:down` (or `:unhealthy`), whose summary then says *it is a MariaDB/MySQL database container, so the database is down too (mariadb:down is not reported separately)*. The containers check has the better finding for that: how the container is run and how to start it. The database check still reports `mariadb:down` when the containers check does not (it is not running in the same job, or it does not report that container - one that has no unit and no restart policy, so nothing says it should run), and the *Database health* template always reports it.

The same goes for the **services** check: root's systemd unit that runs a container (or a pod) that is down is reported by the containers check, and the services check's `unit-failed:UNIT` (or `unit-down` / `unit-flapping`) for that unit is left out and named in that finding (*the services check sees its systemd unit web.service fail too (unit-failed:web.service is not reported separately)*). A failed unit that runs no container stays the services check's.

Service watch and the Health check can both see the same container down: the watch acts on it (within minutes), the Health check records it (once a day). On hosts where Service watch runs, a daily `containers:down` for a watched container usually means the watch could not bring it back.

Troubleshoot (playbooks/troubleshoot.yml)

For "something is wrong on this host": pick the area and the job runs the commands an experienced admin runs first. Each is printed under a line saying what it shows, and the matching health checks run too. Read-only. Findings do not fail this job: you asked for a look, not a verdict.

Area	Commands (what they show)	Checks
overview	uptime, failed units, disk, memory, recent errors, recent reboots	disk, mounts, services, performance, time, logging
disk	space and inodes, biggest directories on the fullest filesystem, deleted-but-open files, files over 1 GiB, journal size	disk, mounts
performance	load, top, vmstat, memory users, disk latency (iostat), load history (sar), OOM kills	performance
service (`ts_service`)	its status, why it stopped (result, exit code, restarts), its log, dependencies, unit file, listening ports	services (plus that unit)
network (`ts_target` host:port)	addresses, routes, DNS servers, name lookup, TCP connect, route taken, firewall, connection counts	network (plus that target)
selinux	mode, today's denials, audit2why, changed booleans, local file labels	selinux
fapolicyd	status, today's denials, rules, enforcing?	fapolicyd
login	sshd log, faillock lockouts, failed logins, SSSD domain status, authselect, clock offset	accounts
time	chrony tracking and sources, timedatectl, configured servers	time
logs	errors, top error senders, rsyslog and its queue, journal size	logging
mariadb (or mysql)	(the MariaDB / MySQL health check)	mariadb

Add `use_act=true` and ACT reads everything collected and explains the root cause (ADDING_ACT.md).

All settings of this job, with their defaults: troubleshoot.

Certificate report (playbooks/cert_report.yml)

The certs check on every host, and **one table**, soonest expiry first: days left, date, host, subject, and where the certificate is. Hand it to whoever renews certificates. It fails the hosts with a certificate inside `check_certs_warn_days`, so a workflow can open tickets (`site_fail_on: []` keeps it green: a report only).

Emailed with `report_email_to` (Linux and Windows: `win_cert_report.yml`). In the email the table is coloured: **red** = expired, **amber** = expires within `check_certs_warn_days` (30), **blue** = within `check_certs_notice_days` (60), the rest plain with a green "ok". Windows: `win_check_certs_warn_days` / `win_check_certs_notice_days`.

Found by itself (the report only; `cert_report_discover: false` turns it off):

- **TLS ports:** every listening TCP port is asked once for its certificate, on the address it listens on, with a 3-second limit (`check_certs_probe_timeout`), at most 25 ports (`check_certs_discover_ports_max`). Ports that are plain text are never touched (`check_certs_discover_ports_skip`: 22, 25, 53, 80, 389, 3306, 5432 ...), nor the ones you listed in `check_certs_endpoints`. Shown as `TLS port 8443, java`

(https://127.0.0.1:8443).

- **Java keystores:** *.jks, *.keystore, *.p12, *.pfx in check_certs_discover_keystore_dirs (/etc, /opt, /srv, Tomcat's folders), and the ones running Java programs use (-Djavax.net.ssl.keyStore=...). A JKS is read without its password; a PKCS12 that needs one is listed under **Found but not checked** - no password is ever guessed. To check it, add it to check_certs_keystores with password_env and a *Keystore password* credential. The JDK's own trust stores (cacerts) are skipped.

The weekly health check (certs in health_checks) does not search by itself, so it opens no tickets for certificates nobody listed; set check_certs_discover_ports / _keystores: true on it when you want that.

Placeholders: postfix makes its own certificate when it is installed, /etc/pki/tls/certs/postfix.pem (subject O=SomeOrganization, OU=SomeOrganizationalUnit, C=--), self-signed for one year and never renewed. On every host with postfix it expires a year after the install. A found one is listed under **Found but not checked** (one row for all the hosts) and is never a finding, a ticket or a CSR, in the report and in the health check (check_certs_placeholder_subject). A real certificate put in its place (a host that relays mail) has a real subject, so it is checked and renewed. A ticket already open for a placeholder gets the usual "passes again" note on the next run (resolved with servicenow_resolve_cleared).

Left out: hosts whose certificates something else renews, such as Satellite and its Capsules (satellite-installer, never by hand). They are listed under **Found but not checked** and get no ticket. Name them in either setting, or both; each takes a list or one string with commas:

Setting	Leaves out	Example
cert_skip_hosts	hosts by name, as in the AAP inventory: any case, * and ? wildcards, a short name matches the full one	[sat01, '*capsule*'] or sat01, *capsule*
cert_skip_groups	every host in these AAP inventory groups (default [sat])	[sat, appliances]

Set them in the template's Variables, or in all.yml so the report and the renewal both get them. The health check still checks these hosts (with [sat]: the Satellite web UI's certificate, playbooks/group_vars/sat.yml).

Certificate renewal: the CSRs (playbooks/cert_renew_csr.yml)

For every certificate **file** the certificate report flags (it expires in 30 days or less, check_certs_warn_days, or has expired), the job makes a new private key and a certificate signing request (CSR) on the host. It attaches the CSR to the certificate's ServiceNow incident. The three use the same window: the report flags, the CSR is made and the incident is opened at 30 days. Change it on the workflow (check_certs_warn_days) and all three follow. The PKI person submits the CSR and attaches the issued certificate to the same ticket. Installing the new certificate is still done by a person: this job changes nothing a service uses.

Per certificate:

1. It reads the certificate file: the subject and the names (subjectAltName: DNS, IP, email, URI).
2. It makes a new key on the host and a CSR with exactly the same subject and names, signed with cert_renew_digest (sha256).
   - RSA keys are at least cert_renew_key_bits (2048); a bigger old key keeps its size.
   - An EC certificate gets a new key on the same curve.
   - It uses the openssl command, so it works on RHEL 8 (OpenSSL 1.1.1) and RHEL 9 (3.x), in FIPS mode.

3. It checks the CSR before keeping it: the CSR verifies, its key is the new key, and its subject and names are exactly the old certificate's. If not, it keeps nothing and says "make this CSR by hand".
4. It keeps the files in `cert_renew_dir` (`/etc/pki/tls/renew`, made 0700):
   - `<name>-<8 characters>.key` (0600);
   - the `.csr`;
   - a `.json` with where it came from, the old serial and expiry, and the AAP job.

The key never leaves the host and never appears in the job output.

5. It finds the certificate's open incident, or opens one. The incident is the one *Certificate watch* opened, found by the same correlation ID (workflow 7). It attaches the CSR (`<common name>-<8 characters>.csr`) and adds a work note: the certificate, where it is, who issued it, the names, where the new key is, and the next steps. Every CSR gets a ticket, whatever `servicenow_min_severity` says: critical (14 days or less, `check_certs_crit_days`, or expired) or warning (15 to 30 days).

Run again:

- The pending key and CSR are reused, so the CSR the PKI already has stays valid.
- A CSR already on its ticket is not attached again. A daily run adds nothing until the certificate changes.
- Once the new certificate is installed, the old one is no longer flagged.
- The folder keeps its files until you remove them.

One certificate in several places (its file, a chain file, served on a port) gets one CSR, and the work note lists every place. The watch opens a ticket for each place; the CSR goes on one of them, and each of the others gets a work note saying which ticket has the CSR.

A certificate served on a port is renewed through its file. The job finds the file when it is in `check_certs_files` or in one of the folders the report searches (`check_certs_discover_dirs`, two levels deep: `/etc/pki/tls/certs`, `/etc/pki/nginx`, `/etc/nginx`, `/etc/httpd/conf.d`, `/etc/stigman`, and `/etc/stigman/nginx/tls` where *STIG Manager - deploy* puts nginx's certificate). A file anywhere else: add its path to `check_certs_files`. To see what the job would do with a certificate that is not expiring yet, run it with **Check**, a **Limit** of that host and `check_certs_warn_days: 400`: it says "would make the CSR" or why not, and changes nothing.

Step 3 of the next steps says to install the issued certificate in place of the file. When something else deploys that file, a copy by hand is overwritten, so `cert_renew_install_notes` writes another step 3 for paths that start with a given folder. The default covers STIG Manager (`/etc/stigman/`): put the issued certificate and the new key in the AAP credential *TLS certificate*, then run *STIG Manager - deploy*. Add your own: `{'/opt/app/certs/': 'App: ...'}`.

Never renewed here (each listed in the job output with the reason):

What	Why
a host in `cert_skip_hosts` or `cert_skip_groups` (`[sat]`)	Satellite renews its own certificates with `satellite-installer`; left out of the certificate report too

What	Why
a path matching `cert_renew_exclude`	something else renews it: Satellite and Capsule (katello, foreman, candlepin, pulp, puppet), and subscription-manager on every registered host (`/etc/rhsm`, `/etc/pki/consumer`, `entitlement`, `product`). Check the report's *Where* column and add your own paths
issued by a CA matching `cert_renew_exclude_issuer` (`O=Katello`, `Puppet CA`)	Satellite's or Puppet's own CA
a CA certificate in a chain file	a CA has no CSR from you: put the CA's new certificate in the chain file
a self-signed certificate (unless `cert_renew_self_signed: true`)	it does not come from a PKI. With the setting on, it gets a CSR, so the PKI issues a real certificate in its place (a STIG wants PKI certificates); `cert_renew_issuer` does not apply to it
a CA's own certificate (a root: key usage *Certificate Sign*)	a CA has no CSR from you, even with `cert_renew_self_signed`
a self-signed certificate only for `localhost`	a PKI issues no certificate for it, even with `cert_renew_self_signed`
a package's placeholder: postfix's `postfix.pem` (subject `O=SomeOrganization`, `check_certs_placeholder_subject`)	not a real certificate: a PKI issues nothing for that subject, even with `cert_renew_self_signed`. A found one is not even checked
served on a port, with no file found	add its file to `check_certs_files`, and it is renewed
in a Java keystore or a PKCS12 file	make its CSR with the keystore's own tool (`keytool -certreq`)
not matching `cert_renew_only` / `cert_renew_issuer`	your filters, e.g. DoD PKI only: `cert_renew_issuer: 'O=U\.S\. Government'`

A certificate whose CSR cannot be made fails the job with the reason, for example a subject openssl cannot write back, or a kind of name it cannot copy. The other certificates are still done.

Earlier than 30 days: the CSR comes when the certificate report flags the certificate. For a longer PKI round trip, set `check_certs_warn_days: 60`, as a survey on the workflow, so the report, the tickets and the CSRs use the same number.

Attachment refused: some instances only allow some file extensions. The CSR then goes in the work note instead, and the job fails and says so. Allow the extension, or set `cert_renew_attach_ext: txt`. The ServiceNow account needs to add attachments to incidents (the `itil` role does).

Try it first:

- Run with **Check** (`--check`): it lists what it would make and attach, and writes nothing on the hosts or in ServiceNow.
- Without the ServiceNow API credential, the job prints each CSR so you can copy it by hand.

All settings of this job, with their defaults: check_certs (`cert_skip_hosts`, `cert_skip_groups`, `cert_renew_*`).

POA&M status (playbooks/poam_status.yml)

Reads `poam/poam.csv` (see poam/README.md for the columns and how to export from eMASS):

Finding	Means
`POA&M 1001 [CAT II]: ... - OVERDUE by 12 days`	open item past its scheduled completion date (critical)
`POA&M 1002 ...: due in 20 days`	due within `poam_due_soon_days` (30)
`... scheduled completion date "TBD" is missing or not a date`	fix the date in the list
`open CAT I finding V-123456 (...) on 2 asset(s) in collection Production has no POA&M item`	STIG Manager cross-check: an open CAT I/II finding that no POA&M item covers

It also lists "maybe ready to close": items whose STIG findings are no longer open in STIG Manager.

The STIG Manager cross-check is optional. It needs a Keycloak client that may use the client-credentials grant, with the scope `stig-manager:collection:read`, and a grant in the collections it reads. **Your realm owner creates that client**; nothing here changes the realm. Then create the *STIG Manager API* credential (token URL `https://<keycloak>/realms/<realm>/protocol/openid-connect/token`, client ID, secret), and set `poam_stigman_api: https://<stigman>/api`.

All settings of this job, with their defaults: poam.

ServiceNow tickets (playbooks/servicenow_tickets.yml)

A workflow step after a check. For each finding at or above `servicenow_min_severity`:

- **no open ticket** for that problem on that host › it opens an incident: short description `[site-health] host: summary`; description with the host, check, finding, the `look:` command, and ACT's analysis if ACT ran; urgency/impact from the severity; assignment group, category and configuration item from the settings.
- **an open ticket exists** › it adds a work note `Still failing at ...` (`servicenow_note_repeats`).
- **a host that was targeted but never reported** › a ticket too (not reached, login failed). The check job stays green for a host it could not reach (`site_unreachable_fails`); this ticket is where it shows.
- **a problem that cleared** (its check ran again and passed) › a work note on its ticket, or it resolves it with `servicenow_resolve_cleared: true`. A finding still there, but below `servicenow_min_severity`, has not cleared: its ticket stays open (a certificate renewal ticket at warning, for one).

Tickets are matched by a correlation ID (`site-health:<host>:<check>:<finding>`, hashed), never by text, so the same problem always updates the same ticket. Without the ServiceNow API credential it only prints what it would open. If no check results reach it at all (the check step died before checking any host), it fails, so a broken schedule shows red instead of green.

All settings of this job, with their defaults: servicenow.

ServiceNow - test ticket (playbooks/servicenow_test_ticket.yml)

Proves the ServiceNow setup end to end, with the same calls and settings as the tickets step. Six steps, each PASS, WARN or FAIL with what usually causes a failure:

Step	What it does	FAIL usually means
1 connection	logs in and reads one incident	certificate (CA), DNS, firewall or proxy; HTTP 401 = user or password; 403 = no `itil` role; 404 = a path after the host name in the Instance URL
2 settings	`servicenow_assignment_group` and `servicenow_caller` exist	the name differs from ServiceNow's
3 open	opens a low-priority incident: `[site-automation test] ... safe to close`	the account may not create incidents (`itil`), or ServiceNow refused a value (it says which)
4 read back	reads it back: number, state, group, caller; prints a link	the account may not read it; WARN if the group did not stick
5 work note	adds a work note	the account may not update incidents
6 resolve	resolves it (`servicenow_test_resolve: false` leaves it open)	WARN only: your instance needs another resolved state or close code (`servicenow_resolved_state`, `servicenow_close_code`)

As *Check* (a dry run) steps 1 and 2 run, and 3 to 6 are only printed. Set up, run and verify it: SERVICENOW_SETUP.md.

ServiceNow health (playbooks/servicenow_health.yml)

From the controller: does the instance answer its REST API, and how fast (`servicenow_slow_seconds`)? Is every MID Server **Up** and **validated**? Hourly is a good schedule. Alert with an AAP **notification** (email) on failure, not with a ticket: a broken ServiceNow cannot take tickets. The MID Server **hosts** (the Linux service, its keystore) are covered by the health check: see `inventories/example/group_vars/servicenow_mid_hosts.yml`.

All settings of this job, with their defaults: servicenow.

Patch hosts (playbooks/patch_hosts.yml)

Installs updates with dnf (`patch_security_only` for security fixes only; `patch_exclude` for packages never updated here). Then:

1. **One host at a time** (`patch_serial`). The rollout **stops at the first host that fails**.
2. Refuses with less than `patch_min_free_mb` free in `/var`.
3. **No automatic restarts** unless `automatic_restarts: true` (the survey or the template's Variables). Without it, a host that needs a reboot is listed ("reboot it by hand, or run again with automatic_restarts: true"). With it, the job reboots only when `needs-restarting -r` says so (`patch_reboot: when_needed`), and **never** hosts in `patch_never_reboot_groups` (`aap_hosts`).

4. After the update (and reboot), **every service that was running before must be running again**. If one is not, the host fails and the rollout stops.
5. Never patches hosts in `no_patch` (vendor appliances), `aap`, `aap_hosts`, `sat`, `netapp` or `netapp_console_hosts`. The AAP host is **always** protected: even a list you write yourself (`patch_never_patch_groups`) cannot remove `aap` or `aap_hosts`. It also refuses a host that is the machine running the job, but only when the connection is `local` or the name is `localhost`; a normal SSH host that happens to be the controller is stopped by the group rule above, not by this one.

Run it as **Job type: Check** first: it lists what would be updated and changes nothing (DRY_RUNS.md).

All settings of this job, with their defaults: patch.

Apply approved ACT fix (playbooks/act_fix_approved.yml)

The step after an approval in the *Fix with approval (ACT)* workflow. It runs **exactly** the commands ACT proposed and a person approved, then re-runs the checks that found the problem. It refuses to run on its own, and skips diagnose-only hosts (`aap_hosts`, `netapp_console_hosts`): there a person applies the fix by hand. See ADDING_ACT.md.

All settings of this job, with their defaults: site_act.

VMware jobs and reports (playbooks/vm_*.yml)

Restart, shut down, snapshot, notes, change VLAN; the secure boot, datastore, snapshot and alarms reports; the snapshot cleanup; ACT's analysis of the alarms. They work through vCenter: setup, every report's sections and troubleshooting are in VMWARE.md.

All settings of this job, with their defaults: vmware_vm.

Windows hosts

Windows has its own playbooks, checks and troubleshooting areas (for example `updates` and `security` in *Windows troubleshoot*). They are described, with their settings, in WINDOWS.md. Settings for them are read the same way as any other: VARIABLES.md.

Not built yet (and why)

Idea	Why not yet
SCAP scan › import into STIG Manager	Needs the same Keycloak client as the POA&M cross-check, plus a decision on SCC vs OpenSCAP content. STIG Manager's own importer (stigman-watcher) is the supported path
Certificate renewal workflow	Depends on how your PKI issues certificates. For STIG Manager, renewal already works: update the TLS credential, run *STIG Manager - deploy*
AAP self-health (job failure rate, execution environment and token age)	Needs an AAP API credential; AAP's own dashboards cover most of it today
Container runbooks (image updates with rollback, drift)	Deferred on purpose; *Service watch* covers container outages
MariaDB backups, security baseline, password rotation	Out of scope for now: health only, as asked

8. Adding ACT (GenAI) later

The runbooks work without ACT. Add it when the checks are running well and you want the next step done for you: **why** it broke and **what** fixes it. This page shows how the runbooks and ACT fit together, how to turn it on per template, and how to write your own check so it works with ACT automatically.

How they fit: checks find WHAT, ACT finds WHY

```
check roles (deterministic, cheap)          site_act (the bridge)             ACT (GenAI)
----------------------------------          ---------------------             -----------
disk, services, selinux, mariadb ...  -->  findings as one line each   -->  root cause, evidence,
each finding: summary + hint command        + the hint commands' output       the exact fix
                                            + your level (explain /           (runs, or proposes,
                                              diagnose / self-heal)            per the level)
```

- The **checks** decide *whether* something is wrong. They are plain Ansible: the same answer every time, and they are free. The model is only called for hosts with findings.
- Every finding carries a **hint**: the read-only command a person would run next. ACT gets those commands' output as evidence. A new check you write is therefore usable by ACT with no extra work.
- `roles/site_act` is the only place that talks to ACT. Every runbook uses it the same way. Turning ACT on is a variable, not a code change.
- Host names, IP addresses and user names are **pseudonymized** before anything reaches the model (ACT 0.6.18 and newer). Its replies are translated back on the host.

The three levels

Level	ACT does	Changes to hosts	Approval step?
`explain`	reads the evidence (the hint commands' output, collected by Ansible) and writes the root cause and a suggested fix. **It runs no commands itself**	none	no
`diagnose`	investigates by running **read-only** commands itself, then proposes exact fix commands. Every change is refused and waits for a person	only after approval	yes: the job fails with `NEEDS APPROVAL` › approval step › *Apply approved ACT fix*
`self-heal`	like `diagnose`, but may **run fixes that match** `site_act_allow` by itself (for example `systemctl restart chronyd`). Anything else still waits for approval. The checks then run again to prove the fix worked	only fixes you listed	only for the other fixes

Start with `explain`. Move a runbook to `diagnose` when ACT's explanations have been right for a while. Use `self-heal` only for fixes you would let a junior admin do unasked: start a stopped service, restart a hung one.

Never fixed automatically, at any level: hosts in `site_act_diagnose_only_groups` (default: `aap`, `aap_hosts`, `sat`, `idm`, `netapp`, `netapp_console_hosts`; `aap` and `aap_hosts` are always kept, whatever list you

write). There `self-heal` becomes `diagnose`, and the apply step skips them: it prints the approved command for a person to run by hand.

Turn it on

1. The ACT model key credential

Create the credential type `ACT model key` (`aap/credential_types/act_model_key.yml`) and one credential with your key: the GenAI key for GenAI.mil, or the AskSage key for Ask Sage (SETUP_AAP.md, steps 2-3). For the GenAI beta proxy, also create `ACT GenAI beta key` (`act_genai_beta_key.yml`). Attach the credential(s) to the **Health check** and **Troubleshoot** templates (and *Service watch - check*), next to `Linux ssh (sudo)`. This is the only place a key lives. The apply steps do not call the model: they run the approved commands themselves, so they need no key.

2. Pick the model provider

ACT can talk to three providers. Pick one with `site_act_provider`:

site_act_provider	Provider	Key (credential)	Also set
`genai` (default)	GenAI.mil	*ACT model key*, **GenAI API key** field	nothing (optional: `site_act_model`)
`asksage`	Ask Sage	*ACT model key*, **AskSage API key** field	`site_act_url`: your organization's Ask Sage API URL. The default is `https://api.genai.army.mil/server/openai/v1/chat/completions`
`genai-beta`	GenAI.mil beta proxy (preview models)	*ACT GenAI beta key* (`aap/credential_types/act_genai_beta_key.yml`)	nothing

The settings:

Setting	What	Default
`site_act_provider`	`genai`, `asksage` or `genai-beta`	`genai`
`site_act_models`	the model **for each provider**, so the model follows the provider you pick, e.g. `{genai: gemini-3.8-flash, asksage: gpt-5.6-sol-gov}`	ACT's default for the provider
`site_act_urls`	the API URL for each provider, e.g. `{asksage: https://<host>/server/openai/v1/chat/completions}`	ACT's default for the provider
`site_act_model`, `site_act_url`	one model / URL for this run, whatever the provider (a survey answer)	empty: use the two above
`site_act_ca`	a CA bundle file **on the hosts**, if the provider's certificate comes from a CA the hosts do not trust yet (for example a DoD CA not in `/etc/pki/ca-trust`)	the host's normal trust store

Where to set them:

- **For everyone**: `playbooks/group_vars/all.yml` in this project (the file has these lines ready as comments). A setting there beats the *inventory's* Variables box in AAP, so put it in one of the two places,

not both. Which place wins in every case: VARIABLES.md.

```yaml
site_act_provider: genai                # the default provider
site_act_models:                        # the model follows the provider you pick
  genai: gemini-3.8-flash
  asksage: gpt-5.6-sol-gov
site_act_urls:                          # only if your Ask Sage is not ACT's default URL
  asksage: https://<your ask sage host>/server/openai/v1/chat/completions
```

To make Ask Sage the default, change the first line to `site_act_provider: asksage`.

- **For one group**: the group's settings file (`playbooks/group_vars/<group>.yml`), or the group's **Variables** box in AAP. For example `site_act_provider: asksage` there, and only that group's hosts use Ask Sage (with its model from `site_act_models`).
- **For one template**: its **Variables** box (*Extra variables*), for example `site_act_provider: asksage`. It beats everything above, for every run of that template.
- **Per launch**: the survey questions in step 3 (a survey answer beats everything too).

The API **key** is never a variable: it comes from the *ACT model key* credential on the template.

These settings apply to every runbook that uses ACT, including *Service watch* and *Apply approved ACT fix*. If the key for the chosen provider is missing, ACT does not run: the job says `ACT did not run: this job has no API key for provider asksage. Attach ...`, and the findings are still reported.

Network: ACT runs **on each host**, not in AAP. So each host must be able to reach the provider's URL over HTTPS. If they go through a proxy, add it: `site_act_env: {HTTPS_PROXY: "http://proxy.yoursite.mil:8080"}`.

Models served only on `/v1/messages` (ACT 0.6.19 and newer). Some gateways offer a model only through the Anthropic endpoint and answer HTTP 400 for it on `.../v1/chat/completions`. ACT tries the other endpoint by itself, with nothing to set. To test your models and set the switches on purpose, follow Both endpoint formats and `:probe` below.

How long ACT may take. `site_act_model_timeout` (default **600** seconds) is the time the model may take for one answer, retries after a provider error included (ACT's `GENAI_TIMEOUT`; ACT's own default, 90, is too short for a long analysis or a busy provider). `site_act_timeout` (default **1200**) is the whole ACT run, several answers. They apply to every ACT job: the runbooks with `use_act`, the VMware alarms and capacity analyses, *Service watch*. A `model turn exceeded the 600s total timeout after a transient provider failure` means the provider was failing and retrying for that long: run it again; if it repeats, raise both in `all.yml` (`site_act_model_timeout: 900`, `site_act_timeout: 1800`).

Streaming. Every ACT job has the model stream its answer (`ACT_STREAM=1`): data keeps flowing while it writes, so a firewall or proxy that cuts quiet connections (`network error reaching API: connection reset by peer`, often on the longer answers only) no longer does. Where a gateway cannot stream, ACT notices and sends normal requests by itself. To turn it off: `site_act_env: {ACT_STREAM: "0"}`.

"ACT gave no analysis: the model answered as a chat". Now and then a model loses the thread and answers like a chat window ("I am ready. What task or command would you like me to assist you with?") instead of analysing the findings; whatever it tried to run then is no fix. The report says so (ACT status `no_analysis`) and nothing is passed on for approval. Run the job again; if it repeats with one model, switch that template to another (`site_act_models`).

Anything else ACT should know about your site goes in the same places (for example `playbooks/group_vars/all.yml`):

```yaml
site_act_extra_instructions: >-
  RHEL 9 servers under DISA STIG. fapolicyd and SELinux are enforcing. Never suggest disabling
    them.
```

All settings, with their defaults: site_act.

3. Survey questions

On **Health check** and **Troubleshoot** › **Survey** › **Create survey question**:

Question	Variable	Type	Choices	Default
Ask ACT (GenAI) about the findings?	`use_act`	Multiple Choice (single select)	`no`, `yes`	`no`
What may ACT do?	`site_act_level`	Multiple Choice (single select)	`explain`, `diagnose`, `self-heal`	`explain`
Which model provider? (optional)	`site_act_provider`	Multiple Choice (single select)	`genai`, `asksage`, `genai-beta`	the one in your settings
Which model? (optional, blank = the provider's model from `site_act_models`)	`site_act_model`	Text, not required		blank

Leave out the last two if everyone uses the same provider. Your setting (section 2) then applies.

4. Try explain by hand

Launch **Troubleshoot** on a host with a known problem: `ts_area` = `service`, `ts_service` = the unit, `use_act` = `yes`, `site_act_level` = `explain`. In the output, read `ACT | what ACT says`. With ACT on, the host's ServiceNow ticket includes this analysis too.

5. diagnose with the approval workflow

Build *Fix with approval (ACT)* (WORKFLOWS_AND_SCHEDULES.md). What happens:

1. The health check finds, for example, `required service chronyd.service is inactive`.
2. ACT (`diagnose`) looks at the unit, its log and the config, and proposes `systemctl start chronyd`. The command is refused, so the job fails: `NEEDS APPROVAL - ... ACT proposes: systemctl start chronyd`.
3. The workflow stops at the approval step. The approver reads ACT's report and approves.
4. **Apply approved ACT fix** runs **exactly** `systemctl start chronyd` itself: no model and no key, so nothing can be reworded between the approval and the fix. A last check refuses a few commands even after approval (reboot, `mkfs`, `usermod` ...; see APPROVED_COMMANDS.md).
5. The same checks run again, without ACT, and the job is green only if the problem is gone. The tickets step notes the ticket as cleared.

6. self-heal for the fixes you trust

playbooks/group_vars/all.yml (or the settings file of one group)

```
site_act_allow:
  - 'systemctl (start|restart) (chronyd|rsyslog|crond)'
```

Each entry is a regular expression that must match the **whole** command. Keep them narrow: name the services, never `.*`. Then pick `site_act_level` = `self-heal`. After ACT applies an allowed fix, the checks with findings run again. The report shows `cleared: ...` or `still there: ...` and reflects the state **after** the fix.

Both endpoint formats and :probe (ACT 0.6.19 and newer)

What this is for. A model gateway can offer a model on two web addresses: the *OpenAI* one (the provider URL, ending in `/v1/chat/completions`) and the *Anthropic* one (ending in `/v1/messages`). Some models answer on only one of them. If ACT prints `HTTP 400 ... bad request` for a model that the gateway lists, this is the usual cause. ACT can now speak both, and `:probe` tests which one each model needs.

Two facts that shape the steps:

- `:probe` is a command you type at the ACT prompt. **It cannot run inside an AAP job** (jobs are not interactive). So you run it **once, by hand, on one lab host**, and then put the result into settings the jobs read.
- In a job, ACT already works out the format by itself (one refused request, then it switches), so the jobs may need no change at all. The settings below only save that one wasted request per run, or point at an Anthropic address that is not the default.

Not verified against a real gateway. These steps were written from ACT's own documentation and tests. Nobody has run them against your gateway yet; treat the first run as the test.

Step 1: check which ACT the project carries

`vendor/act/VERSION` must say `0.6.19` or newer. This release carries **0.6.24** (0.6.24 changes only ACT's own example playbook `podman_app_health.yml`: see CHANGELOG.md). It sends the key only to an `https://` provider URL (an `http://` URL is refused unless `site_act_env: {ACT_ALLOW_HTTP: "1"}`), and more dangerous commands always need a person (see APPROVED_COMMANDS.md). New in 0.6.22 and 0.6.23, all automatic:

- **A bigger output limit for thinking models (0.6.23).** Gemini 2.5 and later, GPT-5 and the o-series think before they answer, and the thinking counts against the output limit. At ACT's old limit (4096 tokens) the thinking could use it all and the answer came back **empty** - on GenAI.mil every Gemini model did that in `:probe`. These models now get 16384 (others keep 4096). It is a ceiling, not a charge: you pay for the tokens used. To force one value: `site_act_env: {ACT_MAX_TOKENS: "8000"}`.
- **Temperature per model.** Gemini 3 models (and GPT-5 / o-series reasoning models) get **no** temperature, so they use their own default (1.0): Google's Gemini 3 developer guide says to keep it at 1.0 and that lower values can make the model loop. Other models keep 0.2. To force a value for every model: `site_act_env: {ACT_TEMPERATURE: "0.2"}`.
- **Fewer empty answers.** A reply cut off at the output limit (the model's thinking used it all) is asked again once with a higher limit; an empty reply is asked again once with a strict JSON schema of ACT's actions; a reply the gateway's content filter blocked is reported as that.
- **Structured output.** When a model refuses ACT's function-calling format, ACT asks for a strict JSON schema instead of "any JSON object", so the gateway itself guarantees a valid answer.

- **HTTP 429 (too many requests):** ACT waits as long as the gateway asks (`Retry-After`). A spent credit quota still stops the run (it would not help to wait).
- **Clearer errors.** A model alias the gateway retired (GenAI.mil retires aliases 60 days after deprecation) is reported as that, not as a missing endpoint or a permission problem; a locked or wrong key says to enter a new one.
- **Interactive only:** replies stream in (a counter shows progress) and **Esc** cancels a reply that is still coming. Jobs do not stream (nothing to watch, and some proxies break streaming).
- **Tool-result turns** (command results sent back the way function-calling models expect): only for models `:probe` confirmed - see step 5. If it is older, update it first: README section

"Updating ACT" (`scripts/update-act.sh`).

Step 2: put ACT on one lab Linux host

Any host that can reach the gateway will do; it does not have to be one of your managed hosts. Copy the file `vendor/act/act` there (WinSCP, or `scp` in PowerShell), then on the host:

```shell
chmod 700 act
```

Step 3: give it your key without writing the key anywhere

Type this on the host. It asks for the key without showing it, and keeps it only in that terminal session (use `GENAI_BETA_KEY` or `ASKSAGE_KEY` for the other providers):

```shell
read -rs GENAI_KEY; export GENAI_KEY
```

Paste the key, press Enter, and do not `echo` it. If the provider URL is not ACT's default, also `export GENAI_URL='https://<your gateway>/v1/chat/completions'`.

Step 4: run :probe

```shell
./act
```

At the ACT prompt:

```
:probe all
```

`:probe all` tries every model the key lists, on both endpoints, with a small request and then ACT's full request. For each it shows what worked and, for a refusal, **the gateway's own reason**. `:probe <model name>` tests just one; `:probe model1 model2` (spaces or commas) tests several. Write down, for each model, which endpoint worked (`openai` or `anthropic`). Leave ACT with an empty line (or Ctrl-D). From 0.6.23 `full OK` means the model gave a usable answer; `full empty (...)` or `full no action (...)` says what came back instead (for example `finish_reason=length`: the model ran out of output room), and a test that needed more room says `OK (needed a higher output limit: N)` - ACT remembers that limit for the model.

From 0.6.22 each model also gets these lines (under its `basic` / `full` lines):

Line	Meaning	What to do
`stream OK` / `stream not supported (...)`	whether replies can stream in (interactive use only)	nothing
`structured output OK (strict)` / `OK (non-strict)` / `not supported (...)`	whether the gateway enforces ACT's JSON schema for this model	nothing: ACT uses the best one that works
`tool results OK` / `tool results not supported (...)`	whether this model accepts command results as tool turns	**write it down**: step 5
`temperature: model default` / `temperature: 0.2`	what ACT sends	nothing (to force a value: `ACT_TEMPERATURE`)
`output limit 16384 (thinking model)` / `output limit 4096`	the most the model may write per answer, thinking included (0.6.23)	nothing
`full empty (finish_reason=length ...)`	even the higher limit was not enough for that test	tell the maintainer; or `ACT_MAX_TOKENS` higher

The result is remembered for the session. If a setup file already exists for your user on that host (`~/.config/act/config.json`), ACT also saves the format there (only the format table, never the key).

Windows. ACT for Windows (`vendor/act-windows/act.ps1`) has the same `:probe` command. Run it the same way on a lab Windows host, in PowerShell, and enter the key with `:setup`. Its result goes into the Windows jobs' settings as above. (Not run by hand for this page.)

Step 5: tell the jobs

A job does not read the lab host's file (it runs as another account, on another host), so put the result in settings, in the same places as the other ACT settings (section 2):

Your probe result	Setting in playbooks/group_vars/all.yml (or a group file)
All the models you use work on the OpenAI endpoint	nothing to set
Some models work only on the Anthropic endpoint	nothing to set (ACT learns it during the run); optional, to skip the one refused request per run when **every** model you use is Anthropic-only: `site_act_env: {ACT_API_FORMAT: anthropic}`
The Anthropic endpoint is at a different address than the OpenAI one with `/messages` in place of `/chat/completions`	`site_act_env: {GENAI_ANTHROPIC_URL: "https://<host>/.../v1/messages"}` (`ASKSAGE_ANTHROPIC_URL` or `GENAI_BETA_ANTHROPIC_URL` for the other providers)
A model you want is listed for the key but works on neither	it is not available to your key; ask the gateway's owner. Choose another model in `site_act_models`
`tool results OK` for the model your jobs use (0.6.22)	optional: `site_act_env: {ACT_TOOL_RESULTS: tool}`. Jobs then send command results as tool turns, which function-calling models handle best on long runs. If a model refuses them anyway, ACT switches that model back by itself
`tool results not supported`	nothing to set (the default sends results as ordinary messages, as before)

`ACT_API_FORMAT: anthropic` applies to **every** model ACT uses in the job. If you use both kinds, leave it out.

Many hosts at once: site_act_concurrency

Every host in one job uses the same key, and GenAI.mil's default quota is **60 requests and 200,000 tokens a minute** per key (and a daily credit budget). A Health check with `use_act` on 20 hosts with findings would otherwise run ACT on as many hosts at once as the job's forks allow, and the gateway answers HTTP 429. `site_act_concurrency` (default **3**) caps how many hosts run ACT at the same time; the rest wait their turn. `0` = no cap. ACT itself also waits when the gateway sends a 429 with `Retry-After`.

How to verify

1. **The probe result:** in step 4 every model you plan to use shows one endpoint that worked.
2. **A job:** launch **Troubleshoot** with `use_act` = `yes`, `site_act_level` = `explain`, and the model in `site_act_model` (or set through `site_act_models`) on a host with a known finding. In the output, task `ACT | what ACT says` holds an analysis, not an `HTTP 400` message.
3. **The old symptom is gone:** search the job output for `400`. A line saying ACT switched to the other endpoint is fine; a final `HTTP 400` for the model is not (then the probe said neither endpoint works: step 5, last row).

If a step fails, the message after `HTTP 400` is the gateway's reason: copy it exactly when you ask its owner for help, but remove the key and host names first.

ACT for vCenter alarms (it runs on the AAP side)

`VM - alarms ACT analysis` (VMWARE.md, "Alarms") uses ACT differently from the health checks: ACT runs **inside the job's execution environment on the AAP node**, not on a managed host, and only reads the evidence the job collected from vCenter (it runs no command). So the ESXi hosts need nothing; the **AAP node** needs HTTPS to the model's URL, and the job checks that first. It uses the same settings as everything else here: `site_act_provider`, `site_act_models`, `site_act_url`, `site_act_env` (a proxy), `site_act_ca` (a CA file **in the execution environment**, e.g. one from the project: `"{{ playbook_dir }}/files/ca/model-ca.pem"`), and the **ACT model key** credential on its template.

Write your own check (it works with the report, tickets and ACT automatically)

A check is a role named `check_<name>` that **adds findings** to `site_findings`. That is the whole contract. As an example, here is a check that `/tmp` is not world-writable without the sticky bit.

1. Create `roles/check_tmp/defaults/main.yml`, with its settings and a comment saying what it checks:

```yaml
---
# TMP: /tmp and /var/tmp must have the sticky bit (mode 1777).
check_tmp_paths: [/tmp, /var/tmp]
```

2. Create `roles/check_tmp/tasks/main.yml`. Collect with read-only commands, and always use `changed_when: false` and `check_mode: false`: read commands then run in a dry run too. Turn what you saw into findings, and add them:

```yaml
---
- name: tmp | modes
  ansible.builtin.command:
    argv: "{{ ['stat', '-c', '%a %n'] + check_tmp_paths }}"
  register: _tmp
  changed_when: false
  failed_when: false
  check_mode: false

- name: tmp | findings
  ansible.builtin.set_fact:
    _tmp_findings: >-
      {%- set f = [] -%}
      {%- for line in _tmp.stdout_lines if line.split()[0] != '1777' -%}
      {%-   set path = line.split()[1] -%}
      {%-   set _ = f.append({'check': 'tmp', 'id': 'tmp:' ~ path, 'severity': 'warning',
                              'summary': path ~ ' has mode ' ~ line.split()[0] ~ ' (expected
                                1777)',
                              'hint': 'stat ' ~ path ~ '; findmnt ' ~ path}) -%}
      {%- endfor -%}
      {{ f }}

- name: tmp | add them to this host's findings
  ansible.builtin.set_fact:
    site_findings: "{{ site_findings | default([]) + _tmp_findings }}"
```

3. Let the health check know it: in `playbooks/health_check.yml`, add `tmp` to `_health_sets` (to `weekly` and `all`), and add `tmp` to the survey choices of the Health check template.
4. Test it on one host: `ansible-playbook playbooks/health_check.yml -l <host> -e health_checks=tmp`. Then break the condition on purpose (`chmod 777 /tmp` on a test host) and check that the finding appears. A check that never fires proves nothing.

That is all. The report prints it, set_stats publishes it, ServiceNow tickets it (the `id` keeps it on one ticket), and with `use_act=yes` ACT receives the summary and runs the `hint` as evidence.

Rules for a good finding:

- `summary`: one line a person understands, with the number and the limit (`/var is 96% full (critical at 95%)`).
- `id`: stable across runs, with no numbers that change (`disk:/var`, not `disk:/var:96`). Otherwise every run opens a new ticket.
- `hint`: **read-only**, runs in under a minute (`timeout 60 ...`), and shows the cause, not just the symptom. It runs as root when ACT is on (Ansible runs it in `explain` mode and hands ACT the output), so never put a command that changes something there.
- `severity`: `critical` = act now (outage, security); `warning` = act soon.

Use ACT in a playbook of your own

Any playbook that fills `site_findings` can hand them to ACT the same way:

```yaml
- name: My check
  hosts: my_hosts
  become: true
  gather_facts: false
  tasks:
    - ansible.builtin.include_role: {name: site_findings, tasks_from: start.yml}
    - ansible.builtin.include_role: {name: site_findings, tasks_from: run_check.yml}
      vars: {_site_check: tmp}                              # runs roles/check_tmp
    - ansible.builtin.include_role: {name: site_act}
      when: use_act | default(false) | bool and site_findings | length > 0
    - ansible.builtin.include_role: {name: site_findings, tasks_from: report.yml}
```

`report.yml` goes last. It prints, publishes (for tickets and the approval workflow), and fails the host when there are findings or a fix waits for approval.

What ACT is told, and what it cannot do

- Its task: the findings (one line each), "find WHY, confirm it with evidence, give the exact fix; several findings may share one cause", plus your `site_act_extra_instructions`.
- `explain`: it runs nothing. Ansible ran the hint commands.
- `diagnose`: only commands ACT can prove are read-only run on their own. Everything else is refused and becomes a proposal.
- `self-heal`: in addition, only commands matching `site_act_allow`.
- Apply step: no model at all. The playbook runs only the approved commands, character for character, in order, and stops at the first one that fails. Run as *Check*, it only shows what it would run (a dry run: see WORKFLOWS_AND_SCHEDULES.md, "Rehearse it").
- Every run is recorded in the AAP job output. The ticket carries ACT's analysis. For the service-watch style incident log, see SERVICE_WATCH_DEMO.md.

9. Windows servers

The Windows runbooks work the way the Linux ones do. Checks find what is wrong and say it in plain words, with a command to look further. The same report, the same ServiceNow tickets and POA&M, and the same workflows and schedules follow. Only the checks themselves are different: each one is a PowerShell script that runs on the server over WinRM.

Nothing extra to install on AAP or on the servers. The playbooks use only Ansible's built-in `script` module, so no Ansible collection is needed. The execution environment needs the Python library for WinRM (`pywinrm`); step 2 shows how to check. The servers need WinRM, which you already use: over HTTPS (5986), or over HTTP (5985) with Kerberos or NTLM encrypting each message (step 1). The check scripts run in Windows PowerShell 5.1 and PowerShell 7, and also in Constrained Language Mode (see step 5).

Template	Playbook	What it does
Windows connection test	`playbooks/win_connection_test.yml`	run it first: can AAP reach the servers, with the right account?
Windows health check	`playbooks/win_health_check.yml`	the checks below; daily and weekly sets
Windows troubleshoot	`playbooks/win_troubleshoot.yml`	on demand: an admin's first PowerShell commands for one kind of problem
Windows certificate report	`playbooks/win_cert_report.yml`	every certificate on every server, soonest expiry first
Windows patch	`playbooks/win_patch.yml`	install what Software Center offers (ConfigMgr); restart only with `automatic_restarts: yes`; prove the server is back
ACT for Windows - install	`playbooks/win_act_install.yml`	put ACT on the servers in `C:\ProgramData\act`, locked down and unblocked; keep it current
ServiceNow tickets, POA&M status	the ones you have	unchanged: they take the Windows results the same way

The checks (`roles/win_check_<name>`):

Set	Check	Finds
daily	`disk`	volumes too full
daily	`services`	required services not running; automatic services that stopped with an error; services that keep crashing
daily	`performance`	CPU, memory or page file too busy; names the busiest processes
daily	`time`	Windows Time not running, not synchronized, synced too long ago, clock off
daily	`network`	no gateway or DNS; names that do not resolve; ports that do not answer; a broken domain trust

Set	Check	Finds
daily	`eventlog`	unexpected shutdowns, crashes, disk errors, a cleared Security log, many errors, full logs
weekly	`security`	Defender real-time protection off or old signatures; endpoint services down; firewall off; SMBv1; RDP without NLA; UAC off; TLS 1.0/1.1
weekly	`audit`	audit policy settings the STIG asks for; event log sizes; the log forwarding agent
weekly	`accounts`	Guest enabled; local passwords old or never expiring; inactive local accounts; unexpected local Administrators
weekly	`certs`	certificates that expire or expired, including the WinRM listener AAP connects through
weekly	`patching`	a restart pending for installed updates; days since the last update

Everything is read-only. The one write is one Application event log entry per finding (source `site-health`), so a SIEM that collects event logs sees the findings too.

Step 1. WinRM on the servers

You already use it. To see it on a server:

```
winrm enumerate winrm/config/listener
```

- **A listener with** `Transport = HTTPS`, `Port = 5986` and a `CertificateThumbprint`: the connection is TLS. The `certs` check warns before that listener's certificate expires: when it expires, AAP can no longer connect.
- **Only** `Transport = HTTP`, `Port = 5985` (often set by a GPO): that works too. Kerberos and NTLM encrypt every message themselves; `ansible_winrm_message_encryption: always` (step 4) makes the connection fail rather than ever send in clear. Never use `basic` sign-in over HTTP.

The firewall must let that port in from the AAP execution nodes.

Step 2. The execution environment

The job runs inside an execution environment (EE), and that EE needs these Python libraries:

- `pywinrm`, always;
- `requests-ntlm`, for NTLM (the usual choice);
- or `requests-kerberos` plus a Kerberos client (`kinit`), for Kerberos.

AAP's *Default execution environment* has them. Your own EE (for example `platforms-ee`) may not. **The Windows connection test tells you**: its first part runs inside the EE and prints `YES` or `no` for each library. If `pywinrm` is missing, you have two options:

- use AAP's Default execution environment for the Windows templates; or
- add the libraries to your EE's definition and rebuild it:

```yaml
dependencies:
  python:
    - pywinrm
    - requests-ntlm
```

Step 3. The credential

Automation Execution › Infrastructure › Credentials › Create credential:

- **Name**: `Windows (WinRM)`; **Credential type**: *Machine*.
- **Username**: `YOURDOMAIN\svc_aap` for NTLM. For Kerberos, use `svc_aap@YOURDOMAIN.MIL`, with the domain in capitals. On a **FIPS** system (STIG) NTLM does not work at all - use Kerberos (see *If something goes wrong*).
- **Password**: the account's password.
- **Privilege escalation**: leave it empty. An administrator's WinRM session is already elevated.

The account must be in the **local Administrators group** on every server, usually through a domain group added by GPO. The checks need that to read the Security log, the audit policy, local accounts and certificate stores.

Step 4. The Windows inventory

Keep the Windows servers in their **own inventory**. Go to **Inventories › Create inventory**, name it `Windows servers`, and paste this in its **Variables** box:

```yaml
ansible_connection: winrm
ansible_port: 5986
ansible_winrm_scheme: https
ansible_winrm_transport: ntlm                  # or kerberos
ansible_winrm_server_cert_validation: validate  # needs your CA bundle in the EE, see below
ansible_winrm_ca_trust_path: /etc/pki/ca-trust/source/anchors/your-ca.pem   # path inside the EE
```

Servers with **only an HTTP listener (5985)**, step 1, take these lines instead (no certificate lines: Kerberos or NTLM encrypts the messages):

```yaml
ansible_connection: winrm
ansible_port: 5985
ansible_winrm_scheme: http
ansible_winrm_transport: kerberos              # or ntlm (not on a FIPS system)
ansible_winrm_message_encryption: always       # encrypted by Kerberos / NTLM, or no connection
```

- **Certificate validation** (HTTPS). `validate` means the EE checks the server's WinRM certificate against the CAs it trusts. EEs usually do not include your organisation's root CAs, so `validate` fails with `CERTIFICATE_VERIFY_FAILED` until you add your CA bundle to the EE and point `ansible_winrm_ca_trust_path` at it (the path inside the EE, as shown above). **Use this setting for real servers.** `ignore` keeps the traffic encrypted but does not check who the server is, so anyone who can intercept the connection could pose as it: use it **only** for a short test on a lab network, then switch back.
- **Hosts.** Add them by full name (`web01.yoursite.mil`); Kerberos needs the full name.

- **Groups.** Use whatever helps: by site, by role (`win_dc`, `win_web`...). The settings below can be set per group. `inventories/example-windows/hosts.yml` shows an example.

Linux and Windows do not mix: the Linux templates skip Windows servers, and the Windows templates skip Linux servers. So a template pointed at the wrong inventory changes nothing and opens no tickets.

Step 5. Test the connection

Create a job template, then launch it with **Limit** set to one server:

Field	Value
Name	`Windows connection test`
Inventory	`Windows servers`
Project	`site-automation`
Playbook	`playbooks/win_connection_test.yml`
Execution environment	one with `pywinrm` (step 2)
Credentials	`Windows (WinRM)`
Limit	empty, **Prompt on launch** ticked

A good result ends with:

```
Connected: WEB01 (Microsoft Windows Server 2022 Standard, build 20348), domain yoursite.mil
PowerShell 5.1.20348.2849, FullLanguage
Logged on as yoursite\svc_aap; local administrator: yes
OK: the Windows runbooks can run here.
```

`ConstrainedLanguage` instead of `FullLanguage` means WDAC or AppLocker restricts PowerShell on that server. The check scripts are written and tested for that mode; only two optional parts are skipped in it (TLS endpoints and the search for missing updates). Whether Ansible itself can run on such a server depends on your ansible-core version and the policy: if this test got through and printed its result, it can.

Step 6. The job templates

For all of these, use: **Inventory** `Windows servers`, **Project** `site-automation`, the EE from step 2, **Credentials** `Windows (WinRM)`, **Limit** empty with **Prompt on launch** ticked. Leave **Variables** empty: with no `target`, they run on every server in the inventory.

Windows health check

Field	Value
Playbook	`playbooks/win_health_check.yml`
Survey: `health_checks`	*Multiple Choice (multiple select)*: `daily`, `weekly`, `all`, `disk`, `services`, `performance`, `time`, `network`, `eventlog`, `security`, `audit`, `accounts`, `certs`, `patching`. Default `daily`

Windows troubleshoot

Field	Value
Playbook	`playbooks/win_troubleshoot.yml`
Survey: `ts_area`	`overview`, `disk`, `performance`, `service`, `network`, `login`, `time`, `logs`, `updates`, `security`. Default `overview`
Survey: `ts_service`	*Text*, not required. "Service name (for area = service), e.g. W3SVC"
Survey: `ts_target`	*Text*, not required. "host:port it cannot reach (for area = network)"
Survey: `ts_hours`	*Integer*, default 2. "How many hours of event logs to read"

Each command's output appears under a `### what it shows` line, followed by the matching health checks. Findings do not fail this job.

All settings of this job, with their defaults: win_troubleshoot.

Windows certificate report

Field	Value
Playbook	`playbooks/win_cert_report.yml`
Survey	none

Active Directory health report

For the domain controllers (group `win_dc`): one report for the whole domain - replication, dcdiag, FSMO, SYSVOL, DNS, time, and what changed in the last day. It needs an account with administrator rights on the DCs, in a credential of its own: ACTIVE_DIRECTORY.md.

Field	Value
Playbook	`playbooks/ad_health_report.yml`
Credentials	`Windows DCs (WinRM)`
Variables	`report_email_to: ad-team@yoursite.mil`

Step 7. Workflows and schedules

These are the same as for Linux, with the Windows templates. The full list, with every step and link, is in WORKFLOWS_AND_SCHEDULES.md (workflows 12 to 17):

Workflow	Steps	Schedule
Windows daily health	Windows health check (`daily`) › ServiceNow tickets (**Run always**)	every day 06:15
Windows weekly compliance	Windows health check (`weekly`) › tickets; Windows certificate report › its own tickets box (both **Run always**)	Monday 07:15

Workflow	Steps	Schedule
Windows patch with a dry run first	Windows health check (`disk`, `services`) › Windows patch as **Check** › *Approval* › Windows patch › Windows health check (`daily`) › tickets	monthly, inside the servers' ConfigMgr maintenance window

The ServiceNow tickets template runs on AAP itself, so it stays as it is.

Start small: the pilot order

Everything Windows here is new, so try it on **one** server that does not matter much, in this order, and widen only when each step looks right:

1. **Windows connection test**: the account, WinRM and PowerShell work.
2. **Windows health check**: the findings make sense for that server.
3. **Windows patch as Check** (the dry run): it lists what Software Center would install.
4. **Windows patch for real**, inside that server's ConfigMgr maintenance window.
5. **ACT for Windows - install** on the same server (tell your security team first, see below).

Every step can be a dry run first: DRY_RUNS.md.

Patching through Software Center

`playbooks/win_patch.yml` does, on each server, what an admin does in Software Center: install every software update your ConfigMgr admins deployed to it ("Install all"), required and available alike. AAP decides nothing about *which* updates: ConfigMgr does, as it does today. Applications listed in Software Center are not installed; only software updates are. Then:

1. **No automatic restarts** unless you ask for them with `automatic_restarts: yes` (the survey, or the template's Variables). Without it, the job installs the updates and lists the servers that need a restart: `A RESTART IS NEEDED to finish the updates ... automatic restarts are off`. A person restarts them (the post-check's `patching` finding keeps them visible). With it, the job restarts a server only when the updates or the ConfigMgr client say so, and never one in `win_patch_never_reboot_groups`.
2. **Prove the server is back** (after a restart, and also when none was needed). A new boot time, and every automatic service that ran before runs again. Delayed services get `win_patch_services_grace_min` (5 minutes) to start.
3. **Look again.** After a restart, servicing-stack and cumulative updates often reveal more updates. There is a second round (`win_patch_max_rounds: 2`), only when it restarted.

Safety.

- One server at a time, and the rollout stops at the first server that fails.
- By default it only patches the group `win_patch_hosts`. Set `target:` in the template's Variables (for example `target: all`) and pick the servers with the Limit at launch.
- Servers in `win_patch_never_patch_groups` (default `no_patch`) are skipped.
- **Maintenance windows still apply.** Outside the server's ConfigMgr maintenance window, ConfigMgr holds the updates. The job then stops with `ConfigMgr will not install these now ... (waiting for a maintenance window)` instead of waiting forever. Run it inside the window, or ask the ConfigMgr admin for a window that covers the patch time.

- **A dry run first:** run the template as *Check*. It rescans and lists what it would install, and changes nothing (DRY_RUNS.md). The *Windows patch with a dry run first* workflow puts that list in front of the approver.

Needs: the ConfigMgr client (Software Center) on the servers, and full PowerShell (not Constrained Language Mode) for the install call. Restarts use `shutdown.exe`, and AAP watches the WinRM port go down and come back. No Ansible collection is needed.

Scheduling. With `target: all` and no Limit, a scheduled run patches every server in the Windows inventory, one after another. Choose that on purpose. Otherwise, make one schedule per group (for example the test servers the week before the others), each with its own Limit.

The job template:

Field	Value
Name	`Windows patch`
Playbook	`playbooks/win_patch.yml`
Credentials	`Windows (WinRM)`
Job type	Run, with **Prompt on launch** ticked (Check = the dry run)
Variables	`target: all` (or a group), and pick servers with the Limit
Survey: `automatic_restarts`	"Restart servers automatically when the updates need it?" Choices `no`, `yes`. Default `no`

If a server fails:

What you see	What to do
`has no working ConfigMgr client`	the SMS Agent Host service (CcmExec) is missing or stopped
`waiting for a maintenance window`	run inside the window, or ask the ConfigMgr admin
`these updates failed: ... (error 0x...)`	`C:\Windows\CCM\Logs\UpdatesDeployment.log` and `WUAHandler.log` on the server
`still not done after 120 minutes`	Software Center on the server; `C:\Windows\CCM\Logs\UpdatesHandler.log`
`did not accept a logon after the restart`	look at the server's console
`... ran before patching and do not run now`	the service did not come back: `Get-Service <name>`, System log

All settings of this job, with their defaults: win_patch.

ACT on the servers (C:\ProgramData\act)

`playbooks/win_act_install.yml` puts ACT for Windows (`vendor/act-windows/act.ps1`) on the servers in `C:\ProgramData\act` (setting `win_act_dir`). Run it again after you update this repository, and every server gets the new version.

- **Locked down.** Ordinary users may create folders and files in `C:\ProgramData` (and in `C:\temp`). Someone could then place, or swap, `act.ps1` before an administrator runs it, and their code would run with administrator rights. So the job:
- takes ownership of `C:\ProgramData\act`;
- lets only Administrators and SYSTEM in (nothing inherited from `C:\ProgramData`);
- checks the file's SHA256 against the one in the repository, every run, and replaces it if it differs.
- **Unblocked.** The file is written on the server itself, so Windows never marks it as "downloaded from the internet", and the job runs `Unblock-File` on it anyway. Every run also checks it is not blocked, and fixes it if it is.
- **Using it on a server:** an administrator opens PowerShell and runs `powershell -ExecutionPolicy Bypass -File C:\ProgramData\act\act.ps1`. The first time, `:setup` asks for their own GenAI or Ask Sage key and keeps it in their profile. No key is put on the server for everyone.
- **Dry run:** run as *Check* and it says what it would install or fix. To remove it: `win_act_state: absent`.
- **Tell your security team before the first run.** The installer is a large PowerShell script that writes a `.ps1` file, and some endpoint-protection products (Defender for Endpoint, Trellix) flag that pattern. Let them know it is coming from AAP, and try one server first.
- **Versions and `:probe`.** ACT for Windows has the same endpoint formats and `:probe` command as the Linux one (see ADDING_ACT.md); the copy in `vendor/act-windows/` is refreshed when the repository's vendored ACT is. On Windows, an unattended run never approves the danger tier of commands: only a person at the prompt can. See APPROVED_COMMANDS.md.
- ACT itself needs full PowerShell. Where WDAC or AppLocker enforce Constrained Language Mode, ACT's own notes (sign it with your code-signing certificate) apply.

All settings, with their defaults: win_act.

Settings: what each check looks at, and how to change it

Every setting is in `roles/win_check_<name>/defaults/main.yml`, with a comment saying what it does. To change one:

- **for all Windows servers**, use the `Windows servers` inventory's **Variables** box;
- **for a group**, use the group's **Variables** box, or a settings file named after the group (for example `playbooks/group_vars/win_web.yml` for a group called `win_web`);
- **for one server**, use the host's **Variables** box.

The settings you will most likely want:

network: what every Windows server must resolve and reach

```
win_check_network_dns_names: [dc01.yoursite.mil]
win_check_network_tcp:
  - {name: LDAPS, host: dc01.yoursite.mil, port: 636}
  - {name: SIEM, host: siem.yoursite.mil, port: 9997}

# security: your endpoint protection's services (for example Trellix / McAfee)
win_check_security_av_services: [masvc, mfemms]

# audit: your log forwarding agent
win_check_audit_forwarder_services: [SplunkForwarder]

# accounts: who may be a local administrator (wildcards allowed)
win_check_accounts_admins_allowed: ['*\Administrator', 'YOURDOMAIN\Domain Admins',
  'YOURDOMAIN\svc_aap']

# services: more services that must run, on a group (for example IIS servers)
win_check_services_required_extra: [W3SVC, WAS]
```

A few defaults worth knowing:

- **disk**: warning at 85% used, critical at 95%. Per volume: `win_check_disk_overrides: {'D:': {warn: 92, crit: 98}}`.
- **services**: `EventLog`, `RpcSs`, `Dnscache`, `W32Time`, `WinRM` and `mpssvc` (the firewall) must run. Automatic services that stop cleanly by design are not reported.
- **security**: Defender is checked only when it is the active antivirus (`win_check_security_av: auto`). With another product, list its services instead.
- **certs**: warning 30 days before expiry, critical 7 days before. Old expired certificates that nothing uses are listed in the report, not raised as findings.
- **patching**: warning when the last update is 35 days old, critical at 60 days.

All settings, with their defaults: win_check_accounts, win_check_audit, win_check_certs, win_check_disk, win_check_eventlog, win_check_network, win_check_patching, win_check_performance, win_check_security, win_check_services, win_check_time.

If something goes wrong

What you see	What to do
`winrm or requests is not installed` / connection test: `no winrm`	the EE has no pywinrm: step 2
`CERTIFICATE_VERIFY_FAILED`	the CA bundle (step 4); `ansible_winrm_server_cert_validation: ignore` only as a short lab test
`the specified credentials were rejected by the server`	username format (`DOMAIN\user` for NTLM), password, or the account is locked
`Connection refused` / `timed out` on 5986	the firewall, or no HTTPS listener (step 1): a server with only HTTP takes port 5985 (step 4)
`kerberos: ... kinit` or `Server not found in Kerberos database`	no Kerberos in the EE, or a short host name: use NTLM, or full names
`ntlm: ... [digital envelope routines] unsupported`	the system is in FIPS mode: NTLM needs MD4 and RC4, which FIPS forbids. Use Kerberos (`svc_aap@YOURDOMAIN.MIL`, `ansible_winrm_transport: kerberos`)
`KDC has no support for encryption type` (Kerberos, FIPS)	the EE's Kerberos follows the host's FIPS policy and offers only AES with SHA-2, while Active Directory's keys are AES with SHA-1. Build your Windows EE with the file `/etc/krb5.conf.d/crypto-policies` removed and a file in `/etc/krb5.conf.d/` holding `[libdefaults] permitted_enctypes = aes256-sha2 aes128-sha2 aes256-sha1 aes128-sha1` (no RC4, no DES); check it inside the EE with `kinit svc_aap@YOURDOMAIN.MIL` - it asks for the password when Kerberos is fixed
`local administrator: NO`	add the account to the local Administrators group
`the <name> check could not run: ...`	the text after it is PowerShell's error. The other checks still ran
`skipping: no hosts matched`	the template's inventory is not `Windows servers`, or the Limit matches no server there
`... is not digitally signed` or `running scripts is disabled on this system`	a GPO sets the PowerShell execution policy for the machine (for example AllSigned), which overrides what Ansible asks for. Set it to RemoteSigned by GPO for these servers, or sign the scripts with your code-signing certificate

Not in this release (next)

- ACT in the Windows jobs: `use_act` on the Windows health check and troubleshoot (explain, propose fixes for approval), like on Linux. ACT is already on the servers (above).
- Service watch for Windows services and IIS application pools.
- Checks for IIS and SQL Server. (Domain controllers have their own report: ACTIVE_DIRECTORY.md.)

10. VMware jobs (vCenter): restart, snapshots, VLAN, reports, alarms and ACT

Small jobs for virtual machines, done through vCenter (they replace the vCenter runbooks of System Center Orchestrator):

Job template	Playbook	What it does
VM - restart	`playbooks/vm_reboot.yml`	restarts VMs. `guest` (default): the operating system restarts itself through VMware Tools. `hard`: a reset, like the reset button
VM - shut down	`playbooks/vm_shutdown.yml`	shuts VMs down and waits until they are off. `guest` (default) through VMware Tools; `hard`: power off at once
VM - snapshot	`playbooks/vm_snapshot.yml`	takes a snapshot **without memory** (the disks only). The AAP job number and user go into its description
VM - delete snapshot	`playbooks/vm_snapshot_delete.yml`	deletes one snapshot by its exact name, or every snapshot of the VM. Lists what the VM has first
VM - notes	`playbooks/vm_notes.yml`	edits the VM's Notes in vCenter: append a line (with date and user), replace, or clear. Prints the old notes
VM - change VLAN	`playbooks/vm_vlan.yml`	moves one network adapter to another port group (VLAN). The VM's other adapters are not touched
VM - secure boot report	`playbooks/vm_secure_boot_report.yml`	read-only: every VM's firmware and secure boot. Lists BIOS VMs and EFI VMs with secure boot off separately
VM - datastore report	`playbooks/vm_datastore_report.yml`	read-only: every datastore's capacity, free space, used and provisioned %; the ones at 85 / 90 % used listed apart
VM - snapshot report	`playbooks/vm_snapshot_report.yml`	read-only: every snapshot of every VM - name, when taken and by whom, age, size, folder - oldest first; the ones 3 days or older listed apart

Job template	Playbook	What it does
VM - snapshot cleanup	`playbooks/vm_snapshot_cleanup.yml`	deletes the snapshots 3 days or older - never "keep" ones, excluded VMs, the AAP server or ones over 1 TB - then reports what it deleted. Best run in the workflow below, after an approval
VM - health report	`playbooks/vm_health_report.yml`	read-only: every datacenter, cluster and host of every vCenter - vSphere HA and DRS working or not (HA agents, failover capacity, DRS recommendations and faults, vCLS retreat mode), hosts not connected or in maintenance, hardware sensors not green, hosts that need a restart, CPU and memory used, configuration issues and alarms - coloured, with what to do for each problem
VM - alarms report	`playbooks/vm_alarm_report.yml`	read-only: the triggered alarms on vCenter, datacenters, clusters and hosts; the alarms that fired in the last 24 hours (also the ones that cleared since); hosts not connected; the last 24 hours' failed logins, VM shutdowns / HA restarts and other errors and warnings; configuration issues - coloured by severity
VM - alarms ACT analysis	`playbooks/vm_alarm_act_analysis.yml`	read-only: the same problems, with ACT's likely cause, fix and confidence for each. ACT runs on the AAP side, never on the hosts
VM - capacity planning	`playbooks/vm_capacity_report.yml`	read-only: overall and per cluster - CPU, memory (also with a host down), vCPUs per core, VMs that still fit, hosts recommended, datastores - with growth from vCenter's own history and the days until each limit; optionally GenAI's estimate next to it
VM - deploy from ServiceNow ticket	`playbooks/vm_deploy_from_ticket.yml`	builds the VM - or VMs, one per name - a ServiceNow ticket asks for, from a template - checked against your rules, placed by site, name rule or free room, the result on the ticket. Best run in its workflow (propose, approval, build)

Job template	Playbook	What it does
VM - deploy settings finder	`playbooks/vm_deploy_inventory.yml`	read-only: every vCenter's datacenters, clusters, templates, datastores, folders and networks, by the names the deploy's settings use - with the lines to paste into the deploy template's Variables
VM - pick up deploy tickets	`playbooks/vm_deploy_poll.yml`	on a schedule: finds new VM request tickets in ServiceNow and starts the deploy workflow for each. Builds nothing itself; the approval stays
VM - VMs created report	`playbooks/vm_created_report.yml`	read-only: how many VMs were created in the last 7 days and each one - when, by whom, how (cloned, deployed from a template, new), the ticket when the AAP deploy built it, where, its size - by person and by day, and the ones created and deleted since. Schedule it weekly (e.g. Fridays at noon)
ESXi - security settings	`playbooks/esxi_security.yml`	on every ESXi host: SSH off, the shell timeouts at 600 s, lockdown mode normal. Changes only what differs, so it can run every night and puts back whatever drifted

They run on the AAP side and talk to vCenter. They never log in to the VMs.

Never the AAP server. Restart, shut down and VLAN change refuse the AAP server's VM. A job would cut off the system running it, and AAP must never be restarted from AAP. The VM is recognised by its name, its guest host name or its IP address, matched against the hosts in your inventory groups `aap` and `aap_hosts`. If the template's inventory has no such group and you have not named the AAP VM (below), these three jobs refuse to run at all. Snapshots, notes and the report are allowed on every VM, since they do not stop it.

Before you start

1. The execution environment must have the VMware collection

The jobs use Red Hat's certified collection `vmware.vmware` (2.11 or later) and its Python libraries (pyVmomi and the vSphere Automation SDK). Find out whether your execution environment (EE) has them. On the AAP server, as the user AAP runs as:

```
podman images | grep -i ee-
podman run --rm <the EE image your templates use> ansible-galaxy collection list vmware.vmware
podman run --rm <the EE image your templates use> python3 -c "import pyVmomi, vmware.vapi;
  print('ok')"
```

- **Both answer:** nothing to do.
- **Not there:** build an EE that has it, with `ansible-builder`. Start from your AAP version's `ee-minimal` image, and pull the collection from your Private Automation Hub when the network is air-gapped.

ansible-builder reads the collection's Python requirements by itself:

execution-environment.yml

```
version: 3
images:
  base_image:
    name: registry.redhat.io/ansible-automation-platform-27/ee-minimal-rhel9:latest
dependencies:
  galaxy:
    collections:
      - name: vmware.vmware
```

Then in AAP: **Execution Environments > Add** (the image you pushed). Pick it on the VM templates.

Do **not** add a `collections/requirements.yml` to this repository for it. AAP would then try to download from Galaxy on every project sync, and on an air-gapped network every sync would fail.

vCenter 7 or 8 is needed: the jobs also use vCenter's REST API.

2. A vCenter account for AAP

A service account with a vCenter role that has only what the jobs need:

Job	vCenter privileges
restart	Virtual machine > Interaction > Reset
shut down	Virtual machine > Interaction > Power off
snapshot / delete snapshot	Virtual machine > Snapshot management > Create snapshot / Remove snapshot
notes	Virtual machine > Change configuration > Set annotation
change VLAN	Virtual machine > Change configuration > Modify device settings, and Network > Assign network on the port groups
ESXi security settings	Host > Configuration > Security profile and firewall (the SSH service), Advanced settings (the timeouts), Change settings (lockdown mode and its exception users). If one is missing, the job's error names it (e.g. `lacks the privilege Host.Config.Settings`)
all (to read), and the alarms report	Read-only, which every vCenter role includes

Give it the role on the folders or clusters of the VMs these jobs may touch, and nowhere else.

3. The credential

AAP: **Credentials > Add**, type **VMware vCenter**: vCenter Host (its name, e.g. `vcsa01.yoursite.mil`), Username, Password. It hands the jobs `VMWARE_HOST`, `VMWARE_USER` and `VMWARE_PASSWORD`. Nothing about vCenter goes in Git.

vCenter's certificate: the jobs check it. If the EE does not trust your CA, a job fails with `certificate verify failed`. The right fix is to add your CA to the EE. Until then, `vmware_validate_certs: false` turns the check off, in the template's Variables (not recommended).

Several vCenters: the reports (alarms, datastores, snapshots and their cleanup, capacity planning) and the ESXi security settings read **every vCenter in one job, into one report**: keep one VMware vCenter credential and list the vCenters in `vmware_vcenters` (see "Several vCenters, one report" below). The jobs

that act on VMs by name (restart, shut down, snapshot, notes, VLAN, secure boot) still use the credential's vCenter only: for a VM in another vCenter, use a second credential with that vCenter's address, or tick **Prompt on launch** for Credentials and pick it at launch.

5. Several vCenters, one report

With two or more vCenters (for example in Enhanced Linked Mode, as one tree in the vSphere Client), list them all in `playbooks/group_vars/all.yml` (for every job) or in a template's Variables:

```yaml
vmware_vcenters: [vc01.yoursite.mil, vc02.yoursite.mil]
```

- **One credential** is enough: the jobs log in to every vCenter with its user name and password. In Enhanced Linked Mode the vCenters share single sign-on, so the same account works on all of them. Give its role as a **Global Permission** (vSphere Client: **Administration > Access Control > Global Permissions**), or on each vCenter's root with **Propagate to children**.
- The credential's own vCenter is always read. Listing it again (or by another name, its IP) is fine: each vCenter is read once.
- A vCenter on another port: `vc03.yoursite.mil:8443`.
- **The report:** one email for all. The **Datacenter** column says which vCenter (`vc02 / DC1`); the footer lists the vCenters read. A cluster or datastore name that exists in two places is shown with its vCenter and datacenter, e.g. `PROD (vc02 / DC1)`, and never added together.
- **A vCenter that cannot be read** (down, a firewall, a refused login) does not stop the others: the report covers the others, lists it in red on top ("vCenters not read") and its title ends with "- 1 vCenter(s) not read". The job stays **green**: the report completed (`vmware_vcenter_fail_unread: true` makes it fail, e.g. for a workflow's failure path).
- **Snapshot cleanup:** each snapshot is deleted on its own vCenter. A VM's ID repeats in every vCenter (both have a `vm-42`), so the report hands the cleanup the vCenter with each snapshot, and the cleanup deletes only that one.
- The ESXi security settings set the hosts of every vCenter listed.

4. Name the AAP VM (and vCenter's)

In your `playbooks/group_vars/all.yml`:

```yaml
vmware_protected_vms: [AAP01, VCSA01]      # the exact VM names in vCenter
```

This covers the case where the AAP VM's name differs from its host name and VMware Tools is not running, so neither the guest host name nor the IP is known. More groups to refuse: `vmware_protected_groups: [aap, aap_hosts, my_group]` (aap and aap_hosts are always refused anyway).

Which VMs a job acts on

You type them. Every job (except the report) asks for **VM names**, one per line: one VM, or several. These are the names as vCenter shows them in its inventory. They are exact, upper/lower case included, with no wildcards.

The job runs on the AAP server and asks vCenter for each name. It does **not** use the template's inventory hosts or a Limit to choose VMs; it uses the inventory only to recognise the AAP server. So a VM does not

have to be in any AAP inventory to be restarted or snapshotted.

- Names that do not exist, or exist twice in vCenter, stop the job before anything is done.
- With several VMs, every check is done for all of them first. If one is refused (the AAP server, no VMware Tools for a clean restart), nothing is done to any of them.
- Several VMs are handled in the order given. Restarts are requested one right after the other, without waiting for each to come back up, so restart the members of a cluster (two database nodes, for example) in separate runs. A shutdown waits for each VM to be off before the next.

The templates

Make one job template per job:

- **Inventory:** your Linux inventory, the one with the group `aap`. The jobs need it for the AAP check; they run nothing on its hosts.
- **Credentials:** the VMware vCenter credential.
- **Playbook:** one of the files in the table at the top.
- **Execution environment:** the one with `vmware.vmware`.
- **Limit:** leave it **empty** and do **not** prompt for it. These jobs run on the AAP side (`localhost`); a Limit, or a workflow's Limit, would make them skip without doing anything.
- **Job type:** tick **Prompt on launch** if you want dry runs. As **Check**, every job says what it would do and changes nothing.

Surveys (**Survey > Add**, then switch the survey **on**):

Template	Question (variable)	Type	Default / choices
all but the report	VM names, one per line (`vm_names`)	Textarea, required	
restart, shut down	How (`vm_power_mode`)	Multiple choice	`guest`, `hard`; default `guest`
snapshot	Snapshot name (`vm_snapshot_name`)	Text, optional	empty = `aap-<date>-<time>`
snapshot	Why / change number (`vm_snapshot_description`)	Text	
delete snapshot	Snapshot name (`vm_snapshot_name`)	Text	
delete snapshot	Delete ALL snapshots (`vm_snapshot_delete_all`)	Multiple choice	`false`, `true`; default `false`
notes	Text (`vm_notes`)	Textarea	
notes	How (`vm_notes_mode`)	Multiple choice	`append`, `replace`, `clear`; default `append`
change VLAN	Network adapter number (`vm_nic`)	Integer	`1`
change VLAN	Port group (`vm_portgroup`)	Text, required	its name as vCenter shows it
any (optional)	Email the result to (`report_email_to`)	Text, not required	empty = no email ("Email the report" below)

The secure boot and datastore reports need no survey. Schedule them weekly with `report_email_to` in their Variables. They stay green when they find VMs or datastores to fix - the report and email say which; `vm_secure_boot_fail: true` / `vm_datastore_fail: true` mark the job failed instead, for a workflow that opens a ticket. `vm_secure_boot_folder` limits the secure boot report to one vCenter folder; `vmware_datacenter` limits either report to one datacenter; `vm_datastore_warn_pct` / `vm_datastore_crit_pct` (85 / 90) set the datastore thresholds.

Snapshots: the report, the cleanup, and the approval workflow

The idea, in one picture

```
1. SNAPSHOT REPORT          looks at every VM and makes a list of its snapshots.
        |                   It changes nothing. It emails you the list.
        v
2. APPROVAL                 AAP stops and waits. A person reads the list and
        |                   clicks Approve (go ahead) or Deny (stop).
        v  (only on Approve)
3. SNAPSHOT CLEANUP         deletes the old snapshots that were ON THAT LIST -
                            nothing else - and emails what it deleted.
```

Think of it like cleaning out a fridge: first someone writes down everything old in it (the report), then a grown-up says "yes, throw those out" (the approval), then the helper throws out **only what is on the paper** (the cleanup). Anything someone put a "keep" label on stays.

What the report shows

For every snapshot of every VM: the VM, the snapshot's name, when it was taken, **who took it**, how many days old it is, how big it is (MB, GB or TB), the VM's folder and the description. Oldest first. **Every snapshot is in that list, whatever its age** - even one taken an hour ago. Then two short lists on top:

- **To delete:** snapshots older than 3 days (`vm_snapshot_max_age_days`).
- **Held back:** old snapshots the cleanup will never delete, each with the reason.

The email's heading and subject always say the limit, e.g. `VMware snapshot report (older than 2 days): 4 to delete (310 GB); 23 snapshot(s) on 17 VM(s)`.

To use 1 or 2 days instead of 3:

Where	Change
In the workflow	the survey answer when you launch it - or for good: the workflow > **Survey** tab > the question > **Default** 2. The report and the cleanup both use it: after the approval, snapshots older than 2 days are deleted.
The report template on its own	Templates > `VM - snapshot report` > Edit > **Variables**: `vm_snapshot_max_age_days: 2`

In the workflow, the survey answer wins over the template's Variables.

"Who took it" comes from vCenter's history. vCenter forgets old history (often after 30 days), so for an old snapshot it may say **unknown**. Snapshots taken by the "VM - snapshot" job also have the AAP user in their description.

What the cleanup never deletes

Never deleted	Why	Change it with (in playbooks/group_vars/all.yml)
A snapshot whose name or description says **keep** or **do not delete**	someone wants it kept	`vm_snapshot_keep_regex`
Any snapshot of the VMs on your do-not-touch list	domain controllers, vCenter, ...	`vm_snapshot_cleanup_exclude_vms: [DC*, VCSA*]` (* = anything)
Any snapshot of the VMs in `vmware_protected_vms`	the same protected VMs as for restart	`vmware_protected_vms`
Any snapshot of the AAP server	AAP must never be put at risk by AAP	(always)
A snapshot bigger than **1 TB**, or whose size is unknown	merging a huge snapshot can take hours and slow the VM: do it by hand, at a quiet time	`vm_snapshot_cleanup_max_size_gb: 1024`
More than 50 snapshots in one run	a typo in the days would otherwise delete everything	`vm_snapshot_cleanup_max: 50`
Anything that was not on the approved list	the approver only said yes to that list	(always, in the workflow)

Set it up (once)

Step 0 - what you need first (see "Before you start" above): the VMware vCenter credential, the execution environment with `vmware.vmware`, and the mail relay in `all.yml` ("Email the report").

Step 1 - your do-not-touch list. In VS Code, in `playbooks/group_vars/all.yml`, add (with your VM names; `*` matches anything) - then Commit, Sync Changes, and sync the project in AAP:

```yaml
vm_snapshot_cleanup_exclude_vms: [DC*, VCSA*, AAP*]
```

Step 2 - the report template. Templates > Create template > Create job template:

- Name: `VM - snapshot report`
- Inventory: your inventory with the `aap` group
- Project: site-automation. Playbook: `playbooks/vm_snapshot_report.yml`
- Execution environment: the one with `vmware.vmware`
- Credentials: your **VMware vCenter** credential
- Leave Limit empty. No survey. Save.

Step 3 - the cleanup template. The same again:

- Name: `VM - snapshot cleanup`
- Playbook: `playbooks/vm_snapshot_cleanup.yml`
- Tick **Prompt on launch** next to **Job type** (this lets you do a dry run - below).
- No survey. Save.

Step 4 - the workflow. Templates > Create template > **Create workflow job template**:

- Name: `VM snapshot cleanup (with approval)`, your organization. Save.
- **Survey** tab > Create survey question - two questions, then switch the survey **on**:

Question	Answer variable name	Answer type	Default
Delete snapshots older than how many days?	`vm_snapshot_max_age_days`	Integer, required, minimum 1	`3`
Email the reports to	`report_email_to`	Text	your team's address

The workflow hands these answers to both jobs, so the report and the cleanup use the same days.

- **Visualizer** (the boxes):

1. Click **Add step**. Node type: **Job template**. Pick `VM - snapshot report`. Save.
2. Hover over that box > **Add step and link** (the plus sign). Node type: **Approval**. Name: `Delete the snapshots listed in the report?`. Timeout: e.g. `1 day` (no answer by then = no). Run type: **Run on success**. Save.
3. Hover over the approval box > **Add step and link**. Node type: **Job template**. Pick `VM - snapshot cleanup`. Run type: **Run on success** (after an approval, that means: approved). Save. **Never Run always here:** the cleanup would then also run when the approval is denied or times out - and it still gets the report's list, so it would delete. (Check the link's label in the Visualizer: green *Run on success*, not blue *Run always*.)
4. Click **Save** at the top of the Visualizer.

Step 5 - who may approve. On the workflow: **User Access** (or Team Access) > Add > pick the people > role **Approve**. They need no other rights. Optional: **Notifications** tab > turn on the approval notification so they get an email when something waits for them.

Step 6 - when it runs. On the workflow: **Schedules** > Create schedule (e.g. every Monday 7:00). Or launch it by hand.

Run it

1. Launch the workflow (or the schedule does). Answer the survey (or keep 3 days).
2. The report runs and **emails the list**: what will be deleted, what is held back, and why.
3. AAP waits at the approval. Open **Automation Execution > Administration > Workflow Approvals** (or the workflow job's view) > the approval > **Approve** or **Deny**.
4. On **Approve**: the cleanup deletes the snapshots from that list that are still there, and still that old, and **emails what it deleted** (and how much space came back). On **Deny** or timeout: nothing is deleted.

The cleanup's email can be sent only when something was actually deleted:
`vm_snapshot_cleanup_email_only_if_deleted: true` (in the cleanup template's Variables).

Dry run: try it, delete nothing

- **The safest first try: say no.** Launch the workflow, read the report email, then **Deny** the approval. The report only reads, so nothing changed anywhere.
- **See exactly what the cleanup would delete, without deleting:** in the workflow's Visualizer, click the `VM - snapshot cleanup` box > **Edit** (pencil) > Job type **Check** > Save, and save the Visualizer. Run the workflow and **Approve**: the cleanup job's output lists `WOULD DELETE ...` for each snapshot and deletes

nothing (and sends no email). Set the box back to **Run** afterwards. (This needs the Step 3 tick "Prompt on launch" for Job type.)

- **Without the workflow:** launch `VM - snapshot cleanup` on its own. With no approved list and no `vm_snapshot_cleanup_confirm: true` it only lists what it would delete - it never deletes.

Without the workflow

To clean up by hand, make a separate template (not the one in the workflow) on `playbooks/vm_snapshot_cleanup.yml` with a survey: `vm_snapshot_max_age_days` (Integer, 3) and `vm_snapshot_cleanup_confirm` ("Delete them?", Multiple choice `false` / `true`, default `false`). With `false` it only lists; with `true` it deletes - with every rule above still in place.

Health: every datacenter, cluster and host

The read-only **VM - health report** answers "is the virtual infrastructure healthy?" for every vCenter (the credential's and `vmware_vcenters`), every datacenter, every cluster and every host - in one email, coloured by severity, with what to do for each problem. The alarms themselves, failed logins and the day's events are in the **alarms report** below: run both every morning (workflow 18 in WORKFLOWS_AND_SCHEDULES.md).

What the report shows

Section	What is in it
Needs attention	every problem found, most severe first: severity, the object (vCenter, datacenter, cluster or host), the problem, **what to do**, and where it is (datacenter / cluster)
Datacenters	each datacenter with everything in it: clusters, hosts, hosts running VMs, in maintenance, not connected, its problems
Clusters	each cluster: hosts running VMs, **vSphere HA** (on or OFF, what admission control reserves, host agents not OK), **DRS** (on or OFF, automation level, score, recommendations waiting), **vCLS** (normal or RETREAT MODE), CPU and memory used, alarms, problems
Hosts	every host, by datacenter and cluster (standalone hosts too): connection, maintenance mode, the **HA agent** (primary, secondary, isolated, partitioned ...), **hardware** (sensors and hardware status not green), CPU and memory used, days up, ESXi version and build, alarms
vCenter	each vCenter's version and the alarms on vCenter itself

On top: the counts in coloured boxes (hosts not connected, HA problems, DRS problems, hosts with hardware faults, triggered alarms). The subject says it in one line, e.g. `[AAP] VMware health report: 2 critical, 3 warning(s) - 3 datacenter(s), 7 cluster(s), 26 host(s)`.

Colour	What
Red (critical)	a host not connected to vCenter (or powered off); a host's HA agent isolated, partitioned, unreachable or in error; vCLS retreat mode on a cluster with DRS (DRS stops); a red hardware sensor; a red alarm; CPU or memory from 95 %

Colour	What
Amber (warning)	HA or DRS off on a cluster of 2 or more hosts; HA host monitoring off; HA failover capacity below what admission control reserves; DRS recommendations waiting, or DRS faults; retreat mode without DRS; a yellow sensor; a host that needs a restart; a configuration issue (SSH on, ...); a yellow alarm; CPU from 85 %, memory from 90 %
Blue (info)	a host in maintenance mode or standby; HA admission control off; DRS in manual mode - worth knowing, not broken

A one-host cluster without HA or DRS is fine, and so is a host in maintenance mode whose HA agent is not configured (that is what maintenance mode does).

Set up the report

1. **Templates > Create template > Create job template:**
   - Name: `VM - health report`
   - Inventory: any (the job runs on the AAP side)
   - Project: site-automation. Playbook: `playbooks/vm_health_report.yml`
   - Execution environment: your VMware one
   - Credentials: your **VMware vCenter** credential. A read-only vCenter account is enough.
   - Variables: `report_email_to: vmteam@yoursite.mil` (and `vmware_vcenters: [...]` for more vCenters, if not already in `all.yml`)
   - Save, then **Launch** it once and read the email.
2. **Schedules > Create schedule:** e.g. every day at 06:00 - or put it in front of the alarms report in workflow 18.

Setting (template Variables)	Default	What it does
`vm_health_require_ha`	`true`	a cluster of 2 or more hosts without vSphere HA is a warning; `false` if that is on purpose
`vm_health_require_drs`	`true`	the same for DRS; `false` where DRS is not licensed (e.g. vSphere Standard)
`vm_health_cpu_warn_pct` / `vm_health_cpu_crit_pct`	`85` / `95`	CPU used (a host, or a cluster's hosts running VMs): amber from, red from
`vm_health_mem_warn_pct` / `vm_health_mem_crit_pct`	`90` / `95`	the same for memory
`vm_health_sensors`	`true`	read the hosts' hardware sensors
`vm_health_fail`	`false`	`true` = the job shows **failed** on a critical problem, for a workflow
`vmware_datacenter`	(all)	one datacenter only

vCLS retreat mode is read from vCenter's advanced settings (`config.vcls.clusters.<cluster id>.enabled = false`). If the account may not read them, the report says so in blue, and retreat mode still shows as vCenter's own alarm on the cluster.

VMs created: the weekly report

The read-only **VM - VMs created report** answers "how many VMs were created this past week?" for every vCenter (the credential's and `vmware_vcenters`): the count up front, then each VM.

Section	What is in it
On top	VMs created, of them by AAP from tickets, deleted since; vCPU, memory and disk added
VMs created, oldest first	when (UTC), the VM, **by whom**, **how** (deployed from template X, cloned from Y, created new), the **ticket** when the AAP deploy built it, cluster, folder, vCPU, memory, disk, operating system, power
By whom	each person (account) and how many VMs - and how many of them came from tickets
By day	how many VMs each day
Created and deleted since	VMs created in the week that are gone now (counted on top too), and when they were deleted

The subject says it in one line, e.g. `[AAP] VMware: 5 VMs created in the last 7 days (3 by AAP from tickets, 1 deleted since)`.

- **When:** the VM's own creation date (vSphere 6.7 and later), else its creation event. **By whom and how:** vCenter's events. vCenter keeps events for a limited time (often 30 days): a week is well inside that. A VM whose event is gone still counts, by its date, with "By" empty.
- **The window** is the last `vm_created_days` (7) days up to the moment the job starts. Run every Friday at noon, each report covers about Friday noon to Friday noon (a job that starts a minute late moves the window by that minute).
- **The ticket** comes from the notes the deploy writes on each VM (*"Deployed from … for INC… by …"*); a VM built by hand has none. Templates are not counted.

Set up the report, every Friday at noon

1. **Templates > Create template > Create job template:**
   - Name: `VM - VMs created report`
   - Inventory: any (the job runs on the AAP side)
   - Project: site-automation. Playbook: `playbooks/vm_created_report.yml`
   - Execution environment: your VMware one
   - Credentials: your **VMware vCenter** credential. A read-only vCenter account is enough (it reads the VMs and the events).
   - Variables: `report_email_to: vmteam@yoursite.mil` (and `vmware_vcenters: [...]` for more vCenters, if not already in `all.yml`)
   - Save, then **Launch** it once and read the email.
2. **Schedules tab > Create schedule:**
   - Name: `Weekly - VMs created`
   - Start date/time: the next Friday, **12:00**, and **Time zone: your site's** (not UTC - so 12:00 is your noon)
   - Repeat frequency: **Week**, every **1** week, on **Friday**

- Save. The schedule's next run shows on the template's Schedules tab.

Setting (template Variables)	Default	What it does
`vm_created_days`	`7`	the window: the last this many days (e.g. `14` for a report every other Friday, `30` for a monthly one)
`vmware_datacenter`	(all)	one datacenter only

Alarms: the report, and ACT's analysis

Two read-only jobs. The **report** shows what is wrong in vCenter now and what happened in the last 24 hours. The **ACT analysis** has ACT (the AI assistant) work out the likely cause and the fix of each of those problems, and say how sure it is. Neither changes anything.

What the report shows

Section	What is in it	Colour
Triggered alarms	the alarms vCenter shows now on vCenter itself, the datacenters, clusters and hosts: severity, object, alarm, since when, acknowledged by whom	red = critical, amber = warning
Alarms that fired in the last 24 h	every alarm that turned yellow or red in the window, one row per alarm and object: how many times it went red and yellow, first and last change, and how it is now - still on, or cleared since (an alarm that comes and goes is worth a look too)	red / amber = still on, blue = cleared
Host connection	hosts not connected to vCenter now, and hosts that lost the connection in the last 24 h and came back	red / amber
Failed logins	wrong user name or password: one row per user, source address and server, with the count and the first and last time	amber; red from 10 failures
Virtual machine events	guest shutdowns and restarts, power-offs, resets, HA restarts and failures, and who did it	blue = someone did it, amber = HA or nobody, red = a failover or power-on failed
Other errors and warnings	every other error and warning event; the same event on the same object counted once	red = error, amber = warning
Configuration issues	what vCenter shows as configuration issues (SSH left on, HA problems, ...)	amber
All hosts	every host: connection, maintenance mode, ESXi version and build, hardware, its alarms and its errors / warnings	red / amber / blue row

On top: the counts in coloured boxes. The subject says the most important ones, e.g. `[AAP] VMware alarms report: 1 critical, 2 warning alarm(s), 47 failed login(s) (events: last 24 h)`.

Set up the report

1. **Templates > Create template > Create job template:**
   - Name: `VM - alarms report`
   - Inventory: any (e.g. your VMware one: the job runs on the AAP side)
   - Project: site-automation. Playbook: `playbooks/vm_alarm_report.yml`
   - Execution environment: your VMware one (this job needs only pyVmomi, which it has)
   - Credentials: your **VMware vCenter** credential. A read-only vCenter account is enough.
   - Variables: `report_email_to: vmteam@yoursite.mil`
   - Save.
2. **Schedules > Create schedule:** e.g. every day at 06:00.

Setting (template Variables)	Default	What it does
`vm_alarm_hours`	`24`	the events of the last this-many hours (168 for a weekly report; 0 = alarms and hosts only)
`vm_alarm_types`	`[vcenter, datacenter, cluster, host]`	whose alarms; add `vm`, `datastore` or `network` for theirs too
`vm_alarm_login_critical`	`10`	this many failed logins of one user from one place, or more, is red
`vm_alarm_fail`	`false`	`true` = the job shows **failed** on a critical alarm or a host not connected, for a workflow
`vmware_datacenter`	(all)	one datacenter only

The ACT analysis

What you get: an email "VMware alarms ACT analysis". First "What to do first", then one row per problem (P1, P2, ...): severity, object, the problem, the **likely cause**, the evidence, the **fix** and a **confidence**. The confidence is ACT's own estimate, from 0 to 100: green 80 and up (the evidence shows it), amber 50-79 (likely), red under 50 (a guess: check first). These are likely causes, not certain ones: check before you change anything. ACT runs no command and changes nothing.

Where ACT runs, and what about the hosts? ACT runs inside this job's execution environment, on the AAP server (or the execution node that runs the job). Not on the ESXi hosts, not on the VMs. So:

- The **ESXi hosts need nothing**: no Python, no network path to ACT.
- **Python** is in every execution environment (Ansible itself runs on it; ACT needs 3.8 or later).
- The **AAP node must reach the model's address over HTTPS** (port 443): `https://api.genai.mil` for GenAI.mil, or your `site_act_url`. The job checks this first and says plainly when it cannot:
- *"cannot reach the model ... Open HTTPS ... or set a proxy"*: ask for a firewall rule from the AAP node to that address, or set your proxy in `playbooks/group_vars/all.yml`: `site_act_env: {HTTPS_PROXY: "http://proxy.yoursite.mil:8080", NO_PROXY: "localhost,.yoursite.mil"}`
- *"Its certificate is not trusted"*: the execution environment does not trust the model's certificate authority. Put that CA certificate (public, `.pem`) at `playbooks/files/ca/model-ca.pem`, add `site_act_ca: "{{ playbook_dir }}/files/ca/model-ca.pem"` to `all.yml`, and add the line `playbooks/files/ca/` to

`.site-local` (so an update keeps it).

Privacy: before anything leaves, ACT replaces host, cluster, datacenter and VM names, IP addresses and user names with placeholders, and puts the real names back into its answer.

Set it up:

1. **The key.** If your other ACT jobs have an **ACT model key** credential, use it. If not: create the credential type from `aap/credential_types/act_model_key.yml`, then a credential of that type with your GenAI.mil (or Ask Sage) key (SECRETS.md).
2. **Templates > Create template > Create job template:**
   - Name: `VM - alarms ACT analysis`
   - Inventory, project, execution environment: as for the report. Playbook: `playbooks/vm_alarm_act_analysis.yml`
   - Credentials: **VMware vCenter** and **ACT model key** (both)
   - Tick **Prompt on launch** next to **Job type** (for the dry run)
   - Variables: `report_email_to: vmteam@yoursite.mil`
   - Save. The provider and model are your usual ACT settings (`site_act_provider`, `site_act_models` in `all.yml`); the default is GenAI.mil.
3. **First run, as a dry run:** Launch, Job type **Check**. It reads vCenter, lists the problems it would give ACT, and calls nothing. Then launch it normally.

Both in one go: a workflow with `VM - alarms report`, then `VM - alarms ACT analysis` linked with **Always**. Each sends its own email. Or schedule just the ACT analysis: its email lists every problem too.

Green or red: nothing wrong = ACT is not called and no email is sent (`vm_alarm_act_email_if_none: true` sends one anyway). The job is **red** when ACT could not run (no key, no network, an ACT error): the email still lists the problems, and says why.

Setting	Default	What it does
`vm_alarm_act_max_items`	`40`	give ACT at most this many problems, the most severe first (the rest are listed as not analyzed)
`vm_alarm_act_config_issues`	`true`	configuration issues too
`vm_alarm_act_check_url`	`true`	check that the AAP node can reach the model first
`vm_alarm_act_model_timeout`	`site_act_model_timeout` (600)	seconds the model may take for its answer (a `GENAI_TIMEOUT` in `site_act_env` wins)
`site_act_timeout`	`1200`	seconds the whole ACT run may take (keep it above the model's)

Capacity planning: overall and per cluster

`playbooks/vm_capacity_report.yml` answers "how much room is left, and for how long?" - for all clusters together and for each cluster. It only reads. The history comes from **vCenter's own daily statistics** (it keeps a year of them at the default statistics level), so it works from the first run: nothing has to be collected first.

What the report shows

Section	What it shows
Overall	all clusters together: CPU and memory capacity, used, used with a host down in each cluster, growth per month and the days until the target; hosts usable and **hosts recommended** (now, and within 12 months); vCPUs per core and room for more VMs; shared storage with its growth and runway
Clusters: CPU and memory	per cluster: hosts usable, CPU and memory used, **memory used if the biggest host fails** (N+1: the cluster must still run everything), vCPUs per core, VMs and their average size, memory growth, **memory runway** (days and date until the target with a host down), **room for more VMs** of the average size (and what limits it), **hosts recommended**, VMs added / removed in the last 30 days
Clusters: storage	per cluster, its shared datastores together: capacity, free, used, provisioned (thin disks at full size), growth, days until 85 % and until full
Datastores: runway	each shared datastore, the one that fills first on top: growth, days (and date) until 85 %, 90 % and full, and how well a straight line fits its history
Math vs GenAI (with `vm_capacity_act: true`)	for every item, the job's runway (the math) next to ACT's own estimate (GenAI), whether they agree, ACT's recommendation and reasoning, and its confidence
Hosts per cluster	every host: state, cores, CPU and memory used, VMs - an unbalanced cluster shows here

Hosts recommended is a count of hosts of the cluster's average size (cores, memory) that keeps memory and CPU at their targets (80 %) with one host down - for example `1 now (memory 59% on average then); 2 within 12 months`. "Within 12 months" applies the current growth. It is a sizing guide in hosts, not a list of models.

How the runway is worked out: a straight line through the last 30 days of vCenter's daily values (memory consumed, CPU used, datastore space used), continued until it reaches the limit. "Not growing" means the line is flat or falling. The **trend fit** (R^2) says how straight the history is: poor means jumps (VMs moved, snapshots deleted, a datastore grown), and the runway is a rough guide then.

Math vs GenAI

With `vm_capacity_act: true`, ACT (the AI assistant, GenAI.mil or Ask Sage) reads the same numbers and makes **its own estimate** for the overall picture, each cluster and each datastore: when it needs action (in days), what to do, why, and how sure it is. The report puts the two side by side:

Math (straight line)	GenAI (ACT)	Agreement
102 days (90 % full)	55 days	GenAI 47 days sooner
0 days (memory)	0 days	agree

They **agree** when they are within 30 days or 25 % of each other. Where they differ, read ACT's reasoning (a recent jump, a limit that comes first, a poor fit), and plan for the sooner one. ACT states needs in hosts, cores, GB and TB, and timing in days or months. It runs on the AAP side and runs no command (as in "The ACT analysis" above: the same key, URL, proxy and CA settings).

Set it up

1. **Templates > Create template > Create job template:**
   - Name: `VM - capacity planning`
   - Inventory: any (e.g. your VMware one). Project: site-automation. Playbook: `playbooks/vm_capacity_report.yml`
   - Execution environment: your VMware one. Credentials: **VMware vCenter** (read-only is enough)
   - Tick **Prompt on launch** next to **Variables** (to try one cluster)
   - Variables: `report_email_to: vmteam@yoursite.mil`
   - Save. Launch it once and read the report.
2. **Per cluster:** the same report for one or a few clusters - Variables `vm_capacity_clusters: [PROD-CL01]` (`*` works: `[PROD-*]`). A second template with that variable gives each cluster its own report.
3. **GenAI's estimate (optional):** add the **ACT model key** credential to the template, and `vm_capacity_act: true` to its Variables. Launch.
4. **Schedule: Schedules > Create schedule**, e.g. every Monday at 07:00 (your time zone).

Setting	Default	What it does
`vm_capacity_clusters`	`[]` (all)	only these clusters
`vm_capacity_failover_hosts`	`1`	hosts down to plan for (N+1); a larger HA admission control failover level wins
`vm_capacity_mem_target_pct` / `_cpu_target_pct`	`80` / `80`	memory / CPU used at most this, with those hosts down
`vm_capacity_vcpu_per_core_max`	`4`	vCPUs per physical core (amber from here)
`vm_capacity_storage_warn_pct` / `_crit_pct`	`85` / `90`	datastore used % (the storage runway is to 90 %)
`vm_capacity_runway_warn_days` / `_crit_days`	`90` / `30`	a runway shorter than this is amber / red
`vm_capacity_horizon_months`	`12`	"hosts recommended within N months"
`vm_capacity_trend_days` / `vm_capacity_history_days`	`30` / `90`	the days the trend is fitted on / read
`vm_capacity_include_local_datastores`	`false`	count a host's own datastores too
`vm_capacity_act`	`false`	GenAI's estimate next to the math

Setting	Default	What it does
`vm_capacity_fail`	`false`	`true` = the job shows failed when a cluster or datastore is critical

Deploy a VM from a ServiceNow ticket

`playbooks/vm_deploy_from_ticket.yml` builds one VM from a vCenter template for a ServiceNow ticket (an incident or a task assigned to your team). You launch it with the ticket number; it reads the ticket, checks the request against the rules you set in AAP, writes on the ticket what it builds, builds it, and writes the result - with the VM's IP address - on the ticket.

> **The short version**
> VM_DEPLOY_GUIDE.md - also a PDF, `docs/pdf/Site-Automation-VM-Deploy-Guide.pdf` - puts the decisions in one place: a standard setup to start with, which number to launch with, what closes the task, where the notes go, where the VM goes, ready-made configurations, where the logic lives - and every `vm_deploy_*` setting with its default, an example and when to set it.

How it flows, step by step

The operator launches **one workflow**, `VM - deploy from ServiceNow ticket (workflow)`, with the ticket number. It runs the deploy job twice - as its *propose* step and its *build* step - with an approval between them:

```
 1. Requester ── writes the ticket in ServiceNow (plain words, or key: value lines)
 2. Operator  ── launches the workflow in AAP with the ticket number          (ONE launch)

 ── workflow step 1: PROPOSE (the deploy job, vm_deploy_phase: propose) ──────────────────
 3. AAP → ServiceNow: reads the ticket
 4. AAP → GenAI:      "copy out what this ticket asks for"       (plain words only; key: value
 5. GenAI → AAP:      names, OS as written, CPU, memory, disks    lines skip 4-6)
                      - each with the ticket's own words; no decisions, no questions
 6. AAP checks every value really is in the ticket, and finds the template from the OS words
       something AAP needs is missing?  → AAP → ServiceNow: its questions to the requester (a
         comment);
                                        the workflow STOPS
 7. AAP checks the rules (templates, site, caps, a free name, the group) and finds the cluster and
    datastore (read-only)
 8. AAP → ServiceNow: "[AAP] GenAI proposal" + "[AAP] Checked - ready to build: ... it would go to
   ..."
    AAP → email: the same, to the approver.                        NOTHING IS BUILT

 ── workflow step 2: APPROVAL ──────────────────────────────────────────────────────────────
 9. Approver  ── reads the email (or the ticket), clicks Approve in AAP     (Deny = nothing built)

 ── workflow step 3: BUILD (the deploy job, vm_deploy_phase: build) ──────────────────────
10. AAP → ServiceNow: reads the ticket and the posted proposal  (GenAI is NOT asked again; a
  ticket
                      changed since the proposal = nothing built)
11. AAP checks the rules again; AAP → ServiceNow: "[AAP] Deploying this VM" (refused = nothing
  built)
12. AAP → vCenter:    clone the template, CPU / memory / disks, customize (or network boot), power
  on
13. AAP → ServiceNow: "[AAP] VM ... deployed" - IP (or MAC for MECM), where it went
14. AAP → email: the result
```

For a demo with no person in between: `vm_deploy_genai_approval: false` on the job template and launch the job itself - one run reads, checks and builds (steps 3-8, then 11-14), when every value is traced to the ticket and within the rules; questions or a refusal still stop it.

Who does what:

Who	Does	Never does
ServiceNow	holds the request; shows the job's notes and questions	-
AAP (this job)	reads the ticket, checks everything, writes the notes, **builds the VM in vCenter**, reports	builds anything the rules do not allow, or without a "Deploying" note on the ticket
GenAI (ACT, evidence mode)	**copies the ticket text out as values** (names, the OS as written, CPU, memory, disks, site), quoting the words each comes from	builds or changes anything, runs a command, sees a password or a credential, sees vCenter, chooses a template version, a cluster or datastore, asks the requester, decides whether to build
vCenter	does what AAP asks: clone, size, customize, power on	-
The operator / approver	launches the workflow; reads the proposal and approves (or denies) in AAP	-

GenAI only copies text out as values. It answers once, in run 1, with values and quotes; AAP checks each quote is really in the ticket, turns the OS words into a template by its own rules, and checks each value against the rules, and writes the result on the ticket for a person to see. Every action - the notes on the ticket and everything in vCenter - is done by AAP, with the VMware vCenter and ServiceNow credentials, which GenAI never sees. Run 2 builds from what is written on the ticket, not from a new GenAI answer.

What the requester writes in the ticket

`key: value` lines, in the ticket's **description** or in a **comment / work note**. A later comment overrides an earlier value (to correct a request, add a comment with the new line):

```
vm_name: app01
template: rhel9-gold
cpu: 4
memory_gb: 16
```

Key	Meaning	Not given
`vm_name`	the VM's name in vCenter and its host name - or **several names** (`vm_name: web01, web02`): one VM each (below)	**required**
`template`	the template, by its name in vCenter	**required**
`cpu`, `memory_gb`	vCPUs, memory in GB	the template's
`disks_gb`	disk sizes in GB, system disk first: `disks_gb: 120, 200` grows disk 1 to 120 GB (never shrinks it) and adds one 200 GB disk (thin)	the template's disks

Key	Meaning	Not given
`site`	which site: its cluster, datastore, folder (and vCenter) come from `vm_deploy_sites`	the ticket's **Location** field, when it is one of the sites
`cluster`, `datastore`, `folder`	where it goes (`datastore` may be a datastore cluster) - overrides the site's. `folder` as a path below the VM folder (`Linux/App`) is that path exactly; a name alone (`App`) is the one folder of that name; the folder the VM goes in - the template's too, when none is given - must be one of `vm_deploy_folders` (when you set that list)	the site's, else `vm_deploy_defaults`, else the template's own
`domain`	the DNS domain set in the guest (Linux)	`vm_deploy_domain`
`customization_spec`	a customization specification from vCenter (**needed for Windows**)	Linux: none needed
`vcenter`, `datacenter`	which one, when the template exists in several	found by the job
`notes`	added to the VM's notes	-

Other lines (a greeting, `Name: Jane Doe`) are ignored. Upper and lower case do not matter (`Memory: 16 GB` works too), and the names people use work as well: `Hostname:`, `Server Name:`, `Computer name:` (the VM's name; for several, `Hostnames:`, `Server names:` ... with the names after it, or one per line below it); `CPUs:`, `vCPU:`, `Cores:`, `Processors:`; `RAM:`, `Mem:`; `Disk:`, `Storage:`, `HDD:`, `Hard drive:`. A unit is read with the value: `Memory: 16384 MB` = 16 GB, `Storage: 100GB, 1TB` = 100 and 1024 GB, `Memory (GB): 16`.

Requesters who do not know template names or `key: value` lines: switch on **plain words** (`vm_deploy_plain_words`, next section, no GenAI) and/or GenAI (`vm_deploy_genai`, below).

What wins when two places give a value, from the least to the most:

	Wins over the ones above it
1	the parent - the **change** a change task belongs to (its description)
2	the ticket's **short description** (plain words only) and **description**
3	the ticket's **work notes**, oldest first
4	the ticket's **comments**, oldest first

So the **change task wins over its change**, and a **comment wins over a work note** (a requester's answer over a team member's note). A `key: value` line and plain words are ranked the same way: where it is written decides, not how - the task's *"named web02 with 6 vCPU"* wins over the change's `vm_name: web01` line. Two answers in the place that wins (*"4 vCPU, or maybe 8 vCPU"*) are a question, never an earlier place's value. `vm_deploy_parent_wins: true` moves the change above the task's short description and description (rows 1 and 2 swap); the task's work notes and comments still win.

What the job does, in order

1. Reads the ticket (`vm_deploy_ticket`: `INC...`, `TASK...`, `SCTASK...`, `RITM...`, `CHG...`, `CTASK...` - or your instance's own prefixes like `INCN...` / `CHGN...`: the job asks ServiceNow's task table which table the ticket is in). For a change task, give the **task's** number: the job reads the task, then its change for the rest of the request; the change must be approved and in Implement.
2. Checks: the ticket is open and assigned to `vm_deploy_assignment_group`; the template is in `vm_deploy_templates`; the cluster, datastore and folder are allowed; CPU and memory are within the caps; the name is valid and **not used in any vCenter** (`vmware_vcenters`) - or it is this ticket's own VM from an earlier run (below). Anything wrong: a work note on the ticket says what, and nothing is built. Several VMs: every one is checked in vCenter before the first is built.
3. Writes **"[AAP] Deploying this VM"** with the exact request on the ticket - and builds nothing if ServiceNow refuses that note.
4. Clones the template (powered off) with the ticket's mark in the VM's notes (`[site-automation ticket=CTASK0010009 state=building]`), sets CPU, memory and disks, applies **guest customization** (Linux: host name = `vm_name`, every network adapter on DHCP; Windows: the named customization specification, its computer name set to `vm_name`), powers it on.
5. Waits for vCenter's customization result (`vm_deploy_wait_customization`, 900 s) and for an IPv4 address from VMware Tools (`vm_deploy_wait_ip`, 600 s).
6. Marks the VM built in its notes (`state=built`; `state=failed` when its customization failed), and writes **"[AAP] VM app01 deployed"** on the ticket: IP address, customization result, where it went, size. On a failure: what failed, and whether a (partly set up) VM exists.
7. Closes the ticket, when you set what closes it (`vm_deploy_done_fields_by_table`, below).

The network is the template's: the template's adapter is on your staging VLAN, and so is the new VM.

Several VMs on one ticket

One ticket may ask for several VMs: one VM per name, the template, CPU, memory, disks and folder the same for each - unless each server has a line of its own that says its own (*A change laid out per server*, below).

```
I need 2 vms created named web-test01 and web-test02
both servers need:
Windows 2022
8 cpus
12gb of memory
storage 180GB disk and 50gb disk
Store in the App-Servers folder.
```

The names, as people write them: `vm_name: web01, web02` (a line), `Hostnames:` with one name per line below it (`- web01`, `1. web01` ...), and in plain words `named web01 and web02`, `named web01, web02 and web03`, `2 VMs: web01 and web02`.

- **Each VM is checked on its own**: its name (valid, not used in any vCenter), its name rule (so two names may go to two clusters), the caps (`vm_deploy_max_cpu` ... are per VM). One VM refused = none is built: the ticket says why. That holds in vCenter too: **every VM is checked there before the first is built** (read-only: its name is free, its template, cluster, datastore and folder are there, its customization is set up) - *"[AAP] Nothing was built: ... 1 of 3 cannot be: - web03: a VM named web03 already exists"*.
- **How many** - `2 VMs`, `two servers`, `both servers`, `3 virtual machines` - must match the names: *"2 VMs named web01"* is a question to the requester, nothing is built. A range (`web01 to web05`, `web01-05`) is

never expanded: a question too - every name is written out.

- **At most** `vm_deploy_max_vms` **(10) per ticket.** More: refused, nothing built (split the ticket, or raise the setting).
- **Built one after the other.** Each VM is placed after the ones before it are there, so automatic placement sees them. The dry run and the propose step show where each would go as if alone: the build may place them differently.
- **On the ticket:** "[AAP] Deploying 2 VMs, one after the other", then one note per VM, written as soon as that VM is done ("[AAP] VM web01 deployed (...) - 1 of 2."). The ticket is **closed only when every VM is built**.
- **One fails, the others are still built.** The ticket stays open with a note per VM and one that sums it up: *"[AAP] 1 of 2 VMs built; this ticket stays open: - web01: built - web02: NOT deployed - ..."*. Fix the cause and **run the job again**: it picks up where it stopped (next).

Run it again: it picks up where it stopped

A job that stopped part way - a VM failed, the job was cancelled or timed out, AAP restarted - is run again for the same ticket, and **picks up where it stopped**. Each VM carries its ticket in its notes from the clone on (`[site-automation ticket=CTASK0010009 state=building]`, then `state=built` once it is up and its customization result is in), so the job knows its own VMs even when a result note never reached the ticket:

The VM in vCenter	Run again
marked **built** for this ticket	left as it is: reported with its IP address
marked **building** for this ticket (cloned, not finished)	**finished where it is**: the CPU, memory and disks it still lacks, the customization, the power on - never cloned again (already powered on: its result is read)
not there	built
there, but not this ticket's (another ticket's mark, or none)	refused, nothing built: *a VM named web02 already exists*

The work notes say which: *"[AAP] Deploying 3 VMs ... Picking up where an earlier run for this ticket stopped: - web01: built already - left as it is - web02: started, not finished - finished in place - web03: to build"*, then each VM's own note (*"Picked up where an earlier run for this ticket stopped: finished in place."*). The ticket is closed once all of them are built.

- One VM tickets work the same: run again, the VM it built is reported, not refused (and the ticket closed, when close fields are set). To build a test VM again, delete it in vCenter first.
- VMs built by an earlier release (no mark; their notes begin *"Deployed from TEMPLATE for CTASK0010009 by"* - the ticket's number written anywhere else does not count): powered on or with the ticket's *"[AAP] VM web01 deployed"* note - left as they are; powered off - finished in place.
- Built, but its **guest customization failed** (`state=failed`): refused - check it, delete it, run again.
- `vm_deploy_resume: false`: the way of earlier releases - a VM the ticket's own *"[AAP] VM web01 deployed"* note names is left as it is; any other VM with the name is refused, and one started is never finished.
- The PXE / MECM case works the same: each VM gets its own MAC address on its own note.
- **The job takes as long as all its VMs.** A template with guest customization waits up to `vm_deploy_wait_customization` (900 s) and `vm_deploy_wait_ip` (600 s) **per VM**: ten such VMs can take hours. Give the deploy template a **Timeout** that allows it (0 = none), or keep tickets small. Network-boot (PXE / MECM) templates do not wait for either.

Setting	Default	What it does
`vm_deploy_max_vms`	`10`	the most VMs one ticket may ask for

A change request: its task does the work, the change must be approved

A common setup: the **change request** holds the requirements and is approved; the team gets a **change task** (a CTASK - or an SCTASK) that may or may not repeat them; the task is what the job runs on, and only the task is closed when the VMs are up. The change itself is never written to - unless you ask for the notes on it too (`vm_deploy_notes_also`, below); it is never closed.

1. Someone raises the change, the request in its description - in plain words like any ticket (with `vm_deploy_plain_words`), or `key: value` lines.
2. The change is approved and goes to **Implement**; the change task is assigned to your group.
3. The job runs on the **change task's number** - launched by hand, or picked up by the schedule.
4. It reads the task and its change, checks that the change is **still approved and in Implement**, builds, writes one note per VM on the task, and closes the task.

The request from the change - nothing to set:

- The job follows the task's link to its change - the field `change_request` (a change task's), else `parent` (`vm_deploy_parent_fields`) - and reads the change's description (`vm_deploy_parent_text_fields`, e.g. add `implementation_plan` if your changes keep the request there).
- Both the change's `key: value` lines and, with `vm_deploy_plain_words`, its **plain words** are read. What the task says itself wins: a task that only says *"deploy the VM of the change"* takes everything from the change; a task that says `memory_gb: 16` changes only the memory. The dry run says where each value came from: `vm_name: web01, web02 (from the words 'named web01 and web02' in the parent CHG0030001 description)`.
- Only a parent that is a **change** is read (`vm_deploy_parent_tables: [change_request]`; `[]` = never read a parent): an incident's parent incident, or a catalog task's request item, is not.
- The change's comments and work notes are not read (CAB notes are not a request), unless you list them in `vm_deploy_parent_text_fields`. GenAI (`vm_deploy_genai`) reads the change's text too.

The change must be approved and in Implement (`vm_deploy_ready_by_table`, on by default):

```yaml
vm_deploy_ready_by_table:
  change_request: {approval: Approved, state: Implement}
```

- Written as the change's form shows them (upper / lower case does not matter); a list allows any of them: `state: [Implement, Scheduled]`. Your instance's labels may differ - the refusal says what the change has: *"the change CHG0030001 (this ticket's parent) is not ready to build: approval is 'Requested', not Approved; state is 'Assess', not Implement"*.
- It applies to the change a task belongs to, and to a change that is itself the ticket. Other tables can have an entry too, e.g. `change_task: {state: [Open, Work in Progress]}`. `{}` = no check.
- It is checked by the dry run, by the workflow's propose step **and again right before the build**: a change sent back while the AAP approval waited is not built.
- A change that **cannot be read** (the ServiceNow account needs read access to `change_request`), or is **closed or canceled**, stops the build too.

- Not ready = nothing built, a note on the task says why. **Run the job again** once the change is ready (a ticket the schedule picked up is not picked up again).

Close only the task when the VMs are up - on the deploy template, **your instance's values** (the values, not the labels: open a closed change task, right-click its header, *Show XML*, and copy what `<state>` and `<close_code>` hold):

```yaml
vm_deploy_done_fields_by_table:
  change_task: {state: "3", close_code: successful, close_notes: "VM deployed by AAP"}
  sc_task: {state: "3", close_notes: "VM deployed by AAP"}        # 3 = Closed Complete
vm_deploy_done_fields_issues_by_table:          # over those when something did not finish
  change_task: {close_code: successful_issues}
vm_deploy_done_fields: {}        # every other table (an incident): left open
```

- The ticket's **table** picks the entry (`sc_task`, `change_task`, `incident` ...), also with your own number prefixes. The change itself is never closed (and has no notes unless `vm_deploy_notes_also`).
- **Successful with issues:** every VM is built, but something did not finish - no IPv4 address within `vm_deploy_wait_ip`, no customization result, the guest's host name not applied, a network adapter not connected at power on, `guestinfo` not set. The task's work notes say what. (A network-boot VM's *"no operating system yet"* is not an issue.)
- **A VM not built:** the task **stays open** - fix the cause and run the job again: the VMs already built are left as they are, the others are built, then the task is closed. Close it *Unsuccessful* by hand if you give up: a closed ticket is never deployed again.
- The close is sent **after** the result note, on its own, and the job checks what ServiceNow kept: when a mandatory field (a data policy) or a business rule stops it, the VMs are still built, the job stays green, and the report and a work note say **"could not be closed"** with ServiceNow's reason. (A value that is not one of the field's choices is not caught: take the values from *Show XML*.)
- A **network-boot** VM (`vm_deploy_pxe_templates`) is closed when it is powered on and booting to MECM - the OS install is not finished yet. To leave those tickets open for whoever checks the install: `vm_deploy_done_fields_pxe: false`.

Picked up by the schedule - on the pickup template:

```yaml
vm_deploy_poll_tables: [change_task]
vm_deploy_poll_prefix: "VM request"     # how your VM change tasks' short description starts:
  never every task of the group
vm_deploy_poll_query: "change_request.approval=approved^change_request.state=-1"
vm_deploy_poll_states: ["1"]            # the task's own state VALUES to pick up (1 = Open)
```

- The query reaches into the task's change (ServiceNow's *dot-walking*): only tasks whose change is approved (`approved`) and in Implement (`-1`) are picked up. These are **values**, not labels: build the filter in the change task list (*Change request.Approval* is *Approved* ...), then right-click the filter and *Copy query*.
- Tasks that do not start with one phrase, e.g. *Task to deploy a staging VM*: no prefix, and the words in the query - `LIKE` is *contains* (any case), `^OR` adds another wording: `vm_deploy_poll_prefix: ""` and `vm_deploy_poll_query: "short_descriptionLIKEto deploy^ORshort_descriptionLIKEdeploy VM^change_request.approval=approved^change_request.state=-1"`. Search for a phrase, not a short word alone: `LIKE` also finds it inside other words (`app` is in *h**app**y*).

- `vm_deploy_poll_states` are the **task's** state values: a change task in a state your instance added (e.g. one shown as *Implementation*) has its own value - copy it the same way.
- Run it once with `vm_deploy_poll_dry_run: true` (or Check): it lists what it would pick up.

Approvals. The change's approval can stand in for the AAP approval step: for change tasks read by the parser, a workflow with the deploy job alone (no approval node) is enough. With GenAI on, keep the AAP approval - the change was approved for its words, not for GenAI's reading of them.

One change task for all the change's VMs works as it is. With **one task per VM**, give each task its own `vm_name:` line: otherwise each task reads every name from the change, and the second task is refused (the names are taken).

The notes on both, and which wins. By default the **task wins over its change** (*What wins*, above): the change's description gives the request, the task's own text corrects it. When the task may repeat the request wrongly, let the change win instead:

YAML

```
vm_deploy_parent_wins: true                 # only if the change must win over the task's
  description (default: the task wins)
vm_deploy_parent_text_fields: [description, comments]   # an answer written as a comment on the
  change counts too
vm_deploy_notes_also: [parent, tasks]       # every note on the change too - or, launched with the
  change, on its task
vm_deploy_notes_tasks_query: "short_descriptionLIKEto deploy"   # which of the change's tasks
  (with vm_deploy_assignment_group)
vm_deploy_assignment_group_by_table: {change_request: ""}   # launched with the change: it may be
  any group's
```

- **Which wins:** off (the default), the change is read first and the task's text wins over it. On, the task's short description and description are read first, then the change's - which wins where both say something; the task fills in only what the change does not say. Either way a work note on the task wins over both, and a comment over a work note. The dry run says where each value came from (`cpu: 4 (from parent CHG0030010 description)`).
- **The notes on both:** every note and comment the job writes - checked / ready to build, deploying, each VM deployed or not, the questions, a GenAI proposal - goes on the ticket **and**:
- launched with the **change task** (the schedule's pickup): on its change (`parent`);
- launched with the **change** (e.g. from a web front end through AAP's API): on the change's open tasks assigned to `vm_deploy_assignment_group` and matching `vm_deploy_notes_tasks_query` (`tasks`), when there are any - none found: the notes are on the change only, and the job says so.

The job says where its notes go and what it closes: *"Notes on CTASK0010009 and CHG0030010. CTASK0010009 is closed when every VM is built."*

- Only the ticket the job was launched with is **closed** (`vm_deploy_done_fields_by_table`) - and, launched with the **change**, its task too with `vm_deploy_close_tasks: true`: the tasks found as above are closed by the `change_task` entry, when every VM is built (*Task CTASK... closed: ...* in the report; one that does not close gets a note saying so). The change is closed only by an entry of its own: keep `change_request` out of `vm_deploy_done_fields_by_table`, and `vm_deploy_done_fields: {}` (it is for every table without an entry - a change too). A note the other ticket does not take (the ServiceNow account cannot update `change_request`) is a warning in the job, not a failure.
- Launched with the **change**, the change itself must pass the rules for a ticket: approved and in Implement, and assigned to `vm_deploy_assignment_group` - or to **any group** with

`vm_deploy_assignment_group_by_table: {change_request: ""}` (a change is often assigned to whoever owns it, not to your team). Its tasks that get the notes are still found by `vm_deploy_assignment_group` (keep it set), and a ticket launched by itself of any other table (a change task, an incident) must still be your group's. A task that has a note of AAP's on it is never picked up by the schedule, so a change built this way is not built again from its task.

Test it, step by step (a test change and a test template):

1. A change in your test instance, its description in plain words (2 VMs, names, OS, CPU, memory), approval **Requested**, state **Assess**; a change task under it, assigned to your group.
2. Launch the deploy template on the **task's** number with **Check**: *"is not ready to build: approval is 'Requested', not Approved ..."* - the labels your instance uses are in that line.
3. Approve the change and move it to **Implement**. Check again: both VMs, each value *"from the words ... in the parent CHG... description"*, *"ready to build"*.
4. Run it for real: the VMs are built, one note each on the task, the task closed with your close code; the change has no new note.

Sites: site A goes to this cluster, site B to that one

YAML

```
vm_deploy_sites:
  SiteA: {cluster: A-PROD-CL01, datastore: A-DSC01, folder: Servers/New}
  SiteB: {cluster: B-PROD-CL01, datastore: B-DSC01, folder: Servers/New, vcenter:
    vc02.yoursite.mil}
```

The ticket says `site: SiteA` (upper / lower case does not matter) - or nothing, when the ticket's **Location** field is one of the site names. The site's values fill in what the ticket does not say (over `vm_deploy_defaults` and the name rules); a `cluster:` the ticket names itself still wins, within the allow-lists. With sites set up, a ticket that names no site is refused (`vm_deploy_site_required: false` uses `vm_deploy_defaults` instead). Any setting can go in a site: `cluster`, `datastore`, `folder`, `vcenter`, `datacenter`, `domain`, `cpu`, `memory_gb`.

The place as people write it. With plain words on, a site's name in the ticket's text picks it (any case, the spaces optional: `Site Two`, `SiteTwo`, `site-two`) - and so do the words you list for it, for every other way people write the place:

YAML

```
vm_deploy_plain_sites:
  "north campus": SiteA          # "Location: North Campus" (also "NorthCampus") is SiteA
  "nc": SiteA
  "site b": SiteB
```

A change that lists its servers one per line puts each VM at the site **its own line** names (`... / ab01web05 / ... Location: North Campus`). The ticket's Location field is read through the same words.

A site in another vCenter (`vcenter: vc02.yoursite.mil`): that vCenter must be in `vmware_vcenters` (the job's vCenter credential works there too: one account for all), and the template must be there under the same name. Give the site its own `cluster` (and `datacenter` when the names repeat across vCenters).

Name rules: the VM's name picks the cluster

When your names say where a VM belongs (a site or room code in the name), let the name pick the cluster - no `site:` or `cluster:` line needed in the ticket. Put your real cluster names in the deploy template's Variables (or `playbooks/group_vars/all.yml`):

```yaml
vm_deploy_name_rules:
  - {match: "*AB01*", cluster: YOUR-CLUSTER-FOR-AB01}            # every name with AB01 in it
  - {match: "*CD02*", cluster: YOUR-CLUSTER-FOR-CD02, folder: Servers/New}
  - {match: "??EF03*", site: SiteB}                              # 2 characters, then EF03: a
    site's settings
```

- `match`: the name pattern - `*` = any characters, `?` = one character; upper and lower case alike. The **first** rule that fits the name is used.
- The rest of the rule fills in what the ticket does not say: `cluster` (or `clusters: [..]` to choose among), `datastore` (or `datastores: [..]`), `folder`, `vcenter` ... - or `site:` a name from `vm_deploy_sites`. A `cluster:` the ticket gives itself, or a site it names, still wins.
- With a name rule matched, the ticket needs no site (even with `vm_deploy_site_required`).
- **A name used at several sites** (the code in the name is the same at two places): `sites:` lists them, and the ticket must say which - by a site's name or its words (`vm_deploy_plain_sites`), on the VM's own line, or the ticket's Location field. Never a guess: with no site, or one not listed, it is a question - *"the ticket does not say where ab01xy05 goes - SiteA or SiteB: write it on its line, e.g. 'Location: SiteA'"*. Keep the placement (`vcenter`, `cluster` ...) in the sites, not in the rule: `- {match: "*XY05*", sites: [SiteA, SiteB]}`.
- A rule's placement (`vcenter`, `datacenter`, `cluster`, `datastore`, `folder`) is never used for a VM that its site, or the ticket, puts in another vCenter.
- The cluster must still be allowed (`vm_deploy_clusters`, when you set that list).
- Datastore: the rule's, else the site's or `vm_deploy_datastores` candidates, else the template's - and when the template's datastore is not on that cluster, the cluster's own least used datastore (shared, under 85 % with the VM). `vm_deploy_datastore_rules` (SQL servers) wins over all of them.
- The dry run (Check) shows it: `cluster: YOUR-CLUSTER-FOR-AB01 (from name rule *AB01*)`.

Automatic placement: the job picks the cluster and datastore

When neither the ticket, its site nor `vm_deploy_defaults` names a cluster (or a datastore), the job chooses one by rules - never from all of vCenter, only among explicit candidates:

- **Candidates:** a site's `clusters` / `datastores` lists, else `vm_deploy_clusters` / `vm_deploy_datastores`. With neither, the VM goes where the template is.
- **Cluster:** the one with the most memory left after this VM, planning one host down (`vm_deploy_failover_hosts`) - it must stay under `vm_deploy_mem_target_pct` (80 %).
- **Datastore:** shared, accessible, not in maintenance, mounted by that cluster, and under `vm_deploy_datastore_max_used_pct` (85 %) with the VM at its **full** size (thin disks counted full). The **least used first**: the one that is the least full with the VM on it. The VMs of one ticket spread out - each counts what the ones before it in the same run put on a datastore.
- The ticket note and the report give the ranking: `cluster chosen: PROD-A2: memory 61.3% used with this VM and 1 host down (target 80%)`, `datastore chosen: A-DS-03: 41.2% used with this VM (at most 85%)`.

```yaml
vm_deploy_sites:
  SiteA: {clusters: [A-PROD-*], datastores: [A-DS-*], folder: Servers/New}
  SiteB: {clusters: [B-PROD-*], datastores: [B-DS-*], folder: Servers/New, vcenter:
    vc02.yoursite.mil}
```

By location, the least used first - for every VM. The location picks the cluster (a site, a name rule, the ticket); the datastore is then the least used of that cluster's. Without datastore lists, switch on `vm_deploy_datastore_balance: true`: a VM nothing else places goes on the least used shared datastore of its cluster (instead of the template's own). A site's or a rule's single `datastore:` still puts it there.

SQL servers on their own datastores (`vm_deploy_datastore_rules`): the VM's name picks its datastores.

```yaml
vm_deploy_datastore_rules:
  - {match: ["*SQL*", "*DBSE*", "*DBV*"], datastores: ["*SQL*"]}   # e.g. ab01sql01, AB01DBSE02,
    ab01dbv03
vm_deploy_datastore_balance: true                                    # every other VM: the least
  used of its cluster's
```

- The **first** rule whose `match` fits the name (`*` and `?`, any case) gives the datastores the VM goes on: the least used of them on its cluster. That wins over a site's, a name rule's or `vm_deploy_defaults`' datastore; a `datastore:` the ticket writes itself must be one of them (else a question: *"ab01sql01 goes on a datastore *SQL\ ...: A-DS01 is not one"*).
- A VM **no rule matches never goes on a datastore a rule names** - they are kept for those VMs, even when it is the template's own (then: the least used of the cluster's others).
- None of them on the VM's cluster, or none with room: nothing is built - *"no datastore among *SQL\ of A-PROD-CL01 has room for 140 GB"*. The check in vCenter before the build finds that, for every VM.

Requests in plain words, without GenAI

With `vm_deploy_plain_words: true` the job also reads the ticket's own words, by rules you set, so a requester can write:

```
Hi, please build a Red Hat 9 server named ab01web05, medium size, for the new portal. Thanks, Jane
```

What	Read from	Your setting
the template	**the operating system as people write it** - found from the template names in `vm_deploy_templates`, nothing to set: a template named *Windows 2022 Standard Template* is found by `Windows 2022`, `Windows Server 2022`, `Server 2022`, `Win2022`; *rhel9-gold* by `RHEL 9`, `RHEL 9.4`, `Red Hat 9`, `Red Hat Enterprise Linux 9` (also Rocky, Alma, CentOS, Oracle Linux, Ubuntu, Debian, SLES). Windows Server by its year alone: `Server 2025`, `a new 2025 VM`, `2022 servers` (not `SQL Server 2019`). An OS no template is for (`Server 2025`, only a 2022 template) is a question that names it - never another version. Two allowed templates for the same OS (a PXE and a Gold Windows 2022): a question. Also the words you list (`vm_deploy_plain_templates`), the template's own name and its short names (`vm_deploy_template_aliases`). Any case; `-`, `_` or nothing between the words; up to two other words between them	`vm_deploy_templates` with the templates' **exact names** (a pattern like `Windows 20*` has no OS to read: list the names, or use `vm_deploy_plain_templates`)

What	Read from	Your setting
CPU and memory	a size word (`small`, `medium`, `large`), and numbers next to their unit: `4 vCPU`, `4 cores`, `CPU - 4`, `Processors: 4`, `2 sockets, 2 cores per socket` (= 4), `CPU : CORE.4`, `16 GB RAM`, `4 gigs of ram`, `RAM - 16GB`, `RAM : 16 GB.`, `Memory (GB): 16`, `16384 MB of memory` - a bare `16 GB` is not read	`vm_deploy_plain_sizes`
disks	`storage 180gb and 50gb`, `storage 180GB disk and 50gb disk`, `a 180GB disk and a 50GB disk`, `Disk 1: 100 GB` / `Disk 2: 200 GB`, `C: 100GB` / `D: 200GB`, `HDD'S C: /120`, `Disks: C: 100, D: 200` (a drive letter after a disk word: GB when no unit), `a 100GB OS disk and a 200GB data disk`, `hard drive 80 gigs`, `disks 1TB + 500gb` - disk 1 first (below)	-
the folder	`the Web-Servers folder`, `folder: Linux/New`, `in folder App`, `the "App Servers" folder` (a name with spaces in quotes)	-
the VM's name	`named ab01web05`, `called ...`, `hostname: ...`, `server name ...`; several: `named web01 and web02`, `named web01, web02 and web03`, `2 VMs: web01 and web02` (how many, when said, must match: above), `spin up new 2025 VM / web01`, `new VM: web01`, `build server - web01` (each name written so is a new VM: two such lines, two VMs) - or else the one word that fits a name rule (`vm_deploy_name_rules`; longer than the rule's own letters, so a site code alone is not a name). Web and email addresses are skipped	the requester gives it
the site	a site's name (`vm_deploy_sites`) as a word of its own - any case, the spaces optional (`SiteTwo` = `Site Two`) - or the words you list for it; with one server per line, each line's own	`vm_deploy_plain_sites`

YAML

```
vm_deploy_plain_words: true
vm_deploy_plain_templates:
  "rhel 9": rhel9-gold
  "red hat 9": rhel9-gold
  "windows 2022": WIN2022-TEMPLATE
vm_deploy_plain_sizes:
  small: {cpu: 2, memory_gb: 8}
  medium: {cpu: 4, memory_gb: 16}
  large: {cpu: 8, memory_gb: 32}
```

Never a guess. Two different answers in the text that says it last (`RHEL 9 or RHEL 8`, `small or medium`, two server names as in *"same as ab01web03, the new one ab01web06"*) are a **question**, written on the ticket as a **comment** (requesters see comments, not work notes): *"[AAP] We cannot build this VM yet: the ticket names more than one operating system ('rhel 9', 'rhel 8'): which one?"*. Nothing is built. The requester answers in a comment (*"RHEL 8 please"*): a later comment wins. Then run the workflow again (the pickup does not take a

ticket twice).

- **What wins:** `key: value` lines always; then the words; then a GenAI proposal; then the defaults. A size word and a number in the same text: the number (`medium with 32 GB RAM` = 4 vCPU, 32 GB). A later comment beats the description.
- **Every value says where it came from** in the job output, the propose note and the approver's email: `template: rhel9-gold (from the words 'red hat 9' in the description)`.
- **Your rules still apply:** the template the words lead to must be in `vm_deploy_templates`, CPU and memory within `vm_deploy_max_cpu` / `vm_deploy_max_memory_gb`.
- **Disks in words:** a disk word with sizes after it is a list, disk 1 first - `storage 180gb and 50gb` = disk 1 grown to 180 GB plus an added 50 GB disk; one size before a disk word is disk 1 (`180gb hard disk`, `a 100 GB system disk`, `C drive 100gb`), or an added disk when the words say so (`50gb second hard disk`, `a 200 GB data disk`, `D drive 200gb`); a numbered disk or a drive letter says which (`Disk 2: 200 GB`, `D: 200GB`). Two sizes for disk 1, or only added disks, are a question (a `disks_gb:` line answers it - or GenAI, when it is on).
- **CPUs:** sockets x cores per socket is the number of vCPUs (`2 sockets, 4 cores per socket` = 8); `2 sockets` alone = 2. Sockets and cores that do not add up (`2 sockets, 4 cores`: 4 or 8?) are a question.
- **The folder** the words name must exist in the datacenter (and be in `vm_deploy_folders`, when set): the dry run says when it does not.
- **With GenAI on too:** GenAI is asked when the words (and lines) do not give both the name and the template, **or the words left a question** (e.g. disks written in words, or two sizes) - and what the lines and words say wins over GenAI: GenAI fills only what is still open.
- A value is never read across a line break: `memory` at the end of one line and `180gb` on the next are not "180 GB of memory".
- **A change laid out per server** - the servers only in its description, one line each - is read as it is: launch the job with the change's **task** (the one assigned to your group); the task's text is read with the change's under it.

```
1.  Build new web tier :  ab01web05 (front end)
a.  Platform team : spin up new 2025 VM / ab01web05 / CPU :  CORE.4  / RAM : 16 GB.  / HDD'S C:
  /120  Location: Site One
1.  Build new web tier :  ab01web06
a.  Platform team : spin up new 2025 VM / ab01web06 /  CPU :  CORE.4  / RAM : 16 GB.  / HDD'S C:
  /120 Location: Site Two
```

= two VMs (`ab01web05, ab01web06`), 4 vCPU, 16 GB, a 120 GB disk 1, Windows Server 2025; each at the site its own line names (`Site One`, `Site Two`: a site's name or its words, `vm_deploy_plain_sites`) - its vCenter and cluster from the site - else placed by its name rule.

- **Each line is its VM's own**: CPU, memory, disks, the operating system, the site. Lines that differ build VMs that differ (`ab01web05: 4; ab01web06: 8` in the dry run); a value only one line gives, or one written once for all (above the lines, or below them after a blank line), is for every VM - but never a site: a line with no Location takes none from another line.
- Two different values on one line (`CPU : CORE.4 / 8 cpu`) are a question about that VM only.
- A comment or work note with a line for all (`cpu: 8`) wins over every line (a comment over a work note).
- A name rule for a name used at several sites (`sites: [Site One, Site Two]`): a line with no Location, or one no site has, is a question - never the rule's vCenter.
- Test it like any change: a dry run (**Check**) on a test ticket shows each value and where it came from.

Requests in plain words: GenAI reads the ticket (proof of concept)

With `vm_deploy_genai: true`, a ticket does not need the `key: value` lines. A requester can write *"We need a Windows Server 2022 box called winapp05 for SiteA, 8 GB RAM, 2 CPUs, a 120 GB system disk and a 200 GB data disk"*, and ACT (GenAI.mil, the same ACT model key credential) reads it.

GenAI only copies; the job decides. ACT is asked to copy out what the ticket says - each new VM's name, the operating system as written (`Windows Server 2025`), CPU, memory, the disks - with the ticket's words for each. It does not choose, judge or ask: a question it adds anyway is only a note in the job's report, never sent to the requester. The job finds the template from the OS words by the same rules as the plain words, the placement from the name rules or the site, and asks the requester only what it needs and cannot read (a name, an OS no template is for, a value it cannot trace to the ticket).

Propose, approve, build. In the workflow (below), the propose step has ACT read the ticket (in evidence mode: it runs no command, sees no credential and no inventory). The job posts what it read on the ticket - `[AAP] GenAI proposal (ticket text sha256:...)`, each value with the words of the ticket it came from - then checks it and finds where it would go, and emails that to the approver. **Nothing is built** until someone approves; the build step then builds **the posted proposal** - GenAI is not asked again - with every check of a normal build. If something is missing or unclear, the questions go to the requester as a **comment** (they see comments, not work notes) and the workflow stops. A value the ticket gives itself (`cpu: 4` in a comment) always wins.

The guardrails:

Guardrail	What it does
Off by default	`vm_deploy_genai: false`; the `key: value` lines work without GenAI
A person approves	the workflow's approval sits between the proposal and the build; the build step builds the proposal posted on the ticket - never a fresh answer, never unseen (`vm_deploy_genai_approval: false` drops the person, for a demo)
The ticket must not change in between	the proposal - and, with or without GenAI, the propose step's *"[AAP] Checked - ready to build"* note - carries a hash of the ticket text (the change's too); editing the description or adding a comment voids it: the build step builds nothing (*"was changed after the approval"*), and the workflow is run again from the start
Every value traceable to the ticket	each value comes with the ticket's words it is from; the job checks those words are in the ticket and support the value: the name word for word, the numbers as written, the **site named in its words**, the **template for the operating system its words (or, when they name none, the ticket) ask for** - anything else is dropped and becomes a question. GenAI's own confidence is not trusted
The template by the job's rules	the operating system GenAI copies gives the template by the plain words' rules; it may also name a key of `vm_deploy_template_catalog` (optional), which must be in `vm_deploy_templates` - a key for another version than the ticket asks for (2022 for *Server 2025*) is dropped. An OS no template is for is a question, never another version
Same rules as any request	allow-lists, CPU / memory / disk caps, name rules, the group, a free name: what the ticket text asks for ("use 64 CPUs") is refused like a typed request
No placement by GenAI	cluster and datastore come from the site, the defaults or the rules above

Guardrail	What it does
Instructions in the ticket are data	the prompt tells the model to ignore them; the two checks above make them harmless anyway
Everything on the record	the proposal, the questions, the "Deploying" note and the result are on the ticket; the job's artifacts keep what GenAI read

Set it up:

1. The deploy job template (Set it up, above) gets the **ACT model key** credential too, and **Prompt on launch** ticked next to **Variables** (the workflow hands each step its `vm_deploy_phase`). Its Variables gain:

```yaml
vm_deploy_genai: true
vm_deploy_template_catalog:      # optional: when the template names do not say their OS
  win2022-pxe: "Windows Server 2022 (installed over the network by MECM)"
  rhel9-gold: "Red Hat Enterprise Linux 9"
```

2. **Templates > Create template > Create workflow job template:** name `VM - deploy from ServiceNow ticket (workflow)`; a **survey** with one question, *Ticket number*, variable `vm_deploy_ticket` (the workflow passes it to every step). Save, then open the **Workflow Visualizer**:
   - **Start › node 1:** job template `VM - deploy from ServiceNow ticket`; its prompt, Variables: `vm_deploy_phase: propose`
   - **node 1 › (On success) › node 2: Approval**, name `Build the VM on the ticket?`, a timeout of, e.g., 8 hours
   - **node 2 › (On success = approved) › node 3:** job template `VM - deploy from ServiceNow ticket`; Variables: `vm_deploy_phase: build`
   - Nothing on *On failure*: a question for the requester, a refusal or a denied approval ends it.
3. Give the approvers the workflow's **Approve** role (the workflow > **User access** > **Add roles**), and `report_email_to` their address on the job template: the propose step's email is what they read.
4. Launch the workflow with a test ticket. The proposal and "ready to build" are on the ticket and in the email; approve in **Automation Execution > Workflow Approvals**; the VM is built.

The same workflow works for tickets with `key: value` lines: step 1 then only checks and places.

Without the workflow (a demo): launch the job template itself with `vm_deploy_genai_approval: false` - one run reads, checks and builds. With `vm_deploy_genai_approval: true` and no workflow, the job proposes and stops; launching it again builds the posted proposal.

A dry run (Check) with GenAI: by default Check does not ask GenAI (it says *GenAI would read the ticket (dry run: not asked)*). With `vm_deploy_genai_in_check: true` on the template (or as an extra variable), Check asks it too and shows its proposal - each value with the ticket's words, what was dropped, the questions it would send - and where the VM would go. It writes nothing on the ticket: no proposal note, no comment to the requester; a real run asks GenAI again.

The ticket text goes to the model (GenAI.mil): only use it for tickets whose text may go there.

Empty templates: network boot to MECM, or boot from a CD (kickstart)

A template with **no operating system** that installs it itself - it boots from the network into an MECM (SCCM) task sequence, or from its CD / ISO into a kickstart: list it in `vm_deploy_pxe_templates` (e.g. `["Windows 20*", "Rhel*"]`). For those, the job:

- names the VM in vCenter as the ticket says (`vm_name`), like every deploy;
- does **no guest customization** (there is no OS yet) and does not wait for an IP address;
- refuses, for a Windows guest, a `vm_name` longer than 15 characters (the Windows / MECM computer name limit);
- gives the VM the ticket's name and number as VM settings the installer can read - `guestinfo.hostname` and `guestinfo.ticket` (`vm_deploy_guestinfo`; vCenter privilege *Change configuration* > *Advanced configuration* - without it the VM is still built and a note on the ticket says the name was not set there);
- powers the VM on - it boots to MECM or its CD - and writes on the ticket its **MAC address** and the computer name to use.

The name inside the OS comes from the installer, not from vCenter:

- **A kickstart (Linux, from the CD):** read the name in `%pre` and use it, for example

```
%pre
name=$(vmtoolsd --cmd "info-get guestinfo.hostname" 2>/dev/null)
[ -n "$name" ] && echo "network --hostname=$name" > /tmp/hostname.ks
%end
%include /tmp/hostname.ks
```

This needs VMware Tools in the installer (`vmtoolsd`, from open-vm-tools); check your ISO (boot it, press Ctrl+Alt+F2 and type `which vmtoolsd`). Without it: a DHCP reservation for the MAC address on the ticket gives the name, or whoever finishes the install sets the name from the ticket.

- **Windows (MECM, from the network):** the name comes from MECM, not from `guestinfo` - Windows PE has no VMware Tools to read it. Use one of the two MECM ways below.

On the MECM side, one of the two usual ways:

1. **Unknown computers:** the task sequence is deployed to *All Unknown Computers* and asks for (or sets) the computer name - use the VM's name from the ticket.
2. **Known computers:** an MECM admin imports the computer (**Assets and Compliance > Devices > Import Computer Information**: name = `vm_name`, MAC = the one on the ticket) into the collection the task sequence is deployed to. (This step can be automated later through MECM's AdminService.)

Network boot first, the disk after the install - the template:

- **Boot order: leave VMware's default, the hard disk first and the network after it.** The empty disk is skipped, so the first boot goes to the network (PXE) and MECM. Once the task sequence has installed Windows, the disk can boot and starts first (with EFI, Windows Setup also puts *Windows Boot Manager* first in the firmware). Nothing to change after the install. With the network first, every restart would ask MECM again.
- **VM Options > Boot Options > Failed Boot Recovery:** on, 10 seconds - a network boot MECM did not answer (e.g. the computer is not imported yet) is tried again instead of stopping at "no operating system".
- **Firmware EFI** (Secure Boot may stay on: MECM's boot images are signed by Microsoft).
- **The adapter:** on the staging VLAN, *Connect At Power On* ticked, and the VLAN forwards PXE to your MECM distribution point (DHCP / IP helper). A **VMXNET3** adapter needs VMware's vmxnet3 driver in the MECM **boot image** (Windows PE has none: it starts, then the task sequence finds no network) - MECM: import the driver (from VMware Tools, `Drivers\vmxnet3`), add it on the boot image's **Drivers** tab and update the distribution points. An **E1000E** adapter works without.

- **The disk:** empty, never initialized.

On the MECM side: the task sequence deployment **Required**, available to *Configuration Manager clients, media and PXE* (Available would wait for someone to press F12). After it succeeded, MECM does not start it again on that computer; to rebuild the same VM: the device in MECM > **Clear Required PXE Deployments**.

For a template that **boots from its CD**: the CD drive must point at the ISO on a **datastore every target cluster's hosts can see** (a clone in a cluster that cannot see it boots to nothing), with *Connect At Power On* ticked, and the **hard disk before the CD** in the boot order (VMware's default): the empty disk is skipped, so the first boot starts the ISO, and once installed the VM starts from its disk. With the CD first it can start the installer again, unless the kickstart ends with `reboot --eject`.

A dry run (Check) reads the ticket, prints the request with where each value came from (`cpu: 4 (from comment 2026-10-08 10:05 - Jane Doe ...)`), checks everything and says where the VM would go. It builds nothing and writes nothing on the ticket. Run it first on every new kind of request.

Find the names first: the deploy settings finder

Not sure what to write for `cluster:`, `datacenter:`, `vcenter:`, the template or a folder? Run the read-only **VM - deploy settings finder** first. For every vCenter (the credential's and `vmware_vcenters`) and datacenter it lists:

- the **clusters and standalone hosts**: hosts usable, memory used, the shared datastores with their free space (the ones automatic placement can use), the networks;
- the **templates**: where each one is (datacenter, cluster, folder), guest OS, CPU, memory, disks, network;
- the **VM folders**, as paths below the datacenter (what `folder:` takes), and the customization specifications;
- what each deploy setting means, with an example from your vCenter;
- what to look at first: a template in several datacenters (each name rule then needs its `datacenter:`), a datacenter with no template, a cluster with no shared datastore;
- **the lines to paste** into the deploy template's Variables: `vm_deploy_templates`, the template catalog for GenAI, and one `vm_deploy_name_rules` line per cluster with its datacenter (and vCenter, with several). Change `CHANGE-ME` to the part of the VM names that goes to that cluster, and delete the lines you do not need.

Set it up: Templates > Create template > Create job template: Name `VM - deploy settings finder`; Inventory: any; Playbook `playbooks/vm_deploy_inventory.yml`; Execution environment: your VMware one; Credentials: **VMware vCenter** (read-only is enough). Optional Variables: `report_email_to` (the email has the same tables), `vmware_datacenter` (one datacenter only). **Launch.** The paste lines are at the end of the output (and in the email, and in the job's artifacts as `vm_deploy_inventory.paste`). It changes nothing.

Set it up

1. **The template:** VMware Tools (open-vm-tools on Linux) installed and running; for Linux with cloud-init, VMware guest customization allowed (`disable_vmware_customization: false`); its network adapter set to **Connect at power on**.
2. **The vCenter account** needs, on the template, the target cluster, datastore(s), folder(s) and the staging port group: *Virtual machine* > *Provisioning* > *Deploy template, Customize guest, Read customization specifications*; *Virtual machine* > *Inventory* > *Create from existing*; *Virtual machine* > *Change configuration* > *Change CPU count, Change memory, Set annotation, Add new disk, Extend virtual disk, Advanced configuration*; *Virtual machine* > *Interaction* > *Power on*; *Resource* > *Assign virtual machine to resource pool*;

Datastore > *Allocate space*; *Network* > *Assign network*. A missing one is named in the error (`lacks the privilege ...`). **A template with a vTPM or encrypted disks** (Windows Server 2022 / Windows 11 templates often have a vTPM, and a vTPM makes the VM encrypted) also needs *Cryptographic operations* > *Clone* (on the template), *Encrypt new* and *Register VM* (on the target VM folder), *Add disk* (on the VMs - for a ticket that adds a disk) and, only while the hosts' encryption mode is not on yet, *Register host* (on the clusters). Not *Manage keys*, *Manage KMS*, *Decrypt* or *Direct Access*. The hosts must reach the key provider (Native Key Provider or KMS) the template's keys come from.

3. **The ServiceNow API credential** (the one the ticket jobs use): its account reads the ticket's table (incident, task ...) and adds work notes to it.
4. **Templates > Create template > Create job template:**
   - Name: `VM - deploy from ServiceNow ticket`
   - Inventory: any. Project: site-automation. Playbook: `playbooks/vm_deploy_from_ticket.yml`
   - Execution environment: your VMware one. Credentials: **VMware vCenter** and **ServiceNow API**
   - Tick **Prompt on launch** next to **Job type** (for the dry run)
   - **Survey:** one question, *Ticket number*, variable `vm_deploy_ticket`, text, required
   - Variables (the rules - a ticket cannot go beyond them):

YAML

```
vm_deploy_templates: [rhel9-gold, rhel8-gold]      # REQUIRED: only these are deployed (* allowed)
vm_deploy_assignment_group: Linux Operations       # only tickets assigned to this group
vm_deploy_clusters: [PROD-*]                       # [] = any
vm_deploy_defaults: {cluster: PROD-CL01, datastore: DSC-PROD, folder: Linux/New}
vm_deploy_max_cpu: 16
vm_deploy_max_memory_gb: 128
vm_deploy_domain: yoursite.mil
report_email_to: vmteam@yoursite.mil
```

5. **First run:** a test ticket assigned to your group with the four lines above; launch as **Check**, read the request and where it would go; then launch it for real.

Setting	Default	What it does
`vm_deploy_templates`	`[]`	the templates a ticket may ask for (**required**: nothing is deployed until set)
`vm_deploy_clusters` / `_datastores` / `_folders`	`[]` (any)	where a ticket may put the VM
`vm_deploy_defaults`	`{}`	what a ticket does not say (cluster, datastore, folder, cpu, memory_gb, domain)
`vm_deploy_max_cpu` / `vm_deploy_max_memory_gb`	`16` / `128`	caps
`vm_deploy_name_regex`	letters, digits, `-`	the names allowed (`'^[a-z0-9-]{1,15}$'` for Windows-safe names)
`vm_deploy_assignment_group`	`""` (any)	only tickets assigned to this group - or to one of a list: `["Group A", "Group B"]`

Setting	Default	What it does
`vm_deploy_assignment_group_by_table`	`{}`	the same for a ticket of one table, in its place: `{change_request: ""}` = launched with a change, any group (its tasks: still `vm_deploy_assignment_group`'s)
`vm_deploy_domain`	`localdomain`	the DNS domain Linux customization sets
`vm_deploy_customize` / `vm_deploy_power_on`	`true` / `true`	guest customization; power on
`vm_deploy_wait_customization` / `vm_deploy_wait_ip`	`900` / `600`	seconds to wait for each
`vm_deploy_done_fields`	`{}`	fields set on the ticket when the VM is up, e.g. `{state: "6", close_code: "Solution provided", close_notes: "VM deployed"}`
`vm_deploy_done_fields_by_table`	`{}`	the same per table, in place of `vm_deploy_done_fields` for it: `{change_task: {state: "3", close_code: successful, close_notes: "VM deployed by AAP"}}` - close the task, never the change ("A change request: its task does the work")
`vm_deploy_done_fields_issues_by_table`	`{}`	over those when every VM is built but something did not finish (no IP in time, no customization result ...): `{change_task: {close_code: successful_issues}}`
`vm_deploy_done_fields_pxe`	`true`	close it also for a network-boot VM (still installing its OS from MECM); `false` = leave that ticket open
`vm_deploy_parent_wins`	`false`	`true`: the change's description wins over its task's short description and description (the task's work notes and comments still win); `false`: the task wins over its change
`vm_deploy_notes_also`	`[]`	every note on these too: `parent` (the change of a task), `tasks` (launched with a change: its open tasks of the group); never closed
`vm_deploy_close_tasks`	`false`	launched with a change: close its tasks found for the notes, by `vm_deploy_done_fields_by_table`'s `change_task` entry - never the change

Setting	Default	What it does
vm_deploy_notes_tasks_query	""	which of the change's tasks get the notes, e.g. short_descriptionLIKEto deploy (with vm_deploy_assignment_group)
vm_deploy_notes_tasks_limit	50	the most change tasks that get the notes (and are closed); a change with more: a warning, the first ones only
vm_deploy_parent_tables / _fields / _text_fields	[change_request] / [change_request, parent] / [description]	read the request from the ticket's parent change too (its lines, and its plain words with vm_deploy_plain_words), under the ticket's own
vm_deploy_ready_by_table	{change_request: {approval: Approved, state: Implement}}	the fields a ticket - or the change it belongs to - must have before anything is built (as the form shows them); checked again right before the build; {} = no check
vm_deploy_ticket_table	from the prefix	the table for a number whose prefix the job does not know (a table of your own); a known prefix keeps its table - a CTASK... is always a change task
vm_deploy_ticket_prefixes	{}	your instance's own number prefixes, e.g. {INCN: incident, CHGN: change_request} - rarely needed: the job asks ServiceNow which table a ticket is in
vm_deploy_sites	{}	per site: cluster, datastore, folder, vcenter ... (a ticket's site: or Location picks one)
vm_deploy_name_rules	[]	the VM's name picks the cluster (or site): [{match: "*AB01*", cluster: ...}], the first match wins; sites: [A, B] = at either site, the ticket says which
vm_deploy_site_required	true	with sites set up, a ticket must name its site
vm_deploy_pxe_templates	[]	empty templates that install their OS themselves - boot from the network (PXE / MECM) or from their CD (kickstart): no customization, MAC and name on the ticket

Setting	Default	What it does
`vm_deploy_guestinfo`	`true`	a VM from an empty template (`vm_deploy_pxe_templates`) gets `guestinfo.hostname` (the ticket's VM name) and `guestinfo.ticket` as VM settings, for its installer to read (`vmtoolsd --cmd "info-get guestinfo.hostname"`). Without the *Advanced configuration* privilege the VM is still built, with a note
`vm_deploy_template_aliases`	`{}`	short names for long template names: `{win2022: "Windows 2022 Standard Template - Site 1"}` - the ticket (and GenAI's catalog) may write `win2022`. A template's own name in other case or spacing is matched too
`vm_deploy_template_specs`	`{}`	the customization specification per template (names or patterns): `{"Windows 2022*": Win2022-Sysprep}` - a Windows template that is not network-boot needs one
`vm_deploy_max_disks` / `vm_deploy_max_disk_gb` / `vm_deploy_max_total_disk_gb`	`5` / `2048` / `4096`	disk caps: how many (disk 1 included), the biggest one, all together - e.g. `vm_deploy_max_disks: 8` on the template
`vm_deploy_mem_target_pct` / `vm_deploy_failover_hosts`	`80` / `1`	automatic placement: a cluster fits when memory with this VM and that many hosts down stays under the %
`vm_deploy_datastore_max_used_pct`	`85`	automatic placement: a datastore fits when it stays under this % with the VM at its full size
`vm_deploy_datastore_rules`	`[]`	datastores by the VM's name, e.g. `[{match: ["*SQL*", "*DBSE*", "*DBV*"], datastores: ["*SQL*"]}]`: those VMs on the least used of them, every other VM never there
`vm_deploy_datastore_balance`	`false`	`true`: a VM nothing else places goes on the least used shared datastore of its cluster (not the template's)
`vm_deploy_resume`	`true`	run again for the same ticket: its VMs built are left, one started is finished in place (the ticket's mark in the VM's notes); `false`: a VM with the name is refused

Setting	Default	What it does
`vm_deploy_plain_words`	`false`	read the ticket's own words by your rules (no GenAI): template words, size words, `named ...`
`vm_deploy_plain_templates`	`{}`	the words for each template, e.g. `{"rhel 9": rhel9-gold, "windows 2022": WIN2022-TEMPLATE}`
`vm_deploy_plain_sizes`	`{}`	size words, e.g. `{small: {cpu: 2, memory_gb: 8}, medium: {cpu: 4, memory_gb: 16}}`
`vm_deploy_plain_sites`	`{}`	the words for each site, as people write the place: `{"north campus": SiteA}` (a site's own name counts too)
`vm_deploy_genai`	`false`	GenAI reads requests written in plain words (propose, approve, build)
`vm_deploy_genai_in_check`	`false`	a dry run (Check) asks GenAI too and shows its proposal and where the VM would go - nothing written on the ticket
`vm_deploy_genai_approval`	`true`	a GenAI proposal waits for a person (the workflow's approval); `false` = one run reads, checks and builds
`vm_deploy_phase`	`""`	the workflow's steps: `propose` (read, check, place - build nothing), `build` (build what the ticket says - never ask GenAI); `""` = the job alone
`vm_deploy_template_catalog`	`{}`	optional: templates GenAI may name by key (the template's exact name), each with a plain description - the job finds the template from the OS words without it
`vm_deploy_genai_model_timeout`	`site_act_model_timeout` (600)	seconds the model may take

All settings of the VMware jobs, with defaults: VARIABLES_REFERENCE.md.

Who can change the request: anyone who can comment on the ticket can change its lines, and an incident or a task has no approval of its own: the person who launches the job is the control, and the "Deploying" note records exactly what was built. When tickets are picked up by a schedule (below), the workflow's approval is that control: a person approves every VM before it is built.

Pick up tickets automatically (a schedule)

`playbooks/vm_deploy_poll.yml` runs on a schedule - every 15 minutes, for example - and starts the deploy workflow above for each new VM request in ServiceNow. Nobody has to type ticket numbers; nobody's approval is skipped.

How it works, each run:

1. It asks ServiceNow for tickets that are **active**, in the states `vm_deploy_poll_states` (New), **assigned to** `vm_deploy_assignment_group`, and **marked as VM requests**: the short description starts with `vm_deploy_poll_prefix` ("VM request", in any case), and/or they match `vm_deploy_poll_query` (e.g. `category=vm_request`). Sorted oldest first.
2. It skips every ticket that already has a note of AAP's on it (`[AAP] ...`: picked up, a proposal, a question, "Deploying", "deployed"): AAP has it already.
3. For each of the others, at most `vm_deploy_poll_max` (3) per run:
   - it writes `[AAP] Picked up by AAP job ... - starting the workflow ...` on the ticket, and reads the ticket back to make sure the note shows (if the account could not see it, the next run would pick the same ticket up again - so the job stops there, red);
   - it starts the workflow `vm_deploy_poll_workflow` with the ticket number (`vm_deploy_ticket`);
   - it writes the workflow's job number and link on the ticket.
4. The workflow then does what it does when you start it by hand: reads the ticket, asks GenAI if needed, checks the rules, finds where the VM goes, writes it on the ticket, emails the approver - and builds only after the **approval**.

The job itself builds nothing, and it never changes a ticket other than adding those two notes (and `vm_deploy_poll_claim_fields`, if you set them). It is green when it ran - also when there was nothing new - and red when ServiceNow or AAP did not answer or a workflow could not be started. It emails (`report_email_to`) only when it started something or something failed, not every 15 minutes.

The guardrails:

Guardrail	What it does
The approval stays	the job only starts the workflow; a person approves every VM before it is built
Only marked tickets	the group is required, and a VM request must be marked (the prefix or the query): an ordinary ticket of the group is never picked up. A query with `^NQ` (a second query that would bypass the group) is refused
A limit per run	`vm_deploy_poll_max` (3): a flood of tickets cannot start 50 workflows; the rest wait for the next run
Once per ticket	the "Picked up" note, read back before the workflow starts; a ticket with any note of AAP's is never picked up again
An off switch	`vm_deploy_poll_enabled: false`: the schedule runs, picks up nothing and says so
A dry run	`vm_deploy_poll_dry_run: true` (or Job type Check): lists what it would pick up - no note, no workflow, no email
Its own AAP user	the job starts the workflow as an AAP user that may do nothing else

Set it up:

1. **The workflow works by hand first:** "Requests in plain words", step 2 above, tested with a ticket. Its survey asks for `vm_deploy_ticket` - the pickup hands the ticket number over that way.
2. **An AAP user that may only start the workflow: Access Management > Users > Create user**, e.g. `svc_aap_pickup` (a normal user, not an administrator). Then open the workflow > **User access** > **Add**

roles > that user > the **Execute** role (WorkflowJobTemplate Execute). Nothing else.

3. **Its credential: Automation Execution > Infrastructure > Credentials > Create credential**, type **Red Hat Ansible Automation Platform** (built in): **Red Hat Ansible Automation Platform** = AAP's own address (e.g. `https://aap.yoursite.mil`, the one in your browser), the user name and password from step 2 (or a token of that user, then no password), **Verify SSL** ticked. The job gets them as `CONTROLLER_HOST`, `CONTROLLER_USERNAME`, `CONTROLLER_PASSWORD` (or `CONTROLLER_OAUTH_TOKEN`); the `AAP_*` names newer AAP versions add work too. If the execution environment does not trust AAP's certificate, set `vm_deploy_poll_aap_ca_path` to its CA file rather than unticking Verify SSL.
4. **Templates > Create template > Create job template:**
   - Name: `VM - pick up deploy tickets`
   - Inventory: any. Project: site-automation. Playbook: `playbooks/vm_deploy_poll.yml`
   - Execution environment: any (it only talks to ServiceNow and AAP)
   - Credentials: **ServiceNow API** and the **Red Hat Ansible Automation Platform** one from step 3
   - Tick **Prompt on launch** next to **Job type** (for a dry run by hand)
   - Leave **Concurrent jobs** unticked (two runs at once could pick up the same ticket)
   - Variables:

```yaml
vm_deploy_assignment_group: Windows Operations     # REQUIRED - the same group as the deploy job
vm_deploy_poll_prefix: "VM request"                # tickets whose short description starts with
  it
vm_deploy_poll_max: 3
vm_deploy_poll_dry_run: true                       # the first week: list only
report_email_to: vmteam@yoursite.mil
```

5. **Launch** it once as **Check**: it lists the tickets it would pick up and says whether AAP can start the workflow (`AAP: workflow "..." (id ...) can be started`).
6. **The schedule:** open the template > **Schedules** > **Create schedule**: a name (`every 15 minutes`), a start time, **Repeat frequency** Minute, every **15**. Save.
7. **A week of dry runs:** read the jobs' output - are these the tickets you expect? Then delete `vm_deploy_poll_dry_run: true` from the Variables. From the next run, tickets are picked up.

Tell the requesters to start the short description with **VM request** (or ask the ServiceNow team for a category or a catalog item, and use `vm_deploy_poll_query`).

Setting	Default	What it does
`vm_deploy_poll_enabled`	`true`	the off switch
`vm_deploy_poll_tables`	`[incident]`	the tables to look in (`incident`, `sc_task`, `task` ...)
`vm_deploy_poll_prefix`	`VM request`	a VM request's short description starts with it (any case)
`vm_deploy_poll_query`	`""`	an extra ServiceNow encoded query, ANDed in (e.g. `category=vm_request`)
`vm_deploy_poll_states`	`["1"]`	the ticket states picked up (`1` = New)

Setting	Default	What it does
`vm_deploy_poll_max`	`3`	the most workflows started per run
`vm_deploy_poll_scan`	`100`	the most tickets read per table and run, oldest first
`vm_deploy_poll_workflow`	`VM - deploy from ServiceNow ticket (workflow)`	the workflow started per ticket
`vm_deploy_poll_dry_run`	`false`	list only: no note, no workflow
`vm_deploy_poll_claim_fields`	`{}`	fields set with the "Picked up" note, e.g. `{state: "2"}` (In Progress)
`vm_deploy_poll_aap_ca_path`	`""`	a CA file for AAP's own certificate

When it goes wrong:

Message	What to do
`attach a credential of type "Red Hat Ansible Automation Platform"`	step 3, on this job template
`AAP refused the login (HTTP 401)`	the credential's user name and password (or token)
`no workflow named "..." that the AAP user may see`	the workflow's exact name in `vm_deploy_poll_workflow`, and the Execute role (step 2)
`takes no ticket number`	the workflow's survey question `vm_deploy_ticket` (or Prompt on launch for Variables)
`AAP did not answer at ...`	AAP's address in the credential; its certificate (`vm_deploy_poll_aap_ca_path`)
`vm_deploy_assignment_group is empty` / `nothing marks a VM request`	set the group, and the prefix or the query
`STOPPED - this account cannot see the work notes it writes`	the ServiceNow account needs to read work notes (the itil role); or set `vm_deploy_poll_claim_fields: {state: "2"}` so a picked-up ticket leaves the New state. Until then `vm_deploy_poll_enabled: false`
`NOT STARTED: ... AAP did not start it`	the reason is on the ticket too; fix the workflow (its templates, inventory, credentials), then start it by hand with that ticket number - the pickup does not try a ticket twice
a ticket is never picked up	the job's output lists every ticket it read and why it skipped it; a ticket not listed at all did not match the query (the `Query:` line): its group, its state, its short description
`only the oldest 100 were read`	tickets AAP already has stay New and count too: set `vm_deploy_poll_claim_fields: {state: "2"}`, or resolve them

Launch it from another tool (AAP's API)

A web front end (or any tool that can send HTTPS) starts the **workflow** with the change's (or the task's) number - the only answer of its survey; the request stays on the ticket, the tool needs no ServiceNow or vCenter access:

```
POST https://<aap>/api/controller/v2/workflow_job_templates/<id>/launch/        (AAP 2.4:
  /api/v2/...)
Authorization: Bearer <a token of an AAP user that may only Execute this workflow>
Content-Type: application/json

{"extra_vars": {"vm_deploy_ticket": "CHG0031234"}}
```

AAP answers `201` with the workflow job's `id`. With the survey on, AAP drops every variable that is not one of its questions: the tool cannot change the rules. What the tool reads back:

What	Where
running, waiting, successful, failed	`GET .../workflow_jobs/<id>/` (`status`)
each step and its job (propose, the approval, build); the approval waits as `workflow_approval`, `pending`	`GET .../workflow_jobs/<id>/workflow_nodes/`
the result, as data: `artifacts.vm_deploy` - `ticket`, `built`, `waiting_for_approval`, `ok`, `error`, `spec` and `sources`, `vms` (each: name, `ok`, `error`, and `vm`: vCenter, datacenter, cluster, IP, MAC, disks, power)	`GET .../jobs/<job id>/`
the report as text, as the job prints it: `artifacts.vm_report.lines`	the same
the whole output	`GET .../jobs/<job id>/stdout/?format=txt`
approve or deny the plan (an AAP user allowed to approve)	`POST .../workflow_approvals/<id>/approve/` (`/deny/`)

The ticket's work notes stay the record (with `vm_deploy_notes_also: [parent, tasks]`, on the change and its task). A request the rules refuse fails its job with the reason in the output (*Not deployed: ...*) and on the ticket. A workflow cannot run as Check: for a preview, launch the job template `VM - deploy from ServiceNow ticket` with `"job_type": "check"` (Prompt on launch for Job type).

ESXi security settings (every night): SSH off, timeouts, lockdown

One job, `playbooks/esxi_security.yml`, sets these on every ESXi host, through vCenter:

What	Setting	Default
SSH off: the service stopped, and set to start manually	`esxi_security_ssh`	`disabled` (`enabled`, or `""` = leave it)
ESXi Shell off (the host's local shell, a separate service from SSH): the same	`esxi_security_shell`	`disabled` (`enabled`, or `""` = leave it)

What	Setting	Default
Timeouts at 600 s (10 minutes): SSH and the ESXi Shell stop by themselves 600 s after someone starts them (`UserVars.ESXiShellTimeOut`), and an idle SSH or shell session is logged out after 600 s (`UserVars.ESXiShellInteractiveTimeOut`)	`esxi_security_settings`	those two, at `600`
Lockdown mode normal: the host is managed only through vCenter; the DCUI (the host's console) still works	`esxi_security_lockdown`	`normal` (`disabled`, or `""` = leave it). **strict is refused**, and a host already in strict is left alone

It changes **only what differs**: a host that already matches is not touched. So it is safe to run every night: it puts back whatever someone changed (SSH turned on, a timeout changed, lockdown turned off) and the email says which host and what. A dry run (Check) changes nothing and lists what it would change. Hosts not connected to vCenter are skipped and listed.

Another timeout? `esxi_security_settings` is a list of advanced settings, name: value. Add or remove lines, e.g. the DCUI and Host Client idle timeouts:

YAML

```yaml
esxi_security_settings:
  UserVars.ESXiShellTimeOut: 600
  UserVars.ESXiShellInteractiveTimeOut: 600
  UserVars.DcuiTimeOut: 600
  UserVars.HostClientSessionTimeout: 600
```

Before the first real run: who logs in to the hosts directly?

Lockdown mode **blocks direct logins** to the hosts: SSH, the Host Client (`https://<host>/ui`), and any tool that connects to a host instead of vCenter. Most often that is the **vulnerability scanner** with ESXi credentials, and sometimes a monitoring or backup tool. Ask those teams first. For each such account, add it to the host's **exception users**: the job adds them **before** it turns lockdown on, and never removes one.

YAML

```yaml
esxi_security_lockdown_exception_users: [svc_scanner, svc_monitor]
```

Exception users must be **local accounts on the host**, or **Active Directory accounts with their own permission on the host**. A `vsphere.local` account or an AD group does not work. Your vCenter service account for AAP is not affected: lockdown keeps vCenter's access.

Set it up

1. **The vCenter account** needs the privileges in "Before you start", step 2 (the ESXi security settings row), on the hosts or their cluster.
2. **Templates > Create template > Create job template:**
   - Name: `ESXi - security settings`
   - Inventory: any (e.g. your VMware one). Project: site-automation. Playbook: `playbooks/esxi_security.yml`
   - Execution environment: your VMware one. Credentials: your **VMware vCenter** credential
   - Tick **Prompt on launch** next to **Job type** (for the dry run) and next to **Variables** (to try it on one host)

- Variables: `report_email_to: vmteam@yoursite.mil` and, if you have them, `esxi_security_lockdown_exception_users: [...]`
- Save.

3. **Dry run:** Launch, Job type **Check**. Read the report: every host and what it would change.
4. **One host for real:** Launch, Job type **Run**, Variables `esxi_security_hosts: [esx01.yoursite.mil]`. Check the host in the vSphere Client: **Configure > System > Services** (SSH and ESXi Shell stopped, policy Start and stop manually), **Advanced System Settings** (the two timeouts), **Security Profile > Lockdown Mode** (Normal). Have the scanner team run a scan of it.
5. **All hosts:** Launch again without `esxi_security_hosts`.
6. **Every night at midnight:** the template's **Schedules** tab > **Create schedule**:
   - Name: `Nightly 00:00`
   - Start date: tomorrow, start time **00:00**
   - **Time zone: yours** (e.g. `America/New_York`). AAP does not assume your local time: with the wrong zone, "midnight" runs at another hour.
   - Repeat: **Daily**, every 1 day. No end. Save.

The nightly email comes only when a host was changed or failed (an empty night sends nothing); `esxi_security_email_only_if_changed: false` emails every night.

A host that needs SSH (a support case)

The nightly run would turn SSH off again. Exempt the host while the case is open, in the template's Variables:

```yaml
esxi_security_exclude_hosts: [esx07.yoursite.mil]
```

Then turn SSH on in the vSphere Client (and lockdown off if the vendor needs the Host Client). Remove the line when the case is closed: the next night sets the host back. To **undo** on purpose for every host: `esxi_security_lockdown: disabled`, and `esxi_security_ssh: ""` to leave SSH as it is.

Setting	Default	What it does
`esxi_security_hosts`	`[]` (all)	only these hosts (names, `*` and `?` allowed; the short name works)
`esxi_security_clusters`	`[]` (all)	only the hosts of these clusters
`esxi_security_exclude_hosts`	`[]`	never these hosts
`vmware_datacenter`	(all)	one datacenter only

Email the report

Every VMware job can email its result: the secure boot report its whole list, the other jobs what they did. Email is off until there is an address to send to.

Step 1 - ask your mail admins for:

- the mail relay's name and port (e.g. `smtp.yoursite.mil`, port 25);
- whether it uses encryption: STARTTLS (usual on 25 and 587), SSL (port 465), or none;

- whether it needs a login, or instead accepts mail from allowed servers. Then the **AAP server's IP address** must be on its allowed list: the email leaves from the AAP server (with separate AAP execution nodes, from the node that runs the job - list those);
- which sender address to use (e.g. `aap-noreply@yoursite.mil`).

Step 2 - the relay, in your `playbooks/group_vars/all.yml` (VS Code, then Commit and Sync Changes; AAP picks it up at the next project sync):

```yaml
report_email_smtp_host: smtp.yoursite.mil       # the relay from step 1
report_email_from: aap-noreply@yoursite.mil     # the sender from step 1
```

Add these lines only when step 1 says so:

The relay	Add to all.yml
port 25 or 587 with STARTTLS	nothing: that is the default
a port other than 25	`report_email_smtp_port: 587` (its port)
SSL on port 465	`report_email_smtp_port: 465` and `report_email_security: ssl`
no encryption at all	`report_email_security: none` (then no login can be used)
its certificate is from your own CA (DoD) and the job says `certificate verify failed`	put the CA certificate (`.pem`, public) at `playbooks/files/ca/smtp-ca.pem` in the repository, `report_email_ca_path: "{{ playbook_dir }}/files/ca/smtp-ca.pem"`, and the line `playbooks/files/ca/` in `.site-local` (so an update keeps it)

Step 3 - only if the relay needs a login: a credential, so the password stays out of Git.

1. AAP: **Automation Execution > Infrastructure > Credential Types > Create credential type**. Name it `SMTP relay`. Paste the `inputs:` part of `aap/credential_types/smtp_relay.yml` into **Input configuration** and the `injectors:` part into **Injector configuration**. Save.
2. **Credentials > Create credential**: type **SMTP relay**, the user name and password. Save.
3. On each VMware template: **Edit > Credentials**, add it next to the vCenter credential.

Step 4 - who gets it. One of:

- every VMware job, always: `report_email_to: [vmteam@yoursite.mil]` in `all.yml`;
- one template: its **Variables** box, `report_email_to: vmteam@yoursite.mil, ops@yoursite.mil`;
- whoever launches it: a survey question, variable `report_email_to`, type **Text**, not required. Left empty = no email.

Several addresses: separate them with commas. Copies: `report_email_cc`, the same way.

Step 5 - test it with the secure boot report: it only reads, so it is safe to run any time. Launch it with `report_email_to` set. In the job output, the task **Email | result** says `emailed "[AAP] VMware secure boot report: ..." to vmteam@yoursite.mil`, and the email arrives.

The other reports (health checks, certificate reports, ...) email the same way: EMAIL_REPORTS.md lists them, and explains when a report job is green or red.

What arrives: a formatted email, subject `[AAP] VMware secure boot report: 29 VM(s) to fix` (or `[AAP] VM restart: web01`, `[AAP] VM snapshot before-patch: web01`, ...). The secure boot report has number boxes (VMs checked, secure boot on / off, BIOS) and a table per problem: each VM with its folder, ESXi host, guest host name, IP, power state and guest OS - so VMs with the same name can be told apart. At the end: the AAP job, who started it, when, and which vCenter. The email also carries a plain-text copy. A dry run (Check) sends nothing; it says `DRY RUN: would email ...`. To change the look of the email: EMAIL_REPORTS.md.

The same report is in the job's **Details > Artifacts** (`vm_report`, and for the secure boot report `vm_secure_boot` with every VM's details), which is also what the next step of a workflow receives.

If it fails, the job fails (so nobody misses it) and says why:

The job says	Fix
`set report_email_smtp_host ... and report_email_from`	step 2 is missing (or not synced to AAP yet)
`could not send the email through ...: [Errno 111] Connection refused` / `timed out`	wrong name or port, or a firewall between the AAP server and the relay
`does not offer STARTTLS`	the relay uses SSL (port 465) or no encryption: see step 2
`TLS with the mail relay ... failed: ... certificate verify failed`	your own CA: the last row of step 2
`refused the login`	the user name or password in the SMTP relay credential
`could not send ...: 530 ... authentication required`	the relay needs a login (step 3), or the AAP server's IP on its allowed list
`refused every recipient` / `(refused: x@...)`	the relay does not accept that address (wrong address, or outside mail not allowed)
`the password would cross the network unencrypted`	a login with `report_email_security: none`: use starttls or ssl

AAP's own **Notifications** (the template's Notifications tab) only say that a job succeeded or failed. This sends the report itself.

What you see

- **restart:** `Restart requested for (operating system restart through VMware Tools): web01`. The job does not wait for the server to come back up; check its services afterwards, or run the health check.
- **shut down:** `Shut down (...): web01.`, once the VM is off (up to `vm_power_timeout`, 600 s). VMs that were already off are listed as such.
- **snapshot:** `Took snapshot "before-patch" (no memory) of: web01`.
- **delete snapshot:** `web01: deleted before-patch`. If the name is not there: `no snapshot named "x" (it has: ...)`.
- **notes:** the notes `BEFORE` and `AFTER`, for each VM.
- **change VLAN:** `web01: Network adapter 2 moved VLAN200`, or `already on VLAN200`.
- **secure boot report:** a count, then one line per VM to fix, with what to do.

- **datastore report, snapshot report, alarms report:** the report as text (the same tables as the email), with the counts in the task name, e.g. `VMware alarms report: 1 critical, 2 warning alarm(s), 47 failed login(s) (events: last 24 h)`.
- **alarms ACT analysis:** `VMware alarms ACT analysis: likely causes and fixes for 12 problem(s)`, then one row per problem with ACT's cause, fix and confidence.

Refusals, and what to do:

- `VMware Tools is not running`: a `guest` restart or shutdown cannot work. Fix Tools, or use `hard` if losing unsaved data is acceptable.
- `No VM is named ...`: names are exact (case included), with no wildcards.
- `2 VMs are named ...`: rename one, or set `vmware_datacenter` if they are in different datacenters.
- `Refused: ... is the AAP server or a protected VM`: by design. Do it in vCenter yourself.

Troubleshooting

Find the message in the job's output (the red task near the end), or in the email, then do what the last column says.

Every VMware job

You see	It means	Do
`No vCenter to talk to: attach a credential of type "VMware vCenter"`	the template has no vCenter credential	Template › Edit › Credentials › add your **VMware vCenter** credential
`vCenter ... refused the login`	the message says which: no domain in the user name (a local vCenter account is `name@vsphere.local`), a wrong user name or password, a locked account or an expired password - or that vCenter does not know the account (the jobs log in to every vCenter of `vmware_vcenters` with it; a `vsphere.local` account works only on the vCenters of its own single sign-on domain, in linked mode)	fix the credential (type **VMware vCenter**); log in to the vSphere Client of **that** vCenter with the same account to test it. The deploy stops when any vCenter refuses: it checks the new name is free in all of them
`vCenter ... accepted the login, but the account has no permission in that vCenter` (before: `NoPermission ... System.View ... group-d1`)	the account logs in (single sign-on) but has no role anywhere in that vCenter, so vCenter refuses the session	give it its role there: a **Global Permission** (Administration > Access Control > Global Permissions, Propagate to children) covers every vCenter of its single sign-on domain. Read-only is enough on a vCenter the job only reads - the deploy reads every vCenter of `vmware_vcenters` to check the new name
`Could not connect to vCenter ...: ... CERTIFICATE_VERIFY_FAILED`	the execution environment does not trust vCenter's certificate	add vCenter's CA to the execution environment; to test only, `vmware_validate_certs: false` in the template's Variables
`Could not connect to vCenter ...: timed out` or `Connection refused`	the AAP node cannot reach vCenter on port 443	a firewall rule from the AAP node (or execution node) to vCenter, port 443
`pyVmomi is not installed in the execution environment`, or `couldn't resolve module/action 'vmware.vmware...'`	the template uses an execution environment without the VMware parts	pick your VMware execution environment on the template ("Before you start", step 1)
`GenAI did not read the ticket: GenAI gave no readable answer`	the model answered, but not with the JSON the job asks for (a chat reply, a refusal, or an answer cut short)	the job output (task *what GenAI proposes*) and the report show what it answered under **WHAT GENAI ANSWERED**: a chat reply ("I am ready...") means the model ignored the task - try another model (`site_act_model`); an answer cut short - a longer `vm_deploy_genai_model_timeout`. The parser (`vm_deploy_plain_words`) works without GenAI

You see	It means	Do
`lacks the privilege System.View on host-...` (a deploy)	the account has no role on that ESXi host: its permission is on the cluster without **Propagate to children**, or the host is in a cluster it has no role on - the deploy reads the hosts of the cluster it builds in, and the host the template is registered on	give the role on those clusters (and the template's) with **Propagate to children** ticked, or as a Global Permission. A host's name for an ID: `https://<vcenter>/mob/?moid=host-...`
A VM, host or datastore you see in vSphere is missing from a report	the vCenter account cannot see it	give its role on that folder or cluster, with **Propagate to children** ("Before you start", step 2)
A whole vCenter's datacenters and clusters are missing (the vSphere Client shows two or more vCenters)	the jobs read the credential's vCenter, plus only those in `vmware_vcenters`	`vmware_vcenters: [vc01..., vc02...]` in `all.yml` ("Before you start", step 5)
Red **vCenters not read** on top, `- 1 vCenter(s) not read` in the title (the job failed at `every vCenter was read` only with `vmware_vcenter_fail_unread: true`)	that vCenter could not be reached, or refused the login; the rest of the report covers the others	the reason is in the row: `refused the login` = the account is not known there (in linked mode: give it a Global Permission); `timed out` / `Connection refused` = a firewall rule to that vCenter, port 443
`Could not find imported module support code for ... site_vmware_... Looked for ... ansible.module_utils.site_vcenters`	the folder `roles/vmware_vm/module_utils/` is missing: an update copied the changed files but not that new folder	copy `roles/vmware_vm/module_utils/` from the release into your repository, commit, sync the project
`the same vCenter as ...: read once` in the artifacts	a vCenter listed twice (a short name and its FQDN, or its IP)	nothing: it is read once
`Refused: ... is the AAP server or a protected VM`	by design	do it in vCenter yourself

Datastore and snapshot reports, snapshot cleanup

You see	It means	Do
The datastore heading still says 80%	`vm_datastore_warn_pct: 80` is set somewhere (it wins over the default 85)	remove it from `all.yml` and the template's Variables, or set the number you want
`vm_snapshot_max_age_days must be a number of days`	a survey or variable answer like `three`	a number: `1`, `2`, `3`
The snapshot report says 3 days although you set 2	in the workflow, the **survey answer** wins over the template's Variables	change the survey's answer or its Default (section "Snapshots")
`Refused: N snapshots are older than ... - more than vm_snapshot_cleanup_max`	more than 50 at once (often a wrong number of days)	check the days; for a big cleanup on purpose, raise `vm_snapshot_cleanup_max`
`NOT DELETED ...: no longer there, or no longer old enough / now kept`	the snapshot changed between the report and the approval	nothing: it is safe. Run the workflow again
`Taken by` says `unknown`	vCenter no longer keeps the event (often after 30 days)	nothing: the snapshot's description may name the AAP user

ESXi security settings

You see	It means	Do
`... the vCenter account lacks the privilege Host.Config.... on this host`	the vCenter role misses that privilege	add it to the role ("Before you start", step 2), on the hosts or their cluster
`lockdown strict is refused`	`esxi_security_lockdown: strict`	use `normal`
`No host matches (hosts: ...)`	a typo in `esxi_security_hosts` / `_clusters`, or `vmware_datacenter`	the names as the vSphere Client shows them
`X: the host has no such setting`	that advanced setting does not exist on this ESXi version	check the name (case matters), or remove it from `esxi_security_settings`
`lockdown exception users: ... - lockdown left disabled, so they are not locked out`	the exception users could not be added, so lockdown was not turned on (on purpose)	fix the cause (often a privilege, or an account the host does not know) and run again
`lockdown is strict: left as it is`	someone set strict on that host	nothing, or set it to normal in vCenter if strict was a mistake
A host is listed as **skipped**	it is not connected to vCenter	reconnect it; the next run sets it
The scanner (or another tool) cannot log in to a host any more	lockdown blocks direct logins	add its account to `esxi_security_lockdown_exception_users` and run again, or exempt the host
SSH is off again on a host you opened for support	the nightly run did its job	`esxi_security_exclude_hosts` while the case is open
The ESXi Shell stays on, and the report does not mention it	`esxi_security_shell: ""` is set somewhere (leave it as it is)	remove it: the default is `disabled`
Every night's email lists the same host as changed	something turns the setting back every day (a host profile, a script, a person)	find what it is: two tools fighting over one setting

Deploy from a ServiceNow ticket

You see	It means	Do
`No ServiceNow connection: attach the "ServiceNow API" credential`	the template has no ServiceNow credential	add it (Set it up, step 4)
`Give the ticket number: the job got none`	the template has no survey, or it is switched off	the Survey tab: *Ticket number*, variable `vm_deploy_ticket`, text, required - and switch it on
`Which table is INCN0012345 in? ...`	your instance numbers its tickets its own way, and the account may not read the task table (or there is no such ticket)	`vm_deploy_ticket_prefixes: {INCN: incident, CHGN: change_request}` on the template; check the number
`ServiceNow has no table "..."` (before: `HTTP Error 400: Bad Request`, `Invalid table change`)	`vm_deploy_ticket_table` or `vm_deploy_ticket_prefixes` names a table ServiceNow does not have	leave both empty: the job finds each ticket's table. `change`, `change request`, `change task`, `catalog task` are now read as `change_request`, `change_task`, `sc_task`
`No change_request ticket INCN... in ServiceNow. vm_deploy_ticket_table ... is set`	`vm_deploy_ticket_table` sends every number with a prefix the job does not know to one table, and this ticket is in another	leave it empty (a `CTASK...` or `INC...` number is never sent there)
`the change CHG... (this ticket's parent) is not ready to build: approval is 'Requested', not Approved; state is 'Assess', not Implement`	the change is not approved, or not in Implement (yet, or any more) - nothing was built	approve it / move it to Implement and run the job again; your instance's labels differ (e.g. *Implementation*): put them in `vm_deploy_ready_by_table`
`this ticket (CHG...) is not ready to build: ...`	the job ran on a change itself, which is not approved and in Implement	the same; or run it on the change's task
`... approval is empty, not Approved`	the account may not read the change's approval field, or the table has none	read access to `change_request.approval`; or take `approval` out of `vm_deploy_ready_by_table`
A change task gets questions ("which operating system ...") though its change says it all	plain words are off (`vm_deploy_plain_words`), or the request is in another field of the change	`vm_deploy_plain_words: true`; `vm_deploy_parent_text_fields: [description, implementation_plan]`
The task closed, but its *Close code* shows a value in grey / not in the list	the close fields sent a label or a value your instance does not have	take the values from a closed task's *Show XML*
`the ticket asks for 2 VMs ('2 vms', ...) but names 1 (web01)`	the words say how many VMs, and the names found are fewer (or more)	the requester writes every name (`named web01 and web02`) in a comment, or a line `vm_name: web01, web02`

You see	It means	Do
`the ticket gives a range of names ('named web01 to web05' ...)`	a range is never expanded	the requester writes every name; a line `vm_name: web01-05` means one VM of that name
`the ticket asks for 12 VMs (...): more than 10 (vm_deploy_max_vms)`	more names than one ticket may build	split the ticket, or raise `vm_deploy_max_vms` on the template
`[AAP] 1 of 2 VMs built (...); this ticket stays open`	one VM failed (its own note says why), the others were built	fix the cause and run the job again: the VMs built are left as they are, the rest are built, the ticket is closed
`a VM named web01 already exists` on a ticket that built it	the VM's notes no longer name the ticket (edited), or the ticket has no "deployed" note for it	check that it is the right VM; delete it to build again, or close the ticket by hand
`No incident ticket INC... in ServiceNow`	a wrong number, or the account cannot see it	check the number; the account needs read access to that table
`ServiceNow refused the read (HTTP 403)`	the account may not read that table	give it read access (e.g. the itil role)
`REFUSED: template '...' is not in vm_deploy_templates`	a template you have not allowed	add it to `vm_deploy_templates`, or correct the ticket
`REFUSED: the ticket is assigned to ...`	not your team's ticket	reassign it, or `vm_deploy_assignment_group`; a change launched by itself, any group: `vm_deploy_assignment_group_by_table: {change_request: ""}`
`the ticket does not say vm_name`	no `vm_name:` line	the requester adds a comment `vm_name: ...`
`Nothing was built: ServiceNow refused the work note`	the account cannot update that table	give it write access to work notes; nothing was built, by design
`a VM named ... already exists in vCenter ...`	the name is taken (or the job ran twice)	another name in a comment, or nothing to do
`template ... is in vc01 / DC1, vc02 / DC1`	the same template name in two places	`vcenter:` (or `datacenter:`) in the ticket
`template ... is Windows and has no customization specification`	an empty template that installs Windows from the network (MECM) is not listed as one - or a template with Windows installed has no Sysprep specification	the empty one: add it to `vm_deploy_pxe_templates` (no customization then); one with Windows: `vm_deploy_template_specs: {"Windows 2022*": <spec>}` - the message lists the vCenter's Windows specifications - or a ticket's `customization_spec:` line
`the ticket does not say which site`	sites are set up, the ticket names none, its Location is not a site, and no name rule fits the name	`site: ...` in a comment; a name rule; or `vm_deploy_site_required: false`

You see	It means	Do
The VM went to the default (or the template's) cluster, not the name rule's	no rule's `match` fits the name (a typo, or `AB01*` when the name starts with other letters)	`*AB01*` (anywhere in the name); check the dry run's `(from name rule ...)`
`site '...' is not one of vm_deploy_sites`	a site name you have not set up	correct the ticket, or add the site
`the ticket does not say where ab01xy05 goes - SiteA or SiteB`	its name rule has `sites:` (a name used at several sites) and its line names no site - or a place no site's name or words match (`Location: N. Campus`, the site's words say `north campus`)	the requester writes the site on the VM's line; or add the way they write it to `vm_deploy_plain_sites`
`ab01xy05 is to go to site SiteC, but its name rule *XY05* is for SiteA or SiteB`	the VM's line (or a `site:` line) names a site its name rule is not for	correct the ticket - or add the site to the rule's `sites`
`for ab01web06: the ticket gives more than one number of CPUs ...`	that VM's own line gives two values (one server per line)	the requester corrects that line, or writes a line for all (`cpu: 4`)
`... ready to build in ... - waiting for the approval`	the propose step (or the job alone with `vm_deploy_genai_approval: true`): by design	approve the workflow (or launch the job again)
`has no GenAI proposal for its text as it is now`	the build step, but the ticket changed after the proposal (or there is none)	run the workflow from the start
`was changed after the approval` / `has no "[AAP] Checked - ready to build" note`	the build step: the ticket (or its change) changed after the propose step checked it, or that step ran with an older release	run the workflow from the start
`Nothing was built: ... 1 of 3 cannot be: - web03: ...`	several VMs: one fails the check in vCenter (name taken, no template, no datastore with room ...) - none is built	fix that one (or the ticket), run again
`goes on a datastore *SQL* ...: A-DS01 is not one` / `is kept for other VMs`	`vm_deploy_datastore_rules`: the ticket names a datastore the VM's rule does not allow, or one kept for other VMs	drop the `datastore:` line (the job picks the least used of the allowed ones)
`no VM folder Test/App in datacenter DC1 (a path is looked up as it is ...)` / `is not one of vm_deploy_folders`	a folder path is that path exactly (never another folder of the same name); the folder found must be allowed	write the path as in vCenter, below the VM folder

You see	It means	Do
`questions for the requester (nothing built)`	the job could not trace a value GenAI gave to the ticket, or something it needs is missing (GenAI's own questions are only `GENAI NOTE` lines, never sent)	the requester answers in a comment (`memory_gb: 16`); run the job again
`DROPPED: vm_name ...`	GenAI gave a value the ticket does not contain (made up, or guessed)	nothing: it is a question now
`DROPPED: template '...': it is for Windows Server 2022, the ticket asks for Windows Server 2025`	GenAI named a catalog template for another version	nothing: never built; see the next row
`the ticket asks for the operating system Windows Server 2025 (...), and no template here is for it`	no allowed template's name (or `vm_deploy_plain_templates` words) is for that version - the job never builds another one	add a template for it to `vm_deploy_templates` (and `vm_deploy_pxe_templates` when MECM installs it); or, when an existing one is right for it (an empty MECM shell): `vm_deploy_plain_templates: {"server 2025": "<template>"}`; or `template: <name>` on the ticket
`GenAI did not run: ...`	no ACT model key, the model not reachable, or a timeout	the ACT model key credential on the template; see docs/ADDING_ACT.md; or `key: value` lines
`no cluster named X in datacenter Y: X is a vCenter, not a cluster - the clusters there: ...` (or *an ESXi host*, *the datacenter*)	the cluster setting names something else: often the vCenter's name in `cluster:`	the job's output shows where it came from (`cluster: X (from ...)`: a ticket line, `vm_deploy_defaults`, `vm_deploy_sites`, `vm_deploy_name_rules`); put one of the clusters listed there (vSphere Client, *Hosts and Clusters*), or no cluster at all - the template's own. A name written in other case is found (with a note)
`no cluster among ... has room`	automatic placement: every candidate would go over the memory target with a host down	another site / cluster, less memory, or a higher `vm_deploy_mem_target_pct`
`no datastore among ... has room`	every candidate datastore would go over 85 % with the VM	another datastore list, smaller disks
`disk 1 cannot shrink`	`disks_gb` asks for a smaller first disk than the template's	the first size at least the template's
A network-boot VM sits at "PXE-E..." / "No boot device"	the VLAN does not reach MECM's PXE, or MECM does not answer this computer	the DHCP / IP helper on the staging VLAN; the computer imported in MECM, or unknown-computer support on the task sequence

You see	It means	Do
`lacks the privilege VirtualMachine.Provisioning...`	the vCenter account's role	grant it (Set it up, step 2)
`lacks the privilege Cryptographer.Clone ... the template is encrypted`	the template has a vTPM (or encrypted disks)	grant the *Cryptographic operations* privileges in Set it up, step 2
Customization `no result within 900 s`	VMware Tools does not run in the template, or cloud-init blocks VMware customization	fix the template; the VM exists - set its host name by hand, or delete it and run again
`no IPv4 address within 600 s`	no DHCP lease on the staging VLAN, or VMware Tools does not run	check DHCP; the IP shows in vCenter when it comes
`the template's ... is not set to connect at power on`	the template's adapter	tick **Connect at power on** on the template

Capacity planning

You see	It means	Do
`not enough history` / `No trend: vCenter has fewer than 7 daily samples`	a new vCenter, or its statistics level or daily interval was changed	wait a week; check vCenter > Configure > General > Statistics: the 1-day interval enabled, level 1 or more
`vCenter's daily statistics interval is turned off: no history`	the 1-day interval is disabled	enable it (same place); usage now is still shown
A runway that jumps from run to run	the trend fit is poor: VMs were moved, snapshots deleted, a datastore grown	read the Trend fit column; use `vm_capacity_trend_days: 60` for a longer, steadier line
`cannot lose a host`	the cluster has as many hosts down (or in maintenance) as it plans to survive	bring hosts back; or set `vm_capacity_failover_hosts: 0` for a cluster that needs no failover
`No cluster matches ...`	a typo in `vm_capacity_clusters`	the cluster's name as the vSphere Client shows it
`ACT did not run: ...`	as for the alarms analysis (key, network, certificate)	the ACT rows above
Math and GenAI differ a lot	ACT weighed something the line does not (a jump, a limit that comes first)	read ACT's reasoning; plan for the sooner of the two

VMs created report

You see	It means	Do
By is unknown	vCenter no longer has the VM's creation event (it keeps them for a limited time), or the VM was created on a host directly	nothing: the VM still counts, by its creation date
Ticket is - for a VM the deploy built	its notes in vCenter were edited, so they no longer say *"Deployed from … for <ticket>"*	nothing for the count; the ticket's own notes say which VM it built
a VM restored from a backup is listed	a restore creates a new VM in vCenter	nothing: to vCenter it is new
`vm_created_days must be a number of days`	the setting is not a number	`vm_created_days: 7`

Health report

You see	It means	Do
`vSphere HA is off` / `DRS is off` on a cluster that should not have them	your design	`vm_health_require_ha: false` / `vm_health_require_drs: false` on the template
`vCLS in retreat mode`	someone set `config.vcls.clusters.<id>.enabled` to `false` in vCenter's advanced settings (often for maintenance) and it was not set back	vCenter > Configure > Settings > Advanced Settings: set it to `true` when the work is done; DRS starts again
`vSphere HA agent: network isolated` / `network partitioned`	the host's management network cannot reach the other hosts (or its isolation address)	the host's management VLAN, uplinks and switch ports; then **Reconfigure for vSphere HA** on the host
`HA can survive 0 host failure(s) now`	the cluster is too full to restart a failed host's VMs	add capacity or move VMs: see the capacity planning report
HA agent ? on every host	vCenter did not say (some versions or a limited account)	the HA state shows in the vSphere Client: cluster > Monitor > vSphere HA
`Could not read vCLS retreat mode` (blue)	the account may not read vCenter's advanced settings	nothing needed: retreat mode still raises vCenter's own alarm on the cluster
`vCenter ... has no datacenter named ...`	a typo in `vmware_datacenter`	the exact name, as the vSphere Client shows it
`Reading the health of datacenters, clusters and hosts from vCenter ... failed: ...`	something this job did not expect from your vCenter	send that line to the people who maintain this repository

Alarms report

You see	It means	Do
`types: unknown object type(s) ...`	a typo in `vm_alarm_types`	use `vcenter`, `datacenter`, `cluster`, `host`, `datastore`, `vm`, `network`
`vCenter ... has no datacenter named ...`	a typo in `vmware_datacenter`	the exact name, as the vSphere Client shows it
An alarm the vSphere Client shows is missing	it is on a VM, datastore or network (not read by default), or in another datacenter	add its type to `vm_alarm_types` (e.g. `[vcenter, datacenter, cluster, host, vm]`); check `vmware_datacenter`
No events at all, or fewer than in the vSphere Client	`vm_alarm_hours: 0`, or the account sees only part of the inventory (vCenter shows an account only the events of what it can see)	give the role at the top of the vCenter inventory, with Propagate to children
`Only the newest 5000 events were read`	a busy vCenter	lower `vm_alarm_hours`, or raise `vm_alarm_max_events`
`Only the newest 2000 alarm changes were read`	an alarm that flaps (on, off, on ...) all day	find it in *Alarms that fired* (the highest counts) and fix its cause; or raise `vm_alarm_max_alarm_changes`
`Only the first 200 are shown`	a long table	the job's **Artifacts** (`vm_alarms`) have all of them; or raise `vm_alarm_max_rows`
The job shows **failed** with `critical alarm(s) ... host(s) not connected`	`vm_alarm_fail: true` is set	expected (for a workflow); remove it for a report only
`Reading alarms and events from vCenter ... failed: ...`	something this job did not expect from your vCenter	send that line to the people who maintain this repository

Alarms ACT analysis

You see	It means	Do
`nothing to analyze`, and no email	nothing is wrong: no alarm, no problem in the window	nothing. `vm_alarm_act_email_if_none: true` sends an email anyway
`A dry run (Check): ACT was not called`	the job ran as Check	launch it with Job type Run
`This job has no API key for provider genai: attach ...`	no **ACT model key** credential on the template, or its GenAI field is empty	add the credential (with the key for your provider) to the template
`This AAP node cannot reach the model at ... (... timed out / Connection refused / Name or service not known ...)`	a firewall, DNS or a missing proxy between the AAP node and the model	ask for HTTPS (443) from the AAP node to that address, or set your proxy: `site_act_env: {HTTPS_PROXY: "http://proxy.yoursite.mil:8080"}` in `all.yml`
`... Its certificate is not trusted: set site_act_ca ...`	the execution environment does not trust the model's certificate authority	put that CA certificate at `playbooks/files/ca/model-ca.pem`, `site_act_ca: "{{ playbook_dir }}/files/ca/model-ca.pem"` in `all.yml`, and `playbooks/files/ca/` in `.site-local`
`ACT failed: ...` with `401` or `Unauthorized`	the key is wrong or expired	a new key in the credential
`ACT failed: ...` with `429` or `Too Many Requests`	the key's quota per minute	run it less often, or lower `vm_alarm_act_max_items`
`model turn exceeded the 600s total timeout` (older releases: `90s`, `300s`), often `after a transient provider failure`	the model (or the provider, retrying after an error) took longer than ACT allows for one answer: `site_act_model_timeout`, 600 s	run it again (a provider hiccup passes); `site_act_model_timeout: 900` in `all.yml` (keep `site_act_timeout` above it); fewer items (`vm_alarm_act_max_items: 20`); or a faster model
`ACT produced no result (it timed out after 1200s ...)`	a slow model, or a lot of evidence	`site_act_timeout: 1800`, or `vm_alarm_act_max_items: 20`
`ACT's answer is below as text`	the model did not use the table format	the answer is still in the email. Run it again, or try another model (`site_act_models`)
A row says `ACT gave no answer for this one`	the model skipped that problem	run it again; fewer problems (`vm_alarm_act_max_items`) help a small model
The job is red, but the email came	ACT could not run; the email lists the problems and says why	fix what the email says, then run it again

Test the way to the model by hand, from the AAP server (the execution environment uses the same network):

```shell
curl -sS -o /dev/null -w '%{http_code}\n' https://api.genai.mil/v1/chat/completions
```

Any number (401, 404, 405) means the way is open. `Could not resolve host`, a timeout, or `SSL certificate problem` is the problem the job reports.

Limits

- **Change VLAN between a standard and a distributed switch.** Moving an adapter from a standard switch port group to a distributed one, or back, fails inside the `vmware.vmware` collection (2.11); the job says so and changes nothing. Do that one in vCenter. Moves between port groups of the same kind of switch work.
- The guest keeps its IP settings after a VLAN change. A new VLAN usually needs new ones.
- `vm_names` are exact VM names. To act on many VMs, list them.
- Tested against vcsim, VMware's vCenter simulator (`tests/vmware/run_vmware_test.sh`), not against a real vCenter. Run each job once as **Check**, then on a test VM, before you use it on real servers. The simulator raises no alarms: the alarms report's alarm handling is tested with recorded data (`tests/test_filters.py`, `tests/test_vmware_modules.py`). The ACT analysis is tested with the real ACT against a scripted model (`tests/test_alarm_act_contract.py`), not a real one.

11. NetApp ONTAP health report

One job, `playbooks/ontap_health_report.yml`, reads every NetApp ONTAP cluster you list and emails one report: what is broken first, then capacity, protection and services. Each row is coloured: red = critical, amber = warning, green = OK, blue = for your information.

It only reads. It sends GET requests to each cluster's REST API (the same API System Manager uses) and nothing else: the code refuses any other kind of request. It runs on the AAP side, like the VMware jobs: no inventory of the clusters is needed, no SSH, no Limit.

What the report shows

At the top: the title with the number of critical and warning findings, then boxes with the main counts (nodes down, health alerts, hardware, full volumes and aggregates, SnapMirror, network, certificates). Then, in this order:

Section	What it shows	Colours
Clusters	each cluster: ONTAP version, system health, nodes up, space used on its aggregates, NTP servers. A cluster that could not be read is listed in red with the reason	health, nodes, space
Nodes	state, uptime (days hours:minutes), model, version, whether the partner could take over (storage failover), service processor, NVRAM battery, the node's clock against the AAP server's	green UP / red DOWN, and each status
System health	ONTAP's health monitors (`system health subsystem show`) that are not OK	amber degraded
Health alerts	what the health monitors raised (`system health alert show`): node, severity, alert, resource, probable cause, corrective action	red critical / major, amber minor
Error events (last 24 h)	ONTAP's event log (EMS): emergency, alert and error events, the same event on the same node counted once	red alert / emergency, amber error
Hardware: controllers and shelves	each controller and disk shelf: power supplies, fans, temperature, failed parts (FRUs)	red failed
Disks	broken disks; disks rebuilding, copying or in maintenance; unassigned disks	red / amber / blue
Spare disks	spare disks per node (a failed disk is rebuilt onto a spare)	red none, amber few
Network interfaces (LIFs)	LIFs down, not on their home port, or disabled	red / amber / blue
Ports down	enabled Ethernet and FC ports that are not up	red in use, blue no cable / not used

Section	What it shows	Colours
Port errors (CRC)	ports with receive errors or CRC errors since the node booted (CRC counters: ONTAP 9.11+)	amber from 1, red from 10,000
Aggregates	every data aggregate, fullest first: node, home node, size, used, available, used %	amber 85 %, red 90 %
Aggregates not on their home node	after a takeover, aggregates still on the partner	amber
Volumes	volumes 85 % full or more, and volumes offline or restricted, with their aggregate, state, type, size, available, used %	amber 85 %, red 90 %, red offline
Inodes (files)	volumes using 85 % or more of their inodes (no new file can be created when they run out)	amber 85 %, red 90 %
Snapshot space	the 25 volumes whose snapshots hold the most: type, volume size, snapshot count, reserve, snapshot used, % of reserve, % of volume	amber from 100 % of the reserve (spilling into the volume)
SnapMirror	relationships that are unhealthy, lagging, broken off, paused or uninitialized, and transfers running now: mirror state, status, healthy, lag, last transfer, progress, progress updated, policy, reason	red unhealthy / 48 h lag, amber 24 h lag, blue transferring
Cluster and SVM peers	cluster peers not available, SVM peers not peered	red / amber
SVMs not running	stopped storage VMs (a stopped DR destination is normal: blue)	red
CIFS (SMB) servers	each CIFS server: enabled or not, and whether its domain controllers answer	red stopped / no DC, amber a DC down
LUNs	LUNs not mapped to any host (forgotten?) and LUNs not online	amber / red
Certificates expiring	certificates expiring within 60 days or expired	amber 60 days, red 14 days

A section with nothing to report says so in one line ("Every LIF is up and at home."). Long tables show the problems first and are cut at 50 rows (`ontap_report_max_rows`); the job's artifacts (`ontap_health`) have every finding.

Before you start

1. A read-only account on every cluster

The report needs an ONTAP account with the built-in **readonly** role, for the **http** application (the REST API). On each cluster, as an administrator (`cluster1` is the cluster's admin SVM, its name):

```
security login create -vserver cluster1 -user-or-group-name svc_aap_ro -application http
  -authentication-method password -role readonly
```

Use the **same user name and password on every cluster**: the job has one credential for all. (An Active Directory account works too, with `-authentication-method domain`, if the cluster is set up for AD logins.)

2. The way from AAP to the clusters

The AAP server (or the execution node that runs the job) must reach each cluster's **management address** on HTTPS, port 443. Test from the AAP server (it asks for the password):

```
curl -sk -u svc_aap_ro "https://cluster1.yoursite.mil/api/cluster?fields=version"
```

A short JSON answer with the version means the account, the network and the API are fine.

3. The cluster's certificate

ONTAP's management certificate is often self-signed, or from your own CA. The job checks it by default. Either:

- put the CA certificate (public, `.pem`) at `playbooks/files/ca/ontap-ca.pem`, add `ontap_ca_path: "{{ playbook_dir }}/files/ca/ontap-ca.pem"` to `playbooks/group_vars/all.yml`, and the line `playbooks/files/ca/` to `.site-local` (so an update keeps it); or
- set `ontap_validate_certs: false` (the password still travels encrypted; the job just does not check who answers).

4. The credential

1. **Automation Execution > Infrastructure > Credential Types > Create credential type**: name `NetApp ONTAP`; paste the `inputs` block of `aap/credential_types/netapp_ontap.yml` into **Input configuration** and the `injectors` block into **Injector configuration**. Save.
2. **Credentials > Create credential**: type **NetApp ONTAP**, name e.g. `NetApp ONTAP read-only`, the user name and password from step 1. Save.

Set it up

1. **Templates > Create template > Create job template:**
   - Name: `NetApp - ONTAP health report`
   - Inventory: any (e.g. your VMware one: the job runs on the AAP side)
   - Project: site-automation. Playbook: `playbooks/ontap_health_report.yml`
   - Execution environment: any with Python 3 (no NetApp collection is needed)
   - Credentials: **NetApp ONTAP read-only**
   - Tick **Prompt on launch** next to **Variables** (to try one cluster first)
   - Variables:

YAML

```
ontap_clusters: [cluster1.yoursite.mil, cluster2.yoursite.mil]
report_email_to: storage-team@yoursite.mil
```

   - Leave Limit empty. Save.

2. **First run: one cluster.** Launch with Variables `ontap_clusters: [cluster1.yoursite.mil]`. Read the report in the job's output. Lines that start with `cluster1: could not read ...` or `... reduced detail ...` under a section say what this cluster's ONTAP version or the account's role did not allow: see Troubleshooting.
3. **All clusters**, then a schedule: **Schedules > Create schedule**, e.g. every day at 06:00 (set **your** time zone).

A dry run (Check) reads and prints the same report, and sends no email.

SnapMirror relationships are read on their **destination** cluster: list the destination clusters too, or the relationships into them are not checked.

Settings

Set them in the template's Variables, or in `playbooks/group_vars/all.yml` for every run.

Setting	Default	What it does
`ontap_clusters`	`[]`	the clusters' management addresses
`ontap_validate_certs` / `ontap_ca_path`	`true` / (none)	check the clusters' certificates; the CA file to check them with
`ontap_volume_warn_pct` / `_crit_pct`	`85` / `90`	volume used % for amber / red
`ontap_aggr_warn_pct` / `_crit_pct`	`85` / `90`	aggregate used %
`ontap_inode_warn_pct` / `_crit_pct`	`85` / `90`	inodes used %
`ontap_snapshot_reserve_warn_pct` / `_crit_pct`	`100` / `150`	snapshot space as % of the snapshot reserve
`ontap_snapmirror_lag_warn_hours` / `_crit_hours`	`24` / `48`	time since the last good SnapMirror transfer
`ontap_cert_warn_days` / `_crit_days`	`60` / `14`	days before a certificate expires
`ontap_time_drift_warn_seconds` / `_crit_seconds`	`60` / `300`	a node's clock against the AAP server's
`ontap_port_errors_warn` / `_crit`	`1` / `10000`	receive or CRC errors on a port, since boot
`ontap_min_spares`	`1`	fewer spare disks than this on a node is amber (0 is red)
`ontap_ems_hours` / `ontap_ems_max`	`24` / `500`	the event log window, and the most events read per cluster
`ontap_report_max_rows`	`50`	rows per table
`ontap_snapshot_top`	`25`	volumes in the snapshot space table
`ontap_report_fail`	`false`	`true` = the job shows **failed** on a critical finding or a cluster not read (for a workflow)
`ontap_timeout`	`30`	seconds for one request

All settings of this job, with their defaults: ontap_report.

Troubleshooting

You see	It means	Do
`No ONTAP account: attach a credential of type "NetApp ONTAP"`	the template has no NetApp ONTAP credential	add it (Before you start, step 4)
`ontap_clusters is empty`	no cluster listed	`ontap_clusters: [...]` in the template's Variables
A cluster **UNREACHABLE** with `timed out` or `Connection refused`	the AAP node cannot reach the cluster on port 443	a firewall rule from the AAP node to the cluster's management address; test with the `curl` above
UNREACHABLE with `Name or service not known`	the name does not resolve on the AAP node	use the name DNS knows, or the IP address
UNREACHABLE with `CERTIFICATE_VERIFY_FAILED`	the cluster's certificate is not trusted	Before you start, step 3
`the ONTAP account was refused (HTTP 401)`	wrong user or password, the account does not exist on **that** cluster, it is locked, or it lacks the `http` application	create it the same on every cluster (step 1); unlock it (`security login unlock`)
`cluster1: could not read alerts - HTTP 403 ... the account's role may not allow this`	the role does not allow that command	use the built-in `readonly` role (step 1)
`... reduced detail - ONTAP refused a field ...`	this ONTAP version does not know a field; the job asked again with fewer	nothing: the section is still shown, a column may say `?`. Send the line to the people who maintain this repository
`CRC counters need ONTAP 9.11 or later`	older ONTAP has no CRC counter in its REST API	nothing: receive errors are still shown
SnapMirror is empty, but you have relationships	they are read on the destination cluster	add the destination cluster to `ontap_clusters`
An aggregate you know is missing	root aggregates are not returned by ONTAP's REST API	by design: root aggregates are not listed
A volume shows `?` for size	it is offline: ONTAP reports no space for it	it is listed in red as offline
The email is too long	many findings	lower `ontap_report_max_rows` / `ontap_snapshot_top`; the artifacts keep everything
`could not read ems - ... only the first 500 records were read`	many error events	raise `ontap_ems_max`, or lower `ontap_ems_hours`

Limits

- The fields the job reads were checked against NetApp's ONTAP REST API reference for ONTAP 9.10 to 9.16. It was tested against a fake ONTAP API (`tests/netapp/`), not a real cluster: run it on one cluster first, and send any `could not read` or `reduced detail` line you see.
- The CLI commands it reads through REST (system health, storage failover, SnapMirror progress) are ONTAP `show` commands at admin level, which the `readonly` role may run.
- It reports; it changes nothing. Fixes are done in System Manager or the ONTAP CLI.

12. Emailed reports: the formatted email and its look

Jobs can email what they found or did, as a **formatted (HTML) report** with a plain-text copy for mail clients that do not show HTML. One email per job run, whatever the number of hosts.

Which jobs email

Template	Playbook	The email
Health check (Linux)	`health_check.yml`	findings of every host (critical first), hosts that did not report, healthy hosts
Windows health check	`win_health_check.yml`	the same, for Windows servers
Database health	`database_health.yml`	the same, for the database check
Certificate report (Linux / Windows)	`cert_report.yml`, `win_cert_report.yml`	findings, plus every certificate, soonest expiry first: red expired, amber within 30 days, blue within 60
POA&M status	`poam_status.yml`	overdue, due-soon and missing items
ServiceNow health	`servicenow_health.yml`	the instance and the MID Servers
Troubleshoot (Linux / Windows)	`troubleshoot.yml`, `win_troubleshoot.yml`	what the checks found
Apply approved ACT fix	`act_fix_approved.yml`	the checks after the fix
VM secure boot report	`vm_secure_boot_report.yml`	VMs with secure boot off, BIOS VMs (folder, host, guest, IP)
VM datastore report	`vm_datastore_report.yml`	datastores 85 / 90 % used or more, then every datastore, fullest first
VM snapshot report	`vm_snapshot_report.yml`	snapshots older than N days, kept ones, then every snapshot with size and age
VM snapshot cleanup	`vm_snapshot_cleanup.yml`	what was deleted (and the space freed), what could not be
VM alarms report	`vm_alarm_report.yml`	triggered alarms, host connection, failed logins, VM events, other errors and warnings, configuration issues, every host - each row coloured by severity
VM alarms ACT analysis	`vm_alarm_act_analysis.yml`	per problem: ACT's likely cause, evidence, fix and confidence (coloured); what to do first
Active Directory health report	`ad_health_report.yml`	domains, domain controllers, FSMO, replication, dcdiag, DNS, time, events, clients in no site, certificates, trusts, sites, then the day's changes (privileged groups, accounts, GPOs, schema and configuration) - problems first, coloured (ACTIVE_DIRECTORY.md)

Template	Playbook	The email
NetApp ONTAP health report	`ontap_health_report.yml`	clusters, nodes, health alerts, events, hardware, disks, network, aggregates, volumes, inodes, snapshots, SnapMirror, SVMs, CIFS, LUNs, certificates - problems first, coloured (NETAPP.md)
VM capacity planning	`vm_capacity_report.yml`	overall and per cluster: CPU, memory with a host down, vCPUs per core, VMs that fit, hosts recommended, datastores with runways; with GenAI, its estimate next to the math
ESXi security settings	`esxi_security.yml`	the hosts it changed (what), the ones that failed (why), every host's settings before the run. By default only when a host was changed or failed
VM deploy from a ServiceNow ticket	`vm_deploy_from_ticket.yml`	the VM (IP or MAC, where it went), the request with where each value came from; in the workflow's first step, where it would go - for the approver
VM deploy settings finder	`vm_deploy_inventory.yml`	the clusters, templates, datastores and folders by the names the deploy uses, what to look at first, and the lines to paste into the deploy template's Variables
VM request pickup	`vm_deploy_poll.yml`	the tickets picked up and their workflow jobs, the ones not started (why), the ones skipped. Only when it started something or something failed
VM restart, shut down, snapshot, delete snapshot, notes, change VLAN	`vm_*.yml`	what was done to which VM

Turning it on

1. **The mail relay, once**, in your `playbooks/group_vars/all.yml` (then Commit, Sync Changes, and sync the project in AAP). A relay that accepts the AAP server by a whitelist needs nothing more:

```yaml
report_email_smtp_host: smtp.yoursite.mil
report_email_from: aap-noreply@yoursite.mil
report_email_security: none          # only if the relay does not offer STARTTLS
```

Encryption, your own CA and a relay login: VMWARE.md, "Email the report".

2. **Who gets each report: on each template.** Templates > the template > Edit > **Variables**:

```yaml
report_email_to: linux-team@yoursite.mil, ops@yoursite.mil
```

Or a survey question with the variable `report_email_to` (Text, not required). Putting `report_email_to` in `all.yml` instead makes **every** job in the table above email those people, each time it runs - usually more than you want.

Not in an AAP inventory **group's** Variables: the email is sent by the AAP side of the job, which never gets a group's variables. The template's Variables, a survey, the inventory's own Variables and `all.yml` all work.

Setting	Default	What it does
`report_email_to` / `report_email_cc`	(none)	who gets it; a list, or text with commas
`report_email_html`	`true`	send the formatted version too; `false` = plain text only
`report_email_html_template`	`report.html.j2`	the look of the formatted version (below)
`report_email_subject_prefix`	`[AAP]`	put before every subject

The subject is the report's title with its counts, e.g. `[AAP] Health check: 3 finding(s) on 2 of 48 host(s)`. Every email ends with the AAP job number, the template, who started it and when. A dry run (Check) sends nothing. A job without `report_email_to` says `No email: report_email_to is not set`.

All settings, with their defaults: site_email.

Green or red: when a report finds problems

AAP shows a job as **successful (green)** or **failed (red)**, nothing in between. What a report job does when it finds problems:

Job	Default	Change it with (template Variables)
VM secure boot report, VM datastore report, VM snapshot report	**green**: the report and email say what to fix	`vm_secure_boot_fail: true` / `vm_datastore_fail: true` = red
VM snapshot cleanup	green; **red** when a deletion failed	-
VM alarms report	**green**: the report and email say what is wrong	`vm_alarm_fail: true` = red on a critical alarm or a host not connected
VM alarms ACT analysis	green; **red** when ACT could not run (no key, no network): the email still lists the problems	-
VM capacity planning	**green**; red when GenAI was asked for and could not run	`vm_capacity_fail: true` = red when a cluster or datastore is critical
ESXi security settings	green; **red** when a host could not be set (the email says why)	-
NetApp ONTAP health report	**green**: the report and email say what is wrong	`ontap_report_fail: true` = red on a critical finding or a cluster not read

Job	Default	Change it with (template Variables)
Health checks, certificate reports, database health, POA&M, ServiceNow health	**red** when there are findings: AAP workflows start tickets (and ACT) from that	`site_fail_on: []` = green, report only

So for a template whose job is to email a report to people, set `site_fail_on: []` and it stays green; the email is the alert. Keep the default on templates that feed a ticket workflow.

Red also means one of these, whatever the setting: a host could not be reached, or the job itself broke (a wrong setting, a missing credential). Those always need a look.

What a report looks like

A title (also the subject) in a coloured band - green, amber, red or blue - then summary boxes with numbers, then sections: each a heading, a line of advice, and a table (or a list). In tables that have a severity, each row is tinted (light red, amber or blue) and the severity cell is filled in its colour, so the problems stand out at a glance. Every value from a host or vCenter is escaped, so a VM name cannot add markup to the email.

Changing the look

The look is one file, `roles/site_email/templates/report.html.j2` (Jinja and HTML, styles inline because Outlook ignores most style sheets). To change it without touching the repository's file:

1. Copy it to `playbooks/files/email/report.html.j2` in your repository and edit the copy: colours, logo text, fonts, a banner line your site requires.
2. In `all.yml`: `report_email_html_template: "{{ playbook_dir }}/files/email/report.html.j2"`.
3. Add the line `playbooks/files/email/` to the file `.site-local` at the top of your repository (create it if it is not there). Without it, the next update from a release **deletes** your copy: the release does not have that file.

Check your copy after an update that changes the report layout (the CHANGELOG says so when that happens).

The **title** of a report (the coloured band, also the subject) is written by the playbook, not the look file: for the VMware reports, `_vm_report_subject` in `roles/vmware_vm/tasks/<report>.yml`. An edit there is undone by the next update, unless that file is in `.site-local` - and then you no longer get the release's fixes to it either.

Emailing a report from another playbook

Describe the report as data, then include the role. For example, at the end of a play that runs on localhost:

YAML

```
- name: Email the report
  ansible.builtin.include_role:
    name: site_email
  vars:
    _report:
      title: "Certificate report: 3 expire within 30 days"   # also the subject
      status: warning                    # ok | warning | critical | info: the colour of the band
      subtitle: "212 certificates on 48 servers"           # optional
      summary:                           # optional: number boxes
        - {label: Certificates, value: 212}
        - {label: Expire within 30 days, value: 3, status: warning}
      sections:
        - title: Expire within 30 days
          status: warning
          text: "Renew them before the date shown."        # optional
          columns: [Server, Subject, Expires]
          rows: "{{ my_rows }}"          # a list of lists, one per row, in the order of columns
        - title: Notes
          lines: [one line, another line]                  # a bullet list instead of a table
      footer: "Source: playbooks/cert_report.yml"           # optional
  when: report_email_to | default([]) | length > 0
```

A section has `columns` and `rows` (a table; an empty list shows "None."), or `lines`, or just `text`. To colour a table, add `row_status` (one status per row: `critical`, `warning`, `info`, `ok` or `''`; tints the row) and `cell_status` (per row, one status per cell; fills that cell), e.g. `row_status: [critical]` and `cell_status: [[critical, '', '']]` for a red row with a red first cell. In a play that runs on many hosts, build the rows from `hostvars` and add `run_once: true` to the include, so one email goes out, not one per host.

Troubleshooting

The job's output says what happened to the email, in the task `Email | result` (or `No email`).

You see	It means	Do
`No email: report_email_to is not set`	no recipient for this template	add `report_email_to` to the template's Variables (or its survey)
`To email the report, set report_email_smtp_host ... and report_email_from ...`	the relay is not set up	step 1 of "Turning it on", in `all.yml`
`the mail relay ... does not offer STARTTLS`	the relay has no encryption on that port	`report_email_security: none` (or `ssl` with port 465)
`TLS with the mail relay ... failed: ... certificate verify failed`	the execution environment does not trust the relay's certificate	`report_email_ca_path` to the CA file (VMWARE.md, "Email the report"); list `playbooks/files/ca/` in `.site-local`
`could not send the email through ...: ... timed out` or `Connection refused`	the AAP node cannot reach the relay	a firewall rule from the AAP node (and every execution node) to the relay's port
`the mail relay ... refused every recipient`	the relay does not accept mail from the AAP node, or for those addresses	ask the mail admins to add the AAP node's address to the relay's allowed list

You see	It means	Do
`emailed "..." to ...` but nothing arrives	the relay accepted it; something after it held it	look in junk / quarantine; ask the mail admins to trace it by the subject; check that `report_email_from` is an address they allow
`DRY RUN: would email ...`	the job ran as Check	nothing: a dry run never sends
The email shows plain text only	`report_email_html: false`, or a mail gateway removed the HTML part	ask the mail admins whether HTML mail is stripped; the text copy has the same tables
The job is red, but the email came	a check report found problems (`site_fail_on`)	by design for ticket workflows; `site_fail_on: []` on email-only templates ("Green or red")

13. Secrets: where they live

Recommendation

Store every secret as an AAP credential, and inject it into jobs as environment variables or files through custom credential types. Nothing secret goes into Git, inventory, job template variables, or surveys.

- AAP encrypts credential values at rest and never shows them again after you save them. People can be allowed to **use** a credential in a job without being able to **see** it.
- If your organization already runs **CyberArk, HashiCorp Vault, or Azure Key Vault**, keep the secrets there and link the AAP credential fields to it with AAP's credential lookup plugins (on the credential's form, the key icon next to a secret field). AAP fetches the value when a job starts; nothing else in this repository changes.
- Use **Ansible Vault** only for playbooks you must also run outside AAP. It is weaker: one vault password decrypts every value, and the encrypted values sit in Git forever (rotating the password does not un-publish old ciphertext).

Where each secret lives

Secret	AAP credential type	Reaches the playbook as
SSH key + sudo for the hosts	built-in **Machine**	the SSH connection (never visible to tasks)
WinRM login for Windows servers	built-in **Machine** (user name and password; Kerberos or NTLM)	the WinRM connection (never visible to tasks). Use a **separate** credential from the SSH one; see WINDOWS.md
Git access for the project	built-in **Source Control**	used only by the project sync
MySQL root / STIG Manager DB passwords	**STIG Manager database** (`aap/credential_types/stigman_database.yml`)	env `STIGMAN_MYSQL_ROOT_PASSWORD`, `STIGMAN_DB_PASSWORD`
TLS certificate, key, CA chain	**TLS certificate** (`tls_certificate.yml`)	files; env `TLS_CERT_FILE`, `TLS_KEY_FILE`, `TLS_CA_FILE` hold the paths
Docker Hub / quay.io login for hosts	**Container registry login (hosts)** (`registry_login.yml`)	env `REGISTRY_*`, used with `podman login --password-stdin`
ACT model key	**ACT model key** (`act_model_key.yml`)	env `GENAI_KEY` / `ASKSAGE_API_KEY` on the controller. **Note:** when ACT runs on a managed host, the job hands the key to that one ACT process on the task's stdin (never on a command line, so not in `ps` or the sudo log) and it is gone when the process ends. So the host's administrators and its process auditing can see it while ACT runs: use a key with a spending limit, and rotate it if a host is untrusted. `VM - alarms ACT analysis` runs ACT on the AAP node itself: there the key never reaches another server
ACT GenAI beta key	**ACT GenAI beta key** (`act_genai_beta_key.yml`)	env `GENAI_BETA_KEY`
Teams webhook URL	**Teams webhook** (`teams_webhook.yml`)	env `TEAMS_WEBHOOK_URL`
ServiceNow API account	**ServiceNow API** (`servicenow_api.yml`)	env `SN_HOST`, `SN_USERNAME`, `SN_PASSWORD` (basic auth on the controller; never sent to hosts)
MariaDB monitor account	**MariaDB monitor** (`mariadb_monitor.yml`)	env `MARIADB_MONITOR_USER` / `_PASSWORD`; the password goes to the client on stdin and reaches it only as `MYSQL_PWD` in its own environment (never a command line, never a fact)

Secret	AAP credential type	Reaches the playbook as
STIG Manager API client	**STIG Manager API** (`stigman_api.yml`)	env `STIGMAN_TOKEN_URL`, `STIGMAN_CLIENT_ID`, `STIGMAN_CLIENT_SECRET` (token request is `no_log`)
Java keystore password	**Keystore password** (`keystore_password.yml`)	env `KEYSTORE_PASSWORD`; the inventory names the variable (`password_env`), never the value; the check reads it on stdin and `keytool` takes it with `-storepass:env` (never a command line)
vCenter account (VMware jobs)	built-in **VMware vCenter**	env `VMWARE_HOST`, `VMWARE_USER`, `VMWARE_PASSWORD` on the controller (the jobs talk only to vCenter)
NetApp ONTAP read-only account (ONTAP health report)	**NetApp ONTAP** (`netapp_ontap.yml`)	env `ONTAP_USERNAME`, `ONTAP_PASSWORD` on the controller; sent only to the clusters' REST API over HTTPS (basic auth); the job sends GET requests only
Mail relay login (emailed reports)	**SMTP relay** (`smtp_relay.yml`), only if the relay needs a login	env `SMTP_USERNAME` / `SMTP_PASSWORD`; sent only over STARTTLS or SSL (refused with `report_email_security: none`)

Create one credential of each type **per environment** (dev / test / prod) so a test job can never use production secrets.

Why environment variables and files, not extra variables

A credential type can inject into `extra_vars`, but then the secret is an ordinary play variable: any `debug` task, any failed task's error message, or `-vvv` output can print it. An environment variable or a file on the controller has to be read on purpose (`lookup('ansible.builtin.env', 'NAME')`), which keeps secrets out of the variable namespace unless a task asks for them. Files are the right choice for certificates and keys: AAP writes them to private temporary files that disappear when the job ends.

Rules for every playbook in this repository

1. `no_log: true` on every task that receives, compares, or writes a secret. On a loop, also set `loop_control: label:` to something harmless, or the item is printed.
2. `diff: false` on any task that writes secret material (copy/template of a key, a `.cnf` with a password). With AAP's *Show changes* enabled, a diff prints the file's contents.
3. **Never put a secret on a command line.** Command lines are visible in `ps`, in audit logs (`execve` records), and in verbose job output. Instead:
   - pass it on **stdin**: `podman secret create NAME -`, `podman login --password-stdin`;

- or in a **file** the program reads (`--defaults-extra-file` for MySQL clients), created with mode `0600` and removed afterwards;
- never `mysql -pPassword` or `curl -u user:password`.
- never through Ansible's `environment:` keyword on a Linux host with become: Ansible puts those variables on the `sudo /bin/sh -c '...'` command line, which `ps` shows and sudo writes to the host's journal / `/var/log/secure` - `no_log` does not prevent that. Send the secret on the task's `stdin:` to a small `sh` that exports it (see `roles/check_mariadb/tasks/health.yml`).

4. **On the hosts, secrets are podman secrets**, not environment lines in unit files:
   - mounted as files where the image supports it (`MYSQL_ROOT_PASSWORD_FILE`, `MYSQL_PASSWORD_FILE`), with `mode=0400` and the container user's uid;
   - injected as an environment variable only when the application cannot read a file (`STIGMAN_DB_PASSWORD`): `Secret=NAME,type=env,target=VAR`.

Podman keeps the value out of the unit file and out of `podman inspect` (checked on podman 5.8; verify on your RHEL's podman). It is still readable by root on the host and inside the container, so protect root and the container.

5. **Private keys on disk:** `0600`, owned by root; certificate and CA files `0644`.
6. **Don't register command output that can contain secrets** without `no_log`, and never `debug: var=` a registered result from such a task.
7. **Fail closed**: assert a secret is present (with `no_log`) instead of letting a task run with an empty password.
8. **Changing a secret in AAP is not the same as rotating it.** For a database password the value inside MySQL must change too (`ALTER USER`). The deploy role refuses to silently swap a deployed database password; rotation is a separate, deliberate run.

The real trust boundary

AAP will not show a stored secret, but **any playbook it runs can print it**. So anyone who can:

- push to the branch or tag the AAP project uses,
- edit a job template (its playbook, credentials, or verbosity), or
- run a template with *Prompt on launch* for variables that change what the playbook does,

can extract every secret that template uses. Therefore:

- **protect the branch** (required reviews) and pin the project to a **tag**;
- give most people **Execute** on templates, not **Admin**; give only automation owners **Use** on credentials;
- keep *Prompt on launch* for variables off on templates that hold production credentials;
- review the **Activity Stream** (who changed which credential or template) regularly.

Keeping secrets out of Git

- `.gitignore` in this repository blocks common key/cert/vault-password file names.
- `.pre-commit-config.yaml` runs **gitleaks** on each commit (`pre-commit install`).
- The inventory in `inventories/example/` holds placeholders only; the real inventory lives in your work Git or as an AAP inventory.
- **No real host names or addresses either.** Every name in the repository is a placeholder (`vc01.example.mil`, `smtp.yoursite.mil`), every IP address is from the ranges kept for documentation

(`192.0.2.x`, `198.51.100.x`, `203.0.113.x`), and the README and the changelog talk about features, never about one site. `tests/check_public.py` checks it in CI and on each commit. Your own site's words (a site code, a vCenter's name, a team, a ticket title) go in `~/.config/site-automation/private-words`, one per line, on the machine you commit from: they are looked for too, and the file never enters the repository.

- If a secret is ever committed: rotate it first, then remove it from history. Removing it from history alone is not enough, because clones and caches keep it.

Rotation checklist

Secret	How
TLS certificate	update the **TLS certificate** credential, re-run `stigman_deploy.yml` (nginx restarts)
Registry token	update the credential; the next run logs in with it
STIG Manager DB password	password-rotation run: `ALTER USER` in MySQL, update the podman secret, restart the API; then update the AAP credential. The playbook for this comes with the day-2 set
SSH key	add the new public key to the hosts, update the **Machine** credential, remove the old key
ACT / Teams	update the credential; nothing else
ServiceNow API account	change the password in ServiceNow, update the **ServiceNow API** credential, then run *ServiceNow - test ticket* (SERVICENOW_SETUP.md) to prove it works
MariaDB monitor account	change it in the database (`ALTER USER`), update the **MariaDB monitor** credential, run a health check with `health_checks=mariadb` (MARIADB.md)
WinRM login	change it in the directory, update the Windows **Machine** credential, run *Windows connection test*

14. Approved commands: what ACT may run by itself

ACT is a tool that runs **on each Linux server** when a job asks it to. This page explains, in plain words, which commands ACT runs by itself, which ones wait for a person, and how you add a command to the "run without asking" list.

Short version. There are three kinds of command:

Kind	Examples	What happens
Reading	`systemctl status chronyd`, `df -h`, `journalctl -u nginx`, `ss -tlnp`	ACT runs it. It never asks. **Nothing on this page limits reading.**
Changing, not on your list	`systemctl restart nginx`, `dnf -y install httpd`	Refused. The job says `NEEDS APPROVAL` and shows the exact command. A person approves it in the workflow.
Changing, on your list	the ones you wrote in `site_act_allow`	ACT runs it by itself (only at level `self-heal`).

The list is one setting: `site_act_allow`. You add a command by adding one line to it.

Read this first: two safety rules that you cannot switch off

1. **Some commands are never run by a job**, whatever you write. ACT itself never pre-approves a *catastrophic* command (recursive delete, disk or volume destruction, power off, reboot). And the step that applies an approved fix (the *Apply approved ACT fix* job, and the service-watch fix step) has its own last check: reboot, shutdown, `kexec`, `mkfs`, `fdisk`, `usermod`, `passwd`, recursive deletes of system folders and a few more are refused **even after a person approved them** - also with a path in front (`/sbin/reboot`) or inside `bash -c '...'` / `$( )`. Patching and restarts have their own job: RUNBOOKS.md, section "Patch hosts".
2. **AAP itself is never fixed by a job.** Hosts in the groups `aap` and `aap_hosts` are diagnose-only: ACT reads and proposes, and the apply step prints the command for a person to run by hand. Nothing you set can remove those two groups (they are built into the code).

Everything below works inside those two rules.

How a command becomes "approved": the three ways

Way 1: a person approves it (the workflow), nothing to configure

This is the default at level `diagnose`.

1. The health check finds a problem. ACT investigates by reading, then proposes the fix.
2. Because the fix changes something and is not on any list, ACT is **refused** and the job fails with `NEEDS APPROVAL - ... ACT proposes: systemctl start chronyd`.
3. The workflow stops at its **Approval** step. The approver reads the proposal and approves or denies.
4. *Apply approved ACT fix* runs **exactly the approved text**, no model involved, then the checks run again.

What the approver sees is what runs. To set this up, see ADDING_ACT.md and WORKFLOWS_AND_SCHEDULES.md.

Two details worth knowing:

- Only a **plain single command** can go through this path (see the character rule below). A proposal such as `cd /x && ./fix.sh` or `something | tee file` is listed in the report as a **manual fix**: a person runs it by hand. That is deliberate.
- The approval covers that command on the hosts that proposed it, once. It does not add the command to any list. Next week the same problem needs the same approval.

Way 2: you put it on the list (site_act_allow), for fixes you trust

Use this for boring, safe, repeated fixes: start a stopped service, restart a hung one. It only takes effect at level `self-heal`. At `explain` and `diagnose`, the list is ignored.

YAML

```yaml
site_act_allow:
  - 'systemctl restart (chronyd|rsyslog|crond)'
  - 'systemctl start (chronyd|rsyslog|crond)'
```

Where to put it, step by step, is in the next section.

Way 3: --allow on the ACT command line (when you run ACT by hand)

If you run `act` on a server yourself, the same idea is `--allow`:

shell

```
act --allow 'systemctl restart (nginx|httpd)' "the web server is down; find out why and fix it"
```

`site_act_allow` is this, done for you: each list item becomes one `--allow` when the job calls ACT. (Full ACT reference: "Approved commands" in ACT's own README.)

At the prompt that ACT shows when you run it by hand (`run? [y]es / [n]o / [e]dit / [q]uit`) there is **no "always allow" answer**. This is on purpose: the list is the only way to make ACT stop asking, and the list is in Git, where a reviewer can see it.

Add an approved command, step by step

You need: VS Code with this repository, and the command as you would type it on the server.

Step 1: write the command exactly as ACT would run it

Try it by hand on a lab server first: `sudo systemctl restart chronyd`. Note the words in between: `systemctl restart chronyd` (drop `sudo`; ACT already runs as root on the host).

Step 2: turn it into a pattern

A pattern is a *regular expression* that must match the **whole** command, from the first character to the last. `systemctl restart chronyd` on its own is already a valid pattern that matches exactly that command and nothing else. Start there. Widen it only when you have to:

You want	Pattern	Matches	Does not match
exactly one command	`systemctl restart chronyd`	`systemctl restart chronyd`	`systemctl restart chronyd.service`
a short list of services	`systemctl restart (chronyd|rsyslog|crond)`	`systemctl restart rsyslog`	`systemctl restart sshd`

You want	Pattern	Matches	Does not match
restart or start	`systemctl (start|restart) (chronyd|rsyslog)`	`systemctl start rsyslog`	`systemctl stop rsyslog`
a unit name with a dot	`systemctl restart chronyd\.service`	`systemctl restart chronyd.service`	`systemctl restart chronydXservice`
a container	`podman restart (mysql|nginx)`	`podman restart nginx`	`podman rm -f nginx`

(In the YAML file, write `|` normally. The `\|` above is only how a table shows a bar.)

Rules for the pattern:

- **The whole command must match.** `systemctl restart .*` looks like "restart anything", but it can only match a plain command. It is also far too wide: it would let ACT restart `sshd` and the network. Name the services.
- **Only a plain single command counts.** The command may contain only letters, digits and these characters: `space _ . / : = @ % + , -`. Anything else is never pre-approved, however you write the pattern: no `; && ||` (chaining), no `|` (pipes), no `> <` (redirection), no `$( )` or backticks, no `*` or `?` (globs), no quotes, no `~`, no line breaks. Such a command still needs a person.
- **A dot in a pattern means "any character".** Write `\.` when the exact dot matters, as in `chronyd\.service`.
- **No file edits.** A pattern covers `run` commands only; ACT changing a file (edit or write) always needs a person.
- **Dangerous commands are never approved by a pattern**, whatever you write: ACT's danger tier (package removal such as `dnf -y remove httpd`, `usermod -aG wheel alice`, `chmod 777`, `podman rm -f mysql`, `rm -f`, ...) and reboots, power-off and disk tools always need a person (tested with ACT 0.6.21). They come back as `NEEDS APPROVAL` / manual fixes.
- **Never put these on the list** anyway: `.*` on its own, `systemctl (stop|disable|mask) .*`, `kill .*`. Those are ordinary changes a pattern *can* approve, and far too wide: `systemctl stop .*` would let ACT stop `sshd`.

Step 3: put it in the right settings file

`site_act_allow` is a setting like any other (VARIABLES.md explains where settings live). Pick the place that matches who should get it:

Who gets the command	Where to write it
Every host	`playbooks/group_vars/all.yml`: add the lines at the end of the file (copy the example below).
One group (for example `stigman`)	`playbooks/group_vars/stigman.yml` (the file name is the AAP group name)
One host	`playbooks/host_vars/<host name as in the inventory>.yml` (create the folder and file if they do not exist)
One template, every run	the job template's **Variables** box (extra variables), YAML: `site_act_allow: ['systemctl restart chronyd']`

A list replaces, it does not add. If `all.yml` has two commands and `stigman.yml` has one command, the stigman hosts get **only that one**. Repeat the ones you keep. (The exact order of who wins is in VARIABLES.md.)

Example, `playbooks/group_vars/all.yml`:

```yaml
site_act_allow:
  - 'systemctl (start|restart) (chronyd|rsyslog|crond)'
```

Use single quotes. Keep the indentation. Spaces, never tabs.

Step 4: save, commit, sync

In VS Code: save, **Commit**, **Sync Changes**. AAP picks the file up on the next run (the project syncs when *Update revision on launch* is ticked).

Step 5: use the level self-heal

Launch the check with `use_act` = `yes` and `site_act_level` = `self-heal` (the survey question on the Health check and Troubleshoot templates; ADDING_ACT.md).

How to verify it worked

Do all three. They take ten minutes and catch the usual mistakes.

1. Check the pattern matches (and only what you meant). No server needed. In PowerShell:

```
$p = 'systemctl restart (chronyd|rsyslog|crond)'      # your pattern, without the quotes' worth of
  YAML
foreach ($c in 'systemctl restart chronyd',            # should be True
               'systemctl restart sshd',               # should be False
               'systemctl restart chronyd; reboot') {  # should be False
  '{0,-40} {1}' -f $c, ($c -cmatch ('\A(?:' + $p + ')\z'))
}
```

Expected output: `True`, `False`, `False`. (Verified. The same test in Python, on a machine that has `vendor/act/act`: load it with `importlib` and call `compile_preapproved`/`preapproved_pattern`; ACT applies exactly `\A(?:your pattern)\Z`.) Add a line for every command you want to *stay* refused and check it prints `False`.

2. Check ACT itself agrees, on a lab server. Log in, put your key in the environment the way SECRETS.md describes, and run ACT in the same mode a job uses. It will not ask anything; it either runs what you allowed or refuses:

```shell
sudo systemctl stop chronyd                # break it on purpose (lab server only)
act --non-interactive --allow 'systemctl (start|restart) chronyd' \
    --result-file /tmp/r.json "chronyd is stopped; find out why and fix it"
echo "exit code: $?"                       # 0 = fixed, 4 = something was refused
python3 -m json.tool /tmp/r.json | less    # see commands[].approval and denied[]
```

In `/tmp/r.json`: `commands[].approval` says `pre_approved` for what your list allowed (with the `pattern` that matched) and `auto` for reads. `denied[]` lists what was refused, each with `pre_approvable`: `true` means "a pattern could approve this" (add one if you want it); `false` means a person has to do it.

3. Check the job. On the same lab server, launch **Health check** with `use_act` = `yes`, `site_act_level` = `self-heal`. In the output:

- `ACT | what ACT says` shows `APPLIED (pre-approved): systemctl start chronyd`;
- the report afterwards says `cleared: ...`.

Then break something that is **not** on your list and check the job says `NEEDS APPROVAL` and does not touch it. A dry run (job type *Check*) does not call ACT, so it cannot test this; see DRY_RUNS.md.

If it does not work

You see	Why	Fix
`NEEDS APPROVAL` for a command you listed	the pattern does not match the **whole** command; or a listed command contains a `;`, `|`, quote or glob; or the level is `explain`/`diagnose`, not `self-heal`	run check 1 with the exact command from the job output; fix the pattern or the level
the command is in `manual fixes`, not `proposed`	it is not a plain single command	ask ACT for a simpler fix, or apply by hand
the setting seems ignored	a more specific place wins (the group or host file, or the template's Variables box)	see the order in VARIABLES.md; the job prints a notice when a settings file overrides the AAP Variables box
an invalid pattern	ACT does not run: the job shows `ACT error ... invalid --allow pattern '...'` and the reason (for instance an unbalanced bracket)	fix the YAML/regex
a command on an AAP host does not run	by design: AAP is diagnose-only	run it by hand from the printed line
the approved command was refused at apply time	the apply-time guard (reboot, shutdown, `mkfs`, `usermod`, recursive delete of a system folder, ...)	do it by hand if you really mean it

Reference: what the guard rails are

Where	Rule	Can you change it?
ACT, every command	proven read-only commands run without asking; everything else asks	no (it is the design)
ACT, `--allow` / `site_act_allow`	anchored to the whole command; plain characters only; never catastrophic; never a file edit	you choose the patterns
ACT, `--auto`	runs reads and ordinary changes without asking, still asks for the danger tier. Not used by these jobs	not used
`site_act_diagnose_only_groups`	hosts where ACT never fixes; the list *you* write replaces the shipped one (`sat`, `idm`, ...), but `aap`, `aap_hosts` are always kept	you choose the groups; you cannot remove `aap`/`aap_hosts`
Apply-time guard (`_apply_refuse`, `roles/site_findings/vars/main.yml`)	last check before an approved command runs	edit the code to change it
Host log	`/var/log/site-automation-fix.log` on each host lists what the apply job ran, when, and for which AAP job and user	

The apply-time guard applies to the **approval** path (Way 1). Commands run by **self-heal** (Way 2) are run by ACT itself, so what protects you there is your list. Keep it short and boring.

Windows

`site_act_allow` is read by the Linux jobs. Windows servers use the approval path only (ACT proposes, a person approves); see WINDOWS.md. If you run ACT for Windows by hand, its `-Allow` option follows the same rules as Linux `--allow`:

	Linux ACT (--allow)	Windows ACT (-Allow)
A pattern that names a danger-tier command (for example `dnf -y remove httpd`, `podman rm -f mysql`)	**does not** approve it	**does not** approve it
A danger-tier command in an unattended run	**always refused**; only a person at an interactive prompt can approve it	the same
Catastrophic commands	never	never
`git`, cmd built-ins, aliases such as `ls` and `dir` in an unattended run	follow the rules above	**still ask**, because they were never on Windows ACT's unattended list
Result file (`--result-file` / `-ResultFile`)	written (since 0.6.16)	written (since 0.6.16; its `tokens` field is filled since 0.6.21)

(ACT 0.6.20 for Linux, released for a few minutes on 2026-09-30, let a pattern approve a danger-tier command; 0.6.21 and later (this project carries 0.6.22) do not. The Windows rows are taken from ACT-Windows 0.6.22 and its self-tests and were not run by hand on a Windows server.)

15. The MariaDB and MySQL check, step by step

There are **two ways to run the database check**, and both check **MariaDB and MySQL** (MySQL 5.7, 8.0 and 8.4, MariaDB 10.x and 11.x, and Percona), installed on the host or **in podman containers** (one, several, found automatically, rootless):

How	Job template	Use it when
Its own template	**Database health - MariaDB / MySQL** (`playbooks/database_health.yml`; SETUP_AAP.md)	the database team wants its own job, schedule and tickets. Runs only this check, on every database server it is pointed at
Part of the Health check	**Health check** with `health_checks` = `mariadb`, `mysql` or `database` (all three are the same check; `daily` includes it)	you already run the Health check and want the databases in it

This page shows how to turn the check on, how to give it a login, and how to see that it works. Nothing here needs you to know Ansible. It changes nothing on the database: the check only asks status questions and never reads your data.

Words used here:

- **container name** - the name podman gives a container. You read it with `podman ps -a`.
- **rootless** - the container belongs to an ordinary user (not root). Only that user can see it.
- **settings file** - a small file in `playbooks/group_vars/` that holds your choices for one group of hosts. See VARIABLES.md.

The short version

1. On the database host, find the container name: `sudo podman ps -a` (rootless: `sudo -u THE_USER podman ps -a`).
2. Put the name in the settings file for your MariaDB group (section 3).
3. Ask the DBA for a read-only monitoring account and put it in an AAP credential (section 4).
4. Run the **Database health** job (or the *Health check* with `health_checks` = `mariadb`) and read "How to verify" below.

1. Which hosts the check runs on

The check runs on a host when **any** of these is true:

- the host is in one of the inventory groups `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb` or `mysql` (any of the five works, for either engine), or
- the host has a container name set (section 2), or
- you set `check_mariadb_enabled: true` for the host or its group.

The **Database health** template does not need this: every server it runs on is checked. It runs on the groups `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb` and `mysql` (since 0.6.0; before that only the first three). A **Limit** only narrows that list; it cannot add a server that is in none of them. For a server outside those groups, put `target: <group or host>` in the template's **Variables** box, or add the server to one of those groups. If `target` matches no host, the job stops with a message that lists the groups your inventory does have. On every other host, in the Health check, the job prints `skipped: the MariaDB/MySQL check is`

`off for <host>` and moves on. That is normal; it is not an error.

2. Tell it where MariaDB is

Pick **one** of the four ways. Put the lines in the settings file (section 3).

Your situation	Setting
MariaDB or MySQL is installed on the host itself (a service)	nothing to set. The check finds the service `mariadb`, `mysqld` or `mysql` and tells the engine and version apart by itself. To name the service: `check_mariadb_service: mariadb`
In a podman container, with **no** database server installed on the host, or started by a unit that runs podman (`podman compose up -d`, `podman start` - such a unit shows `active (exited)` or `inactive` while the database runs)	usually nothing to set (0.7.0 and later): the check sees that and finds the container by its image. Naming it is still clearer: `check_mariadb_container: mariadb`
One container	`check_mariadb_container: snow-mariadb`
Several containers on the same host	`check_mariadb_containers: [snow-mariadb, other-db]`
You do not want to list them: find them	`check_mariadb_container_discover: true`

Details that matter:

- **Discovery** looks at every container on the host, running or not, and picks the ones whose **image name** matches `check_mariadb_container_image_regex` (default `(mariadb|mysql|percona)`). If your image has another name, change the pattern, for example `'(mariadb|snowdb)'`. Discovery that finds nothing is reported as a warning `mariadb:no-container`, so a wrong pattern is not silent.
- With **several** instances on one host, each is checked on its own. Their findings start with the container name, for example `[snow-mariadb] 143 of 151 connections in use`.
- The check runs the database's own client **inside** the container (`podman exec`), so you do not need a client on the host and you do not open any network port.
- The engine (MariaDB or MySQL) and its version are read from the server, so the same settings work for both.
- A container that is named but does not exist is reported as `mariadb:down` ("MariaDB/MySQL is not running (container ... is missing)"), not skipped.

Rootless containers

When the containers belong to a user, name that user:

```yaml
check_mariadb_container_user: appuser
```

The check then runs podman **as that user** (the job needs privilege escalation, which the templates in SETUP_AAP.md already have). If that user does not exist on the host you get the critical finding `mariadb:user`, with the hint `getent passwd <user>`. Leave the setting empty for containers that belong to root.

3. Where to put the settings

Open `playbooks/group_vars/mariadb.yml` (MySQL: `playbooks/group_vars/mysql.yml`) in VS Code. It already holds the lines below as comments. Remove the `#` in front of the ones you need and replace `CHANGE-ME`:

```yaml
check_mariadb_enabled: true
check_mariadb_container: snow-mariadb
# check_mariadb_container_user: appuser      # only for rootless containers
```

A settings file applies to the group with the same name as the file. `mariadb.yml` is for the group `mariadb`, `mysql.yml` for `mysql`. If your hosts are in a group with another name (the example inventory uses `mariadb_hosts` and `mysql_hosts`), copy the file to `playbooks/group_vars/<your group name>.yml`. The check accepts all five group names, but the *settings* are only read from the file that matches the group. (The **Database health** template runs on the `_hosts` groups only: for a group called `mariadb` or `mysql`, give it a `target`, see section 1.) One host that differs from the rest: `playbooks/host_vars/<host name>.yml` (create it). Which place wins, and how to set the same thing in AAP's own Variables boxes, is in VARIABLES.md.

Then commit, push, and run **Sync** on the project in AAP.

4. The login: the "MariaDB monitor" credential

Where the database runs	What to do
MariaDB installed on the host	Usually nothing. Run with privilege escalation and the check logs in as root through the local socket (RHEL's default).
MySQL installed on the host	Usually the credential: MySQL's root account normally has a password.
In a container (either engine)	Use the credential. The official images give root a password, and without it the job reports a finding that contains `Access denied` and ends with `attach the "MariaDB monitor" credential`.

Steps:

1. Ask the DBA to run this **inside that database** (the account needs no data access). The privileges differ by engine:

```sql
CREATE USER 'aap_monitor'@'localhost' IDENTIFIED BY '<password>';

-- MySQL 5.7 / 8.0 / 8.4:
GRANT PROCESS, REPLICATION CLIENT ON *.* TO 'aap_monitor'@'localhost';
-- MySQL Group Replication only, also:
GRANT SELECT ON performance_schema.* TO 'aap_monitor'@'localhost';

-- MariaDB 10.5.9 and newer:
GRANT PROCESS, REPLICA MONITOR ON *.* TO 'aap_monitor'@'localhost';
-- MariaDB before 10.5.9:
GRANT PROCESS, REPLICATION CLIENT ON *.* TO 'aap_monitor'@'localhost';
```

On MariaDB 10.5.9 and newer, `REPLICATION CLIENT` **is not enough:** it no longer covers the replication status, so the account needs `REPLICA MONITOR`. `PROCESS` lets the check see other sessions (for long queries). Without the replication privilege the check still works and reports `mariadb:replication-query` (below) with

the grant to ask for. The host is `localhost` because the client runs inside the container.

2. In AAP create a credential of type **MariaDB monitor** (the type is in `aap/credential_types/mariadb_monitor.yml`; SETUP_AAP.md shows how to load it). Type the account name and password into the credential. **Never** put the password in a settings file, the inventory or Git: SECRETS.md.
3. Attach the credential to the **Database health**, **Health check** and **Troubleshoot** job templates. Credentials attach to templates, not to groups; the check only uses it on hosts that have a database.

The password reaches the client only through its environment (`MYSQL_PWD`, and `podman exec -e MYSQL_PWD` for containers), never on a command line. If your client needs other options, for example a socket path or a TCP address, use `check_mariadb_client_args: ['--socket=/var/lib/mysql/mysql.sock']`.

5. What it checks, and the limits you can change

Replication is read with the statement that fits the engine: `SHOW ALL SLAVES STATUS` (MariaDB), `SHOW REPLICA STATUS` (MySQL 8.0.22 and newer, 8.4) or `SHOW SLAVE STATUS` (older MySQL). The job's status line names the engine, for example `MySQL 8.4.11 - container snow-mysql (podman, image ...): up 312.4 h, 23/151 connections, ...`.

Each limit is a setting. The defaults are in the table; change one the same way as in section 3.

Finding	Turns on when	Setting (default)
`mariadb:down`	the service or container is not running, or the named container does not exist	-
`mariadb:connect`	it runs but the status query failed (login, hang)	-
`mariadb:restarted`	restarted less than this many minutes ago	`check_mariadb_restart_warn_minutes` (60)
`mariadb:connections`	connections in use, percent of `max_connections`	`check_mariadb_conn_warn_pct` (80), `check_mariadb_conn_crit_pct` (95)
`mariadb:refused`	clients were refused because the limit was reached	-
`mariadb:threads-running`	queries executing at this moment	`check_mariadb_threads_running_warn` (32)
`mariadb:long-queries`	a query ran longer than this many seconds (never the query text)	`check_mariadb_long_query_seconds` (300)
`mariadb:bufferpool`	percent of reads served from memory (judged after 1 million reads)	`check_mariadb_bufferpool_hit_warn` (95)
`mariadb:replica:<name>`, `mariadb:replica-lag:<name>`	replication stopped, or behind by seconds	`check_mariadb_replica_lag_warn` (300), `check_mariadb_replica_lag_crit` (1800)

Finding	Turns on when	Setting (default)
`mariadb:replication-query`	the replication status could not be read (usually a missing privilege; the finding names the grant)	section 4
`mariadb:galera`, `mariadb:galera-size`	Galera / XtraDB Cluster node not healthy; cluster smaller than expected	`check_mariadb_galera_size` (0 = not Galera; set 3 for three nodes)
`mariadb:group`, `mariadb:group-size`	MySQL Group Replication: a member is not ONLINE (or none is listed); fewer ONLINE members than expected	`check_mariadb_group_size` (0 = do not check the size; set 3 for three members)
`mariadb:group-query`	the Group Replication members could not be read (needs `SELECT` on `performance_schema`)	section 4
`mariadb:errorlog`	`[ERROR]` lines in its log this recent (days)	`check_mariadb_error_since_days` (1)
`mariadb:disk`	the disk holding the data directory is this full (percent)	`check_mariadb_disk_warn_pct` (85), `check_mariadb_disk_crit_pct` (95)
`mariadb:user`	`check_mariadb_container_user` names a user that does not exist	-
`mariadb:no-container`	discovery found no container with a matching image	`check_mariadb_container_image_regex`
performance (warnings): `mariadb:slow-queries`, `tmp-disk`, `full-joins`, `table-locks`, `row-locks`, `deadlocks`, `bufferpool-wait`, `log-waits`, `aborted-connects`, `table-cache`	averages since the server started, judged after it has run 2 hours: slow queries per hour, % of temporary tables on disk, joins without an index per hour, table lock waits, average row lock wait, deadlocks per hour ...	`check_mariadb_performance` (true), `check_mariadb_perf_min_uptime_hours` (2), `check_mariadb_slow_per_hour_warn` (60), `check_mariadb_tmp_disk_pct_warn` (25), `check_mariadb_full_join_per_hour_warn` (100), `check_mariadb_table_lock_wait_pct_warn` (1), `check_mariadb_row_lock_avg_ms_warn` (500), `check_mariadb_deadlocks_per_hour_warn` (1), `check_mariadb_innodb_waits_per_hour_warn` (1), `check_mariadb_aborted_connects_per_hour_warn` (60), `check_mariadb_table_open_per_hour_warn` (600)
performance, now (warnings): `mariadb:lock-waits`, `long-transactions`, `purge-lag`, `open-files`	a session waiting this many seconds for a lock; a transaction open this long (idle ones are named); InnoDB history list length; files open, % of `open_files_limit`	`check_mariadb_lock_wait_seconds` (30), `check_mariadb_long_trx_seconds` (600), `check_mariadb_history_length_warn` (1000000), `check_mariadb_open_files_warn_pct` (85)

The finding ids keep the `mariadb:` prefix for MySQL too, so existing tickets and filters still match. What each finding means and the usual cause are in RUNBOOKS.md. Every setting with its default is in VARIABLES_REFERENCE.md. With several containers on one host, the finding id includes the container

name, for example `mariadb:snow-mariadb:connections`.

All settings of this job, with their defaults: check_mariadb.

How to verify

1. **The names are right.** On the database host: `sudo podman ps -a` (rootless: `sudo -u THE_USER podman ps -a`). The name you put in the settings file is in the NAMES column.
2. **The account works.** On the host, run the client inside the container as the monitoring account (it asks for the password; type it here, not in a file):

```shell
sudo podman exec -it snow-mariadb mariadb --user=aap_monitor -p -e "SHOW GLOBAL STATUS LIKE
  'Uptime'"
```

A number back means the login and the `PROCESS` rights work. (Some images have `mysql` instead of `mariadb`.)

3. **Run the job.** In AAP launch **Database health - MariaDB / MySQL** (first as *Check*, then as *Run*; SETUP_AAP.md), or the **Health check** with `health_checks` = `mysql` (or `mariadb`, or `database`) and a Limit of the database host. In the output, find the task `MariaDB | status`. It prints one line per instance, like `MariaDB 11.8.9 - container snow-mariadb (podman, image ...): up 312.4 h, 23/151 connections, 1 running, 0 long queries, replication connections 0`. That line proves the check reached the database. Each host then prints `HEALTHY` or its findings, and the job ends with `N healthy, M with findings`.
4. **No findings** means the database is healthy by these limits. To see a finding on purpose, set a limit that is sure to trip, for example `check_mariadb_conn_warn_pct: 1`, run again, look for `mariadb:connections`, then remove the line.

What was tested, and what was not

Tested against real containers (throwaway rootless podman)	Tested only against a fake podman
MySQL 8.0.46, MySQL 8.4.11, MariaDB 11.8.9: the engine and version are recognised; a replication source that never connects is reported as broken; the error log is read; a login with no credential gives `mariadb:connect` with the grant to ask for; the `REPLICA MONITOR` grant and revoke on MariaDB behave as described above	MySQL 5.7, and MySQL Group Replication (`mariadb:group`, `group-size`, `group-query`). They follow the documented statements but have not met a real server. The **performance** findings (0.15.0): their queries use documented status variables and `information_schema` tables of both engines, but only the fake has answered them so far.

The fake-podman test (`bash tests/mariadb/run_mariadb_test.sh`) covers one container, several, discovery, a stopped and a missing container, a rootless user that does not exist, MySQL 5.7, 8.0 and 8.4, replication, Group Replication, a denied grant, and that the password never appears on a command line. Nothing here has been run against **your** database: step 3 is the real proof.

If it does not work

You see	Cause	Fix
`skipped: the MariaDB/MySQL check is off for <host>` (Health check only)	the host is in none of the five groups and no container is named	section 1, or use the **Database health** template
`mariadb:replication-query`	the account lacks the replication privilege (MariaDB 10.5.9+: `REPLICA MONITOR`, not `REPLICATION CLIENT`)	section 4
`mariadb:no-container`	discovery matched nothing, or podman is run as the wrong user	`podman ps -a` as the right user; set `check_mariadb_container_image_regex` or `check_mariadb_container_user`
`mariadb:user`	the rootless user name is wrong	`getent passwd <user>` on the host
`status query failed: ... Access denied ...` and `attach the "MariaDB monitor" credential`	container root has a password	section 4
`... is not running (container ... is missing)` but it is running	the name is wrong, or the containers belong to another user	compare with the `podman ps -a` output; `check_mariadb_container_user`
The check runs on the wrong host, or not at all after a rename	a settings file only applies to the group named like the file	section 3
`MariaDB is not running (service mariadb is inactive)` - or a login error from the host's client - but `podman ps` shows the database container up	the host's `mariadb.service` only starts the container (`podman compose up -d`); before 0.7.0 the check took it for the database	0.7.0 and later find the container by themselves; or name it: `check_mariadb_container: mariadb`
Database health says `skipping: no hosts matched` (0.5.0), or `target '...' matches no host`	the server is in none of `mariadb_hosts`, `mysql_hosts`, `database_hosts`, `mariadb`, `mysql` (a Limit cannot add it)	section 1: `target` in the template's Variables, or add it to one of those groups
Your setting has no effect	another place wins	VARIABLES.md

16. Active Directory health report

One job, `playbooks/ad_health_report.yml`, reads every domain controller (DC) in the Windows inventory and emails one report for the whole domain: what is broken first, then replication, DNS, time and SYSVOL, then **what changed** in the last 24 hours (privileged groups, accounts, Group Policy, schema and configuration). Each row is coloured: red = critical, amber = warning, green = OK, blue = for your information.

It only reads. Each DC runs a PowerShell script over WinRM (the same way the Windows health check does) that queries Active Directory, runs `dcdiag` tests that only read, `repadmin /showrepl`, `/queue`, `/showutdvec` and `/showbackup`, `w32tm /query` and `/stripchart`, `nltest /sc_verify`, and reads the event logs. It changes nothing: it does not run dcdiag's `DnsDynamicUpdate` test (that one writes a test record), and it does not run `repadmin /syncall`, `/replicate` or anything else that acts.

No second hop. Every DC asks only itself (`localhost`). AD is replicated, so the DC that reads the domain has every object in its own copy, and the replication metadata says on which DC a change was made. So the job works over WinRM with NTLM or Kerberos and needs no delegation or CredSSP.

What the report shows

At the top: the title with the number of critical and warning findings, then boxes with the main counts (DCs read, replication, dcdiag, services, SYSVOL / GPO, FSMO / DNS / time, privileged changes, accounts created / deleted, GPO changes). Then, in this order:

Section	What it shows	Colours
Domains	each domain: functional levels, schema version, DCs checked / in AD, the PDC emulator, tombstone lifetime, Recycle Bin, RID pool left, the oldest partition's last backup, the krbtgt password's age, the SYSVOL migration state	amber Recycle Bin off, krbtgt older than 180 days, FRS; red RID pool under 2 %, backup older than 30 days or the tombstone lifetime
Domain controllers	each DC: site, OS, GC / RODC, the FSMO roles it holds, LDAP answering (and synchronized), DC services not running, dcdiag's failed tests, SYSVOL, its clock, the least free space of its database, log and SYSVOL volumes, uptime. A DC in AD that is not in the inventory is listed in blue	red unreachable, LDAP down, not synchronized, a DC service stopped
Operations masters (FSMO)	who holds each role, whether that holder still exists as a DC, whether it answers on LDAP	red holder gone or not answering
Inbound replication	links with a problem only (`repadmin /showrepl` on each DC): partition, partner, failures, last success, last error with what the code means	amber 6 h without success, red 24 h, a failing link, or longer than the tombstone lifetime; amber 50+ requests in the queue

Section	What it shows	Colours
Replication from every DC (up-to-dateness)	how long ago each DC's latest change reached the DC that read the domain (`repadmin /showutdvec * /latency`): this also catches a DC that is **not in the inventory** and stopped replicating	amber 6 h, red 24 h
dcdiag	the failed tests and what dcdiag said. `SystemLog` is skipped by default (the Events section covers it)	red Connectivity, Advertising, NetLogons, Replications, SysVolCheck, Services, KnowsOfRoleHolders, RidManager, MachineAccount; amber the others
DNS	dcdiag's DNS tests that only read (basic, forwarders, delegation, record registration), and the SRV records `_ldap` / `_kerberos` (`dc._msdcs`) and `_ldap._tcp.gc`: every DC must be registered	red record missing; amber a DC not registered, a DNS test failed
Time	each DC's source, stratum, last sync, offset: a DC against the PDC emulator, the PDC against its NTP source	red the PDC on its own clock, 60 s off; amber 5 s off, no sync for 24 h
Events	the Directory Service, DFS Replication and DNS Server logs, and the Netlogon, KDC, time and Group Policy entries of the System log: the events that matter on a DC, with what they mean (2042, 1388 / 1988 lingering objects, 2087, DFSR 2213 / 4012, LDAP signing 2887, Netlogon 5719 ...), and every other error in blue	per event
Clients in no site	client addresses in no AD subnet (`netlogon.log` NO_CLIENT_SITE), and subnets linked to no site	amber
Certificates (LDAPS and Kerberos)	the newest LDAPS and KDC certificate of each DC	amber 30 days, red 7 days; blue none
Trusts	every trust; outbound and two-way trusts verified (`nltest /sc_verify`)	red does not verify
Sites	each site, its DCs and subnets	blue no DC or no subnet
Privileged group changes	members added to or removed from Domain / Enterprise / Schema Admins, Administrators, Account / Server / Print / Backup Operators, Group Policy Creator Owners, Key Admins, Cert Publishers, DnsAdmins (and your own): when, which DC, **by whom**	amber (`ad_privileged_change_severity`)

Section	What it shows	Colours
Privileged groups	their direct members now	amber a group in `ad_groups_should_be_empty` with a member
Users, groups and computers created or deleted	when, where, by whom	blue
Accounts enabled, disabled, locked or unlocked	when, which DC, by whom; the accounts locked out now	blue
Group Policy changes	GPOs created, changed, deleted; OUs, domains and sites whose links changed (and what is linked there now)	blue
Group Policy: AD against SYSVOL	GPOs with no SYSVOL folder, GPOs whose version differs in AD and SYSVOL, SYSVOL folders with no GPO	amber / blue
Schema and configuration changes	schema version and new classes / attributes, sites, subnets, site links, DCs added or removed, new trusts	amber schema and trusts; blue the rest (`ad_dc_change_severity`)
Security log: notable events	the Security log cleared on a DC (red), the DSRM password set, trusts created / removed, the domain policy changed	per event
Not read	anything a script could not read, with the reason	

When and **which DC** come from AD's replication metadata, which every DC keeps: no audit policy is needed for them. **By whom** comes from the Security log of the DC the change was made on (events 4720, 4722, 4725, 4726, 4728-4733, 4741, 4743, 4756, 4757, 4767), so it is filled in only when *Audit Security Group Management*, *Audit User Account Management* and *Audit Computer Account Management* are on (the DISA STIG asks for all three), and that DC is in the inventory.

The job's artifacts (`ad_health`) have every finding, for a workflow or ServiceNow.

Before you start

1. The DCs in the Windows inventory

The DCs go in the group `win_dc` of your Windows inventory (the example has it: `inventories/example-windows/hosts.yml`). WinRM is set up as for every Windows server (WINDOWS.md, steps 1 to 5). Test it with *Windows connection test* and Limit `win_dc`.

Put **every** DC in the group if you can. A DC that is not there is still covered by the domain-wide sections (replication from every DC, SRV records, FSMO, the changes), but its own services, dcdiag, events, time and disks are not read, and changes made on it show no "by whom".

2. The account

`dcdiag` and `repadmin` need administrator rights **on the DCs**, and the local Administrators group of a DC is the domain's built-in Administrators group: there is no smaller group that lets them run. So the job needs an account in the domain's **Administrators** (or **Domain Admins**) group.

- Make a **dedicated** account for it (e.g. `svc_aap_adhealth`), used by nothing else.
- Treat it as tier 0: allow it to log on to the DCs only (*Log on to* / an authentication policy silo), and add it to **Protected Users** if your AAP uses Kerberos (Protected Users blocks NTLM).
- Its password lives only in AAP, in a **Machine** credential of its own, e.g. `Windows DCs (WinRM)` (same fields as the `Windows (WinRM)` credential of WINDOWS.md, step 3).

Reading the **Deleted Objects** container (for "deleted" in the changes) also needs these rights; with less, that part is listed under *Not read* and the rest of the report is complete.

3. On the DCs

- The **ActiveDirectory** PowerShell module (`RSAT-AD-PowerShell`): installed with the AD DS role, so every DC has it unless it was removed. Without it the DC's LDAP checks and the domain-wide part are skipped and the report says so (amber).
- **English** dcdiag output: the job reads dcdiag's `passed test` / `failed test` lines. On a DC with another display language, dcdiag's results are reported as *not run* (amber).
- Windows Server 2012 R2 and later; Windows PowerShell 5.1. Constrained Language Mode (WDAC, AppLocker) is fine: the scripts are tested in it.

Set it up

1. **Templates > Create template > Create job template:**
   - Name: `Active Directory health report`
   - Inventory: `Windows servers`. Project: site-automation. Playbook: `playbooks/ad_health_report.yml`
   - Execution environment: your Windows EE (WINDOWS.md, step 2)
   - Credentials: **Windows DCs (WinRM)** (the account of step 2)
   - Tick **Prompt on launch** next to **Variables** and **Limit**
   - Variables:

```yaml
report_email_to: ad-team@yoursite.mil
```

   - Leave Limit empty (with no `target`, it reads the group `win_dc`). Save.
2. **First run: one DC.** Launch with Variables `target: dc01.yoursite.mil`. The domain-wide part is read on that DC too. Read the report in the job's output; anything under *Not read* says what that DC or account could not read (see Troubleshooting).
3. **All DCs**, then a schedule: **Schedules > Create schedule**, every day at 06:30 (set **your** time zone). Keep `ad_report_hours` (24) equal to the time between runs, so every change is in exactly one report.

A dry run (Check) reads and prints the same report, and sends no email. It takes one to three minutes per DC (mostly dcdiag); the DCs are read in parallel (the template's **Forks**).

The **PDC emulator** reads the domain-wide part when it is in the job; otherwise the first DC of the domain whose LDAP answered. The job output says which (`corp.example.mil: read by ...`).

Settings

Set them in the template's Variables, or in `playbooks/group_vars/all.yml` (or `win_dc.yml`). Ready to paste, with every setting switched off: aap/extra_vars/ad_health_report.yml.

Setting	Default	What it does
`ad_report_hours`	`24`	the changes of the last this-many hours (match the schedule)
`ad_event_hours`	`24`	the event log entries of the last this-many hours
`ad_security_events`	`true`	read the Security log for "by whom"
`ad_privileged_groups_extra`	`[DnsAdmins]`	more groups to watch (names or SIDs)
`ad_groups_should_be_empty`	`[]`	groups that must have no member, e.g. `[Schema Admins]`
`ad_privileged_change_severity`	`warning`	a privileged group change: `warning`, `critical` or `info`
`ad_schema_change_severity` / `ad_dc_change_severity`	`warning` / `info`	the schema changed; a DC added or removed
`ad_dcdiag_skip`	`[SystemLog]`	dcdiag tests to skip
`ad_dcdiag_dns`	`true`	also dcdiag's DNS tests that only read
`ad_trust_verify`	`true`	verify outbound and two-way trusts
`ad_repl_warn_hours` / `_crit_hours`	`6` / `24`	a replication link without success for this long (keep warn above your inter-site schedule)
`ad_repl_queue_warn`	`50`	requests in a DC's inbound replication queue
`ad_time_warn_seconds` / `_crit_seconds`	`5` / `60`	a DC's clock offset
`ad_time_sync_warn_hours`	`24`	no time sync for this long
`ad_disk_warn_pct` / `_crit_pct`	`15` / `5`	free % of the database, log and SYSVOL volumes
`ad_cert_warn_days` / `_crit_days`	`30` / `7`	the LDAPS / KDC certificate expires
`ad_backup_warn_days` / `_crit_days`	`7` / `30`	the last system-state backup of a partition
`ad_krbtgt_max_days`	`180`	the krbtgt password's age (the STIG asks for 180)
`ad_rid_warn_pct` / `_crit_pct`	`10` / `2`	RID pool left
`ad_no_client_site_warn`	`1`	client addresses in no subnet before it is amber

Setting	Default	What it does
`ad_max_events` / `ad_changes_max`	`2000` / `500`	the most event log entries read per log and DC; the most changed objects per kind
`ad_report_max_rows`	`50`	rows per table (problems first; the artifacts have all)
`ad_report_fail`	`false`	`true` = the job shows **failed** on a critical finding or a DC not read (for a workflow)

All settings of this job, with their defaults: ad_report.

Troubleshooting

You see	It means	Do
A DC **UNREACHABLE** with `ntlm: ... rejected` / `timed out`	WinRM to that DC failed	*Windows connection test* with Limit that DC; WINDOWS.md, "If something goes wrong"
LDAP **not checked** and `ActiveDirectory PowerShell module` under *Not read*	the module was removed from that DC	`Install-WindowsFeature RSAT-AD-PowerShell`
dcdiag **not run**: `no test result in its output (is dcdiag in English?)`	dcdiag prints another language	run the job with an account whose display language is English, or accept it (the other sections do not need dcdiag)
dcdiag: `The term 'dcdiag.exe' is not recognized`	the host is not a DC	take it out of `win_dc`
`objects deleted (reading Deleted Objects needs ...)` under *Not read*	the account may not list Deleted Objects	use an account in Domain Admins, or give the account read access to `CN=Deleted Objects` (Microsoft documents it: `dsacls` with `/takeownership`, then `/G <account>:LCRP`)
"By" is `-` everywhere	the change was made on a DC not in the job, or the audit policy does not record it	add the DC to `win_dc`; turn on the audit subcategories above
Replication **amber** every night on site links	the link replicates on a schedule longer than `ad_repl_warn_hours`	raise `ad_repl_warn_hours` above the site link's interval
Many **5719** / **5807** events	clients in subnets AD does not know	add the subnets in *Active Directory Sites and Services* (the *Clients in no site* table lists them)
GPO **version AD x, SYSVOL y**	SYSVOL has not caught up with an edit, or a GPO is broken on this DC	wait for the next run; if it stays, check DFSR on this DC (`dfsrdiag backlog`)

Limits

- Group membership is **direct**: a member added to a group that is itself in Domain Admins shows as a change of that group only if that group is watched (add it to `ad_privileged_groups_extra`).
- "Created" and "changed" times of accounts are AD's own; "changed" GPOs use the reading DC's `whenChanged` (a GPO whose links changed shows under *links*, not as changed).
- A forest with several domains: put the DCs of every domain in `win_dc`; each domain is read by one of its own DCs. Schema and forest-wide FSMO roles are reported by the forest root domain.
- The report covers what the DCs say about themselves and the domain. It does not test logons from clients, and it does not read Azure AD / Entra ID.

Appendix: credential types to paste into AAP

Each block below is one file in `aap/credential_types/`. In AAP: **Automation Execution > Infrastructure > Credential Types > Create credential type**. Paste the part under `inputs:` into **Input configuration** and the part under `injectors:` into **Injector configuration** (without those two top-level lines). Copy from the repository file where you can; PDF copy can change spaces.

ServiceNow API (servicenow_api.yml)

Used by playbooks/servicenow_tickets.yml, servicenow_health.yml and servicenow_test_ticket.yml (REST Table API, basic auth only). The same variable names as the certified servicenow.itsm collection, so this credential also works if you move to that collection later.

YAML

```yaml
inputs:
  fields:
    - id: instance
      type: string
      label: Instance URL
      help_text: e.g. https://servicenow.example.mil
    - id: username
      type: string
      label: API user name
    - id: password
      type: string
      label: API password
      secret: true
  required: [instance, username, password]
injectors:
  env:
    SN_HOST: "{{ instance }}"
    SN_USERNAME: "{{ username }}"
    SN_PASSWORD: "{{ password }}"
```

MariaDB monitor (mariadb_monitor.yml)

For the mariadb health check. Needed when MariaDB runs in a container (the official images give root a password); optional on a host install, where the check can log in as root through the local socket. The password reaches the client only through its environment (MYSQL_PWD), never a command line. The account needs no data access at all (docs/MARIADB.md section 4 has the GRANT).

YAML

```yaml
inputs:
  fields:
    - id: username
      type: string
      label: Monitor account
    - id: password
      type: string
      label: Password
      secret: true
  required: [username, password]
injectors:
  env:
    MARIADB_MONITOR_USER: "{{ username }}"
    MARIADB_MONITOR_PASSWORD: "{{ password }}"
```

Keystore password (keystore_password.yml)

Optional, for the certs check on a PKCS12 keystore (JKS keystores list without a password). In the inventory, name the variable, never the password: check_certs_keystores: [{path: /path/to/keystore, password_env: KEYSTORE_PASSWORD}]

YAML

```
inputs:
  fields:
    - id: password
      type: string
      label: Keystore password
      secret: true
  required: [password]
injectors:
  env:
    KEYSTORE_PASSWORD: "{{ password }}"
```

STIG Manager API (stigman_api.yml)

Optional, for the STIG Manager cross-check in playbooks/poam_status.yml. The client is created by your Keycloak realm owner (client-credentials grant, scope stig-manager:collection:read, and a grant in the collections to read) - this repository never changes the realm.

YAML

```
inputs:
  fields:
    - id: token_url
      type: string
      label: Token URL
      help_text: https://keycloak.example.mil/realms/<realm>/protocol/openid-connect/token
    - id: client_id
      type: string
      label: Client ID
    - id: client_secret
      type: string
      label: Client secret
      secret: true
  required: [token_url, client_id, client_secret]
injectors:
  env:
    STIGMAN_TOKEN_URL: "{{ token_url }}"
    STIGMAN_CLIENT_ID: "{{ client_id }}"
    STIGMAN_CLIENT_SECRET: "{{ client_secret }}"
```

ACT model key (act_model_key.yml)

```yaml
inputs:
  fields:
    - id: genai_key
      type: string
      label: GenAI API key
      secret: true
    - id: asksage_key
      type: string
      label: AskSage API key
      secret: true
injectors:
  env:
    GENAI_KEY: "{{ genai_key }}"
    ASKSAGE_API_KEY: "{{ asksage_key }}"
```

ACT GenAI beta key (act_genai_beta_key.yml)

Only for ACT with site_act_provider=genai-beta (the GenAI.mil beta proxy). A separate type, so an "ACT model key" type that is already in use does not have to change (AAP refuses to change the fields of a credential type that credentials use). Attach it next to "ACT model key".

```yaml
inputs:
  fields:
    - id: genai_beta_key
      type: string
      label: GenAI beta API key
      secret: true
  required: [genai_beta_key]
injectors:
  env:
    GENAI_BETA_KEY: "{{ genai_beta_key }}"
```

STIG Manager database (stigman_database.yml)

```yaml
inputs:
  fields:
    - id: mysql_root_password
      type: string
      label: MySQL root password
      secret: true
      help_text: Used only when the database is first created (MYSQL_ROOT_PASSWORD_FILE).
    - id: stigman_db_password
      type: string
      label: STIG Manager database user password
      secret: true
  required: [mysql_root_password, stigman_db_password]
injectors:
  env:
    STIGMAN_MYSQL_ROOT_PASSWORD: "{{ mysql_root_password }}"
    STIGMAN_DB_PASSWORD: "{{ stigman_db_password }}"
```

TLS certificate (tls_certificate.yml)

AAP writes each value to a private temporary file inside the job's execution environment and puts the file's PATH in the environment variable; the playbook copies the files to the host.

YAML

```
inputs:
  fields:
    - id: certificate
      type: string
      label: Certificate (PEM; server certificate first, then intermediates)
      multiline: true
    - id: private_key
      type: string
      label: Private key (PEM)
      multiline: true
      secret: true
    - id: ca_chain
      type: string
      label: CA chain (PEM) - also used to trust your Keycloak/other internal services
      multiline: true
  required: [certificate, private_key]
injectors:
  file:
    template.cert: "{{ certificate }}"
    template.key: "{{ private_key }}"
    template.ca: "{{ ca_chain }}"
  env:
    TLS_CERT_FILE: "{{ tower.filename.cert }}"
    TLS_KEY_FILE: "{{ tower.filename.key }}"
    TLS_CA_FILE: "{{ tower.filename.ca }}"
```

Container registry login (hosts) (registry_login.yml)

For podman on the MANAGED HOSTS (the built-in "Container Registry" credential type is for pulling execution-environment images on AAP itself, not for the hosts).

YAML

```
inputs:
  fields:
    - id: registry
      type: string
      label: Registry (e.g. docker.io, quay.io)
    - id: username
      type: string
      label: Username
    - id: password
      type: string
      label: Password or access token
      secret: true
  required: [registry, username, password]
injectors:
  env:
    REGISTRY_HOST: "{{ registry }}"
    REGISTRY_USERNAME: "{{ username }}"
    REGISTRY_PASSWORD: "{{ password }}"
```

Teams webhook (teams_webhook.yml)

```yaml
inputs:
  fields:
    - id: webhook_url
      type: string
      label: Teams Workflows webhook URL
      secret: true
  required: [webhook_url]
injectors:
  env:
    TEAMS_WEBHOOK_URL: "{{ webhook_url }}"
```

SMTP relay (smtp_relay.yml)

Only when your mail relay needs a login to send. The jobs that email a report (report_email_to, roles/site_email) read SMTP_USERNAME / SMTP_PASSWORD from it. A relay that accepts mail from the AAP server without a login needs no credential.

```yaml
inputs:
  fields:
    - id: username
      type: string
      label: SMTP user name
    - id: password
      type: string
      label: SMTP password
      secret: true
  required: [username, password]
injectors:
  env:
    SMTP_USERNAME: "{{ username }}"
    SMTP_PASSWORD: "{{ password }}"
```